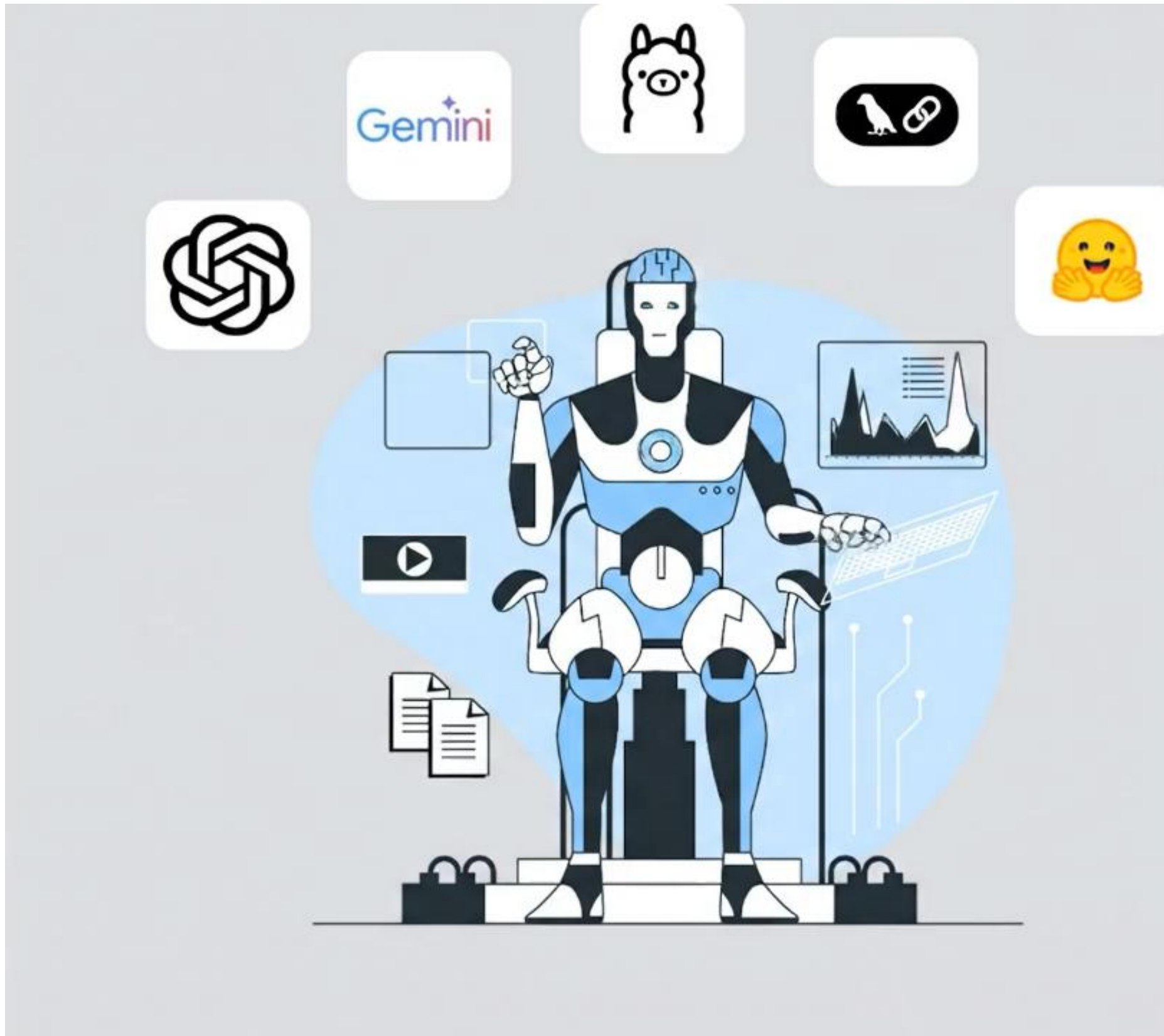


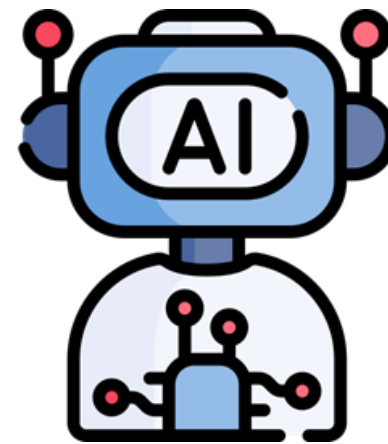
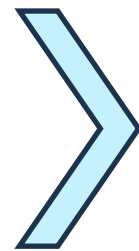


Building Autonomous Agentic AI Systems for Beginners Hands-On

- By Thinknyx Technologies LLP

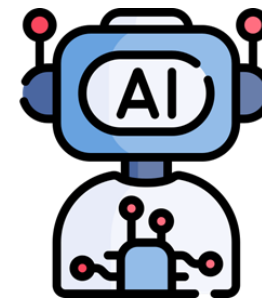


**Prompt-based
System**



**Agentic AI
System**

- ✓ Reason
- ✓ Plan
- ✓ Use Tools
- ✓ Take Actions Autonomously



Agentic AI

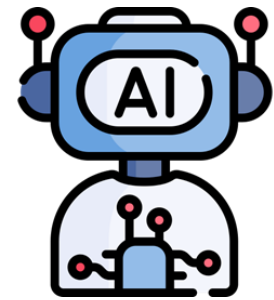
AI system that can independently set goals, make decisions, and execute actions to achieve outcomes with minimal human intervention



**Agentic AI
Fundamentals**

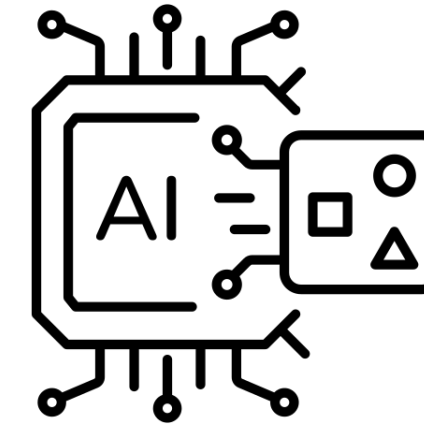
Building Agentic AI

**Enhancing Real-world
Agents**

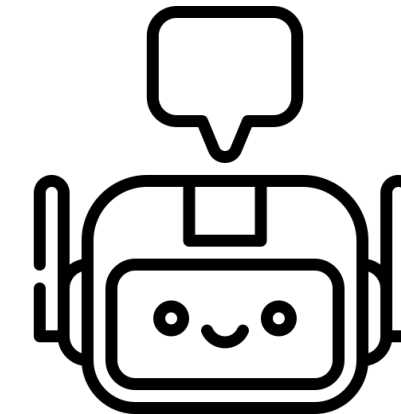


Agentic AI

How Agentic AI Works?



Design



Build



Yogesh Raheja



Amanpreet Singh



Dheeraj Sain



Deepthi Narayan



Yogesh Raheja





Understand Agentic AI and how it differs from traditional AI

Foundations of Agentic AI: LLMs, agents, memory, and RAG

Build hands-on agents using modern frameworks

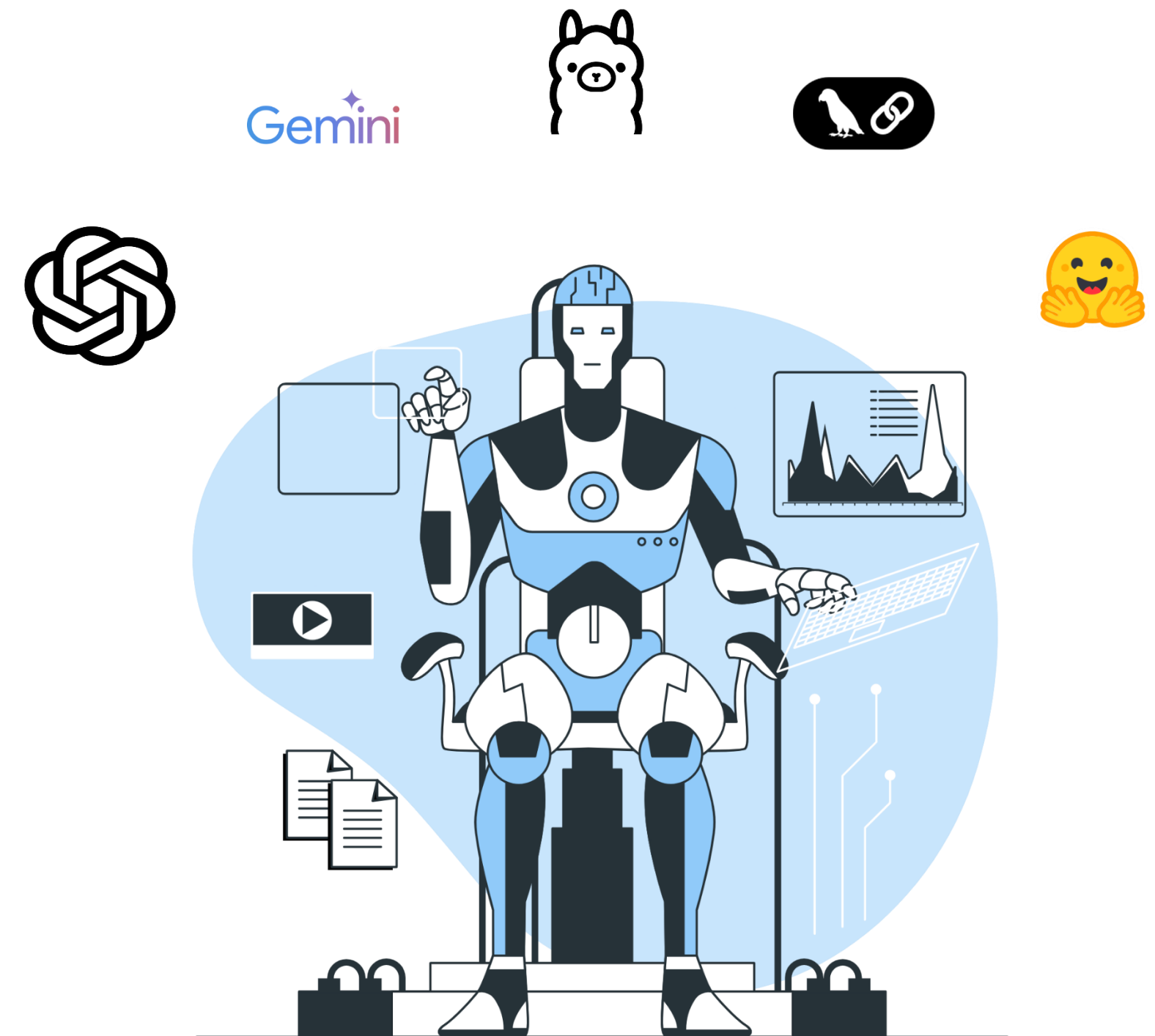
Integrate agents with tools and workflows (MCP, n8n)

Develop multi-agent and RAG-enhanced systems

Apply deployment, optimization, and safety best practices

Section 1.0

Introduction to Agentic AI



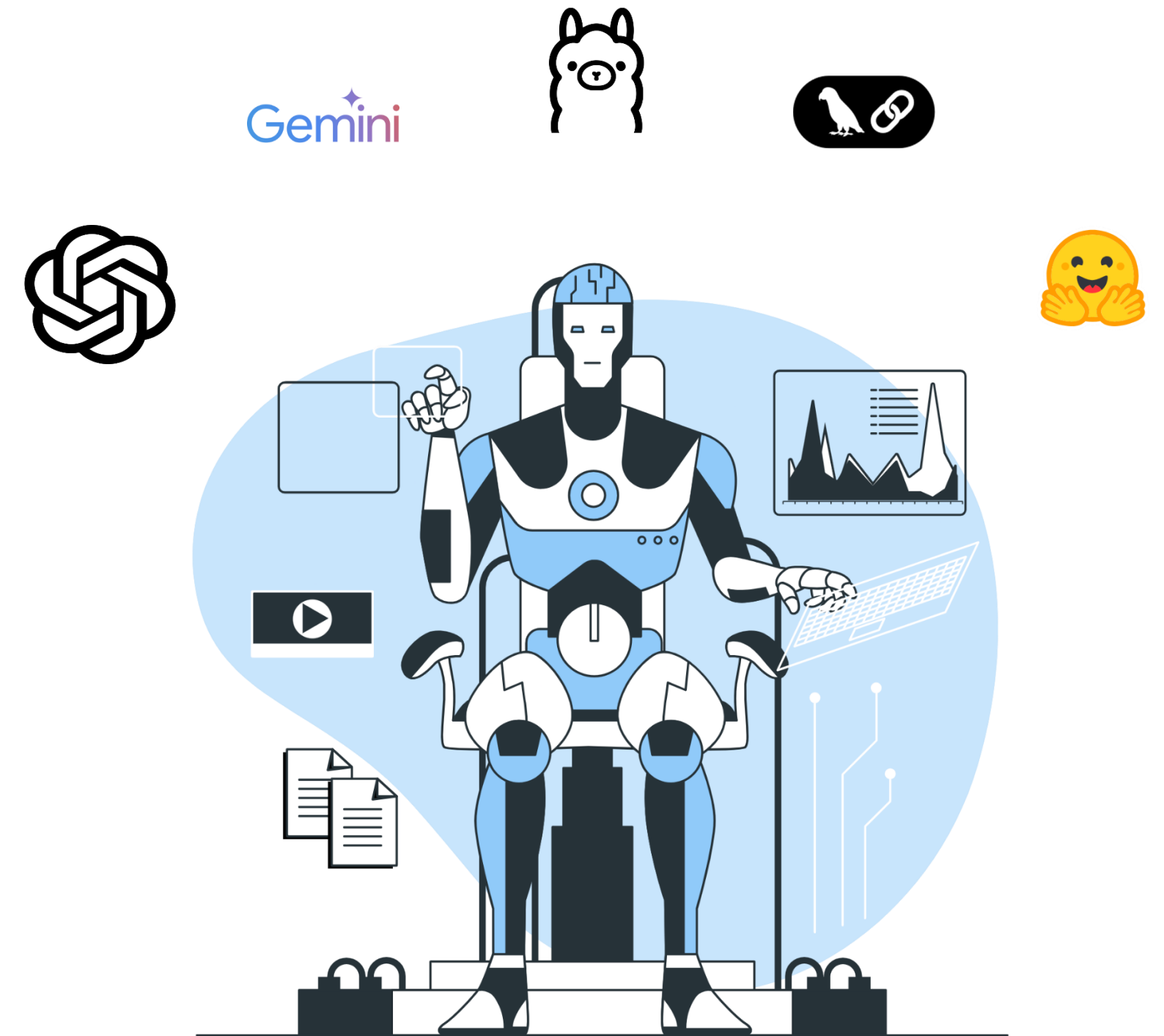
Introduction to Agentic AI

Section Overview

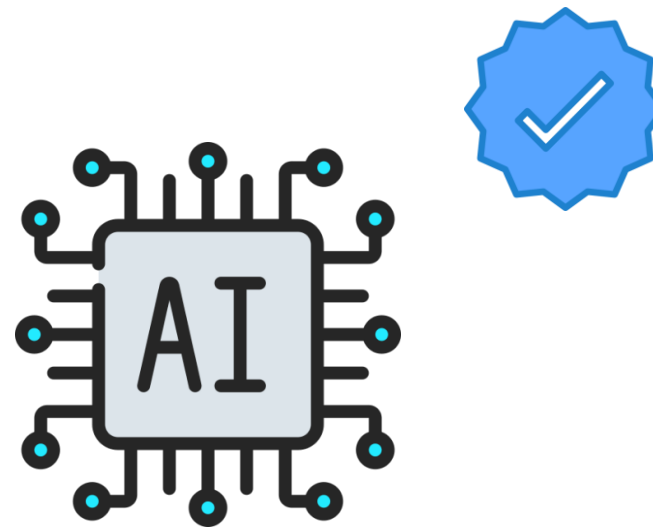
- ❑ What is Agentic AI
- ❑ Understand how it differs from Generative AI and AI Agents
- ❑ Building blocks of an Agentic AI system
- ❑ Explore Industry Impact of Agentic AI
- ❑ Understand the Limitations of Agentic AI

Section 1.1

What is Agentic AI?

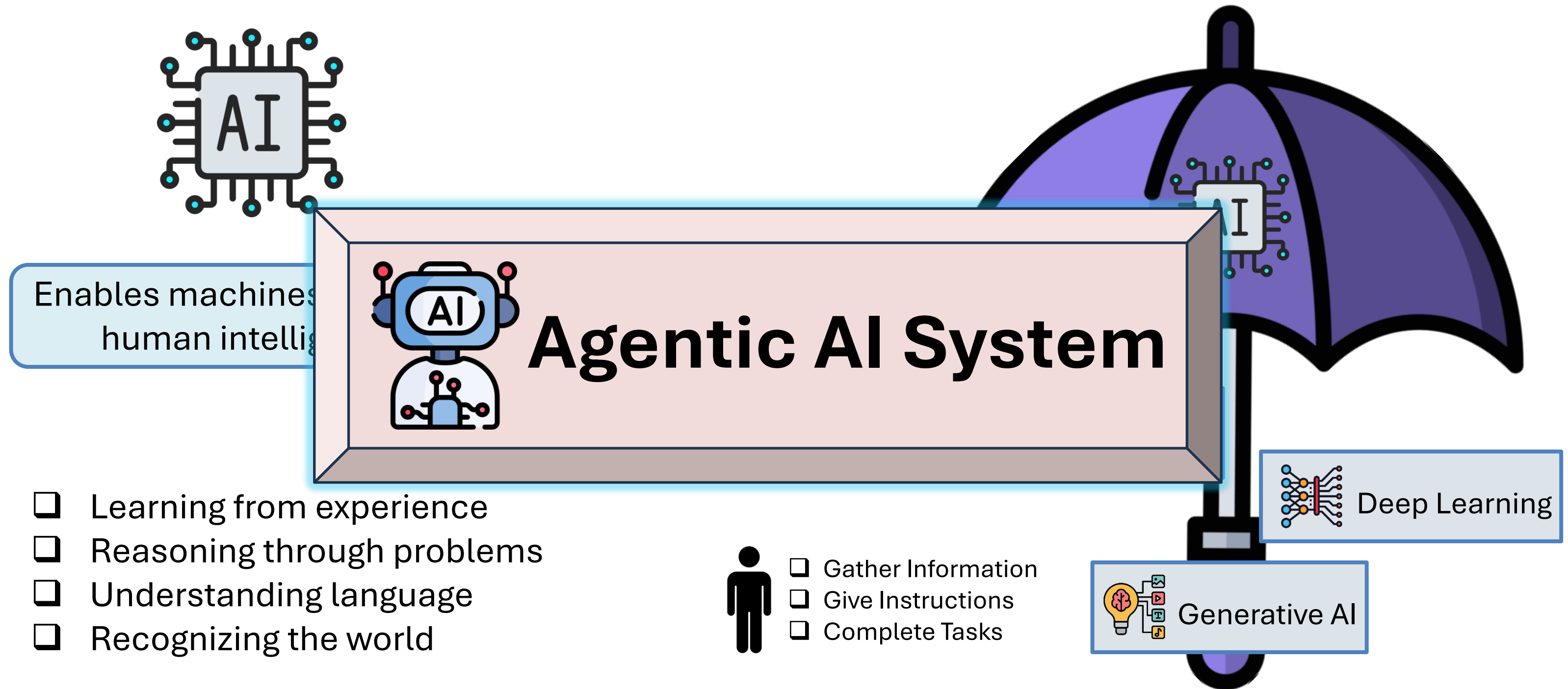


What is Agentic AI?

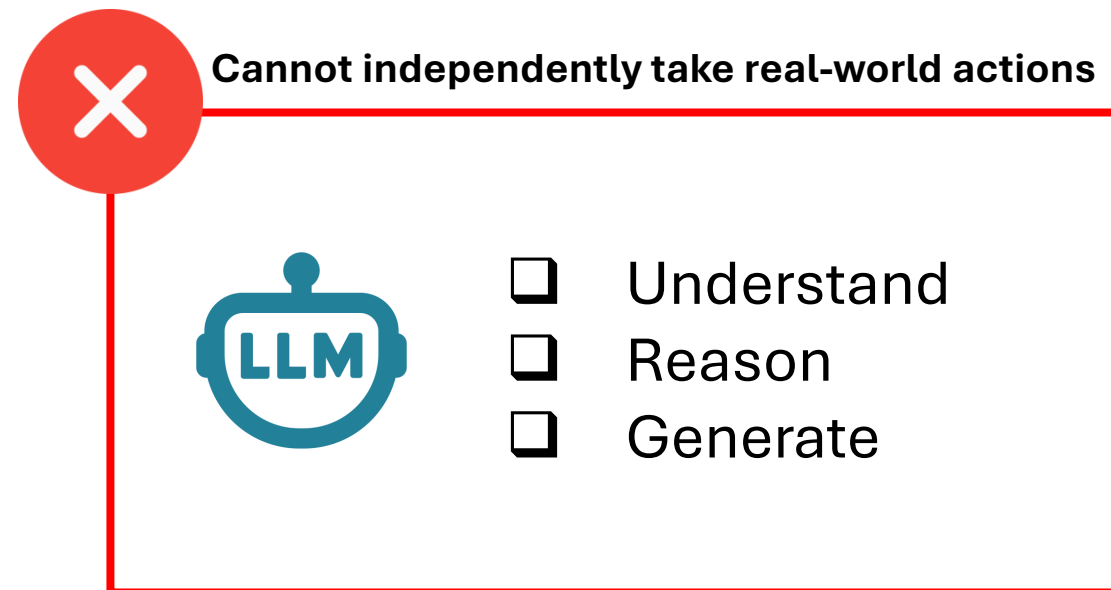
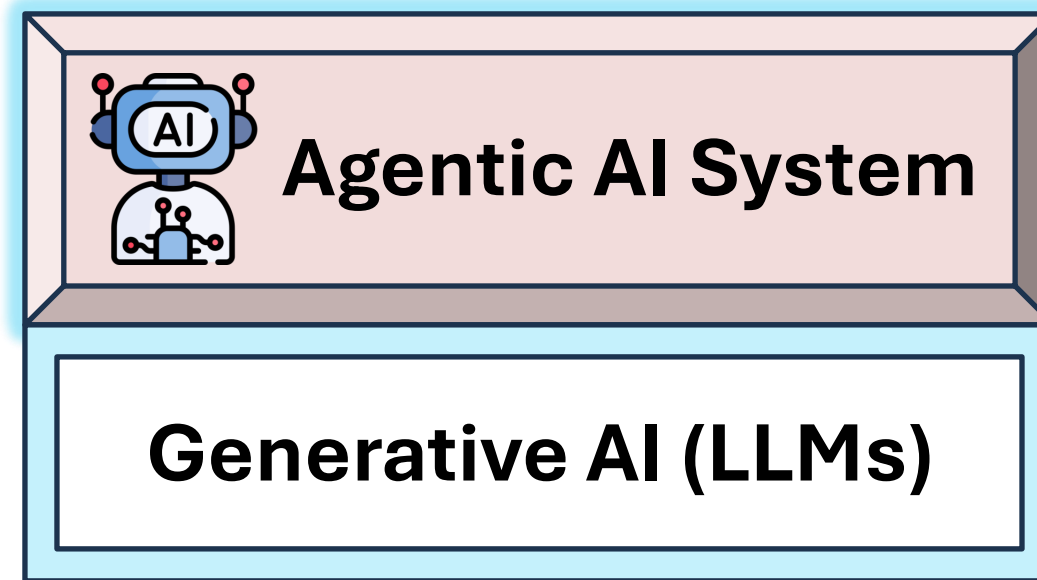


Artificial Intelligence

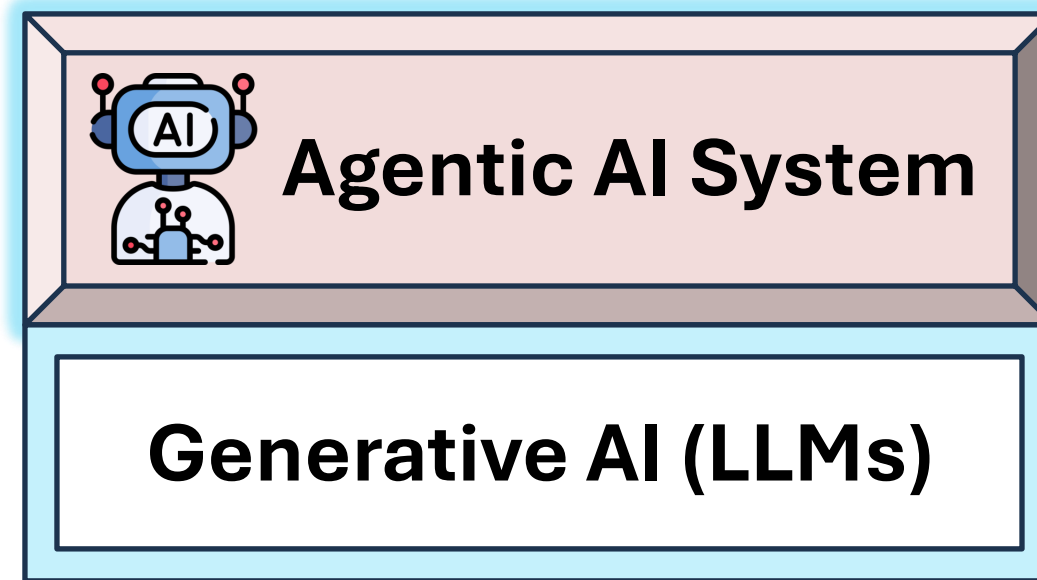
What is Agentic AI?



What is Agentic AI?



What is Agentic AI?



Agentic AI



Decision-making systems

- ☐ Calling APIs
- ☐ Searching the web
- ☐ Querying Databases
- ☐ Monitoring Information

What is Agentic AI?

Online Food Order



Open the App

Search Restaurants

Find Dish

Check for Discounts

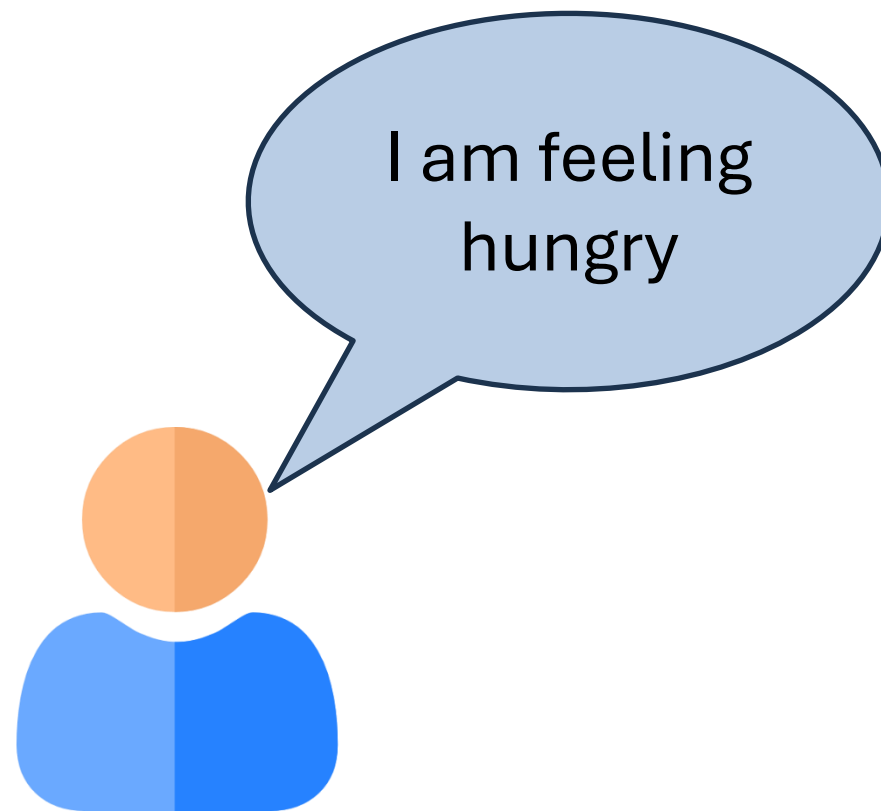
Add to the Cart

Make Payment

Track the Order

What is Agentic AI?

Online Food Order



Agentic AI System

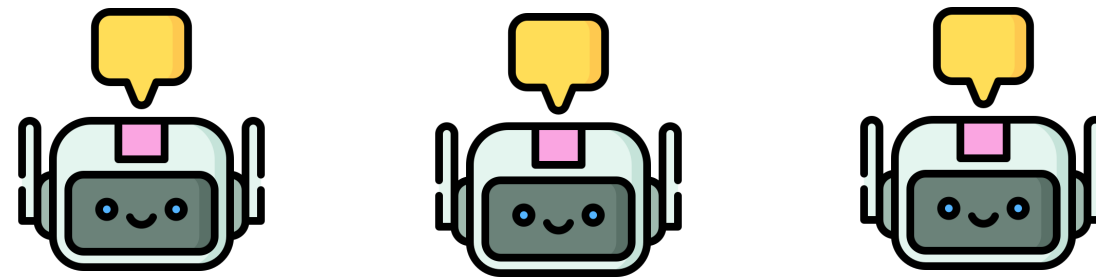
- ✓ Chooses the App
- ✓ Picks Favourite Restaurant
- ✓ Chooses Food
- ✓ Applies Best Discount
- ✓ Places the Order
- ✓ Make Payment
- ✓ Track the Order

What is Agentic AI?

Agentic AI

Artificial intelligence system designed to achieve specific goals with minimal human supervision

AI Agents

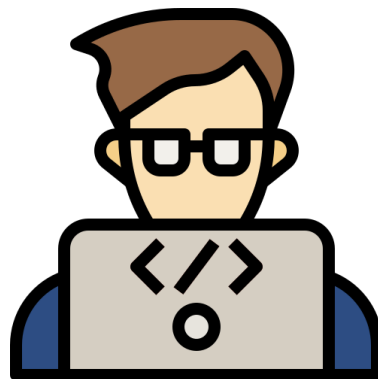


Intelligent models that mimic human decision-making for real-time problem solving

What is Agentic AI?

Setting up Development Environment

Traditional Way



Install Tools

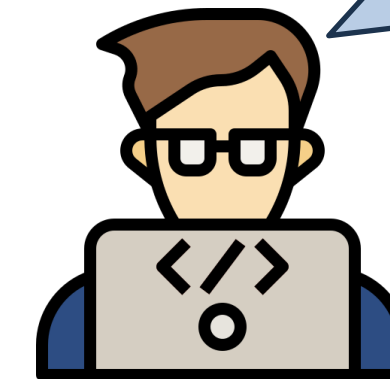
Configure IDE

Setup Docker

Download Dependencies

Connect Project Codebase

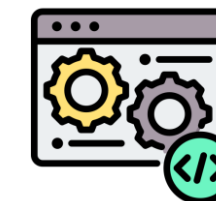
Agentic AI



Set up my environment for Project X



Installs Tools



Configures Environment

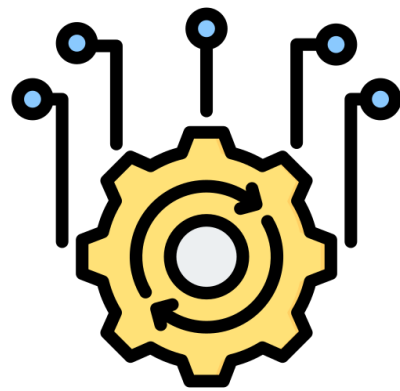


Pulls Code

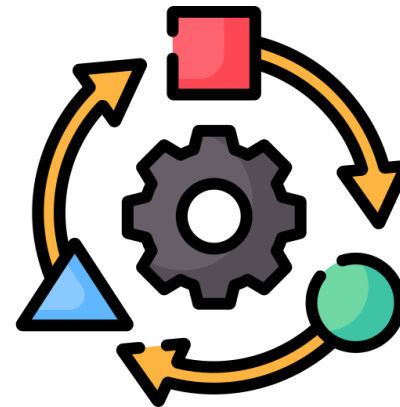


What is Agentic AI?

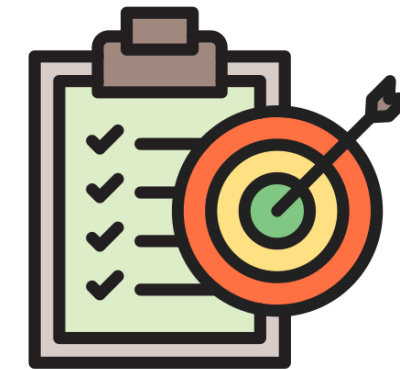
➤ Agentic AI



Act Autonomously

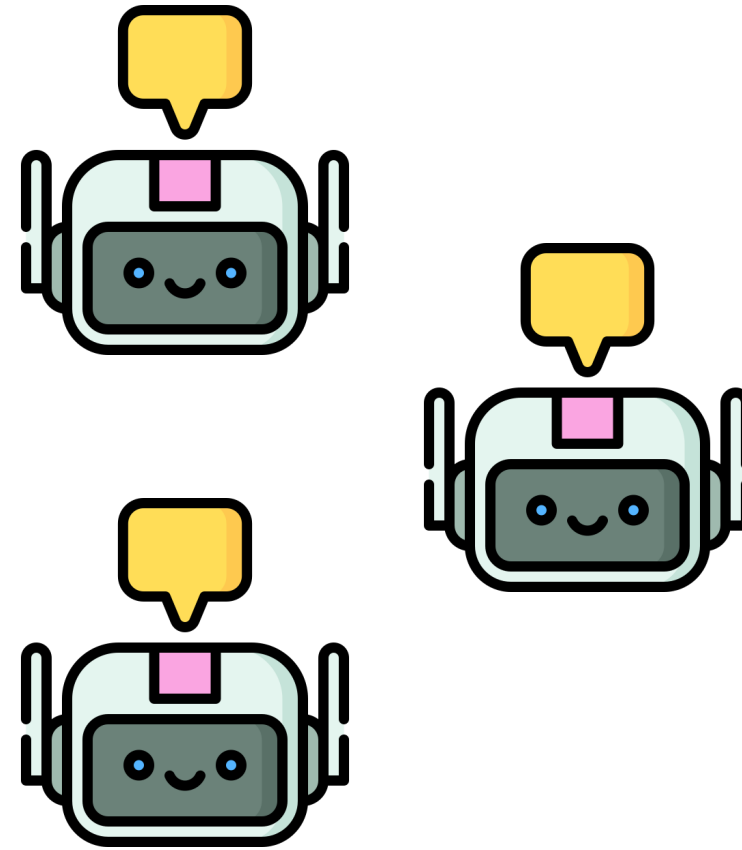


Adapt to real-time
situations



Pursue specific goals
with min. supervision

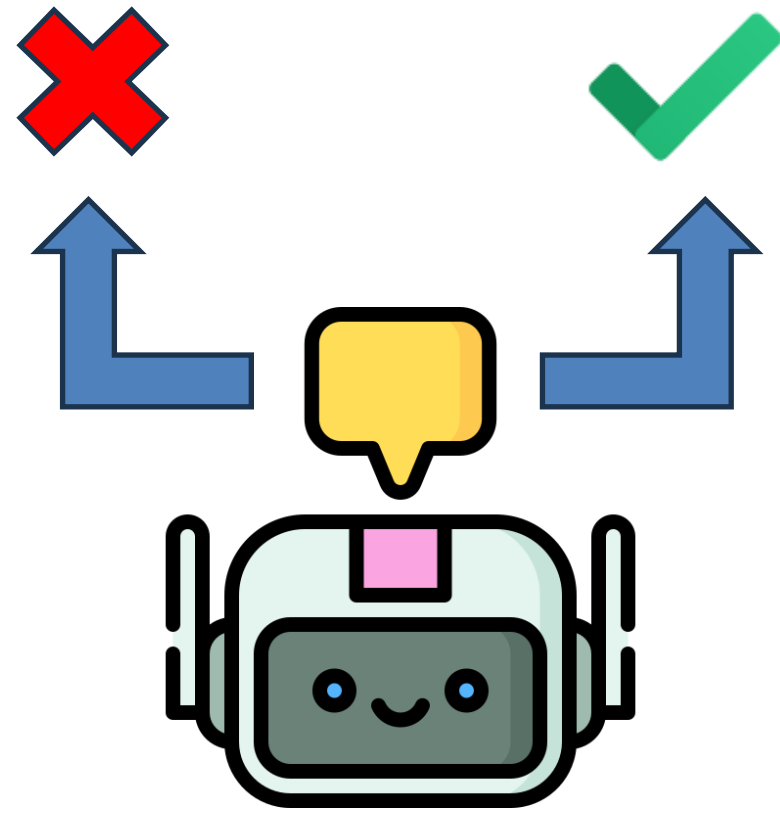
What is Agentic AI?



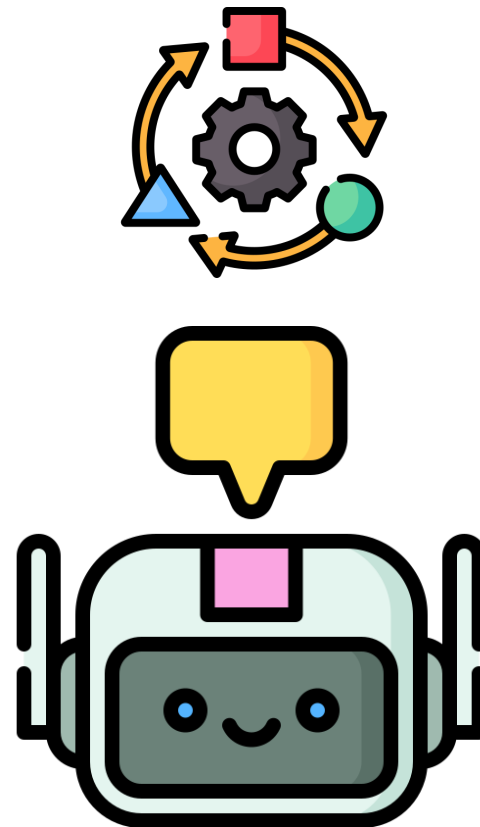
AI Agents

Machine learning models that mimic human decision making to solve problems in dynamic environments

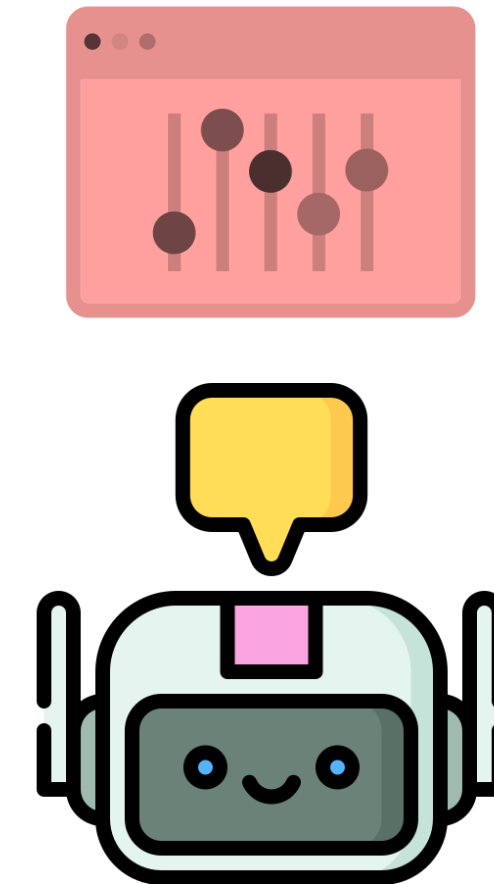
What is Agentic AI?



Independent Decisions



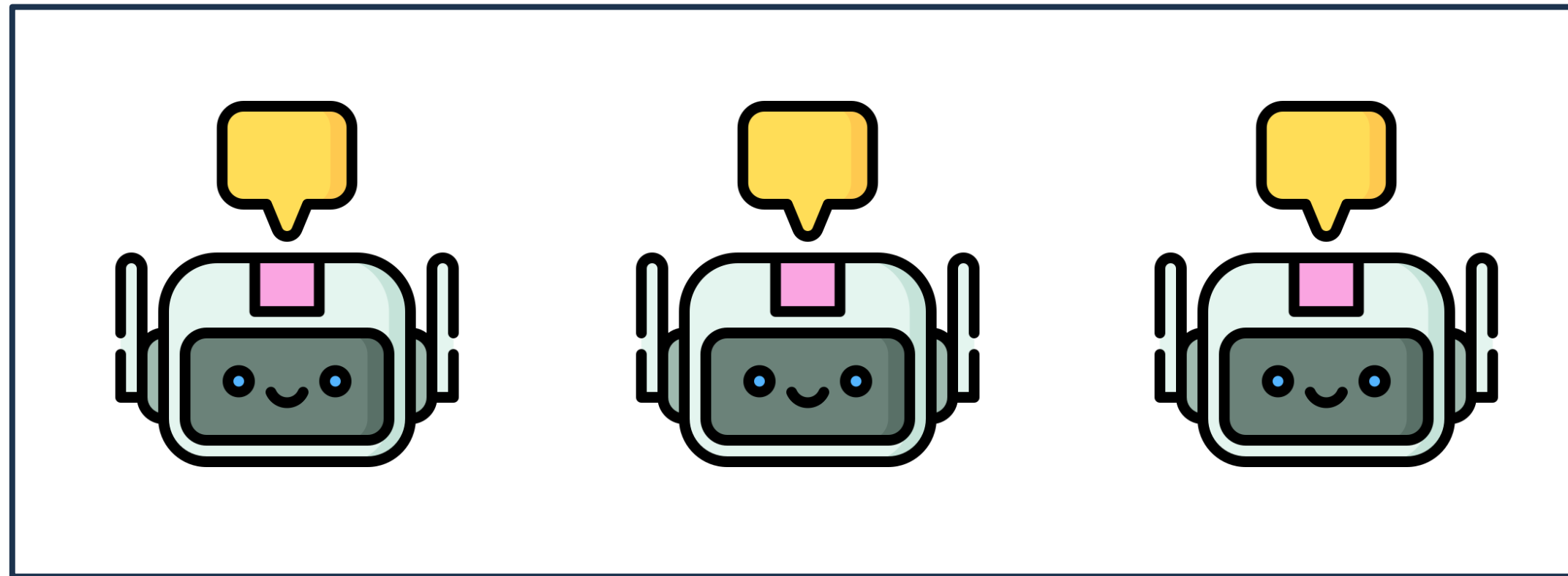
Adapt to Changing Inputs



Optimize Actions

AI Agents

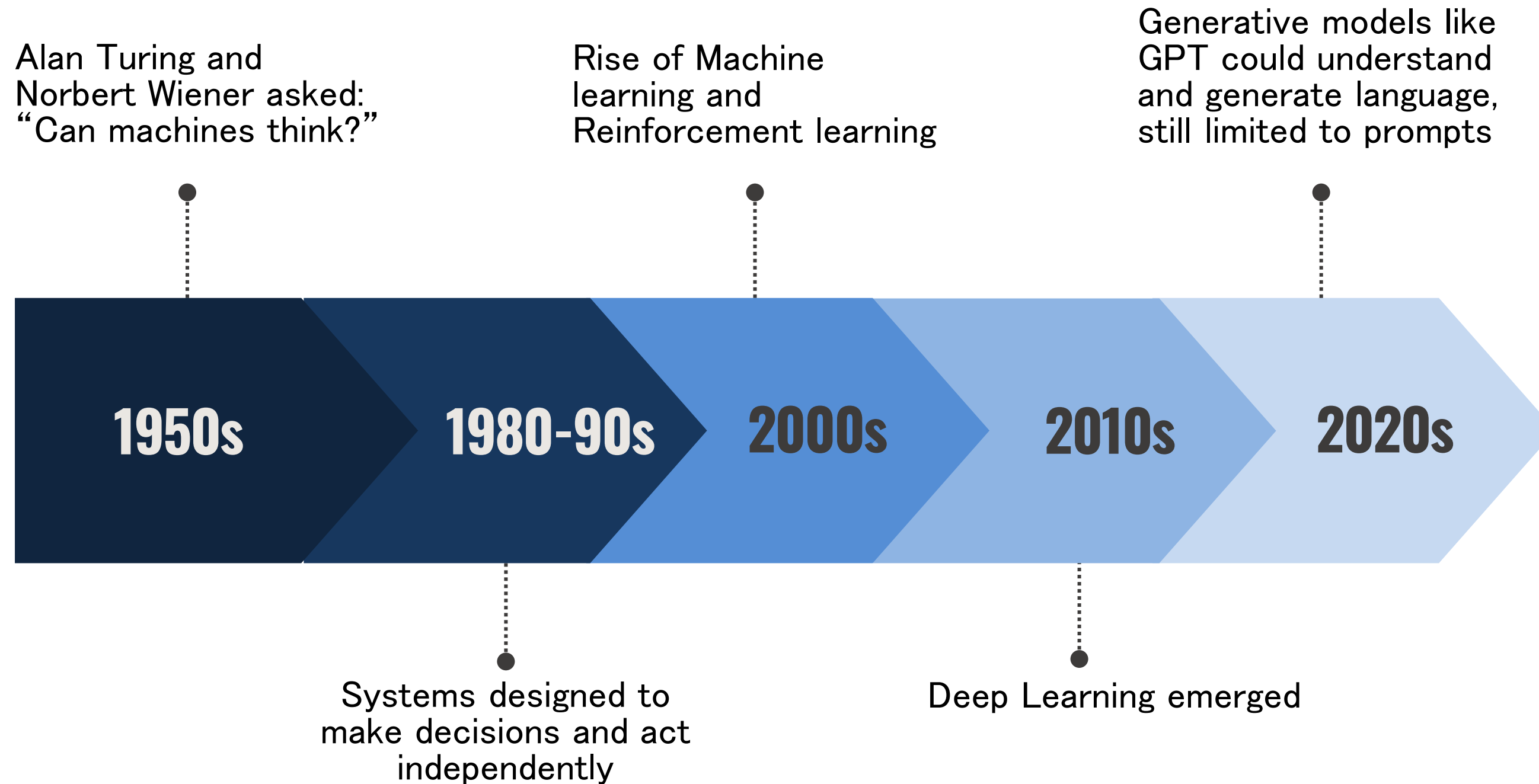
What is Agentic AI?



Agency - The ability to take purposeful action

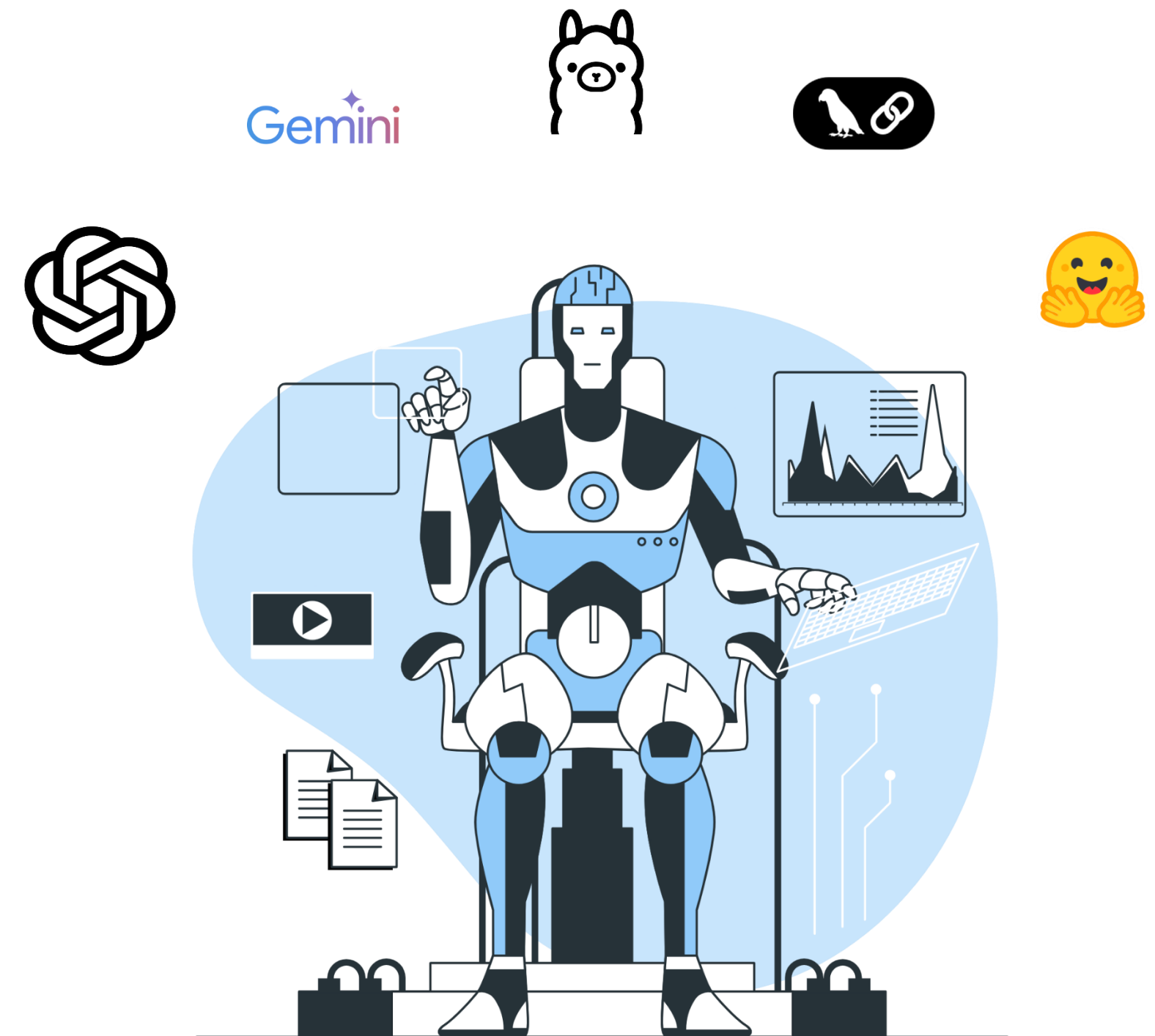
What is Agentic AI?

➤ History

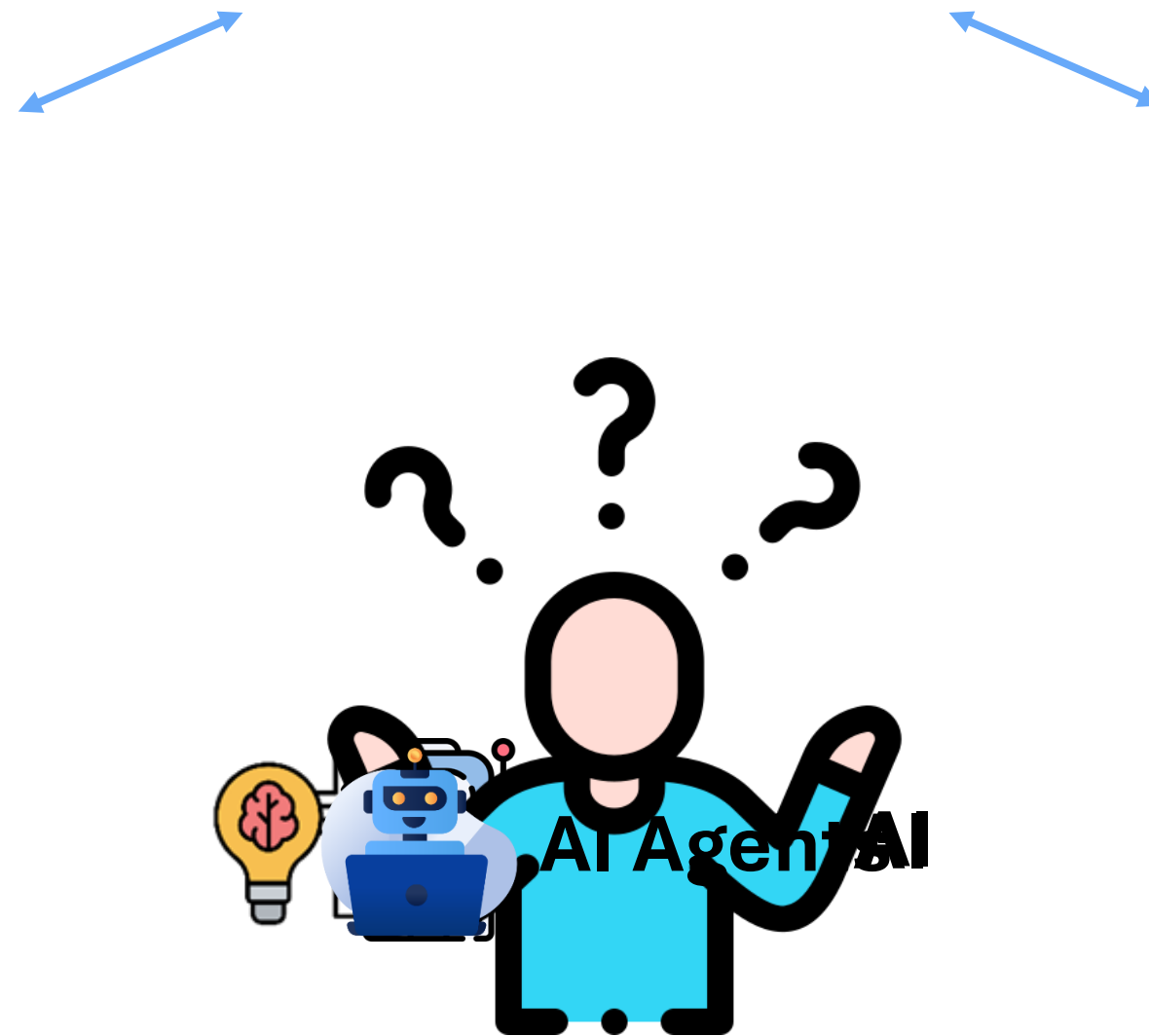


Section 1.2

How Agentic AI is different from Generative AI and AI Agents?



How Agentic AI is different from Generative AI and AI Agents?



How Agentic AI is different from Generative AI and AI Agents?

Imagine:

- ☐ It is Friday evening
- ☐ Hungry & tired
- ☐ Want to order Pizza

How Agentic AI is different from Generative AI and AI Agents?

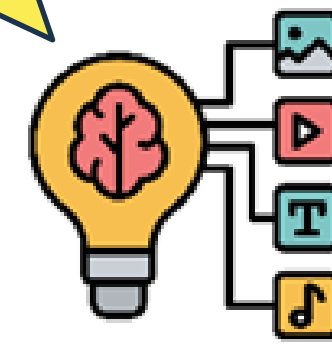


Generative AI (Content Creation)

I'm in the mood for
pizza



Here's a recipe for
mushroom pizza with garlic
butter crust and here's
what it would look like
fresh out of the oven!



Imaginative

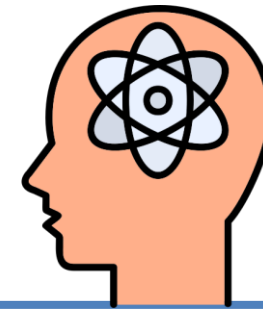
Creative

Doesn't order the pizza for you

How Agentic AI is different from Generative AI and AI Agents?



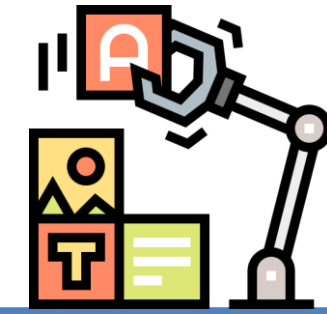
Generative AI



Think



Design



Create

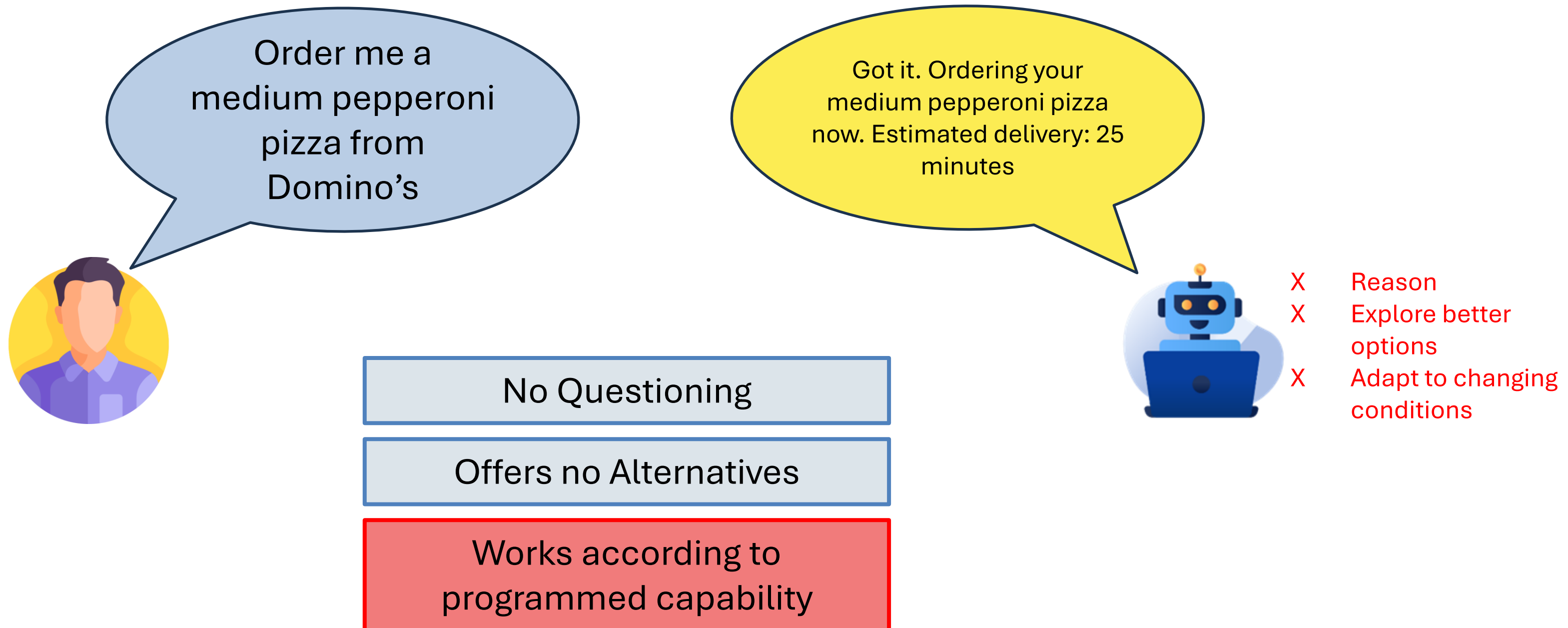


Real-world Action

How Agentic AI is different from Generative AI and AI Agents?



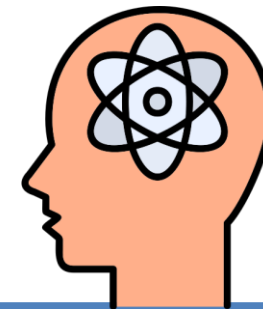
AI Agents (Action-oriented but Instruction-bound)



How Agentic AI is different from Generative AI and AI Agents?



AI Agents



Efficient

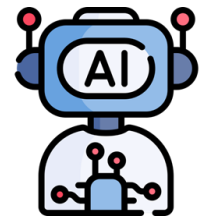


Reliable



Not Curious or Creative

How Agentic AI is different from Generative AI and AI Agents?



Agentic AI (Goal-Oriented)

Intent

Context

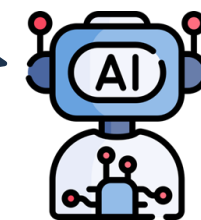
Preferences



I'm hungry

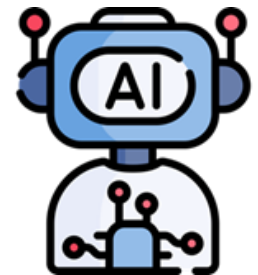
Noted

- ☐ You usually order pizza on Fridays
- ☐ Your favourite place is closed tonight, but there's a new pizzeria nearby with great reviews
- ☐ I'll order their wood-fired margherita, it's lighter since you had a burger for lunch
- ☐ Would you like me to start your favourite show while you eat?



Agentic AI

How Agentic AI is different from Generative AI and AI Agents?



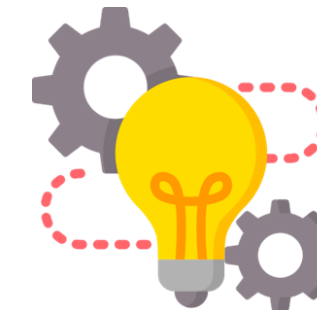
Agentic AI



Initiatives



Plan



Reason



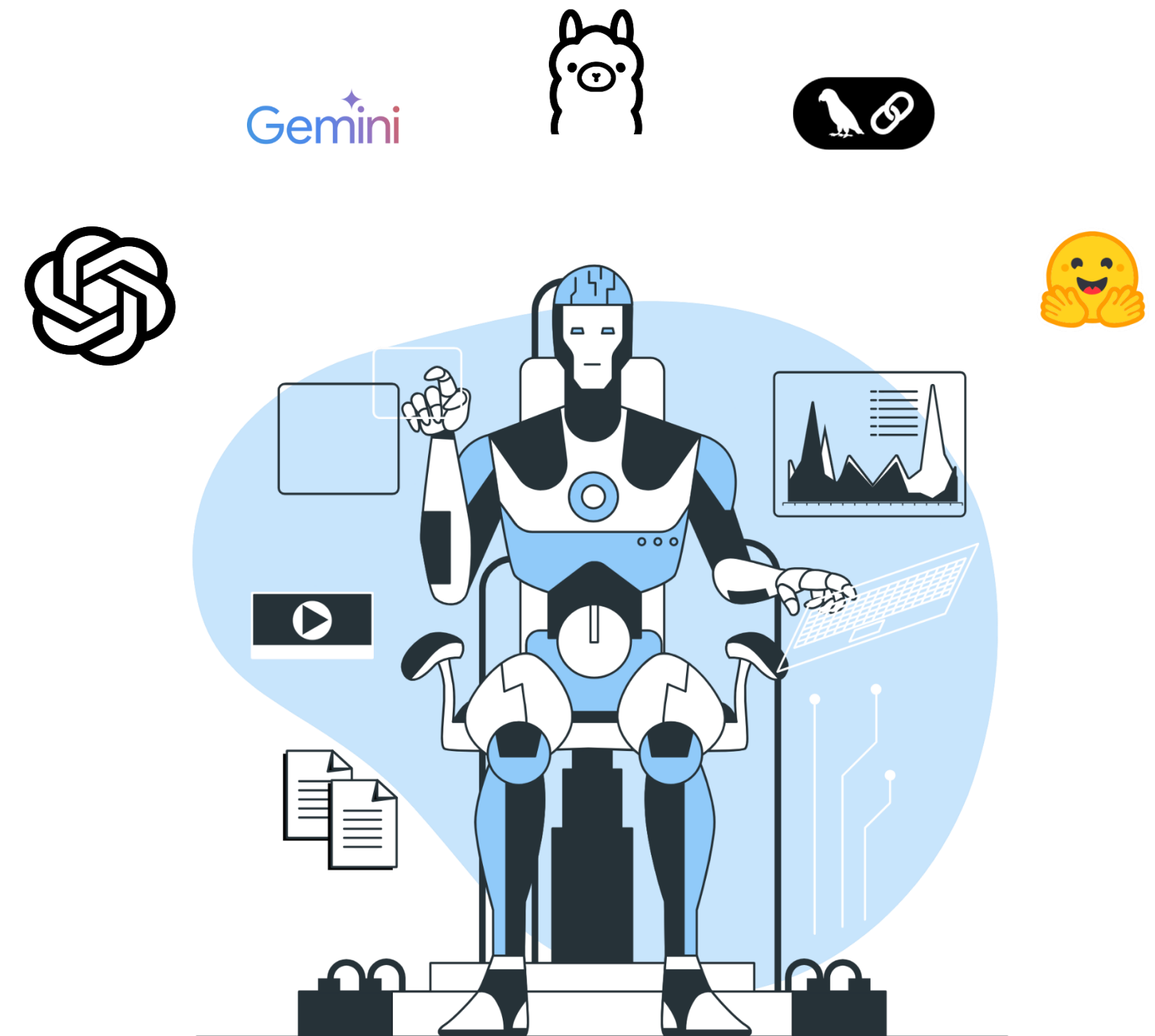
Act

How Agentic AI is different from Generative AI and AI Agents?

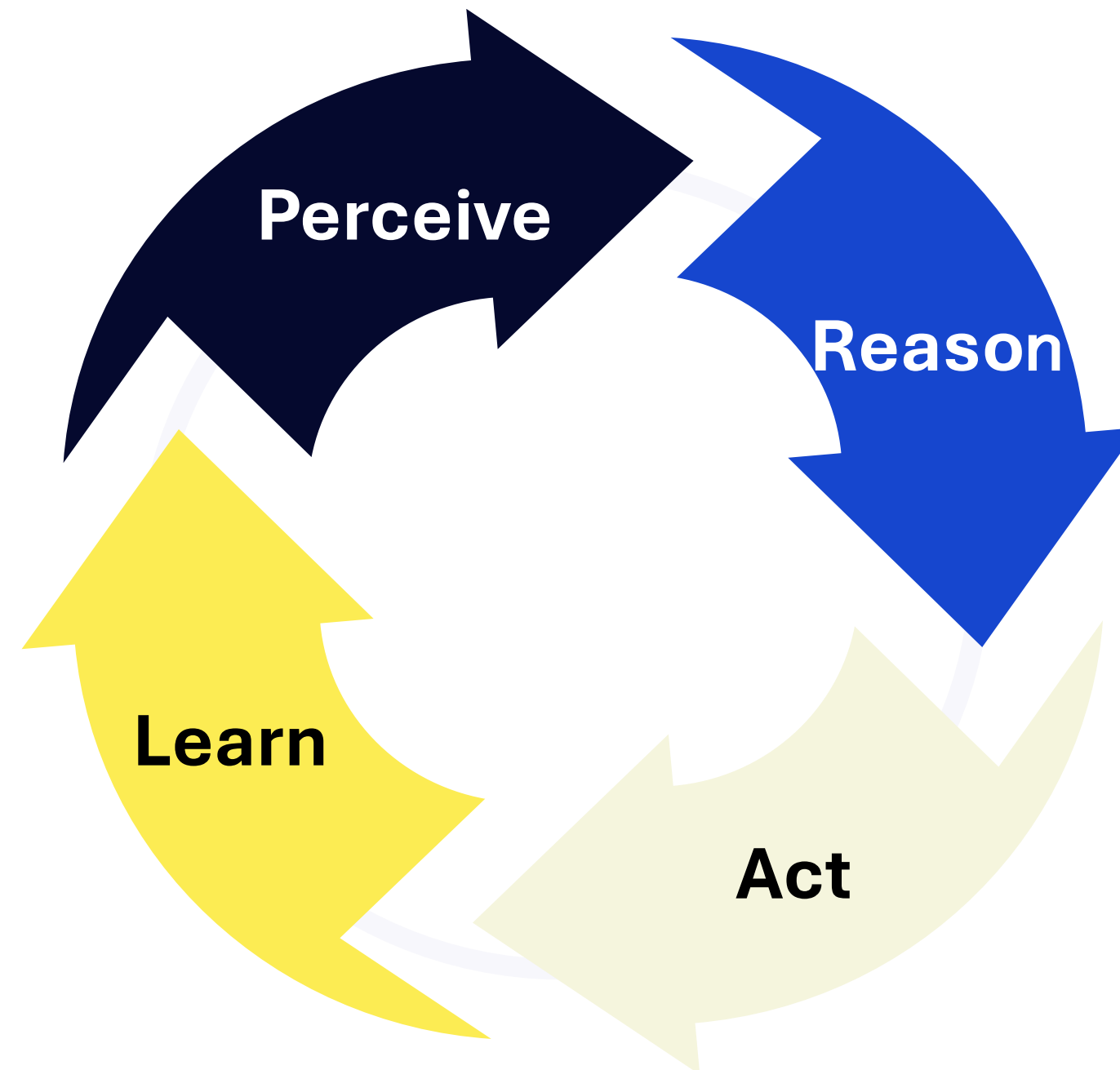
Features	Generative AI	AI Agents	Agentic AI
Purpose	Create Content	Perform Specific Task	Achieve Complex Goals
Autonomy	Low	Medium	High
Adaptability	Limited	Task-aware	Dynamic & Evolving
Initiative	Reactive	Task-driven	Goal-driven
Examples	ChatGPT, Gemini	Chatbots, Smart Vacuums	Autonomous vehicles, AI Traders

Section 1.3

Building Blocks of an Agentic AI System

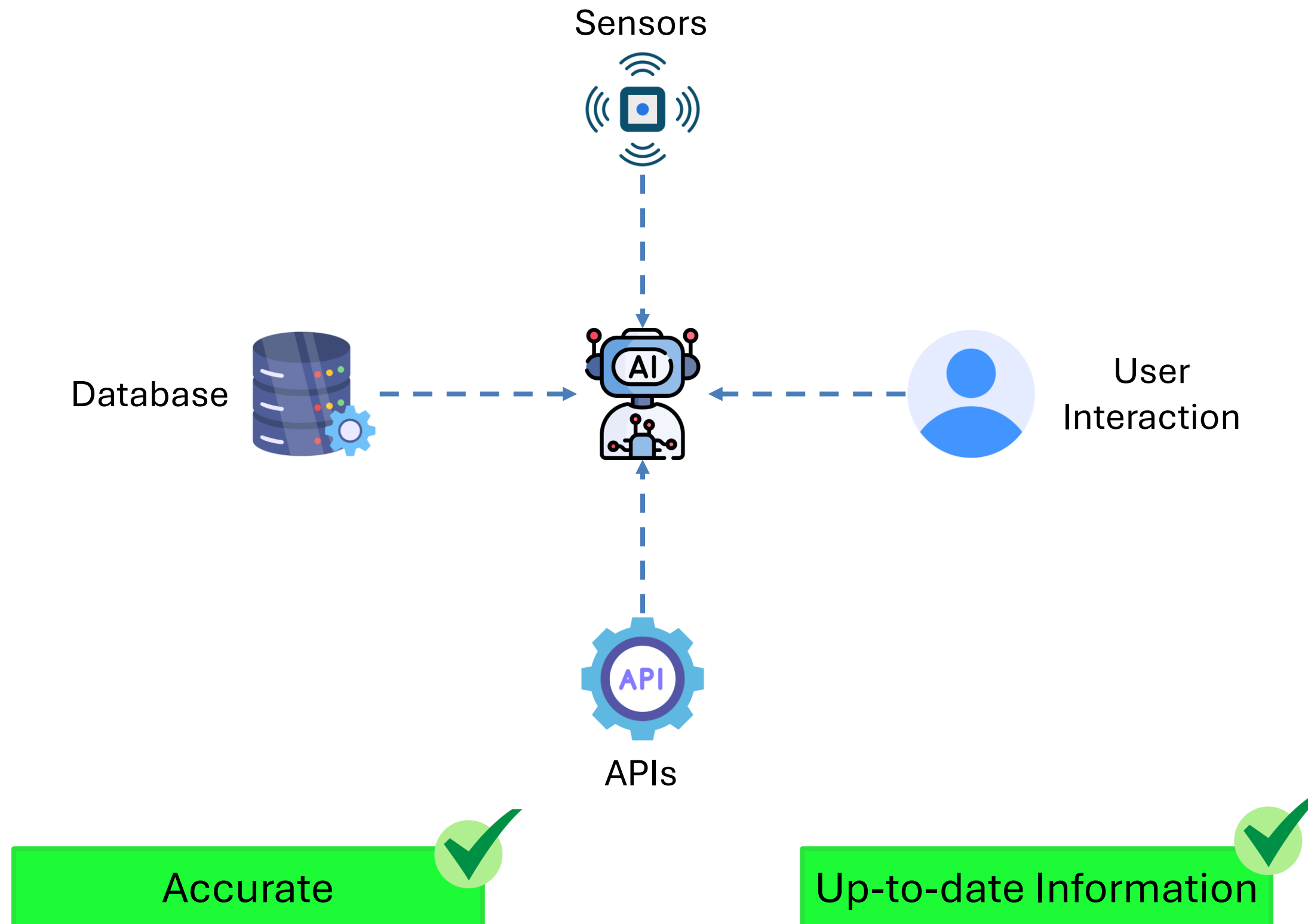


Building Blocks of an Agentic AI System



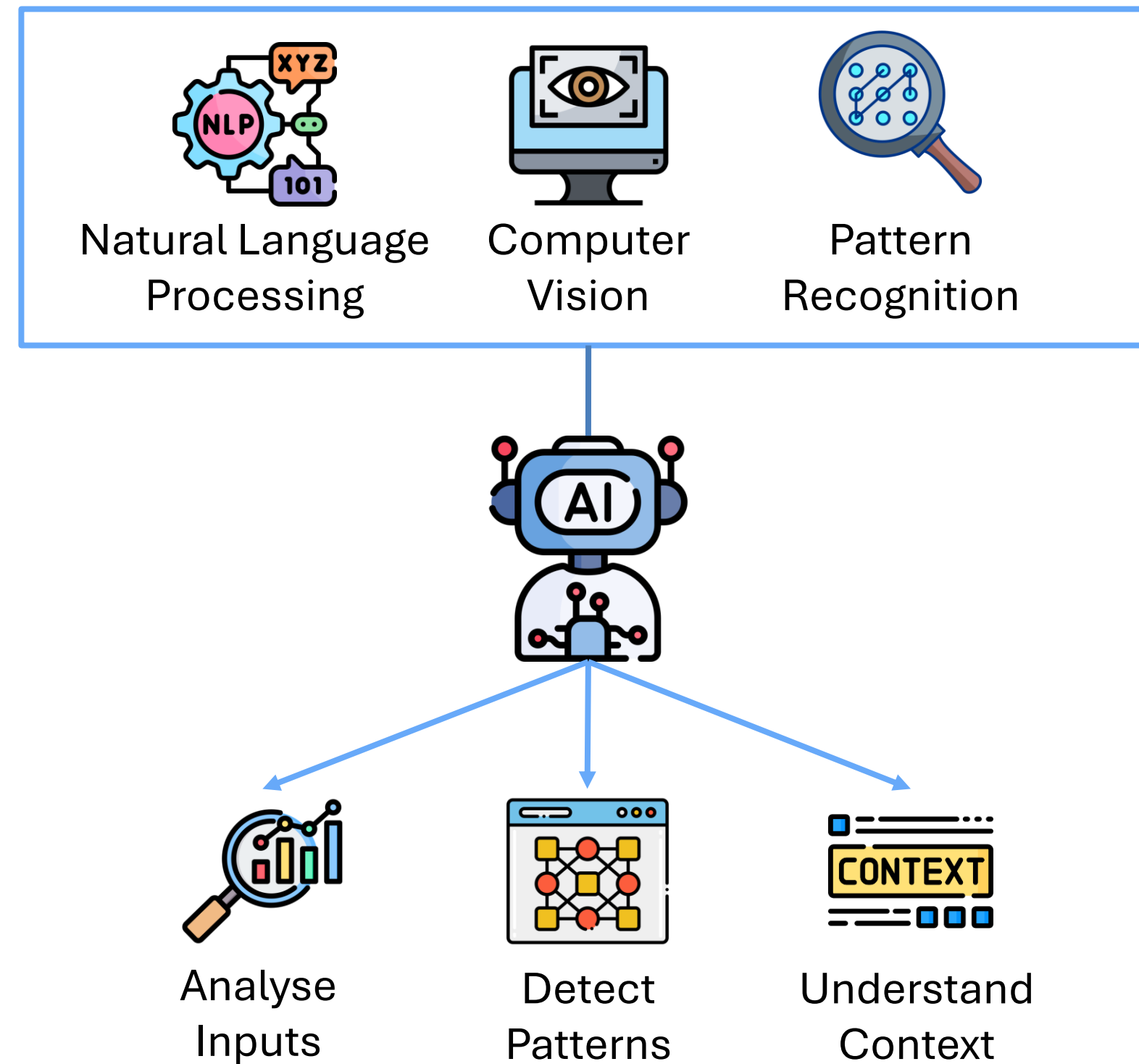
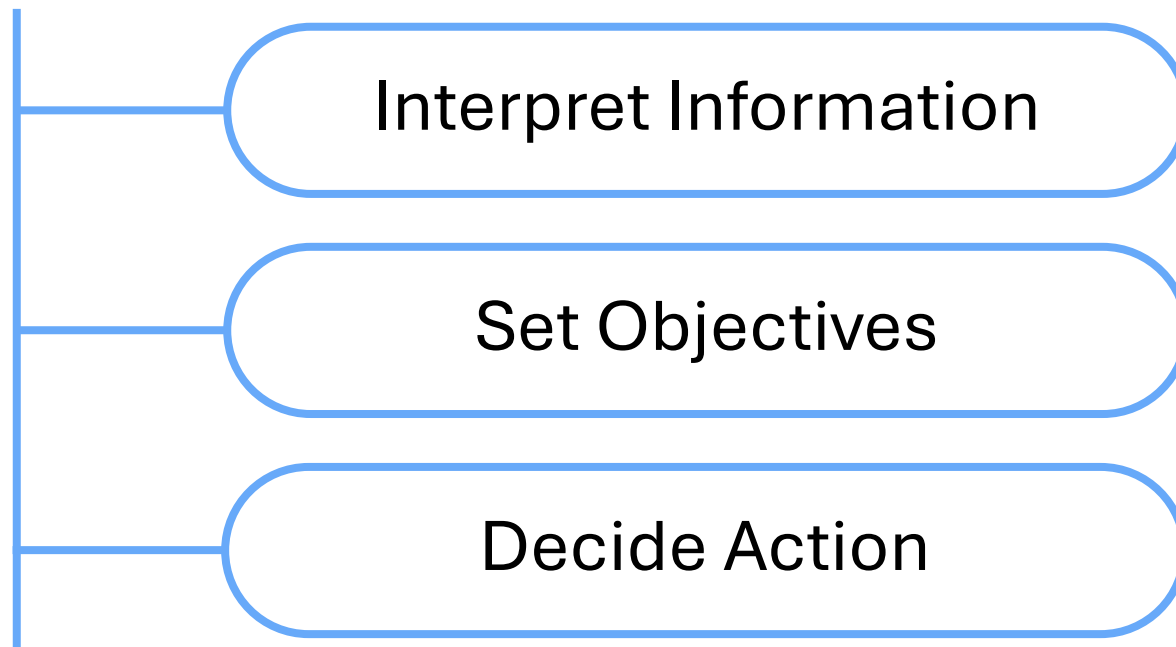
Building Blocks of an Agentic AI System

➤ Perception



Building Blocks of an Agentic AI System

➤ Reason



Building Blocks of an Agentic AI System

➤ Reason

Predicting Outcomes

Probabilistic Models

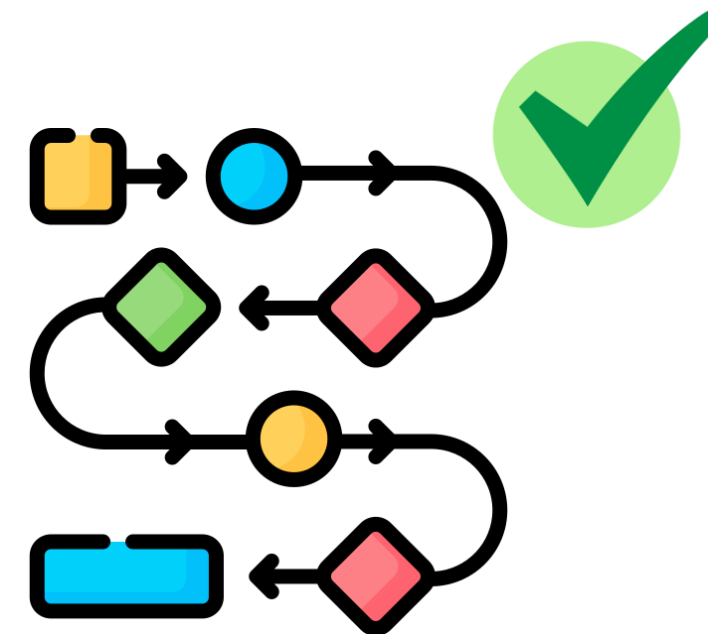
Decision Trees

Reinforcement Learning

Efficiency

Accuracy

Effectiveness



Building Blocks of an Agentic AI System

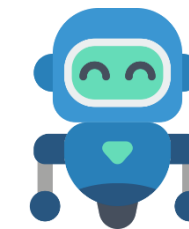
➤ **Act**



APIs



Database



Robot



Users

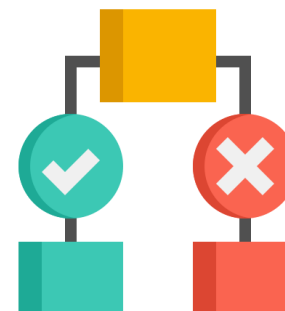
Building Blocks of an Agentic AI System

➤ Learn

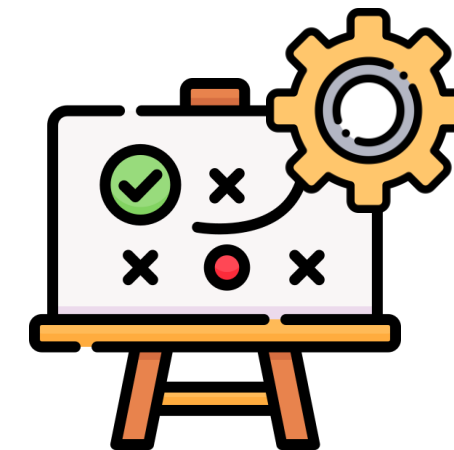
Feedback & Adaptation



Assess
Decisions

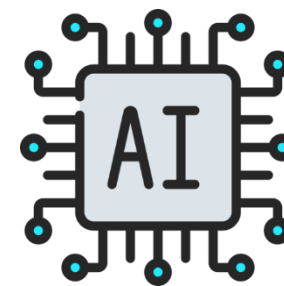


Evaluates
Performance



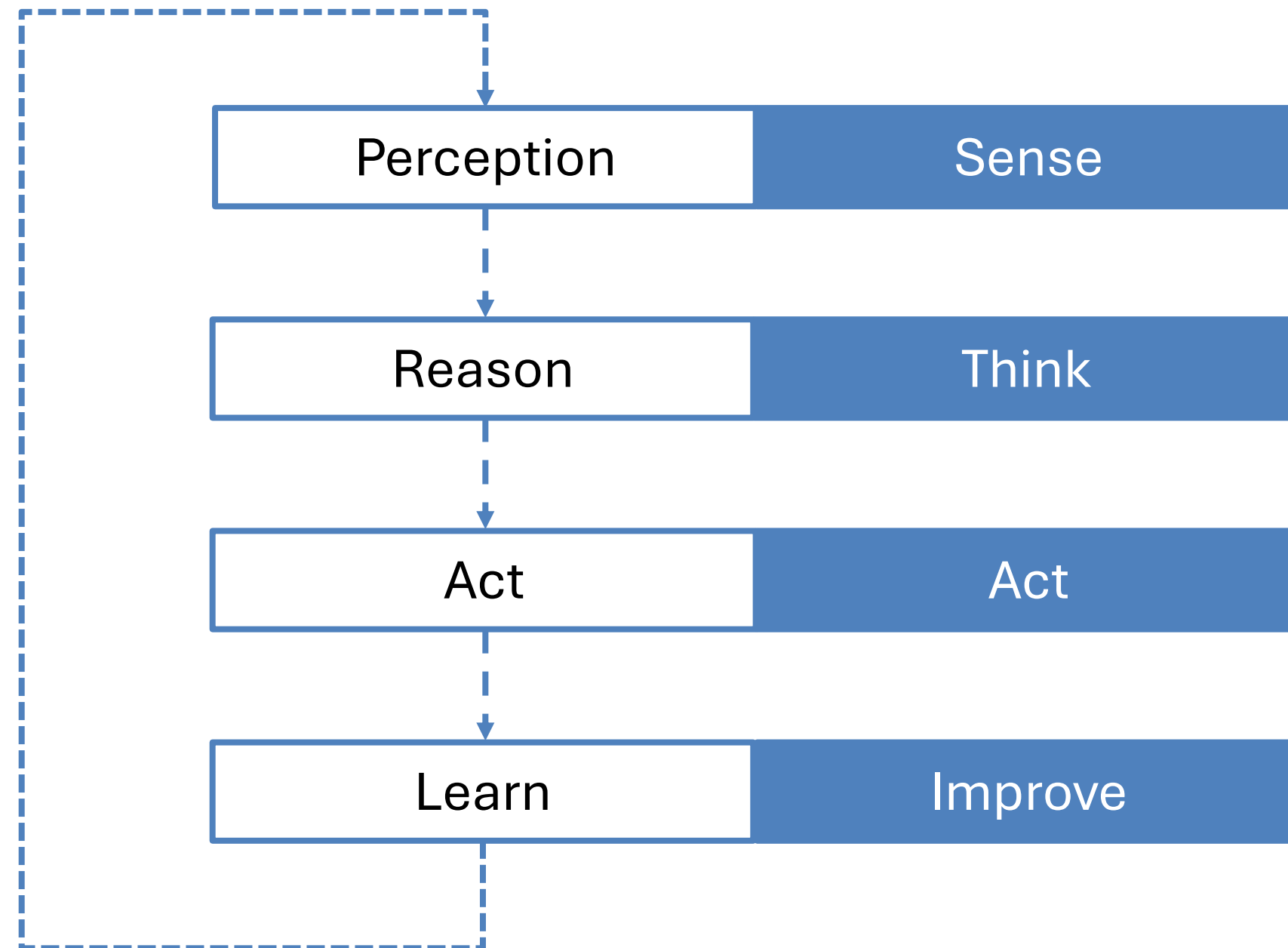
Refine
Strategies

**Reinforcement &
Self-Supervised
Learning**



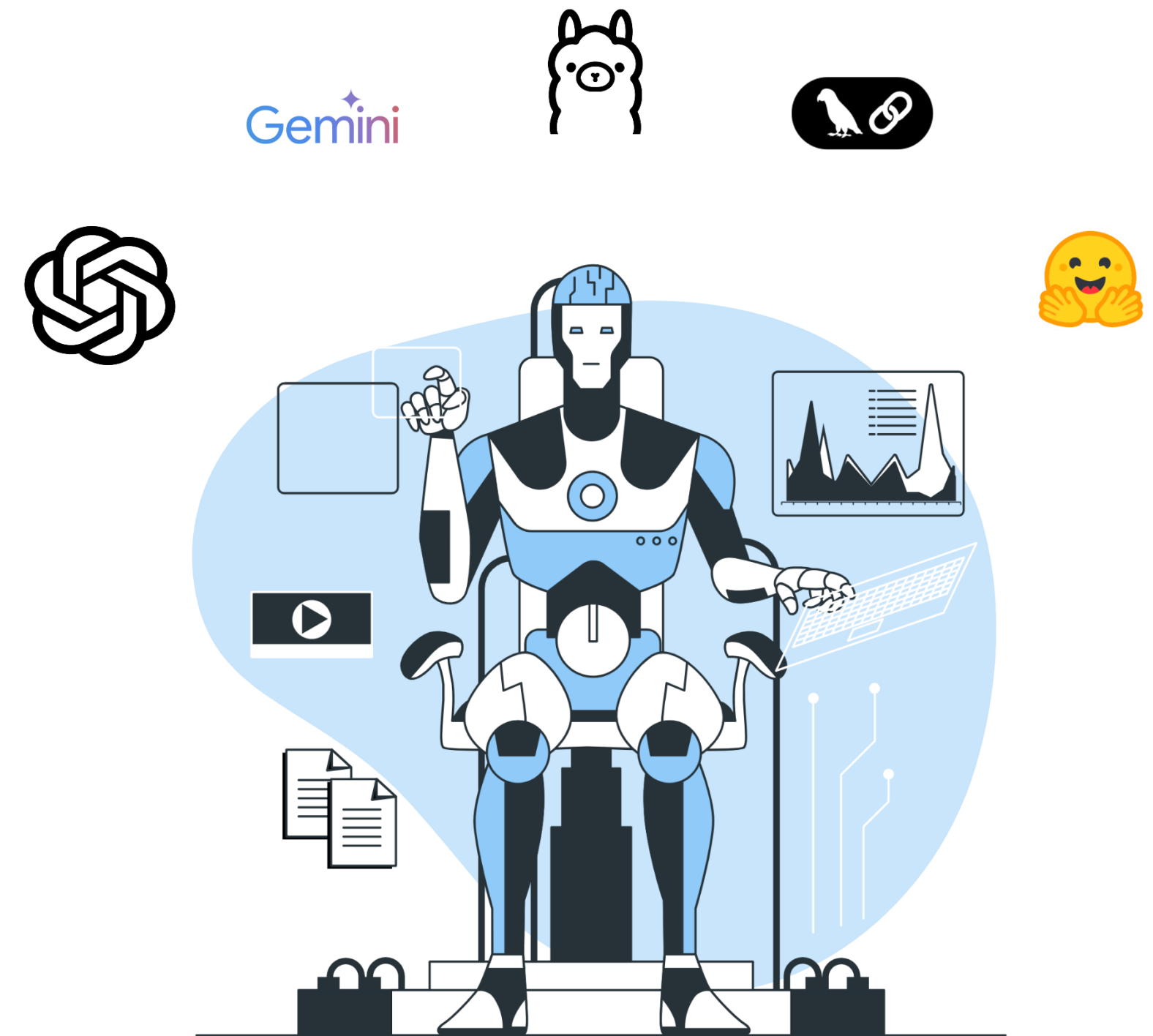
**Increased
Efficiency and
Effectiveness**

Building Blocks of an Agentic AI System

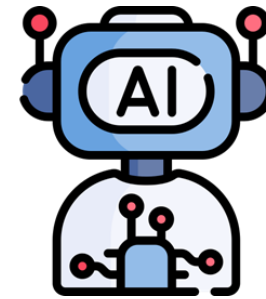


Section 1.4

Industry Impact & Limitations of Agentic AI



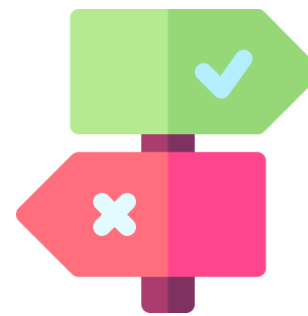
Industry Impact & Limitations of Agentic AI



Agentic AI



Automate Tasks



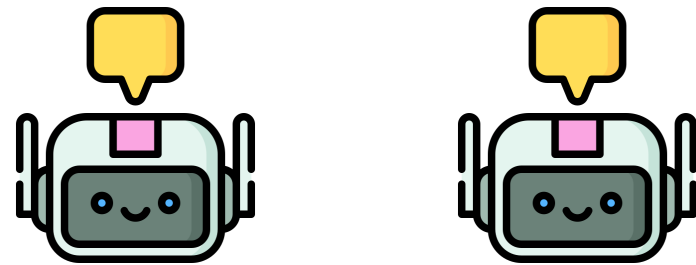
Enhance Decision Making



Improved Efficiency

Industry Impact & Limitations of Agentic AI

➤ Business Process Automation



Repetitive Workflows

Operational Tasks

Examples:

- ☐ Automatically updating CRM records
- ☐ Scheduling follow-up meetings or sending emails
- ☐ Generating daily reports or summaries



Saves Time



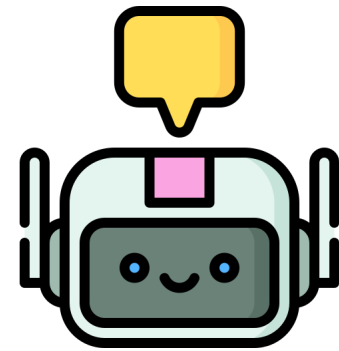
Reduced Error



Increased Productivity

Industry Impact & Limitations of Agentic AI

➤ Software Development and IT Operations



Coding

Deployment

Monitoring

Troubleshooting

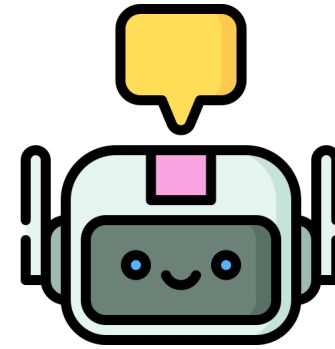
Examples:

- ☐ Writing or reviewing code
- ☐ Managing CI/CD pipelines
- ☐ Detecting and resolving system alerts automatically

Autonomous DevOps Assistants

Industry Impact & Limitations of Agentic AI

➤ Customer Support and Experience



- ✓ Understand queries
- ✓ Take context-aware actions
- ✓ Learn from interactions

Personalized

Always-available Support

Examples:

- ☐ Handling customer queries via chat or email
- ☐ Processing refunds or claims within defined limits
- ☐ Escalating complex cases to human agents

Faster responses & consistent customer experiences at scale

Industry Impact & Limitations of Agentic AI

Limitations

1

Limited Understanding of
Complex Contexts

Dependency on Data Quality

2

3

Limited Autonomy and Decision
Boundaries

Integration Challenges

4

5

Security and Privacy Concerns

Need for Continuous Monitoring
and Updates

6

Industry Impact & Limitations of Agentic AI

➤ In Summary

- ☐ Automate tasks, supports development, and enhances customer experiences
- ☐ These systems are not fully autonomous
- ☐ Faces limits in context understanding, data quality, integration, and security
- ☐ Human supervision is required to guide goals, validate outputs, and ensure safe operation

Section 1.5

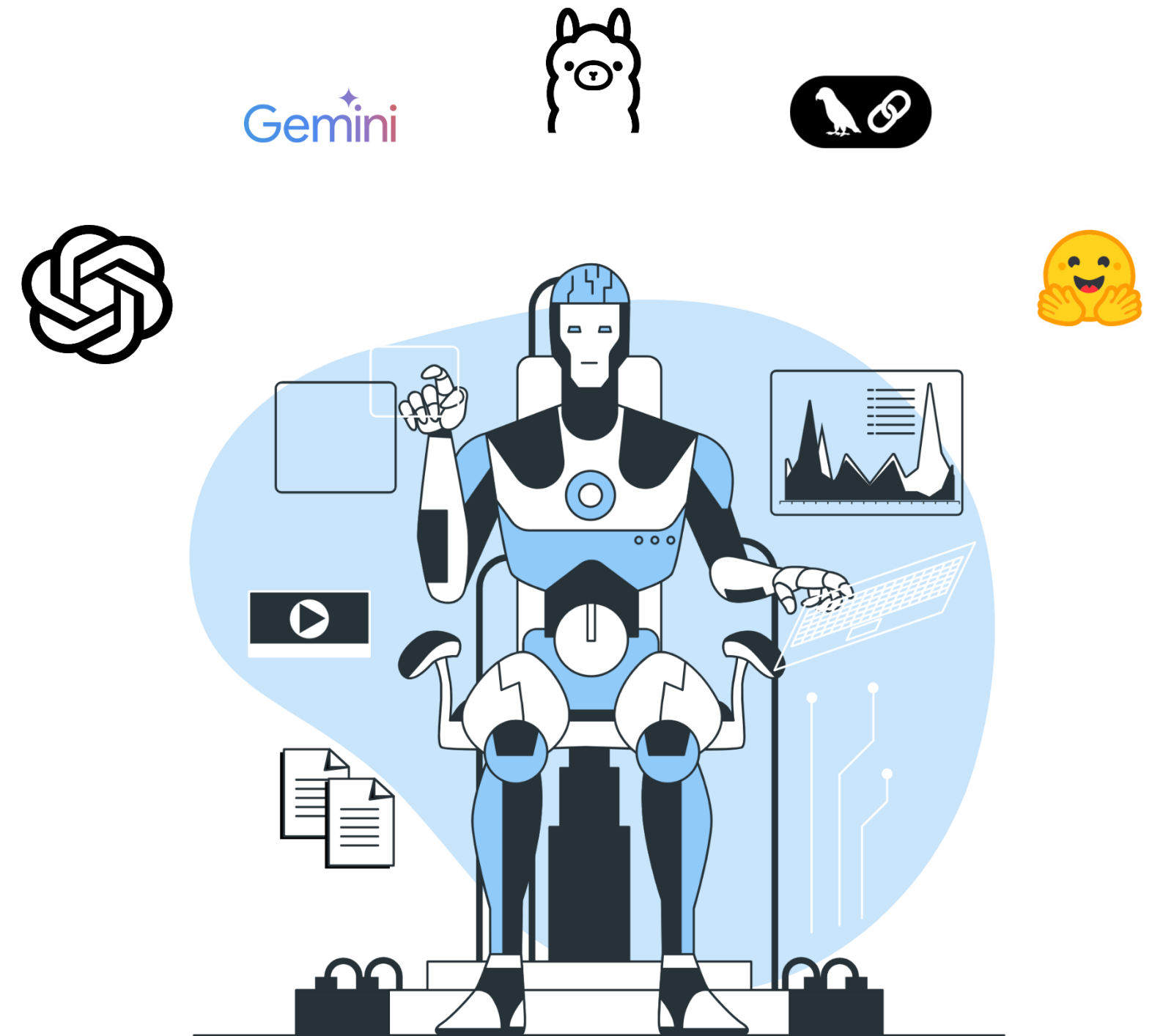
Summary

- ✓ Explored the foundation of Agentic AI
- ✓ Compared Agentic AI with traditional AI and basic AI agents
- ✓ Understood the key building blocks of Agentic AI system
- ✓ Discussed the industry impact of Agentic AI
- ✓ Learnt about the limitations of Agentic AI systems

Thank You

Section 2.0

Foundation of Agentic Systems



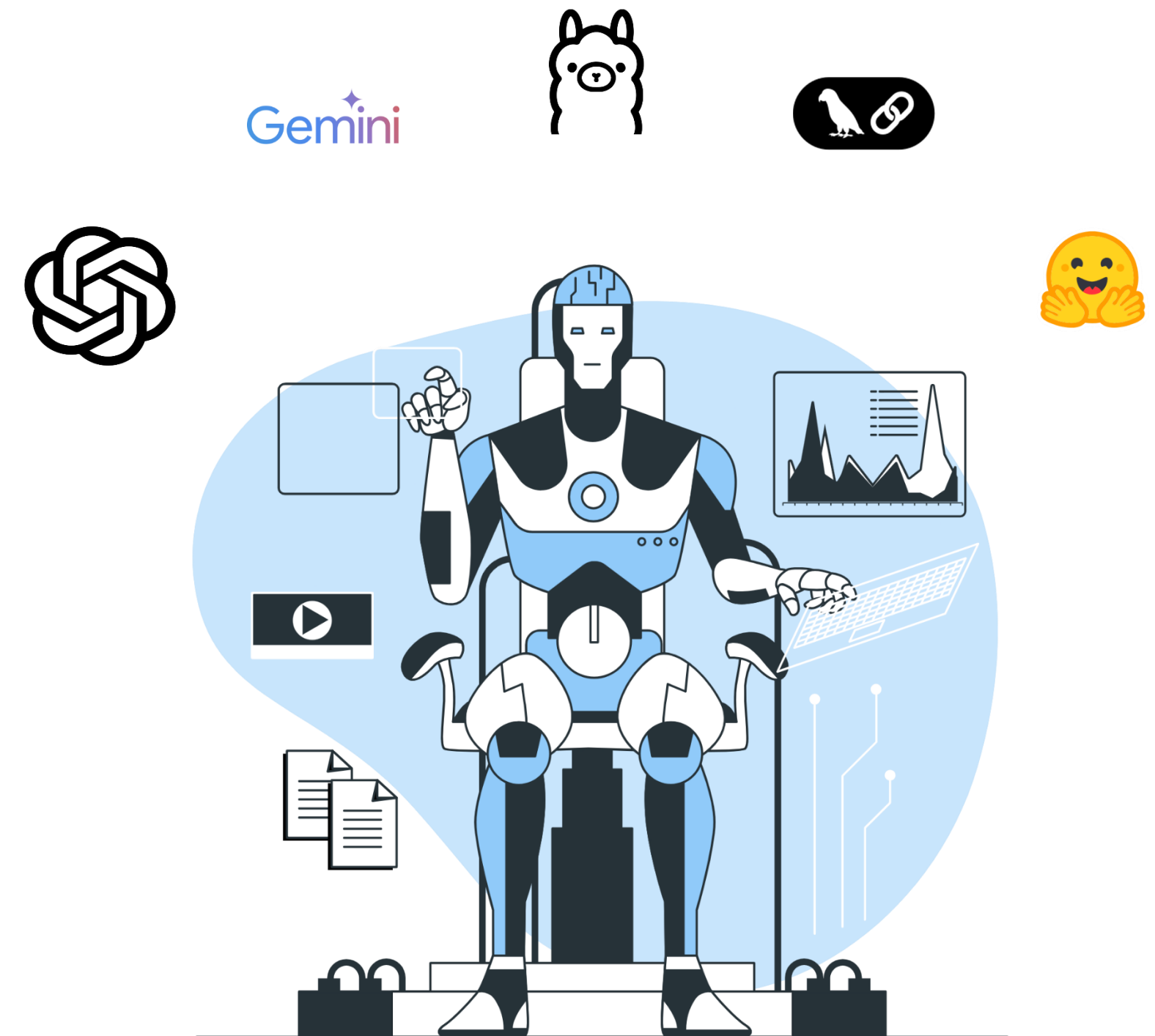
Foundation of Agentic Systems

Section Overview

- ☐ Large Language Models (LLMs)
- ☐ AI Agents
- ☐ Demonstration: Building AI agent from scratch
- ☐ RAG, Embeddings & Vector Representations, & Memory Systems
- ☐ Prompt Engineering & Task Orchestration
- ☐ Agentic AI Workflow
- ☐ Demonstration: AWS Instance Launcher

Section 2.1

Large Language Models (LLMs)

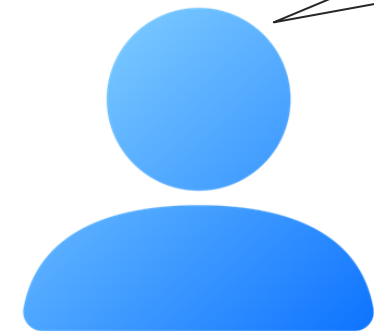


Large Language Models (LLMs)



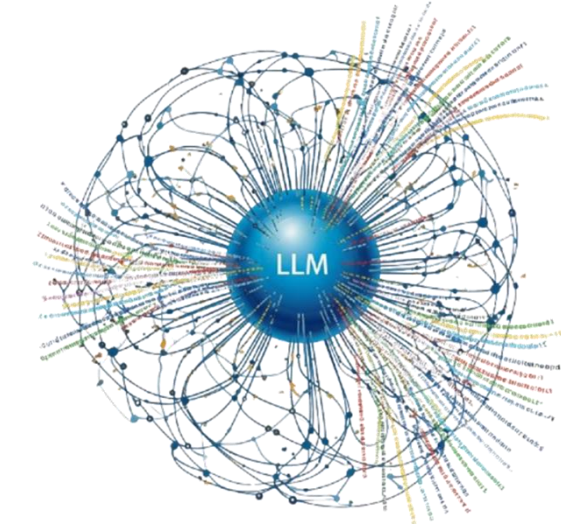
- ❑ Helps LLM to learn how language is structured, how grammar works, and how different words relate to each other

Large Language Models (LLMs)



“The sun rises in the ---”

East



Predicts what word should
come next in a sentence



Transformer

Large Language Models (LLMs)



Transformer

Emails

Stories

Blogs

Translate
Languages

Summarize large documents

Answer complex questions

Generate computer code

Popular Tools



OpenAI

Google
Gemini

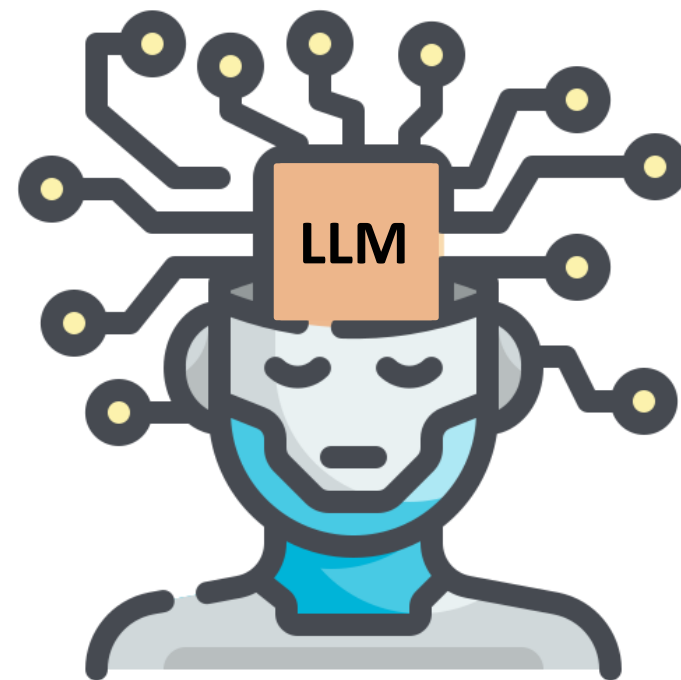


Claude



LLaMA

Large Language Models (LLMs)

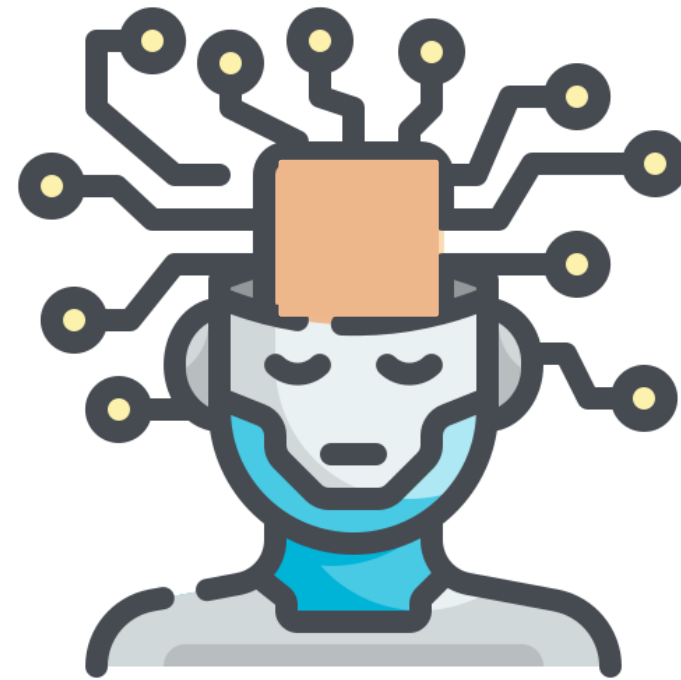


Understands Language

Reasons Information

Makes context-aware
decisions

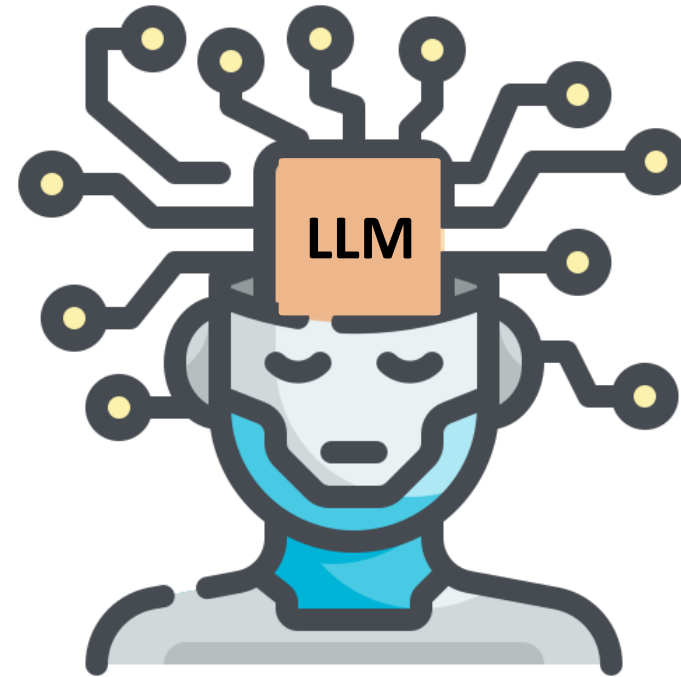
Large Language Models (LLMs)



LLM

Set of programmed instructions

Large Language Models (LLMs)



- ✓ Understand Intent
- ✓ Dynamically Reason
- ✓ Take Actions

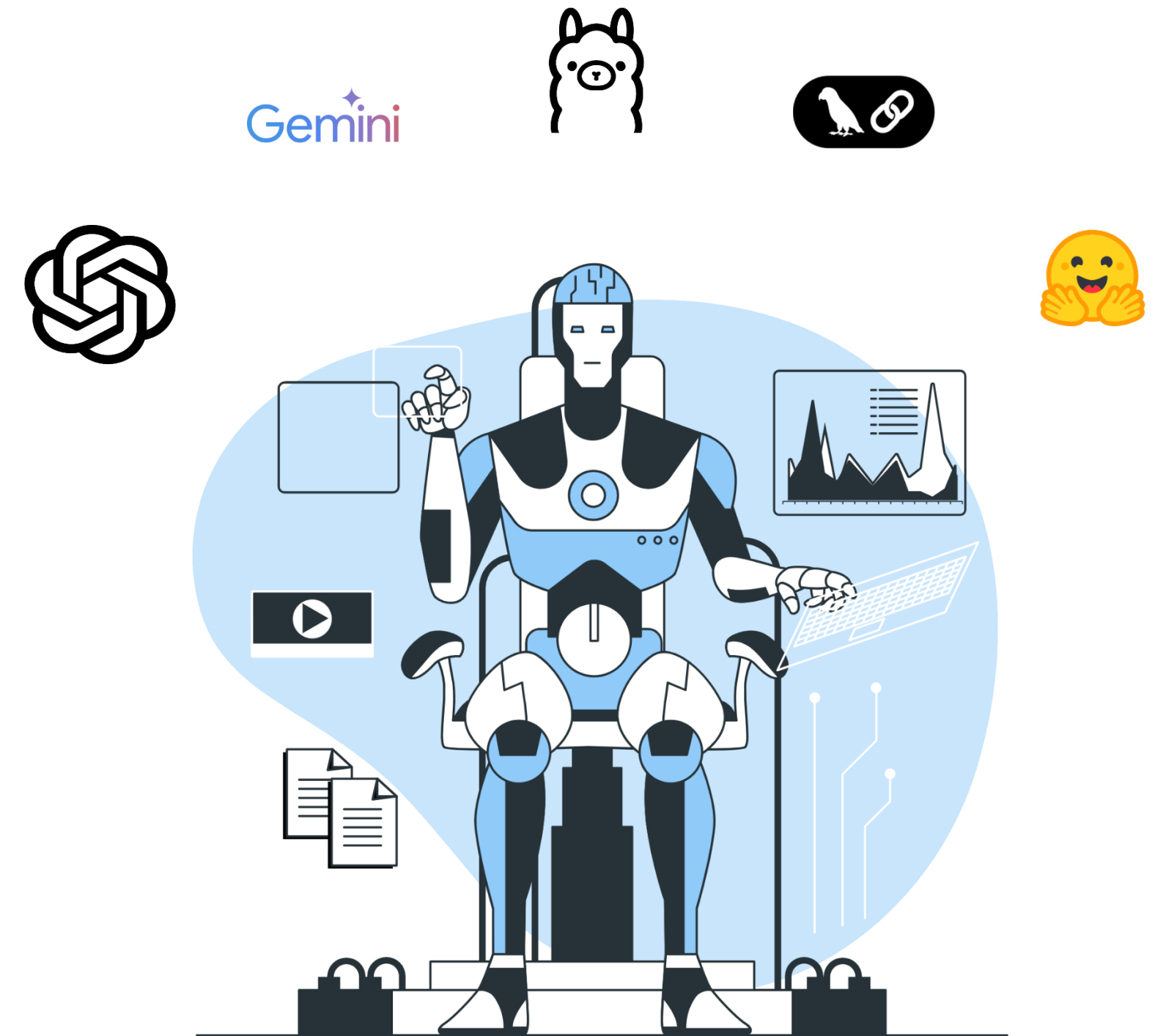
Interpret Goals

Adapt to New Situations

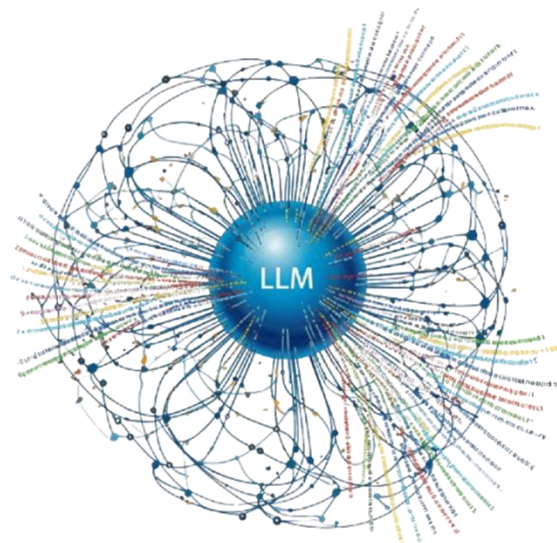
Respond Intelligently like Humans

Section 2.2

Understanding AI Agents



Understanding AI Agents

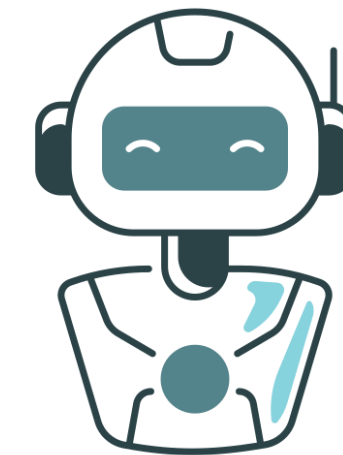


Large Language Models
(Cognitive Core)

Take Decisions

Perform Tasks

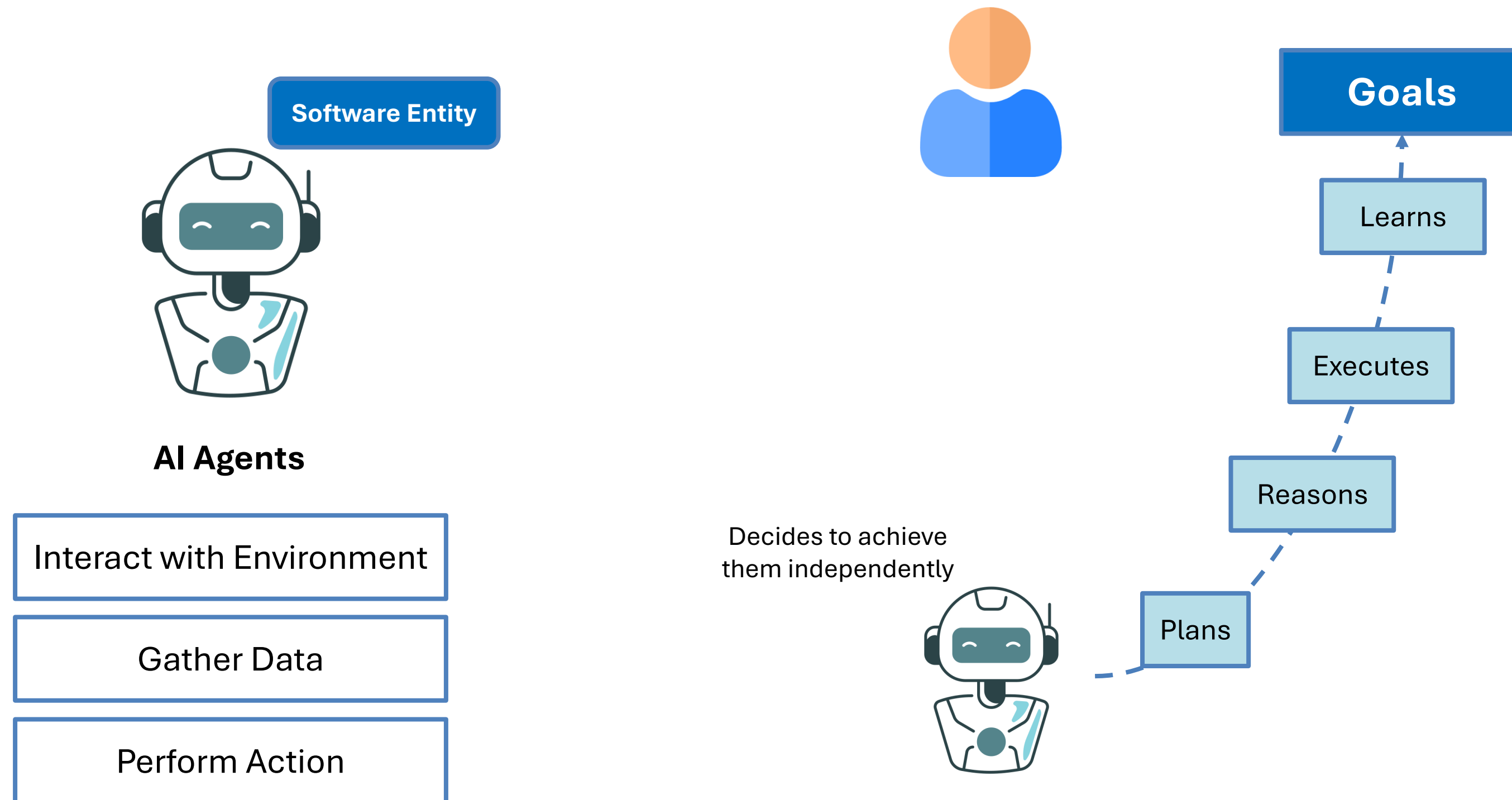
Interact with Environment



AI Agents
(Operational Layer)

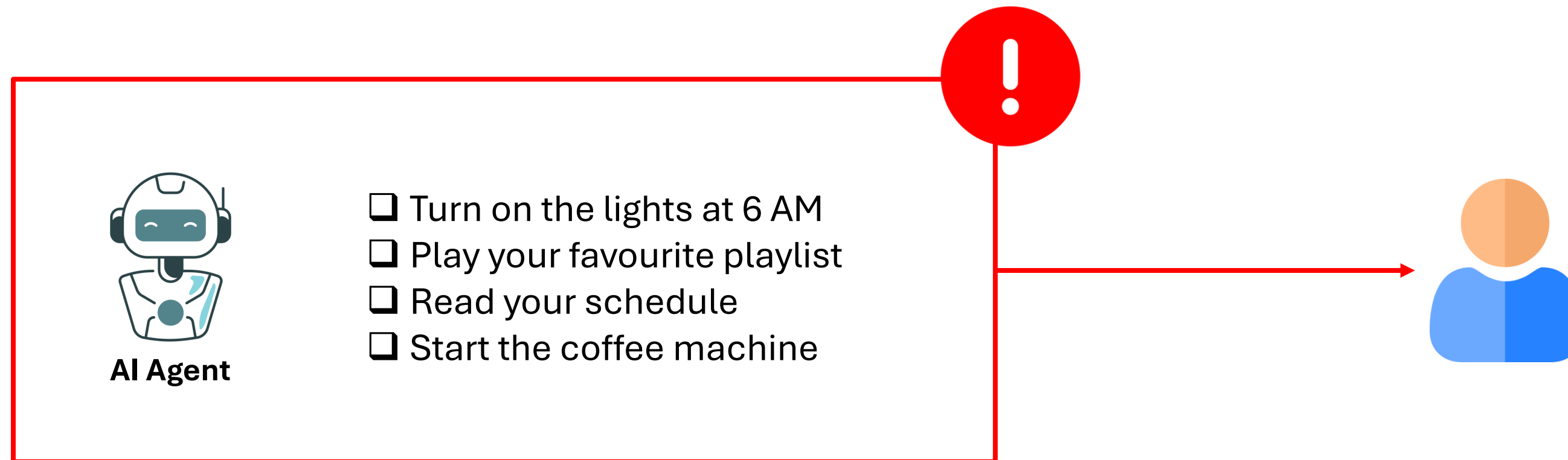
Understanding AI Agents

Role of AI Agents



Understanding AI Agents

Role of AI Agents



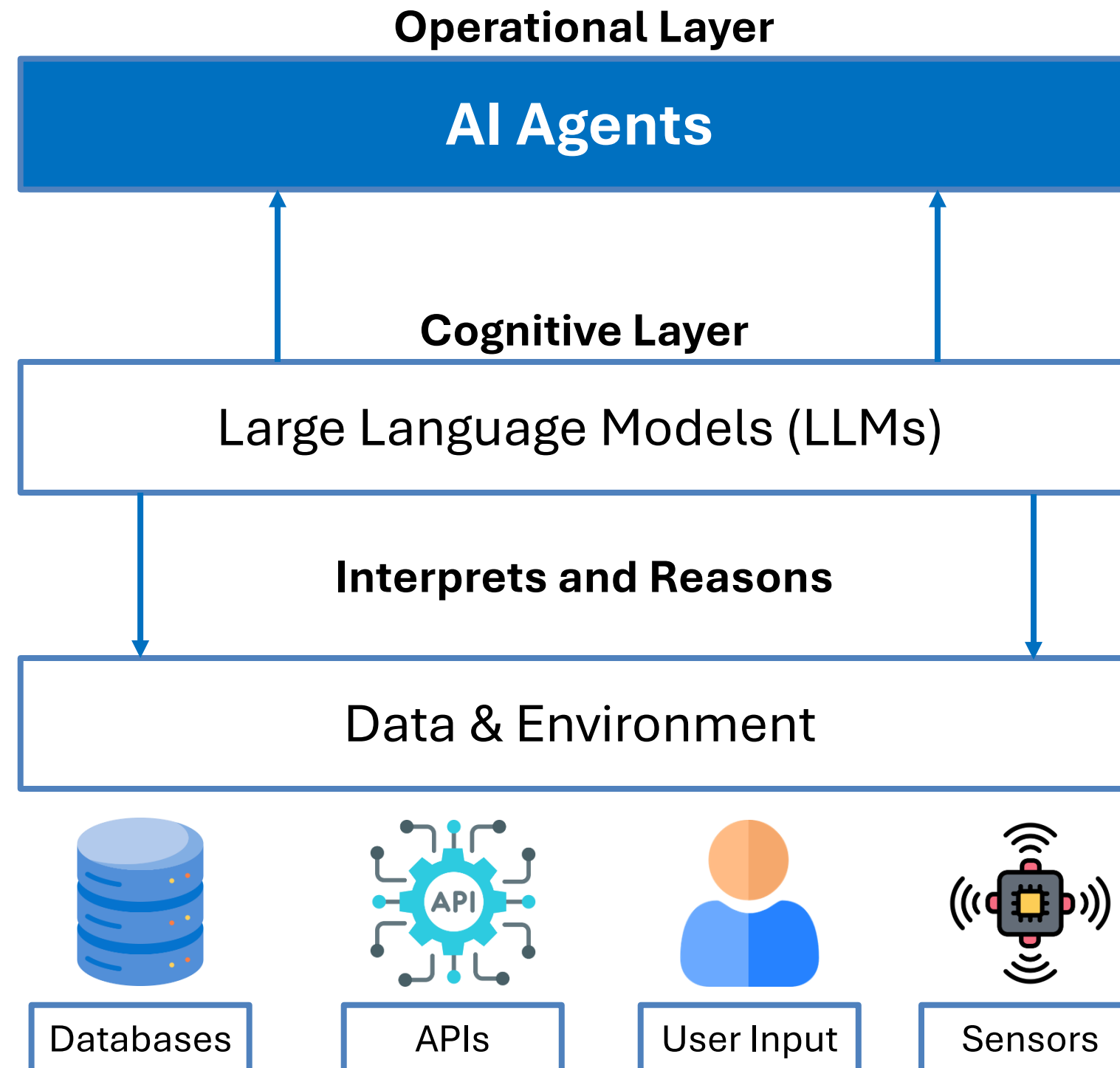
Think

Act

Adapt

Operational Powerhouse

Understanding AI Agents



Understanding AI Agents

Executes Instructions

Automation

- ❑ Follows **fixed, rule-based workflows**
- ❑ Handles repetitive as programmed
- Examples:** sending a welcome email on signup or updating a database entry when a condition is met
- ❑ Fast and reliable for routine work
- ❑ Fails to adapt when unexpected situations occur

Executes Goals

AI Agents

- ❑ **Dynamic and adaptive** in their behavior
- ❑ Understand context and adjust actions based on changing conditions
- ❑ **Examples:** analyzes sentiment, personalizes responses, or initiates chats when needed instead of predefined email

Understanding AI Agents

Reactive

- ❑ Responds directly to stimuli or inputs without long-term memory or planning
- ❑ Example - Chatbot that answers predefined FAQs

Proactive

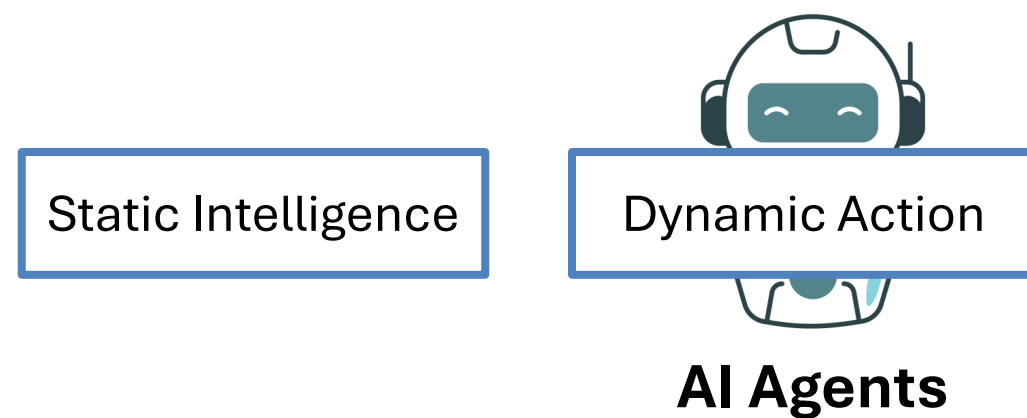
- ❑ Plans ahead, analyze options, and makes goal-aligned decisions
- ❑ Example - Scheduling assistant that manages calendars & reschedules meetings

Collaborative

- ❑ Work alongside humans or other agents, communicating and coordinating actions
- ❑ Example - Support agent that assists humans with live chat suggestions

Understanding AI Agents

Why AI Agents Matter?



Turn LLMs into goal-driven agents that can:

- ☐ Automate workflows
- ☐ Access external tools and APIs
- ☐ Manage multi-step reasoning tasks
- ☐ Maintain short-term or long-term memory
- ☐ Collaborate across systems and users

Section 2.3

Demonstration | Weather AI Agent from Scratch

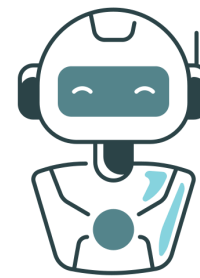


Demonstration | Weather AI Agent from Scratch

AI Agents

Agentic AI

Pipeline



AI Agents
(Core)

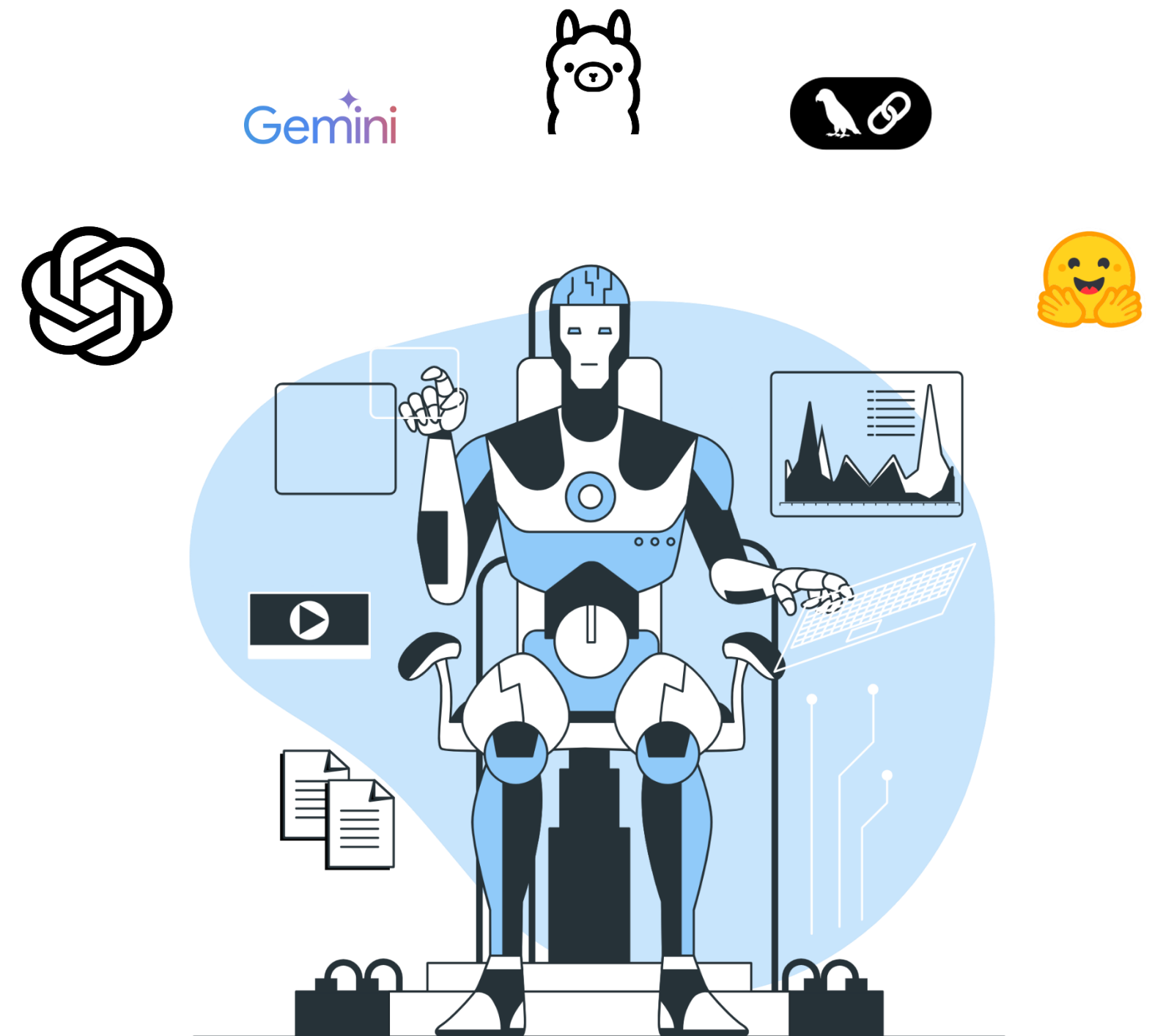
Demonstration | Weather AI Agent from Scratch

Weather Agent

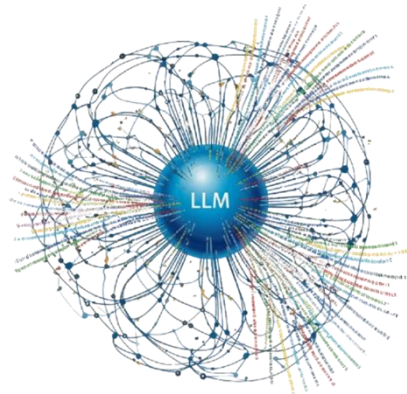
- ❑ Understands user intent from natural language questions like “What’s the weather in Paris?”
- ❑ Extracts key information such as the city, date, or task
- ❑ Understand how an agent links language, reasoning, and tools to act intelligently

Section 2.4

Retrieval-Augmented Generation (RAG)



Retrieval-Augmented Generation (RAG)



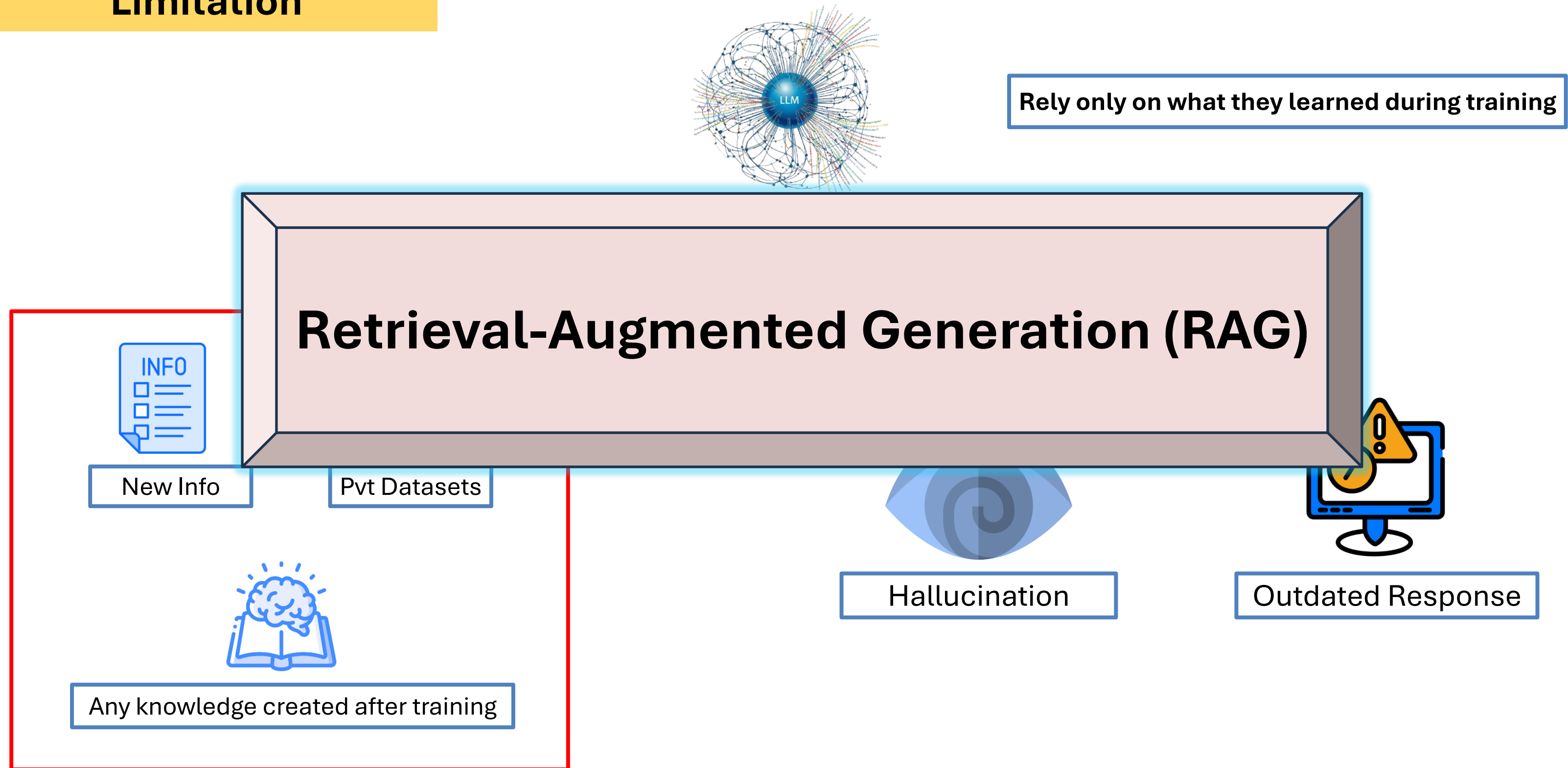
Large Language Models
(Brain)

- ☐ Trained on massive datasets
- ☐ Perform tasks like answering questions to writing content or summarizing information

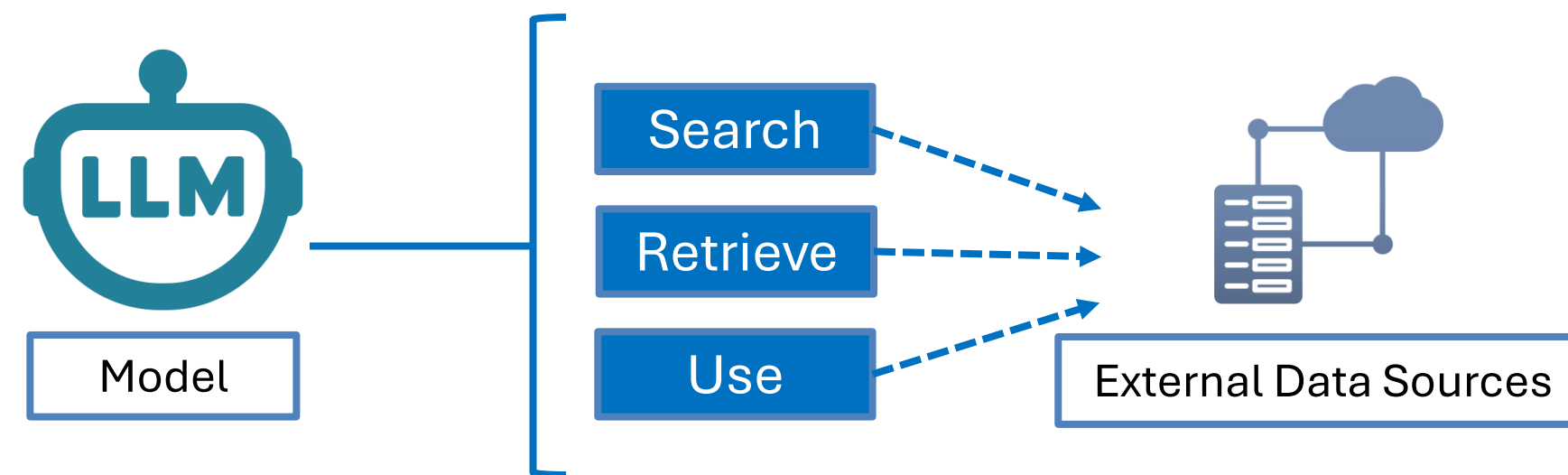


Retrieval-Augmented Generation (RAG)

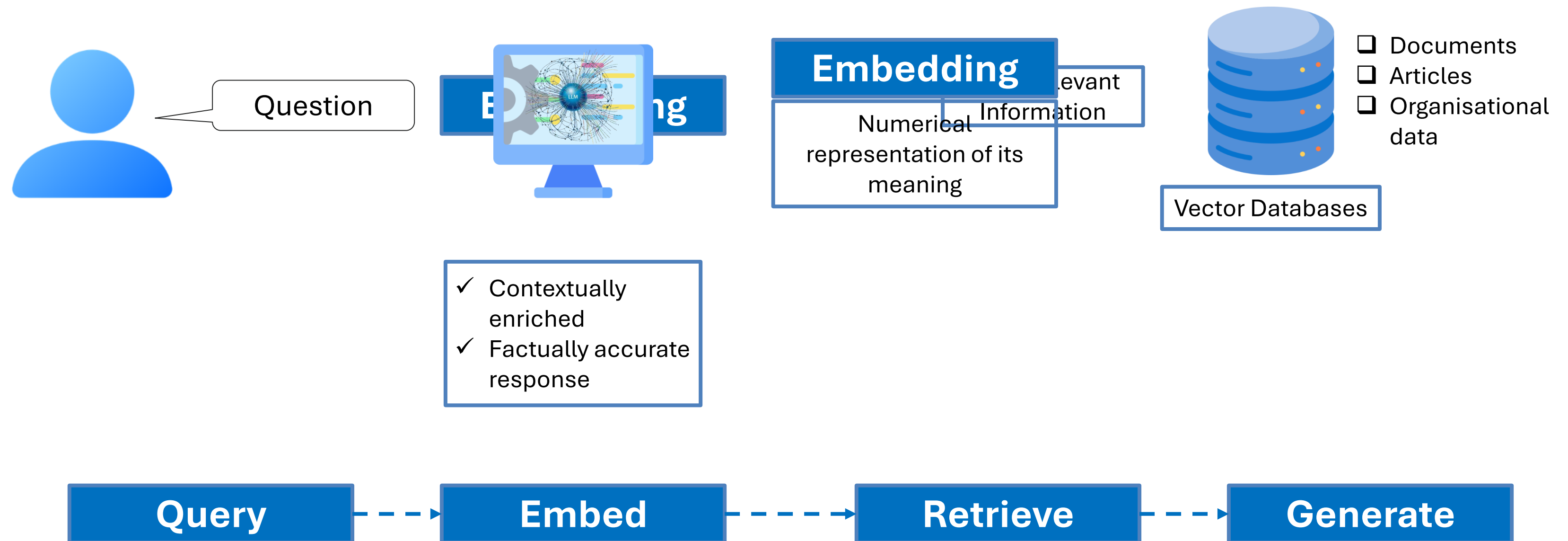
Limitation



Retrieval-Augmented Generation (RAG)



Retrieval-Augmented Generation (RAG)

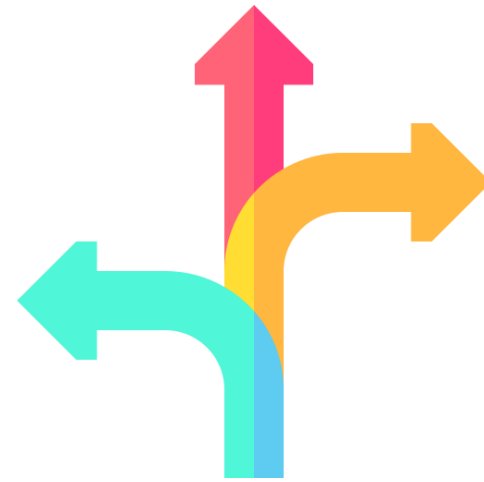


Retrieval-Augmented Generation (RAG)

The Value of RAG



Improved Accuracy



Flexibility



Updated Knowledge



Transparency



Cost-Effectiveness

Retrieval-Augmented Generation (RAG)

Real-World Application

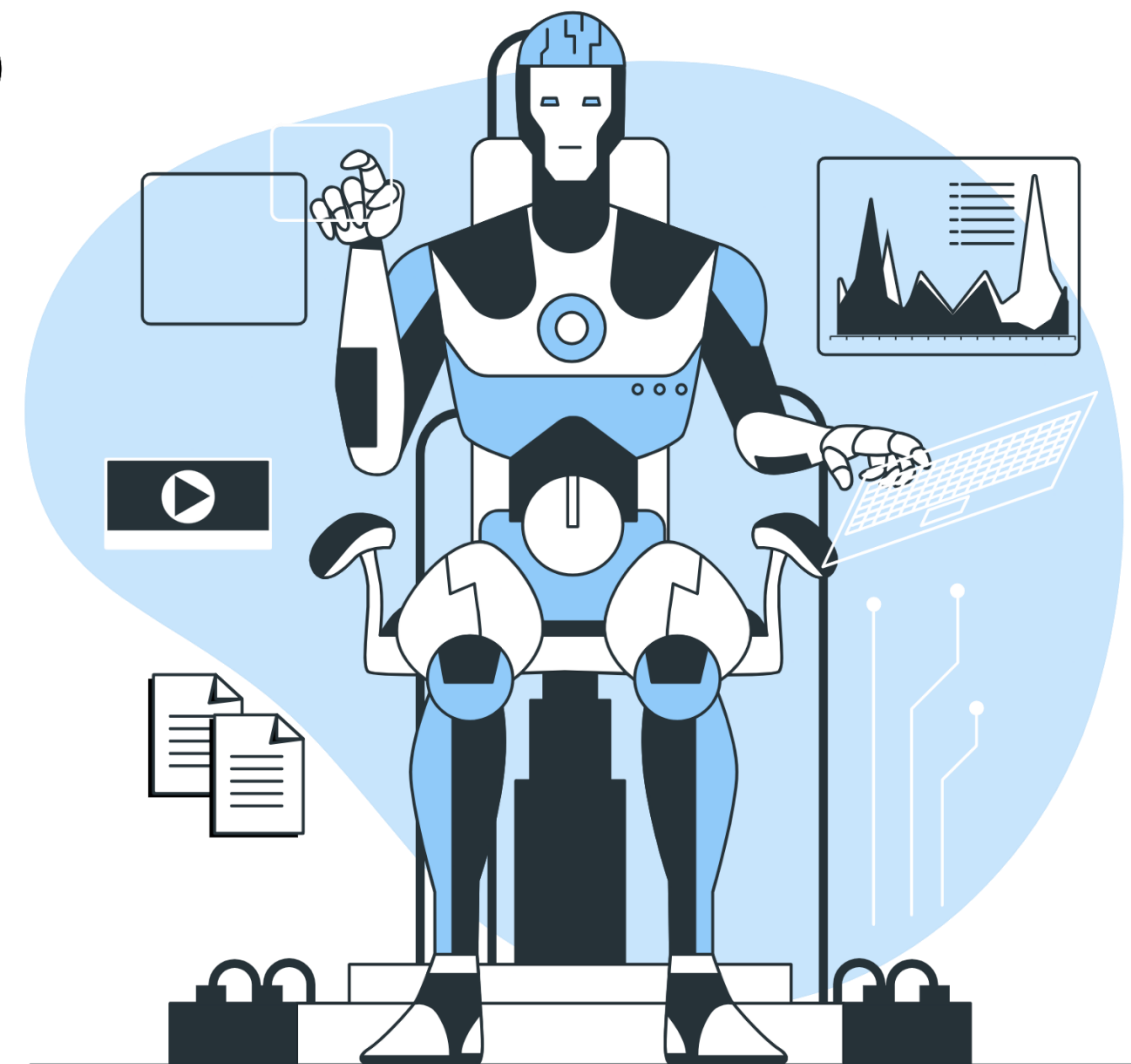
- ❑ **Customer Support:** Retrieves accurate answers from FAQs and documents
- ❑ **Legal/Medical:** Quickly searches laws, regulations, and research
- ❑ **Academia:** Summarizes and cites academic papers
- ❑ **News/Enterprise:** Provides real-time summaries and insights

Section 2.5

Embeddings & Vector Representations



Gemini



Embeddings & Vector Representations



How an AI system actually understands the meaning?

Car

Automobile



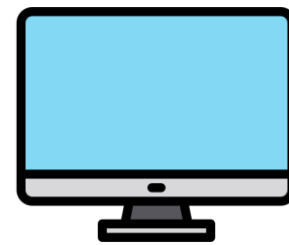
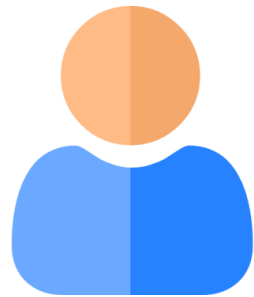
Cloud Computing

DevOps

Embeddings

Embeddings & Vector Representations

“Artificial Intelligence is transforming industries.”



The meaning is clear

- ☐ Just a sequence of characters
- ☐ Doesn't understand the meaning
- ☐ Sentences → Numbers

“Artificial” → [0.12, 0.89, 0.44, 0.76]

“Intelligence” → [0.14, 0.91, 0.47, 0.79]

These numbers show how words relate by meaning and context, not just spelling or frequency

Word

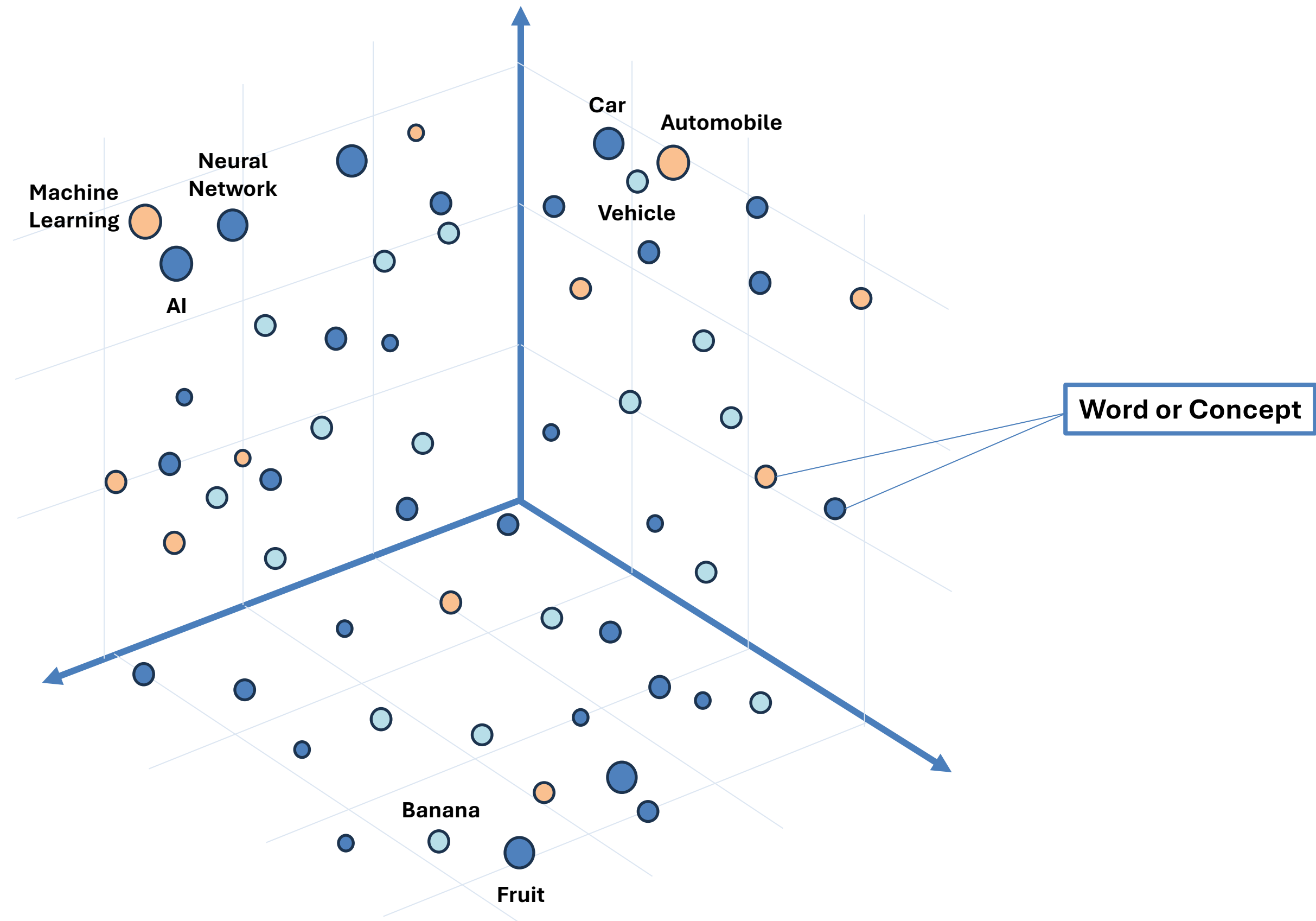
Phrase

Document

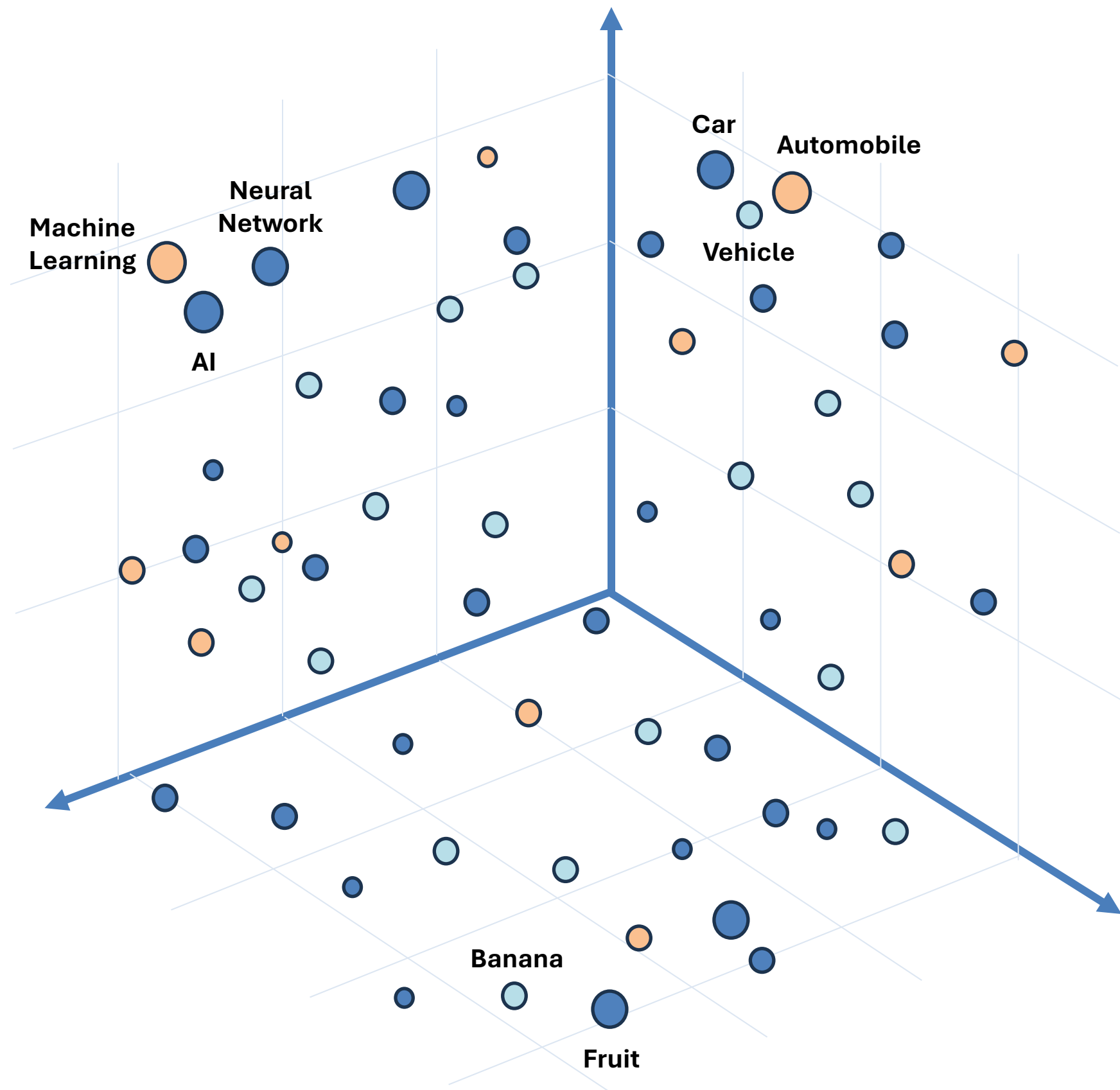
Vector

Numerical Representation in Multi-dimensional Space

Embeddings & Vector Representations



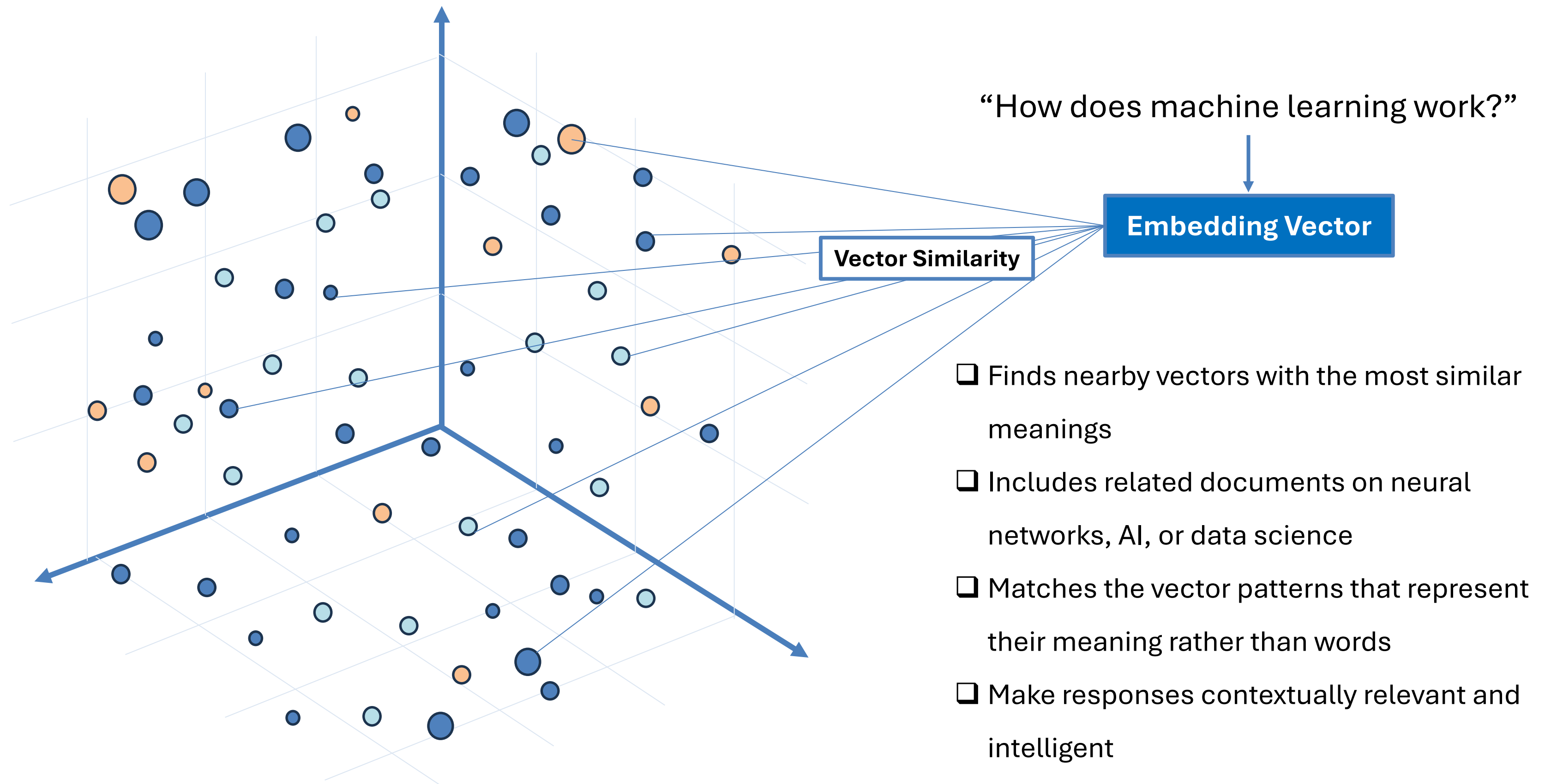
Embeddings & Vector Representations



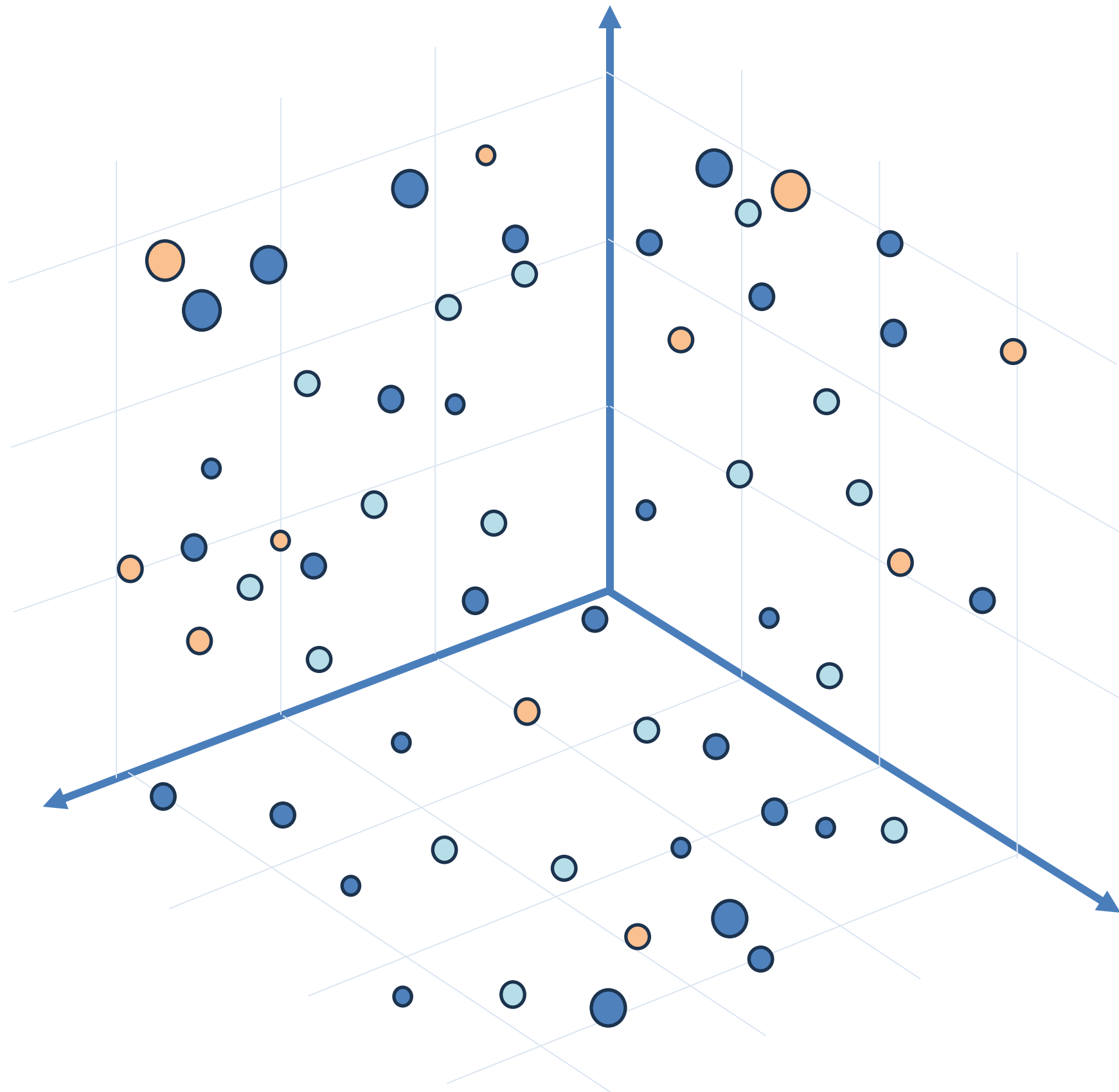
Vector Space

Helps AI models understand semantic similarity or meaning-based relationships

Embeddings & Vector Representations

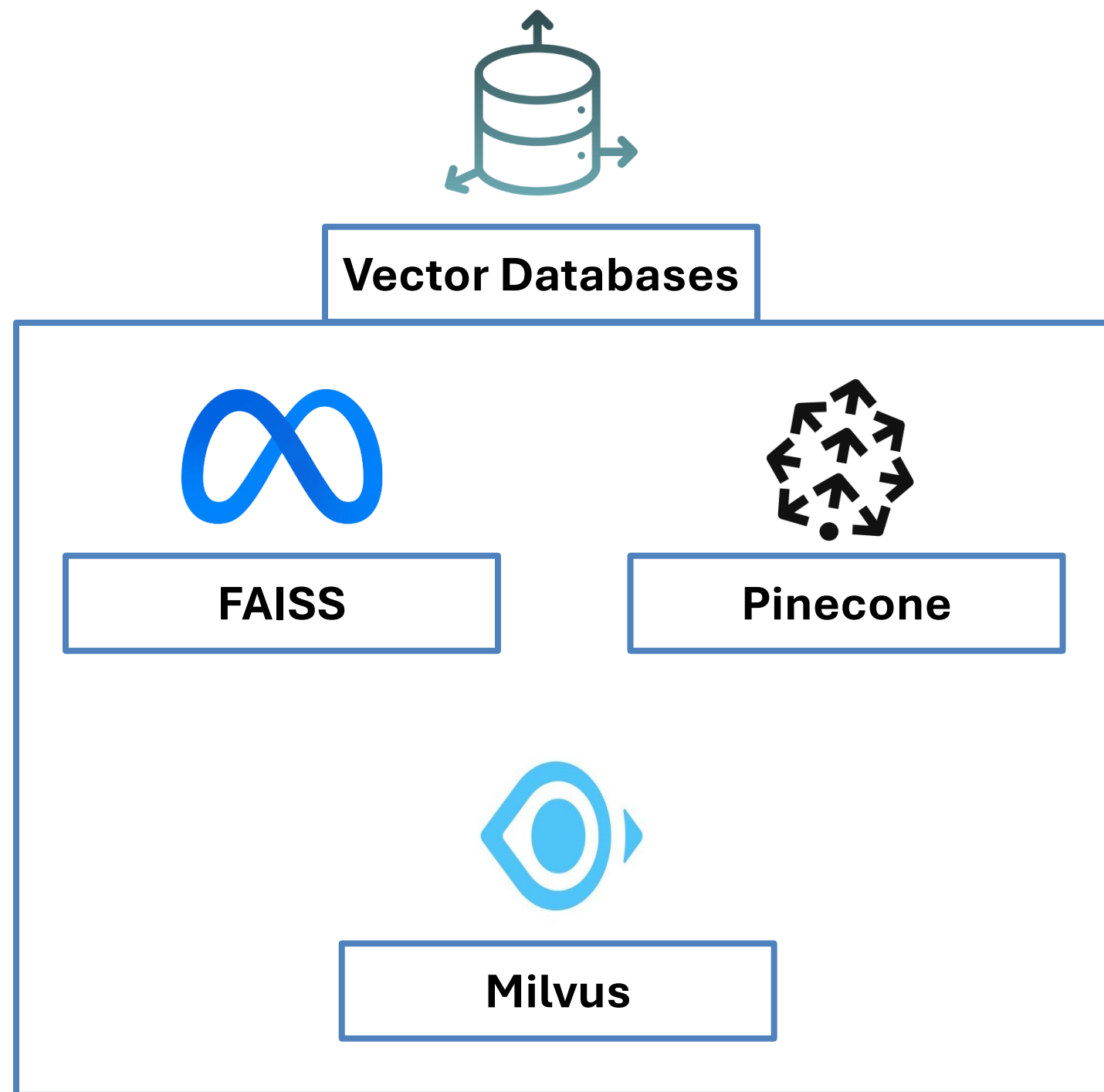


Embeddings & Vector Representations

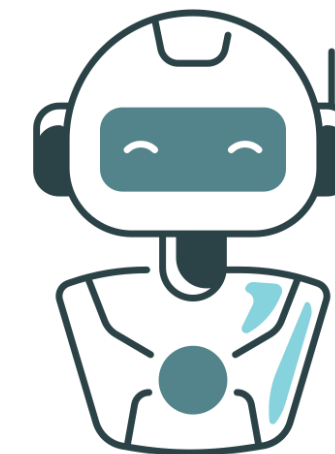


- ❑ This mechanism underpins RAG, long-term memory, and semantic search in agentic systems
- ❑ Enables agents to remember context and recall past interactions
- ❑ Agents can retrieve the right information at the right time using embeddings

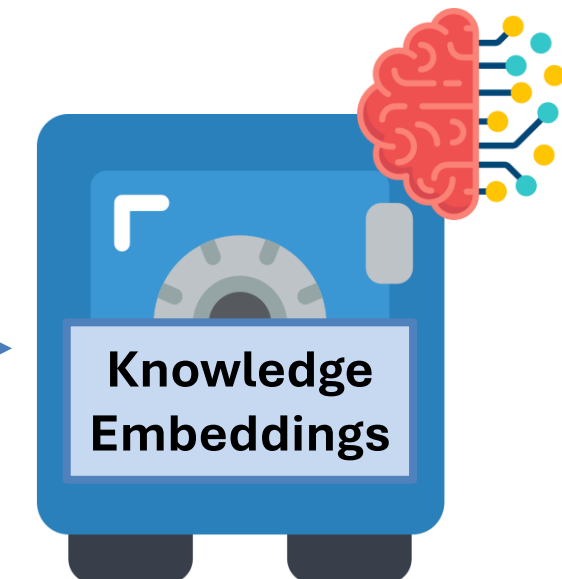
Embeddings & Vector Representations



Fast & Scalable Similarity Searches



AI Agents



Knowledge
Embeddings

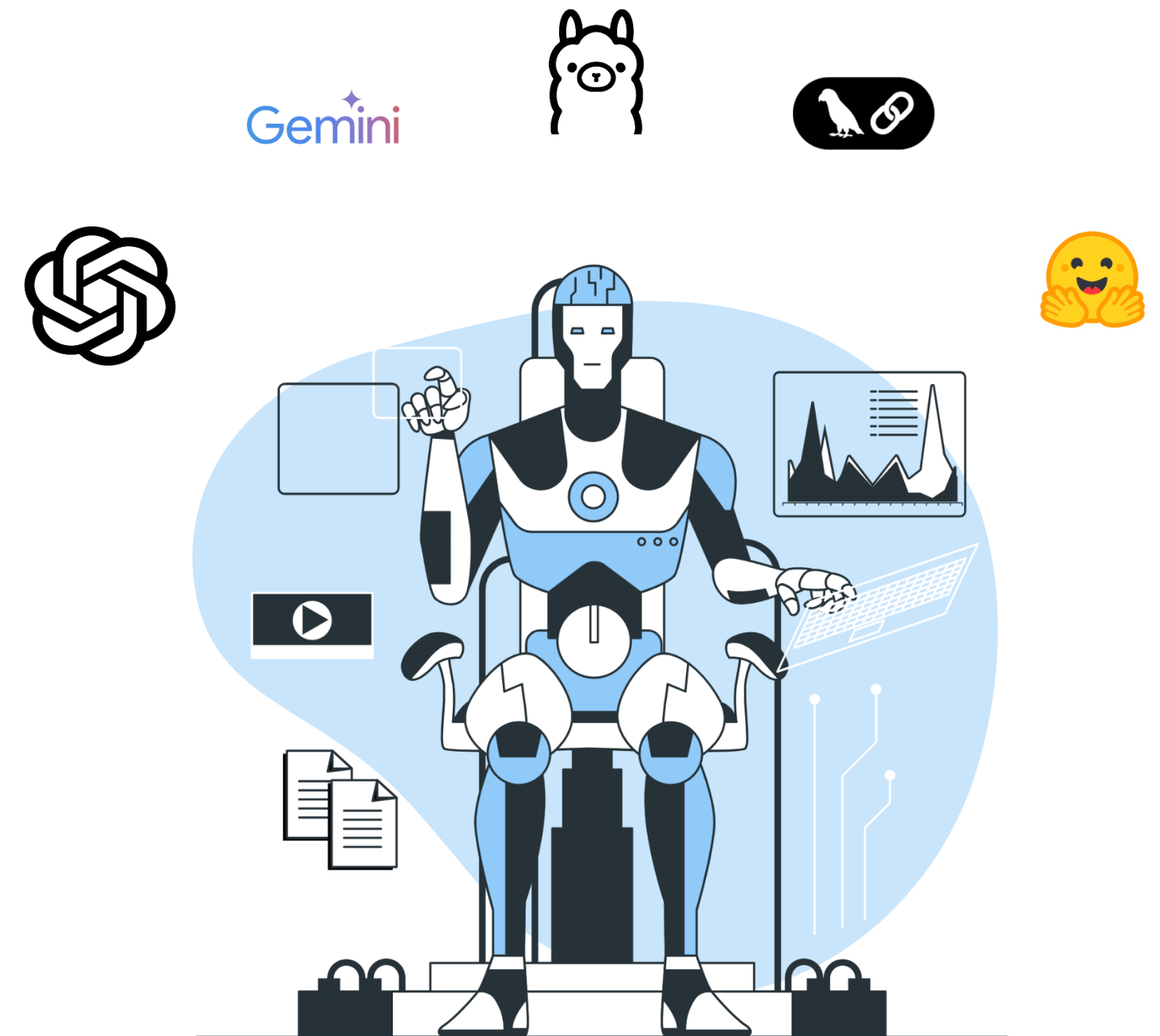
Embeddings & Vector Representations

Summary

- ❑ Acts as the semantic backbone of Agentic AI
- ❑ Allows systems to:
 - ✓ Understand meaning
 - ✓ Connect ideas
 - ✓ Recall knowledge

Section 2.6

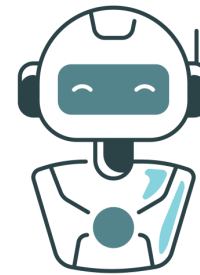
Memory Systems in Agentic AI



Memory Systems in Agentic AI



Memory



AI Agents

Learn

Adapt

Evolve

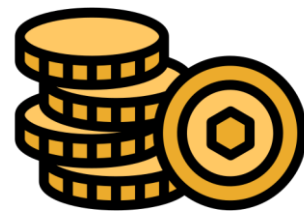
Memory Systems in Agentic AI



LLMs Limitation in handling Information



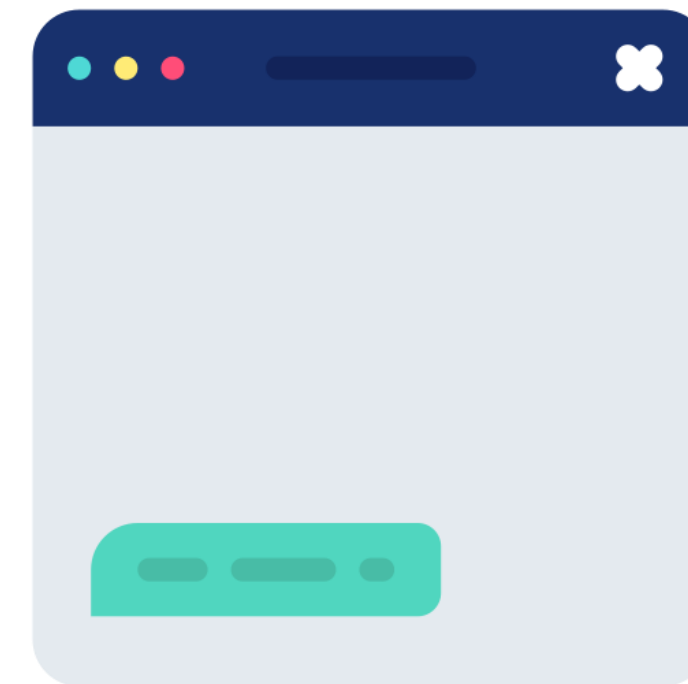
Interaction with Model



Tokens



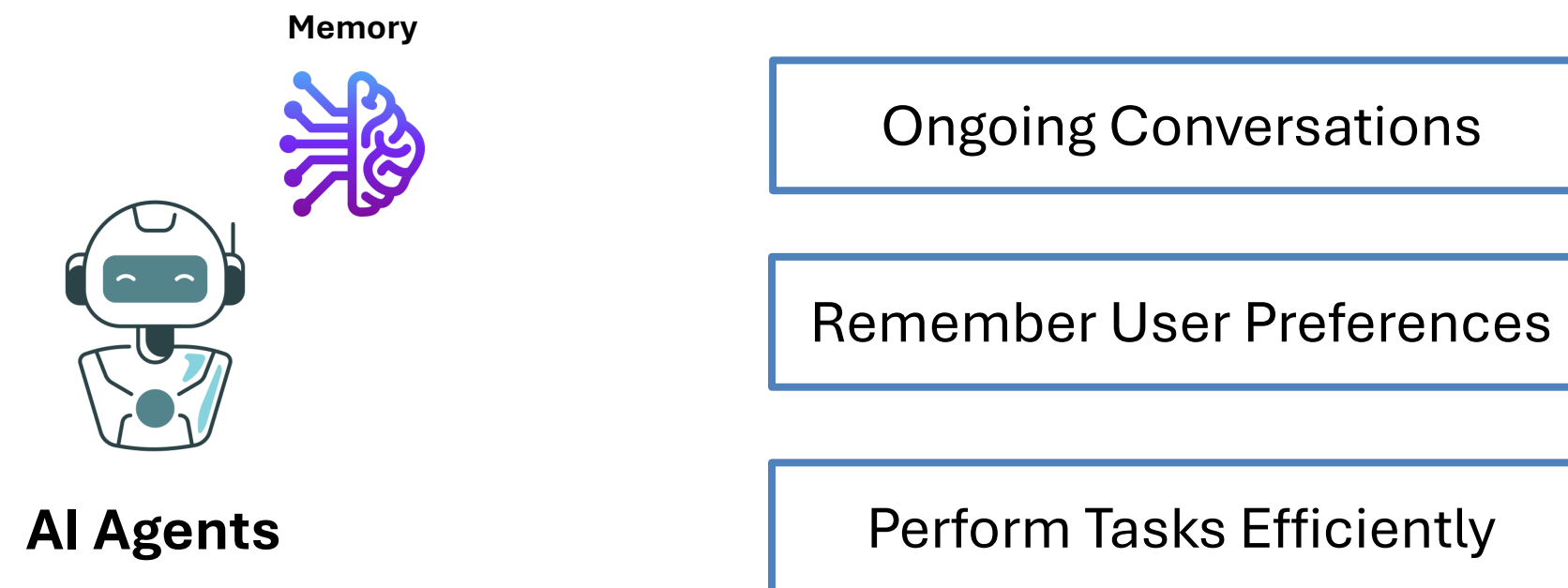
Maximum Token Limit



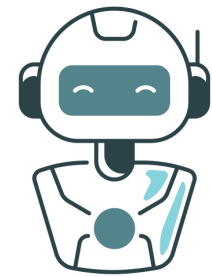
Context
Window

Model's built-in memory is temporary and limited to that session

Memory Systems in Agentic AI



Memory Systems in Agentic AI



AI Agents

Treat every new interaction
as it's first interaction

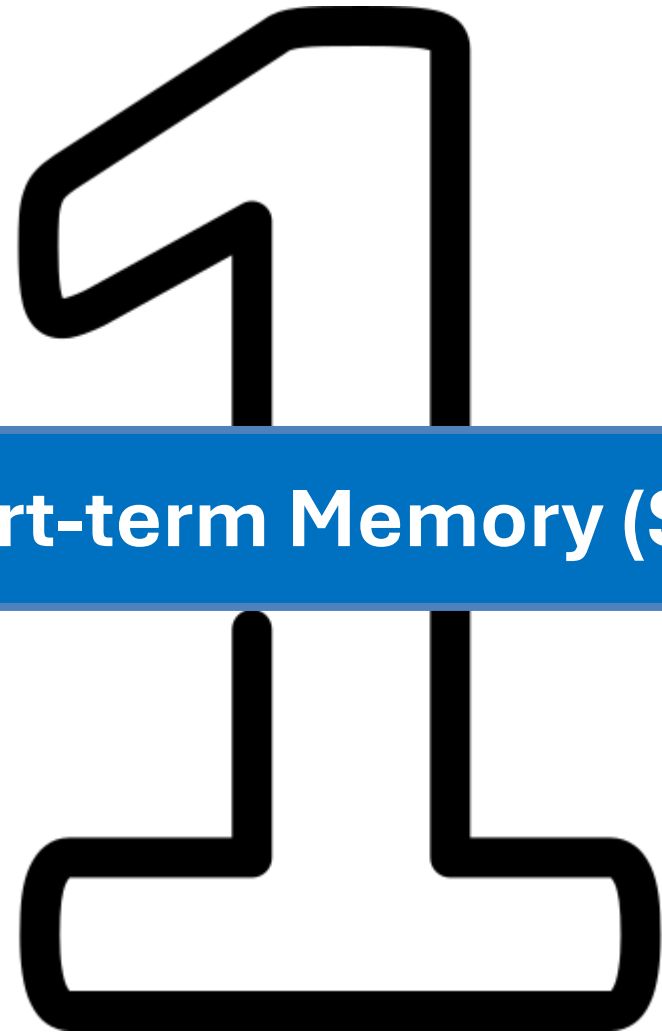
Context

Continuity

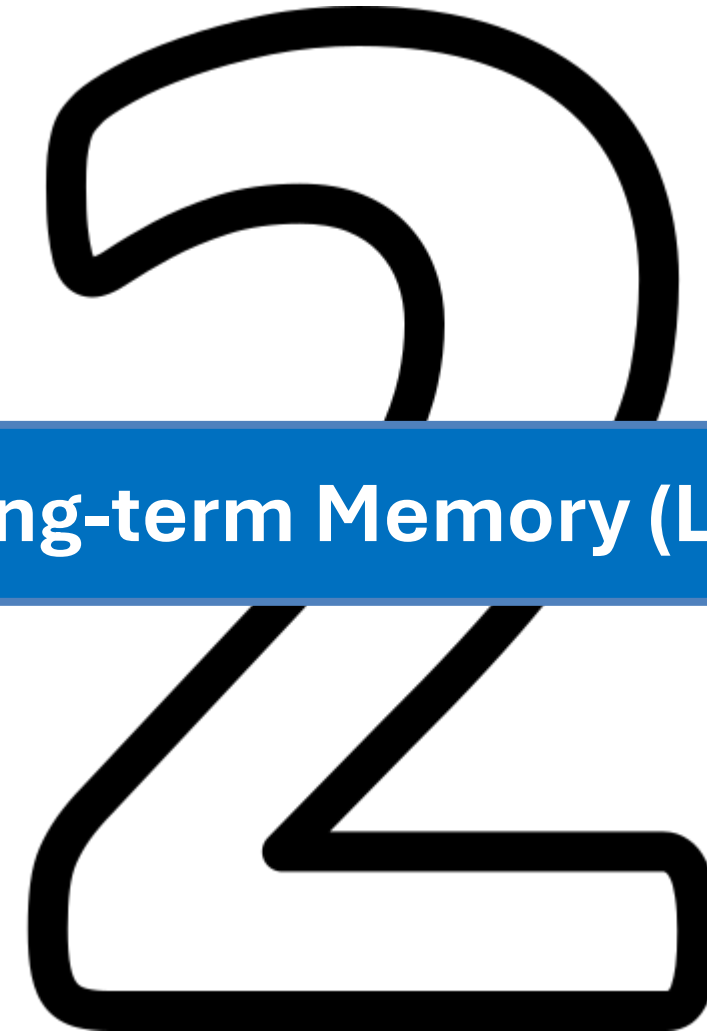
Personalization

Memory Systems in Agentic AI

Types of Memory

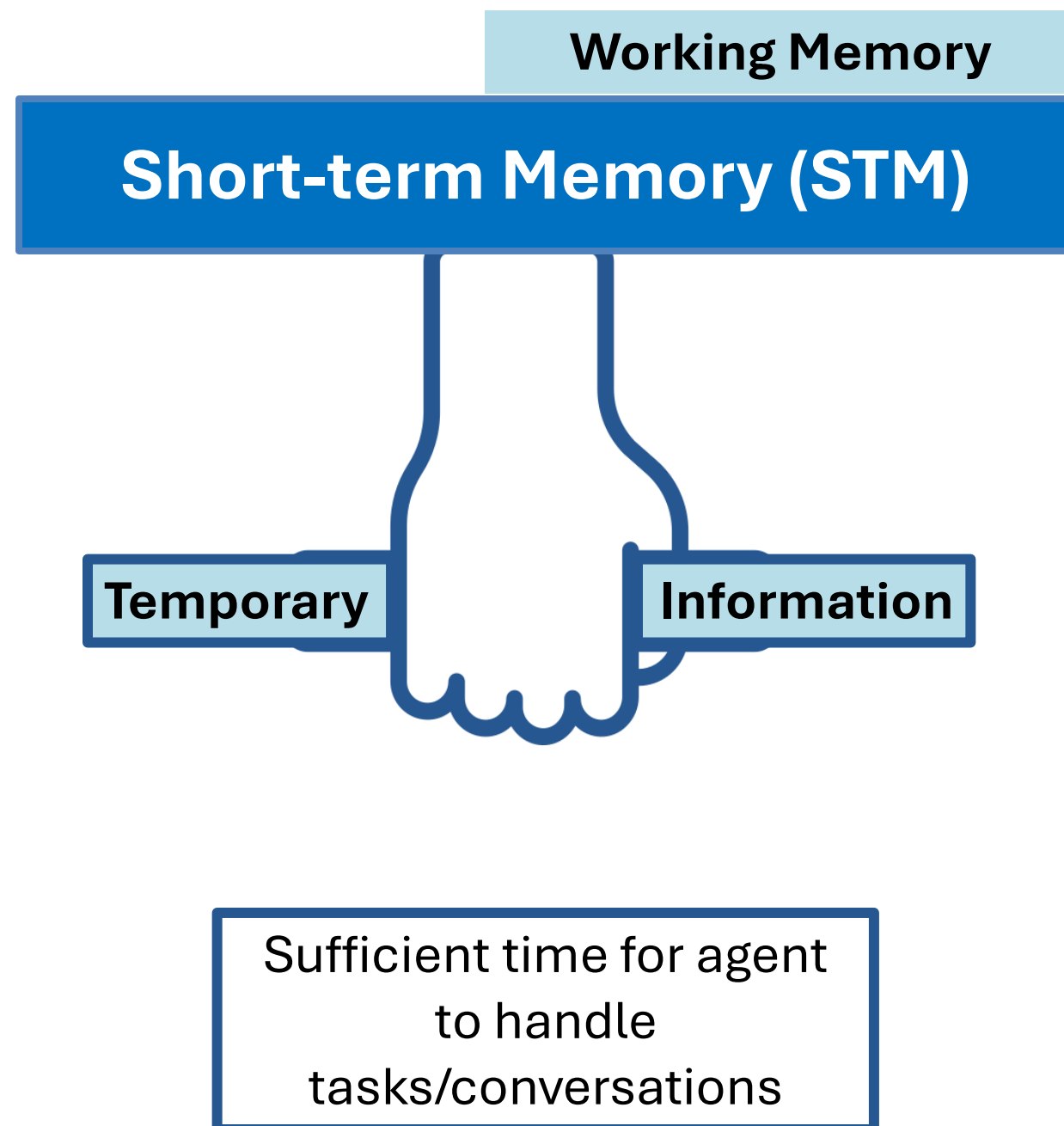


Short-term Memory (STM)



Long-term Memory (LTM)

Memory Systems in Agentic AI



For Example:

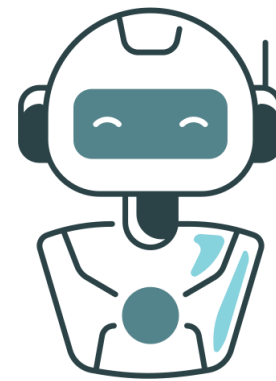
- ❑ If a user says, "Schedule a meeting with Sarah tomorrow at 3 PM," and then says, "Make it 4 PM instead," the agent knows "it" refers to that meeting
- ❑ In a sequence like "Create a report and then email it," the agent remembers both steps during this interaction



- ❑ Stores the most recent messages or actions
- ❑ Model maintains flow and relevance
- ❑ Once the session ends, this memory is cleared

Memory Systems in Agentic AI

Long-term Memory (LTM)



AI Agents

Persistent

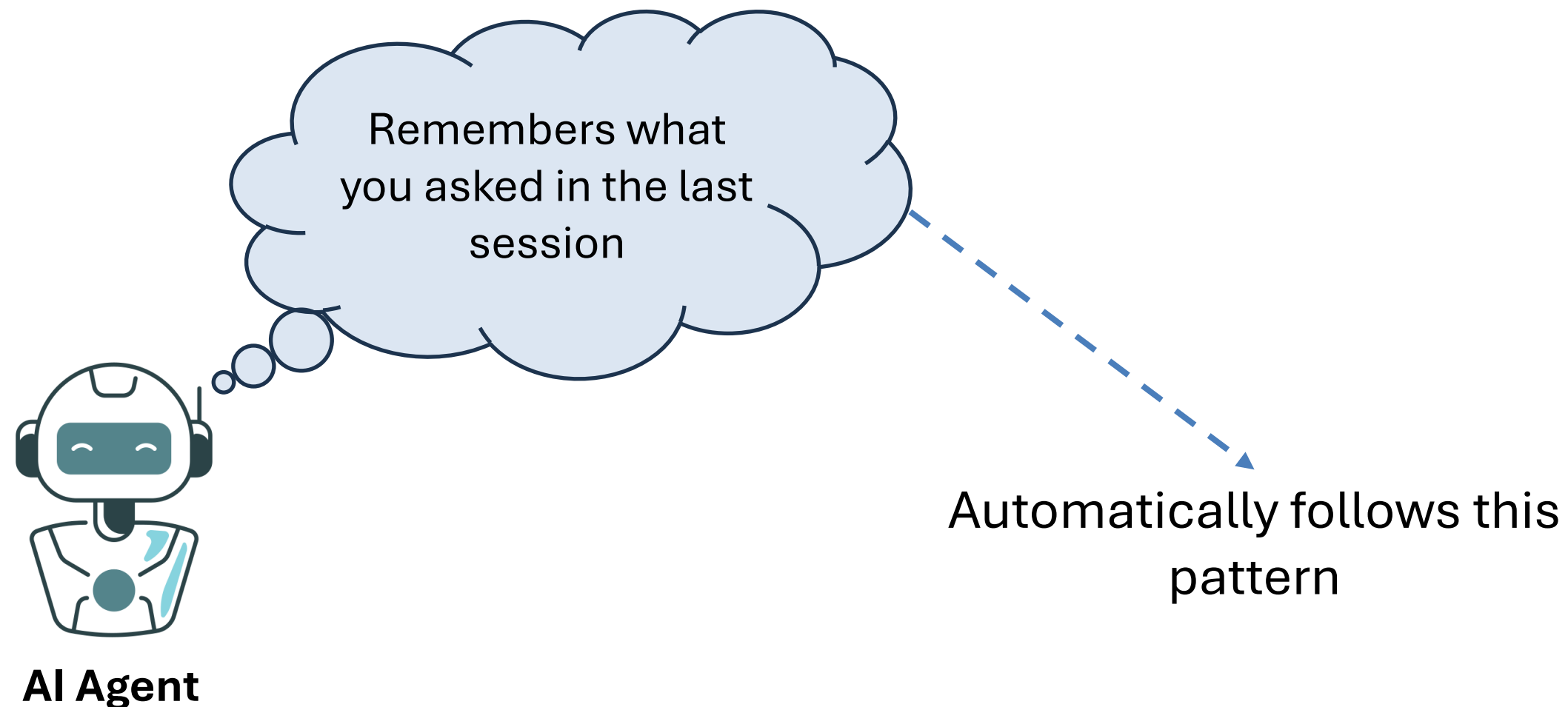
Evolving

- ❑ Stores knowledge and experiences across sessions
- ❑ Allows agents to recall old interactions, user habits, and past learnings

Memory Systems in Agentic AI

Episodic Memory

Stores specific personal experiences from past interactions



Memory Systems in Agentic AI

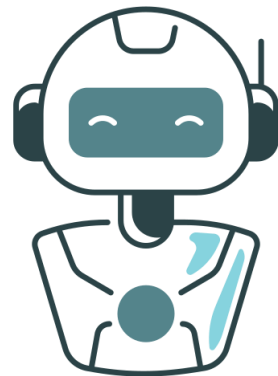
Semantic Memory

Holds general knowledge and factual information that the agent can reuse across contexts



- ✓ “Docker is a containerization tool”
- ✓ “Python is commonly used for machine learning”

---> **Vector Embeddings**



AI Agent

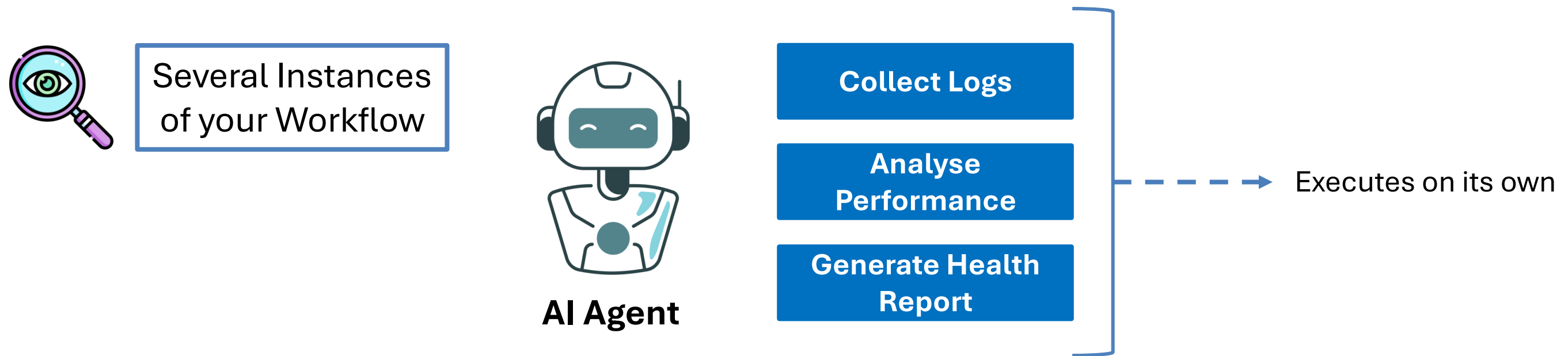
Instantly Recall

Connect Related
Concepts

Memory Systems in Agentic AI

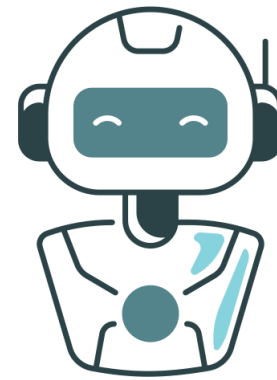
Procedural Memory

Stores skills, rules, and step-by-step processes that the agent has learned over time



Memory Systems in Agentic AI

How Memory Types Work Together



AI Agent

- ✓ Uses short-term memory to understand the current conversation
- ✓ Retrieves long-term memories like user preferences or related facts
- ✓ Applies procedural memory to execute the task efficiently
- ✓ Generates a response that feels consistent, context-aware, and personalized

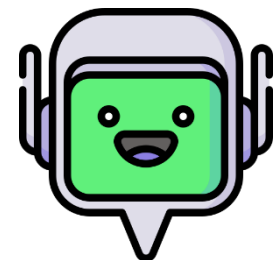
Memory Systems in Agentic AI

Real-World Examples



Smart Home Agent

- ✓ Remembers “Dim the lights after 10 PM” and does it automatically every night



Personalized Tutor

- ✓ Knows your learning style – **Procedural**
- ✓ Remembers what topics you covered – **Episodic**
- ✓ Uses its knowledge base to explain concepts - **Semantic**

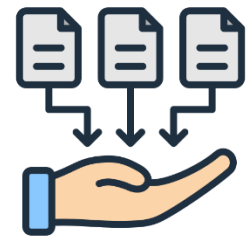


Customer Support Bot

- ✓ Keeps the chat thread active - **Short-term**
- ✓ Recalls the last issue – **Episodic**
- ✓ Refers to internal FAQs - **Semantic**
- ✓ Follows standard procedures - **Procedural**

Memory Systems in Agentic AI

How Memory is Implemented



**Data
Capture**



**Vector
Embeddings**



**Vector
Database**



**Retrieval
Process**



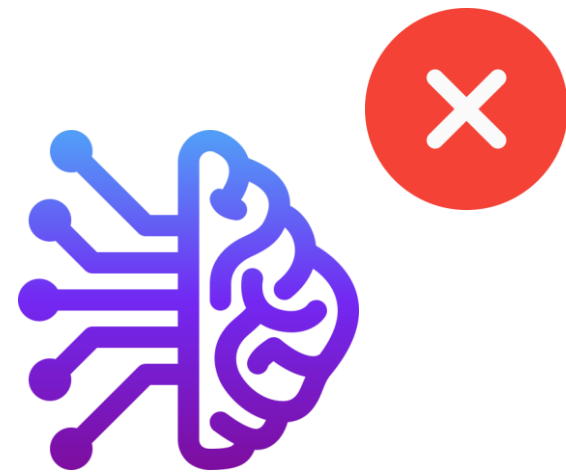
**Memory
Pruning &
Updating**



**Memory-
Augmented
Prompts**

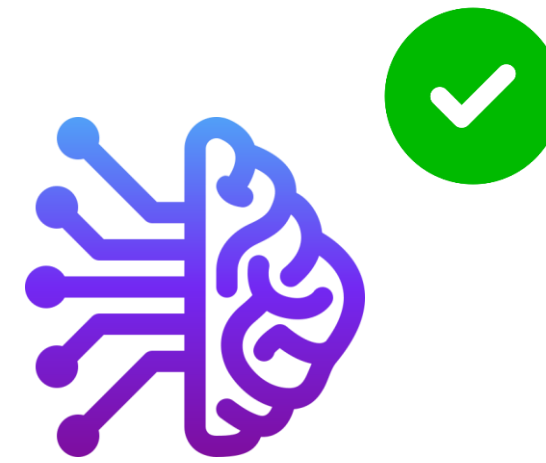
Memory Systems in Agentic AI

Why Memory Matters



Memory

- ☐ AI agent is stateless
- ☐ Forgets everything once the chat ends



Memory

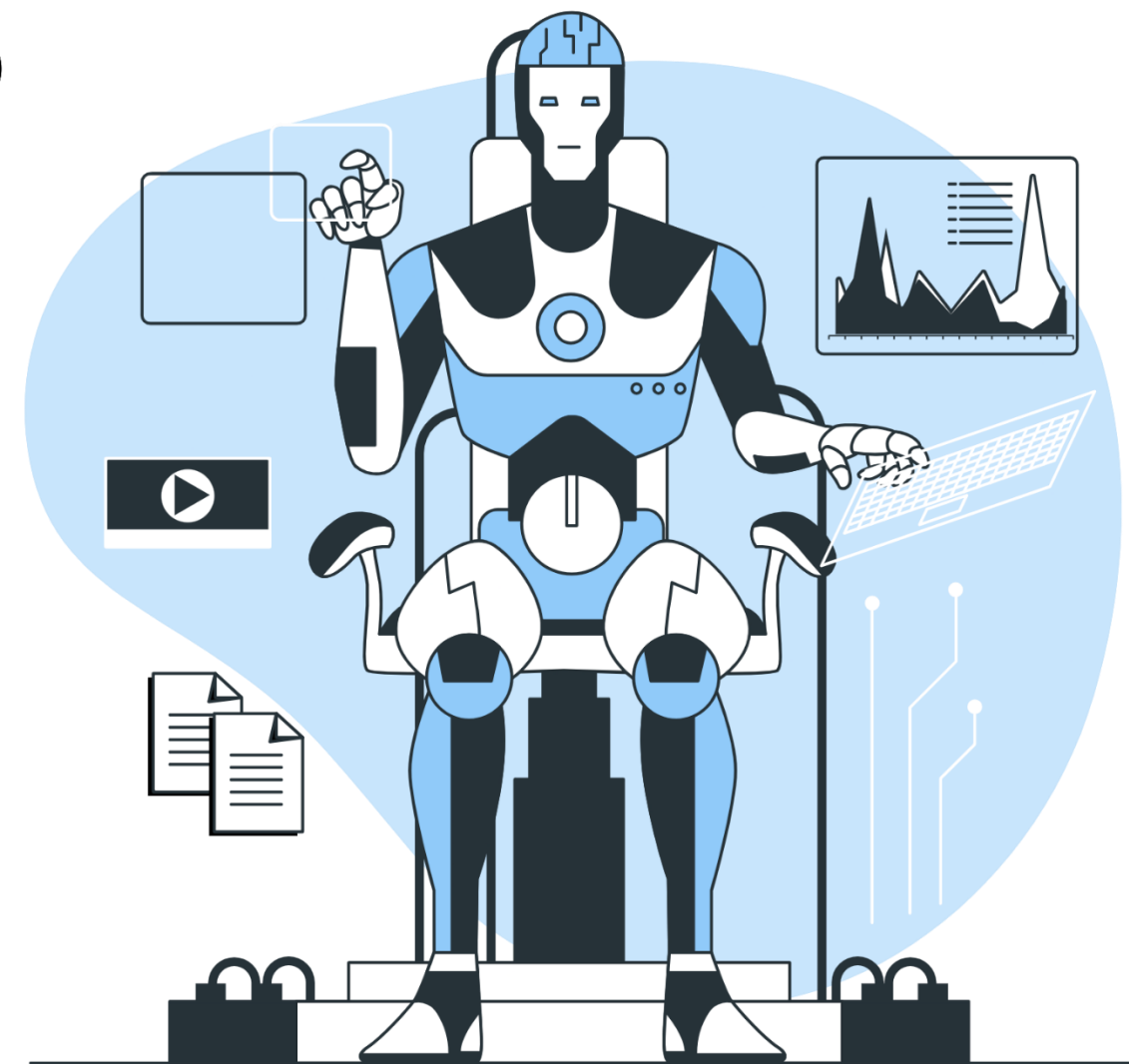
- ☐ Becomes stateful
- ☐ Capable of personalization, learning, and decision-making

Section 2.7

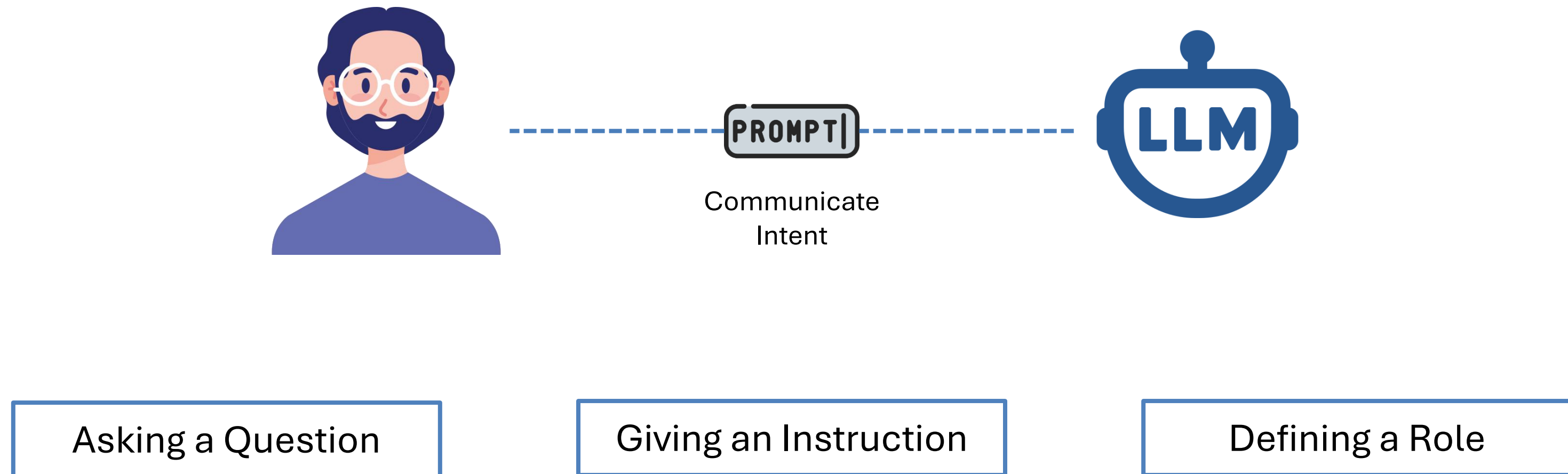
Prompt Engineering & Task Orchestration



Gemini

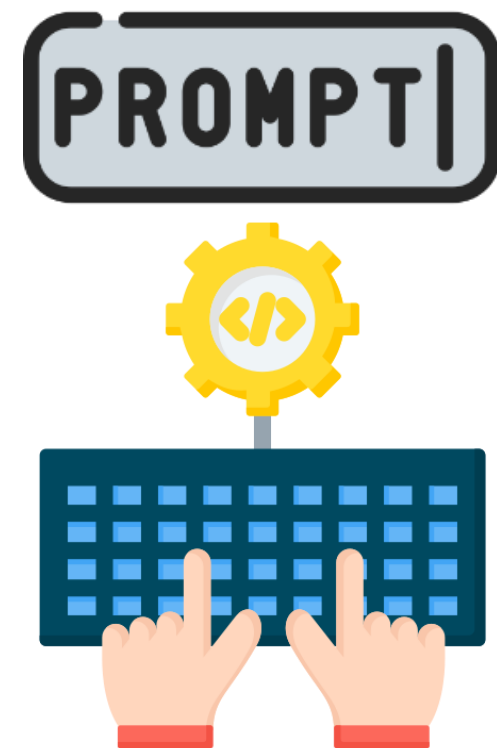


Prompt Engineering & Task Orchestration



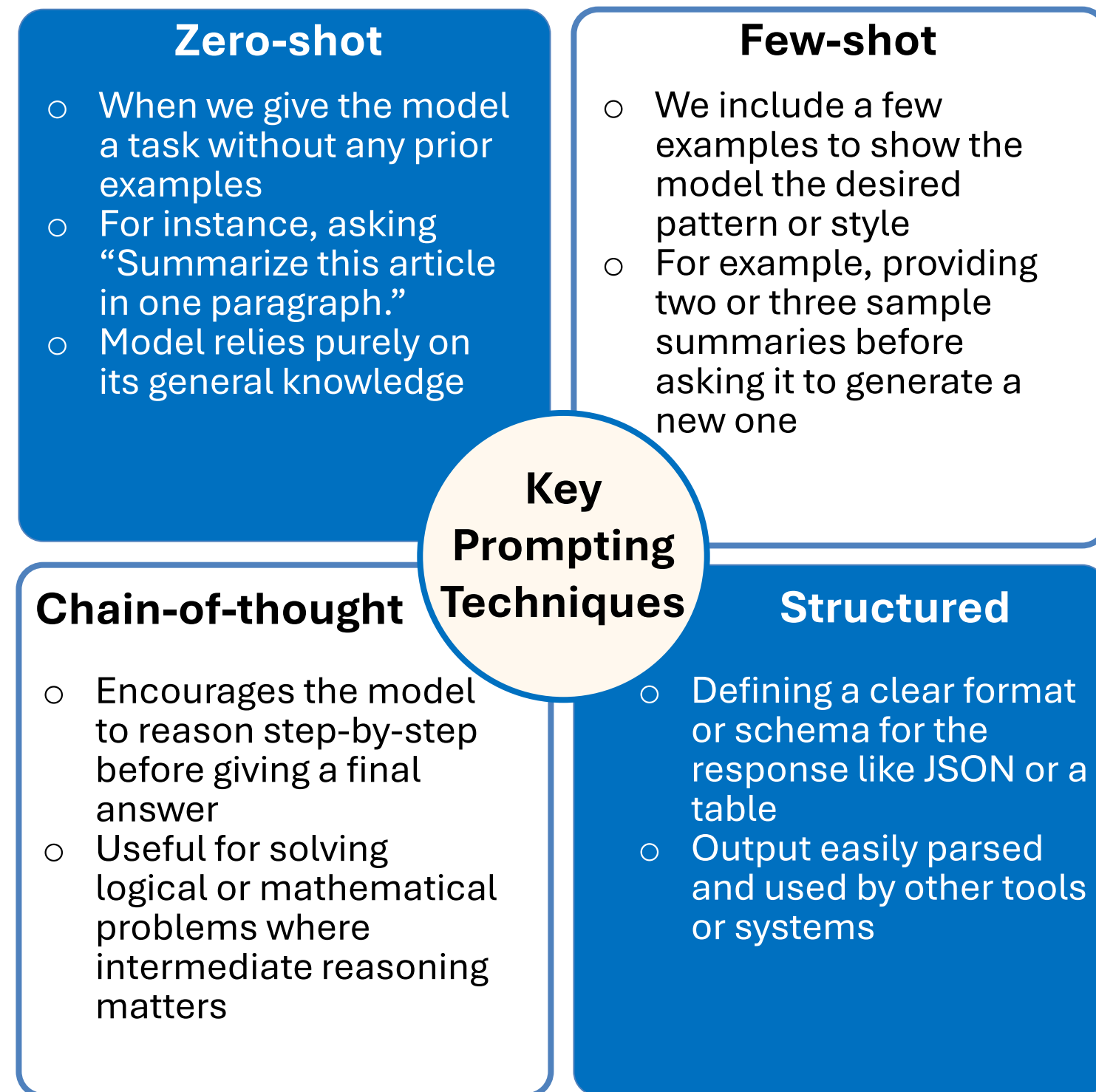
Think of prompts as the “**programming language**” of **LLMs**, the better we craft them, the better and more accurate the responses we’ll get

Prompt Engineering & Task Orchestration



Designing prompts in a way that guides the model to produce the most relevant and reliable output

Prompt Engineering & Task Orchestration



Prompt Engineering & Task Orchestration

Agentic System

PROMPT|

PROMPT|

PROMPT|

PROMPT|

PROMPT|

PROMPT|

PROMPT|

PROMPT|

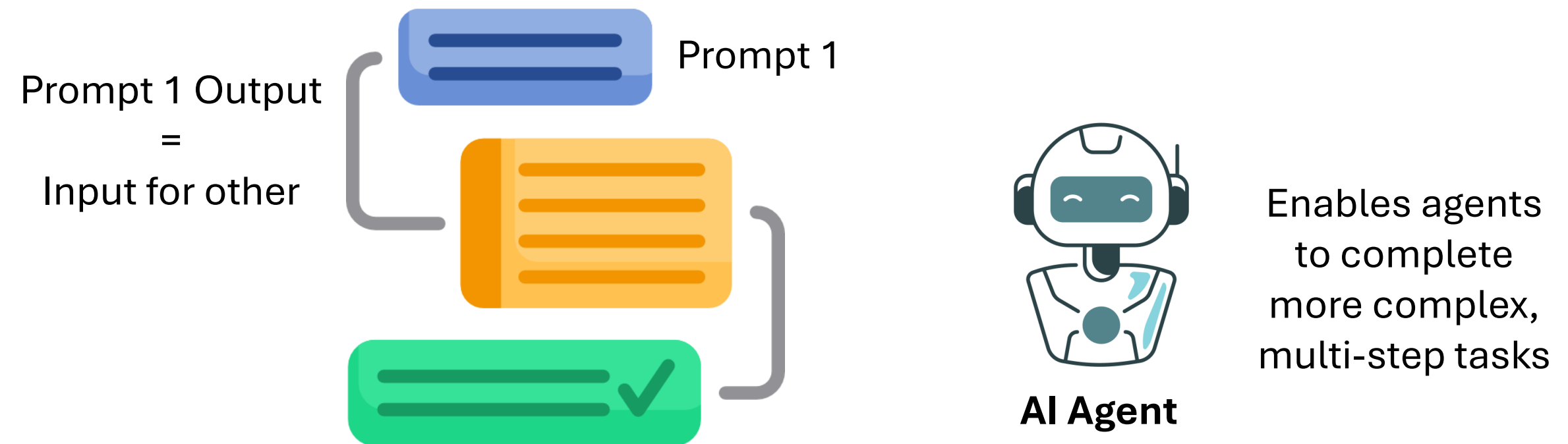
PROMPT|

Task Orchestration

Prompt Engineering & Task Orchestration

Task Orchestration

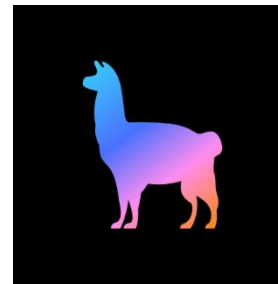
Refers to how we chain multiple prompts together, manage different tools or APIs, and maintain logical flow between them



Prompt Engineering & Task Orchestration



LangChain



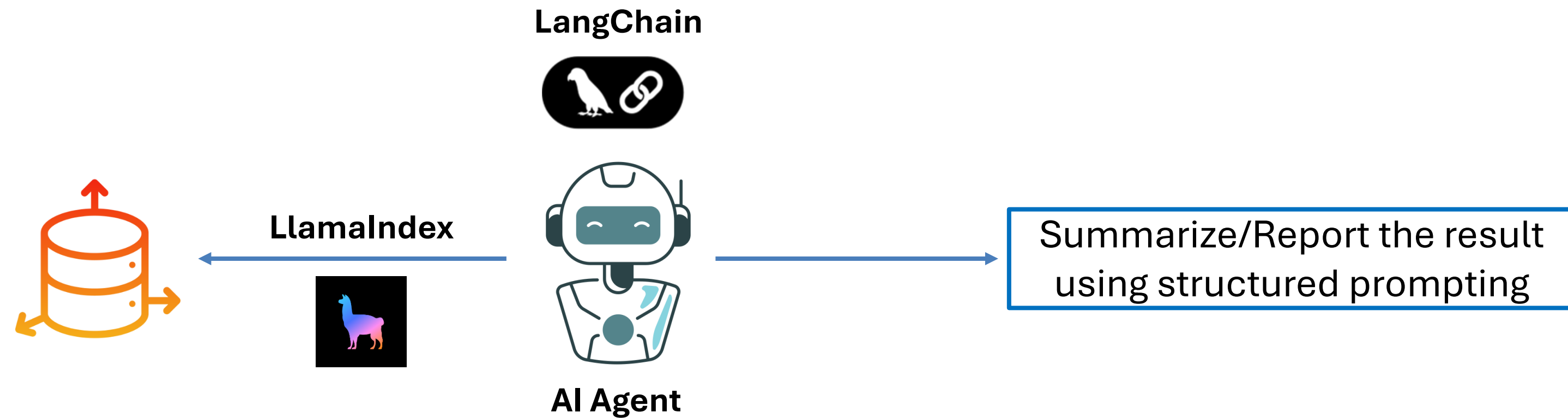
LlamaIndex



Semantic Kernel

Help developers design pipelines that combine LLM reasoning with external tools like databases, APIs, or custom logic

Prompt Engineering & Task Orchestration



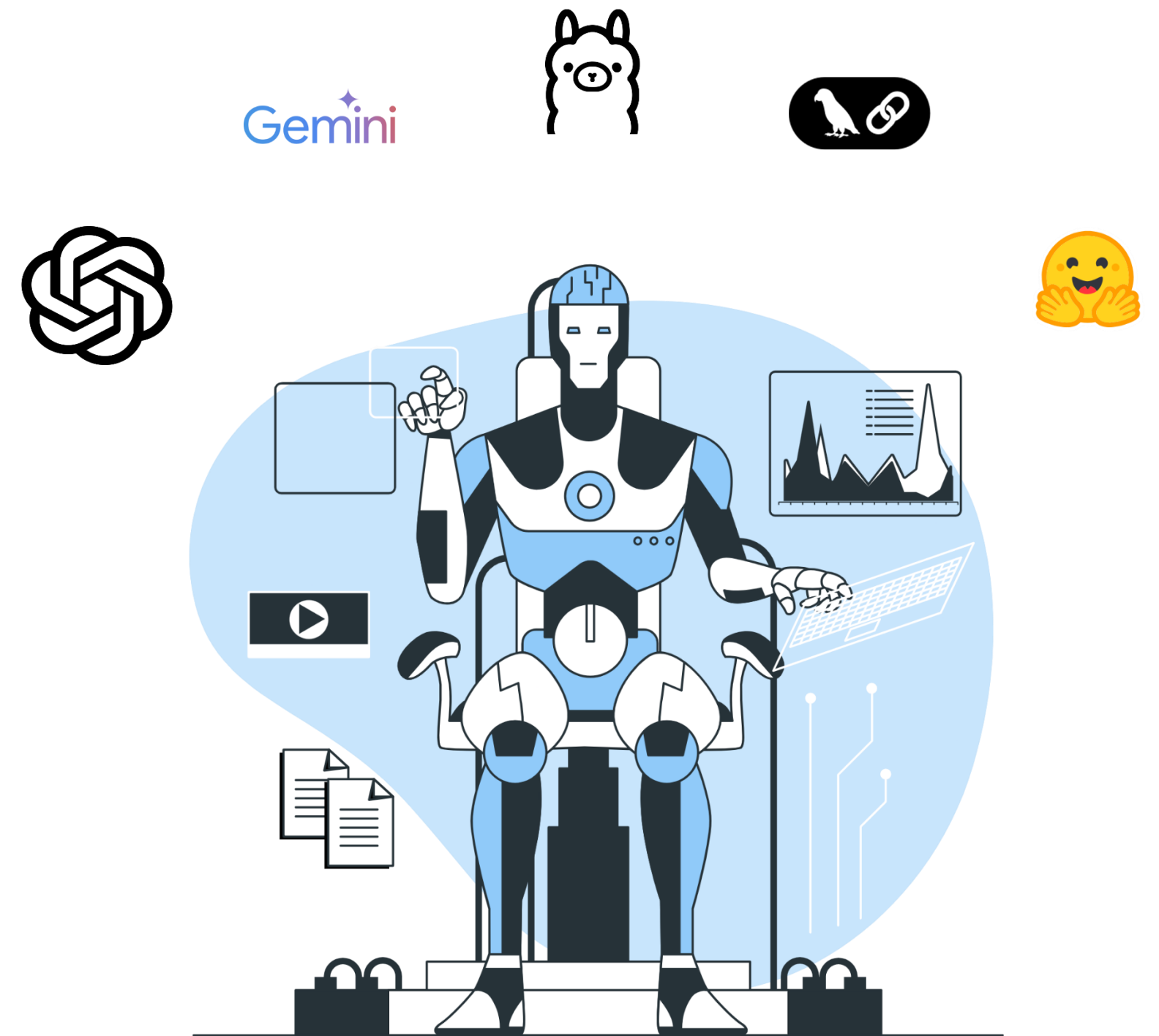
Prompt Engineering & Task Orchestration

Control Layer of Agentic Systems

Prompt Engineering	Task Orchestration
Define what to do	Define how to get it done

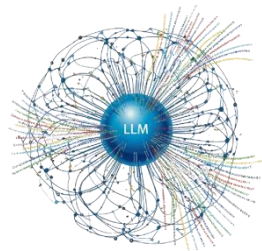
Section 2.8

Agentic AI Workflow



Agentic AI Workflow

Cognitive Layer



LLM

Interprets user's input

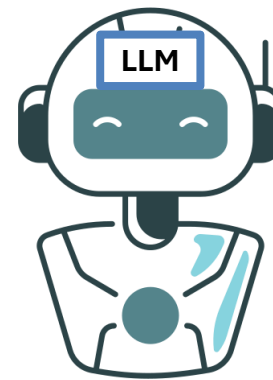
Break it down

Determines objective

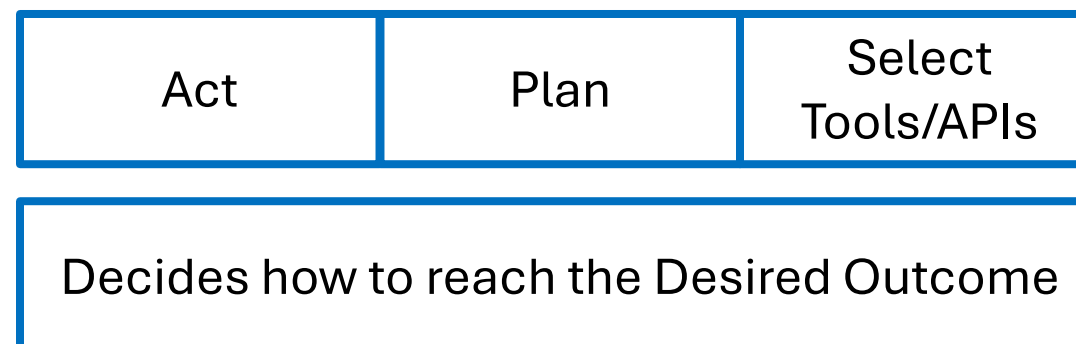


Agentic AI Workflow

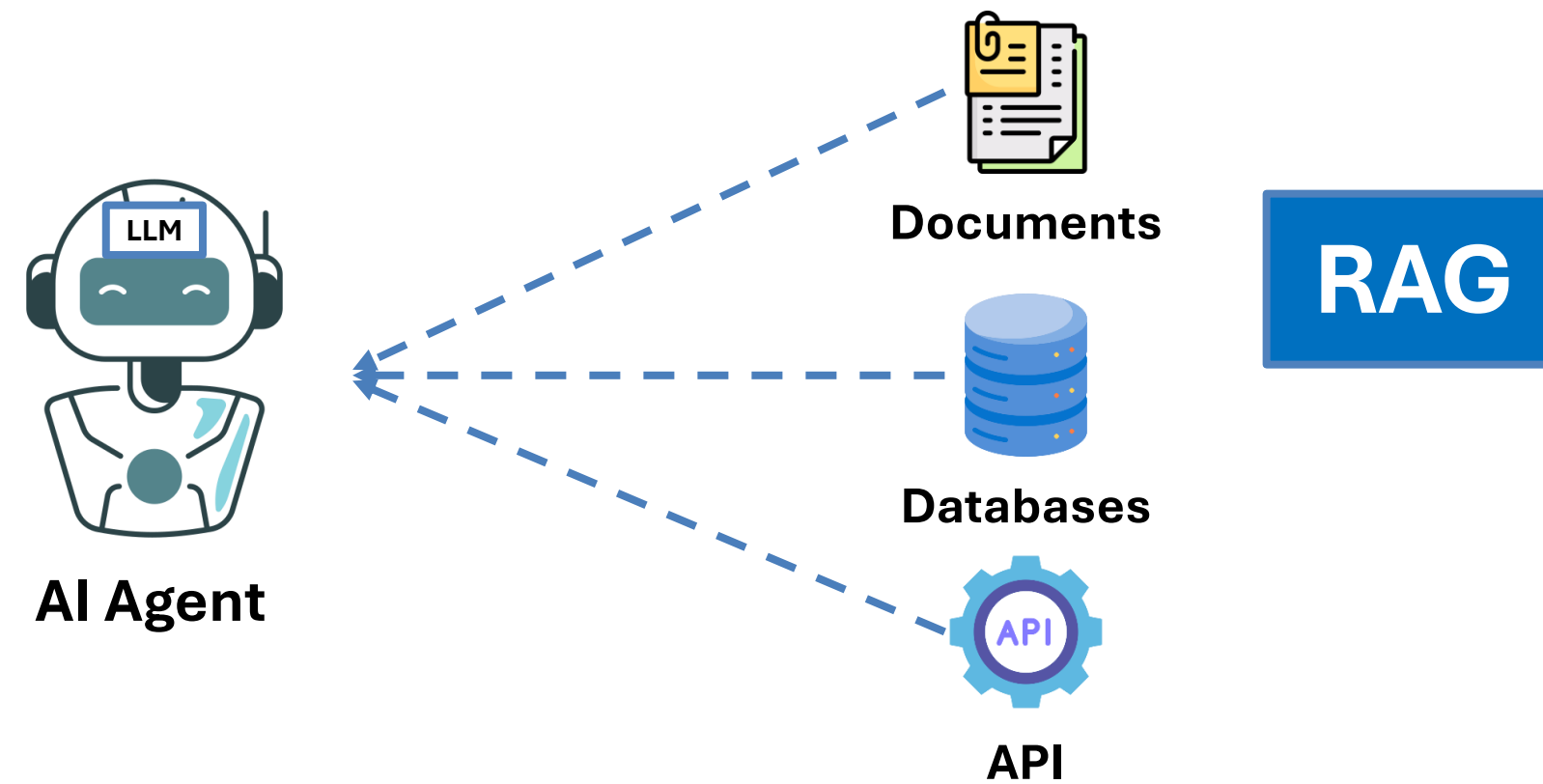
Operational Layer



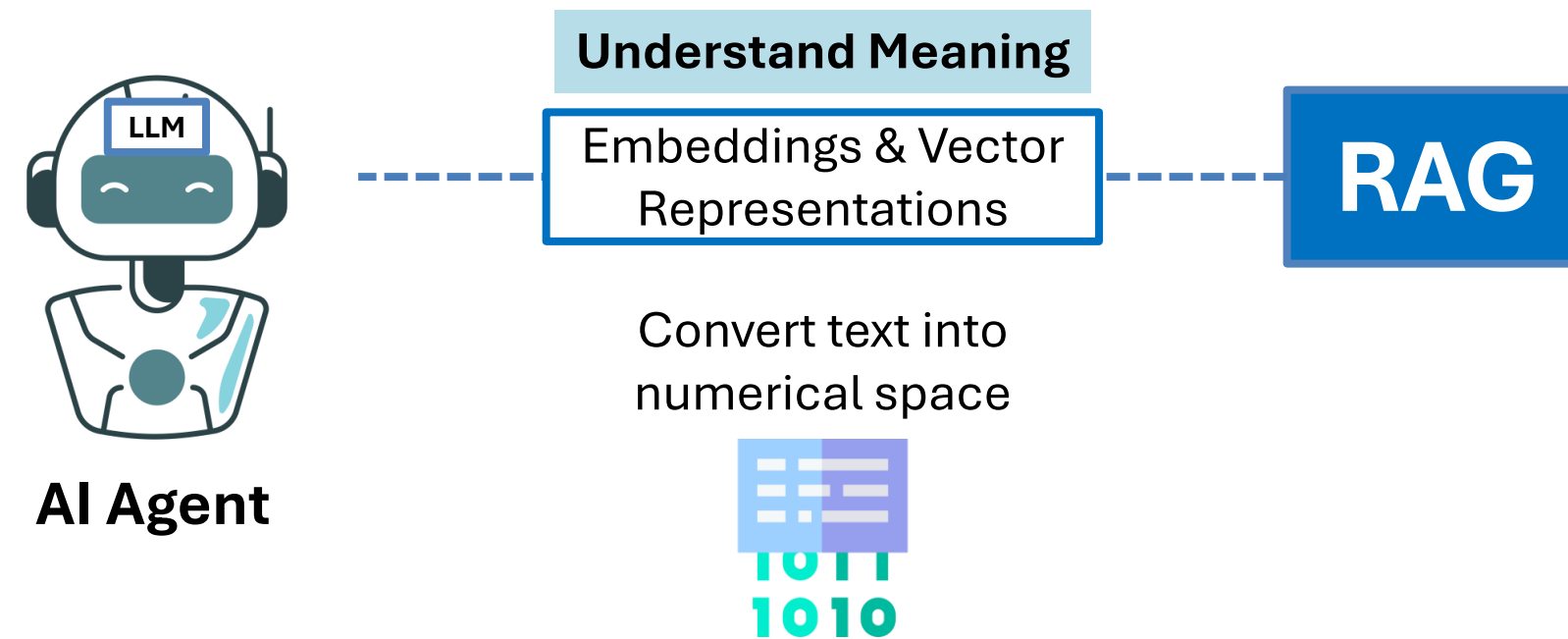
AI Agent



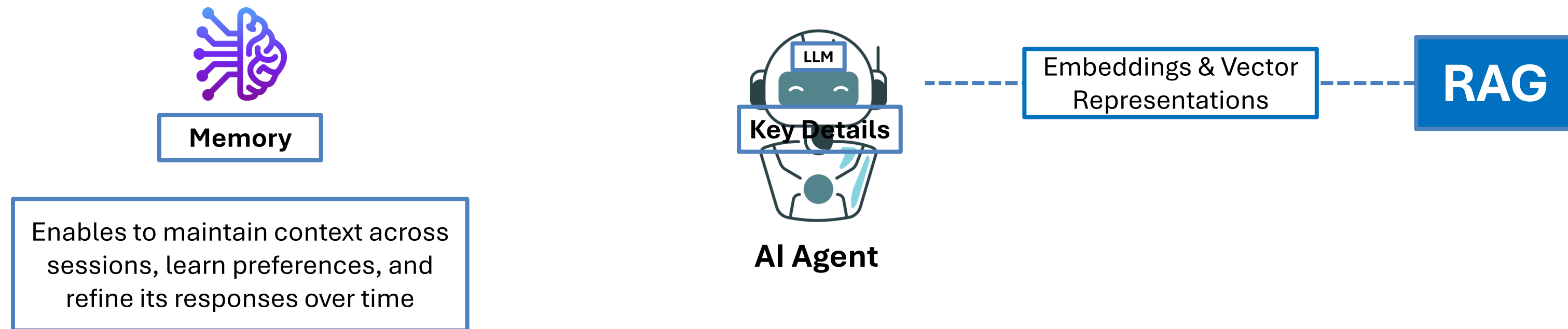
Agentic AI Workflow



Agentic AI Workflow



Agentic AI Workflow



Agentic AI Workflow

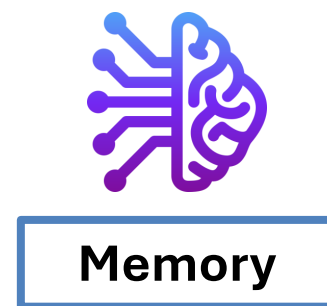
- ✓ Carefully designed prompts guide the LLM to reason, plan, and act systematically

Prompt Engineering

Task Orchestration

Conductor of Orchestra

- ✓ Task orchestration ensures sub-tasks run in the right sequence with the right dependencies to achieve the goal

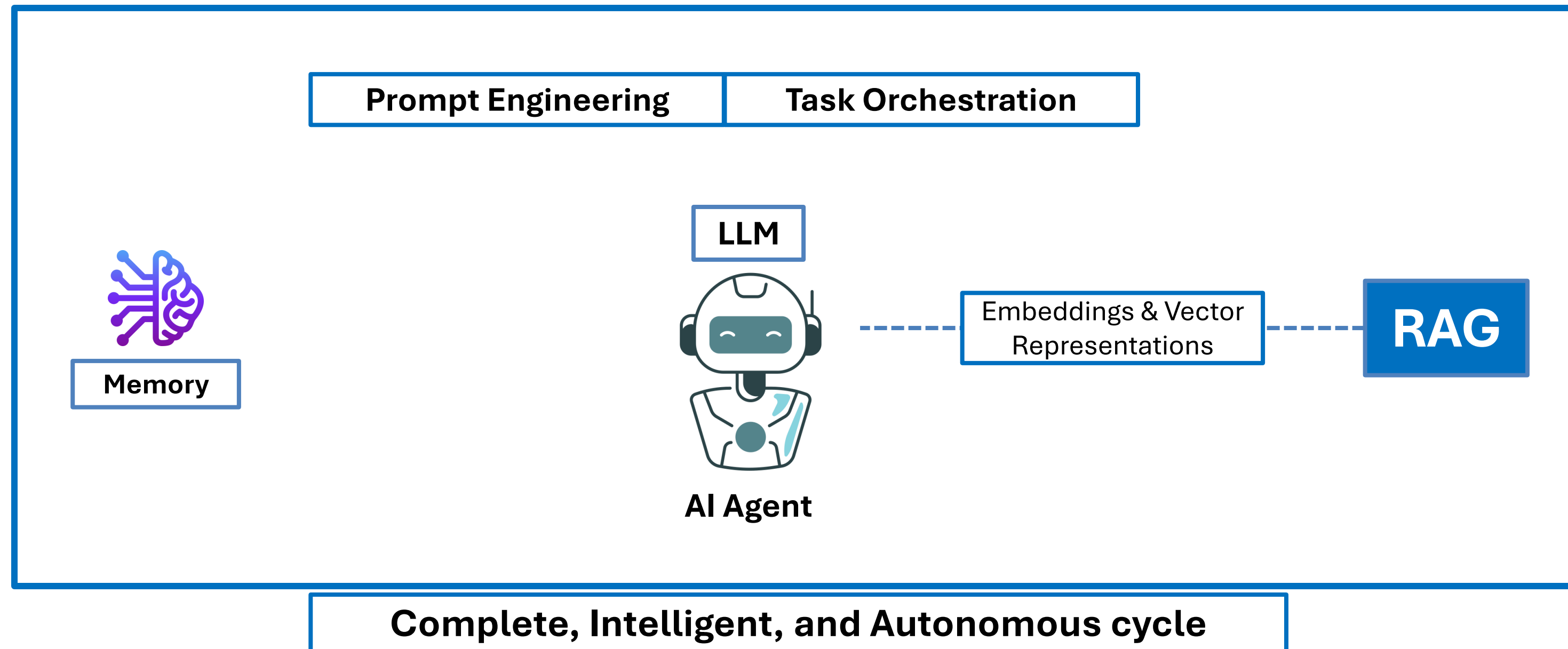


AI Agent

Embeddings & Vector
Representations

RAG

Agentic AI Workflow



Section 2.9

Demonstration: AWS Instance Launcher



Section 2.10

Summary

- ✓ Large Language Models (LLMs)
- ✓ AI Agents
- ✓ Demonstration: Weather AI Agent from scratch
- ✓ Understood RAG, Embeddings & Vector Representations
- ✓ Memory systems for continuity
- ✓ Prompt engineering & Task orchestration
- ✓ Workflow of Agentic AI system
- ✓ Demonstration: AWS Instance Launcher Agentic system

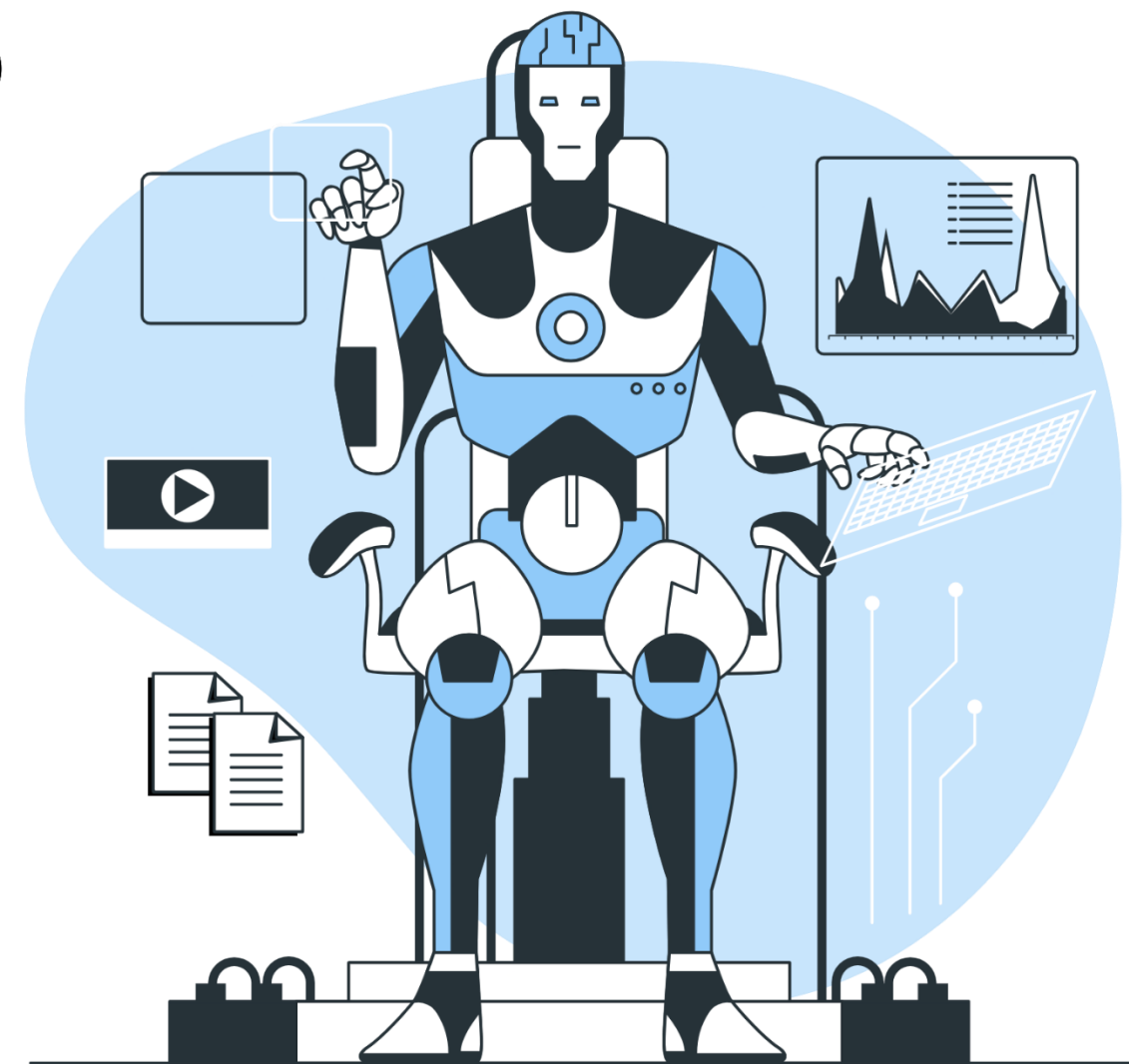
Thank You

Section 3.0

Developing Agentic AI Systems



Gemini



Developing Agentic AI Systems

Section Overview

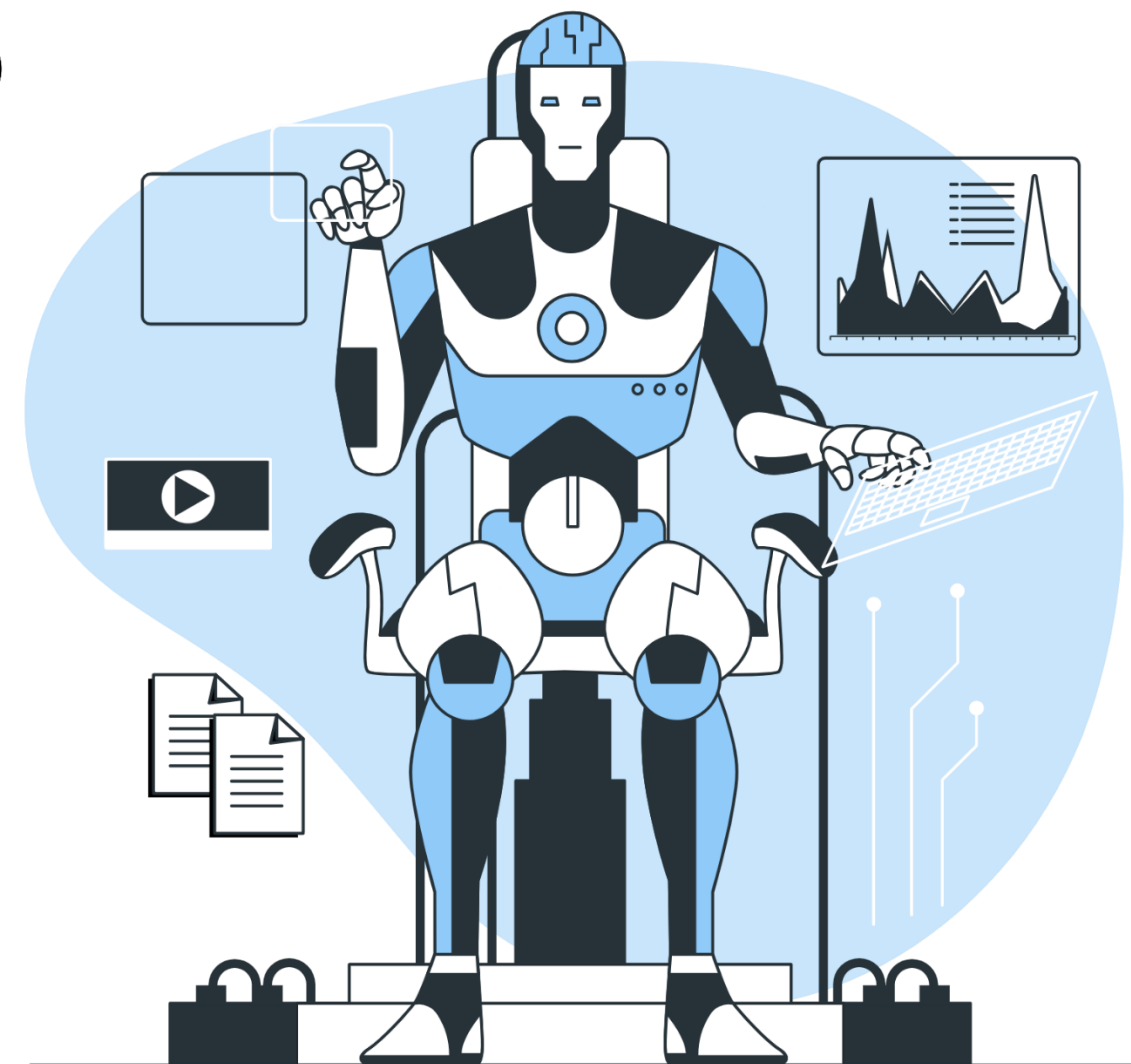
- ❑ Explore Frameworks and Development Platforms
- ❑ Hands-on Demonstration using **Smolagents**:
 - ❑ Introduction
 - ❑ Weather Agentic AI
 - ❑ SQL workflows
- ❑ Demonstration: Low-Code and No-Code agent with **n8n**

Section 3.1

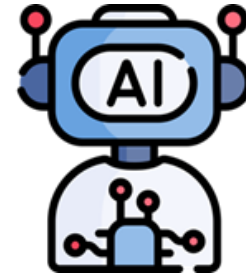
Frameworks & Development Platforms



Gemini



Frameworks & Development Platforms



Agentic AI

Foundation Models

Supporting Tools

Frameworks

Frameworks & Development Platforms

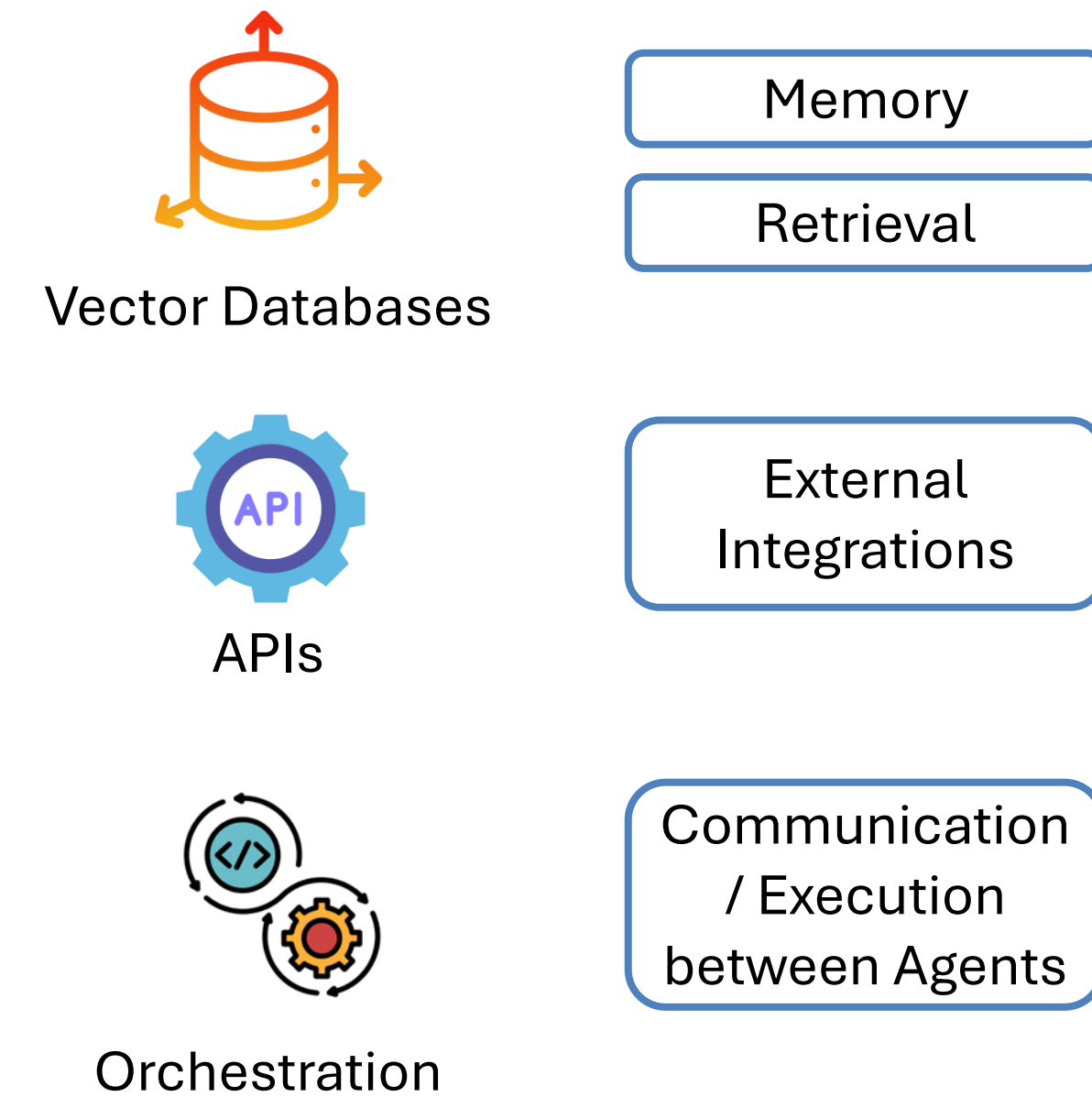
Foundation Models



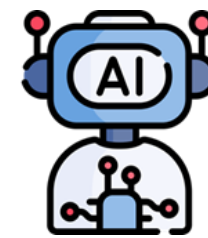
Reasoning

Language Understanding

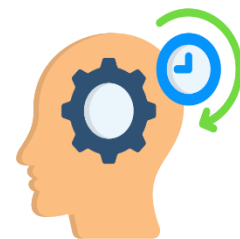
Supporting Tools



Frameworks & Development Platforms



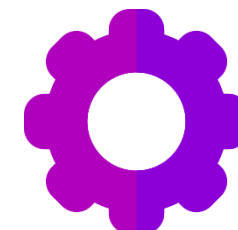
Agentic AI



Memory



Reasoning
Loops



Tool Usage



Multi-agent
Coordination

Frameworks & Development Platforms

Frameworks

Code

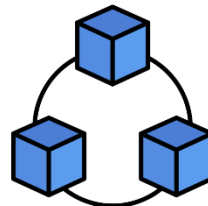


Flexibility



Control

Low Code



Pre-built Components

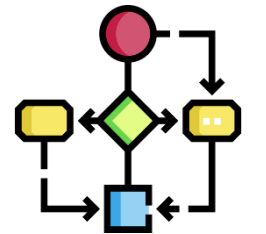


Minimal Scripting

No Code



Visual Interfaces



Drag-Drop Workflows

Frameworks & Development Platforms

Code



LangGraph

Enables complex multi-agent systems with clear state and flow control



LlamaIndex

Connects agents to external data for context-aware reasoning

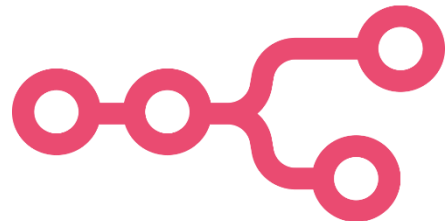


Smolagents

Offers a lightweight framework for quick experimentation and easy setup

Frameworks & Development Platforms

Low Code



Enables visual workflow automation to integrate agents with tools, APIs, and real-world pipelines



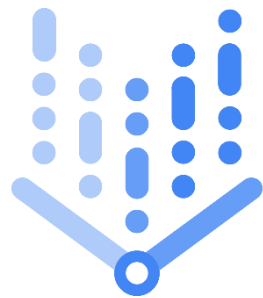
Enables collaborative multi-agent systems with efficient teamwork and minimal coding



Streamlines agent orchestration with an intuitive, configuration-based setup and minimal coding

Frameworks & Development Platforms

No Code



Enables users to deploy and manage agents directly through its UI



Enable custom agent creation through natural language instructions for rapid experimentation



Offers visual no-code platform for building agents for integration of LLMs with external tools and services

Frameworks & Development Platforms

Ecosystems



Hugging Face



OpenAI

- ❑ Offer direct access to models, embeddings, and tools
- ❑ Easy integration into agentic architectures for seamless development
- ❑ Reliable and well-supported ecosystems with stable APIs and SDKs
- ❑ Production-ready endpoints for building intelligent real-world applications

Frameworks & Development Platforms

Frameworks for this course



Smolagents



CrewAI



n8n

Section 3.2

Demonstration | Building our first Agentic AI using smolagent



Section 3.3

Demonstration: SQL with Agentic AI with smolagent



Section 3.4

Demonstration: Creating low code agent with n8n



Section 3.5

Demonstration: Creating Low-Code and No-Code agent with n8n



Demonstration: Creating Low-Code and No-Code agent with n8n



Writing Code

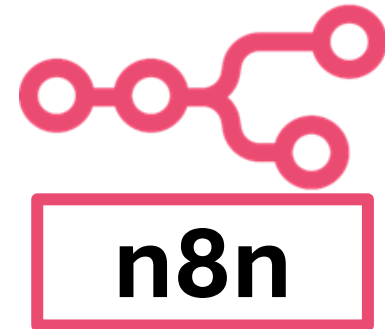
Powerful

Complex



n8n

Demonstration: Creating Low-Code and No-Code agent with n8n



- ❑ An open-source workflow automation platform enabling users to visually create workflows with a node-based interface
- ❑ It has over 400 integrations that you can connect your workflow

Features

Low-Code & No-Code Interface

- ✓ Convenient and accessible for both technical and non-technical users

Custom Code Execution

- ✓ Allows custom logic integration within workflows

Version Control

- ✓ Supports version control and workflow history

Scalable Automations

- ✓ Offers secure self-hosted and managed cloud options

Demonstration: Creating Low-Code and No-Code agent with n8n



- ☐ Build workflows visually using drag-and-drop nodes
- ☐ Each node performs a specific action
- ☐ Enables to create multi-step AI agents with minimal or no coding

Demonstration: Creating Low-Code and No-Code agent with n8n

Our Project Workflow

- ❑ Create an automation using connected nodes called **Workflow**
- ❑ Nodes are steps that perform actions (API, email, AI, etc.)
- ❑ **Trigger Node** – Nodes that trigger the entire workflow
- ❑ **Regular Nodes** – Nodes that runs afterwards for specific actions

Section 3.6

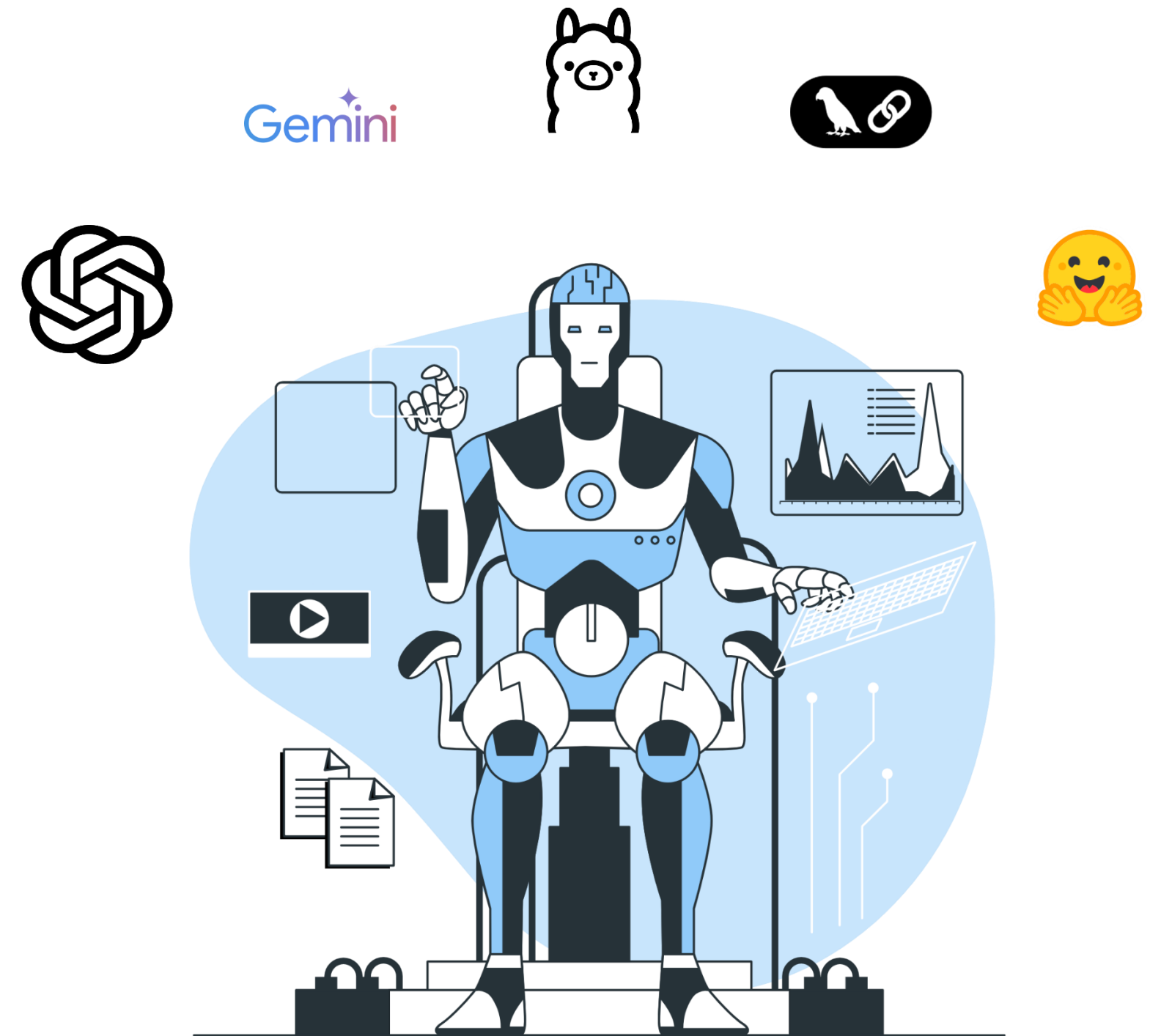
Summary

- ✓ Frameworks and development platforms
- ✓ Hands-on demonstrations with **Smolagents**:
 - ✓ Weather Agentic AI
 - ✓ SQL workflows
- ✓ Hands-on demonstrations: Low-Code and No-Code agent using **n8n**

Thank You

Section 4.0

Getting started with MCP



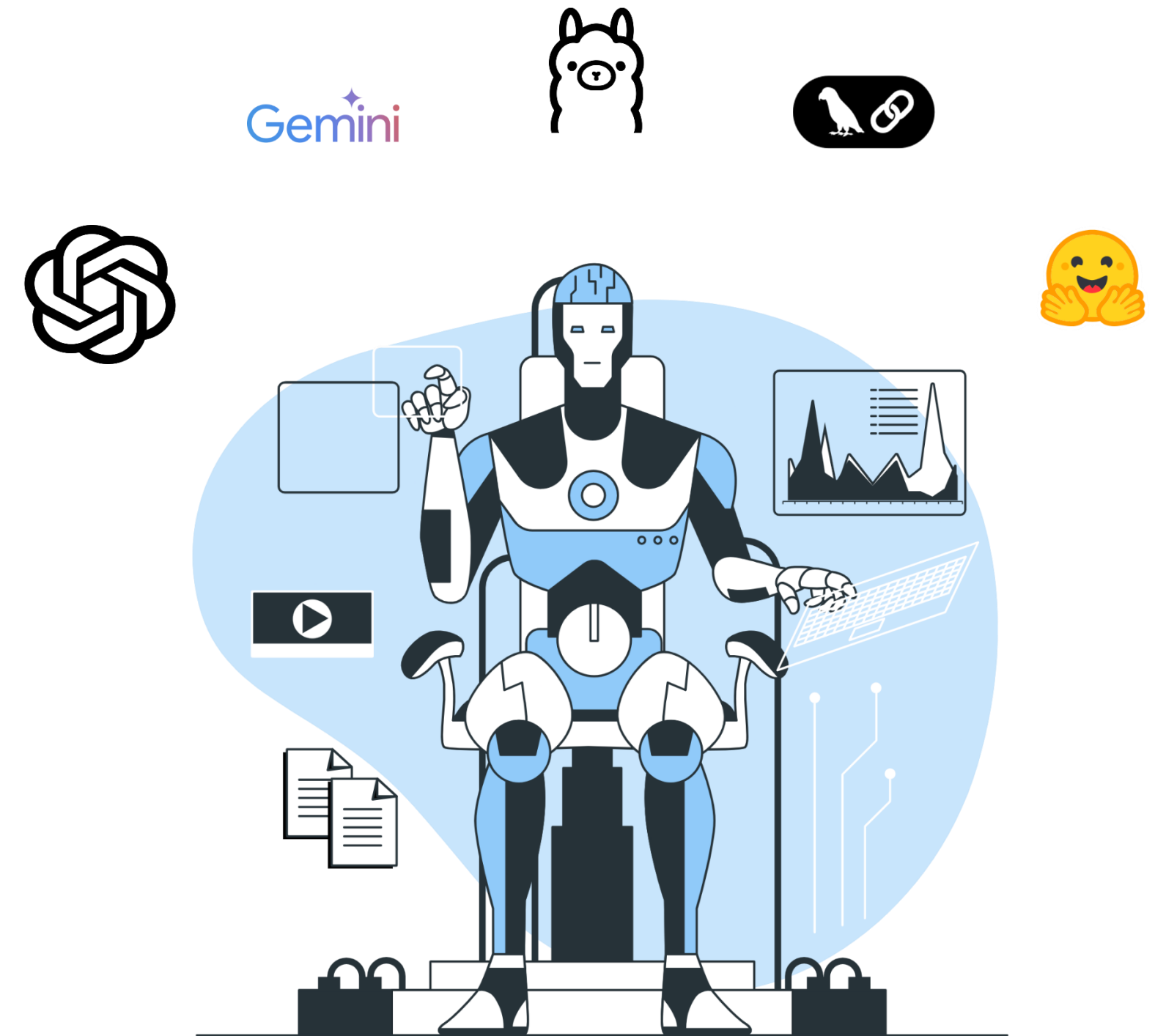
Getting started with MCP

Section Overview

- ❑ Understand MCP & its working
- ❑ Hands-on demonstration:
 - ❑ Building MCP server
 - ❑ Integrating agent with MCP server
 - ❑ Setting up Docker MCP

Section 4.1

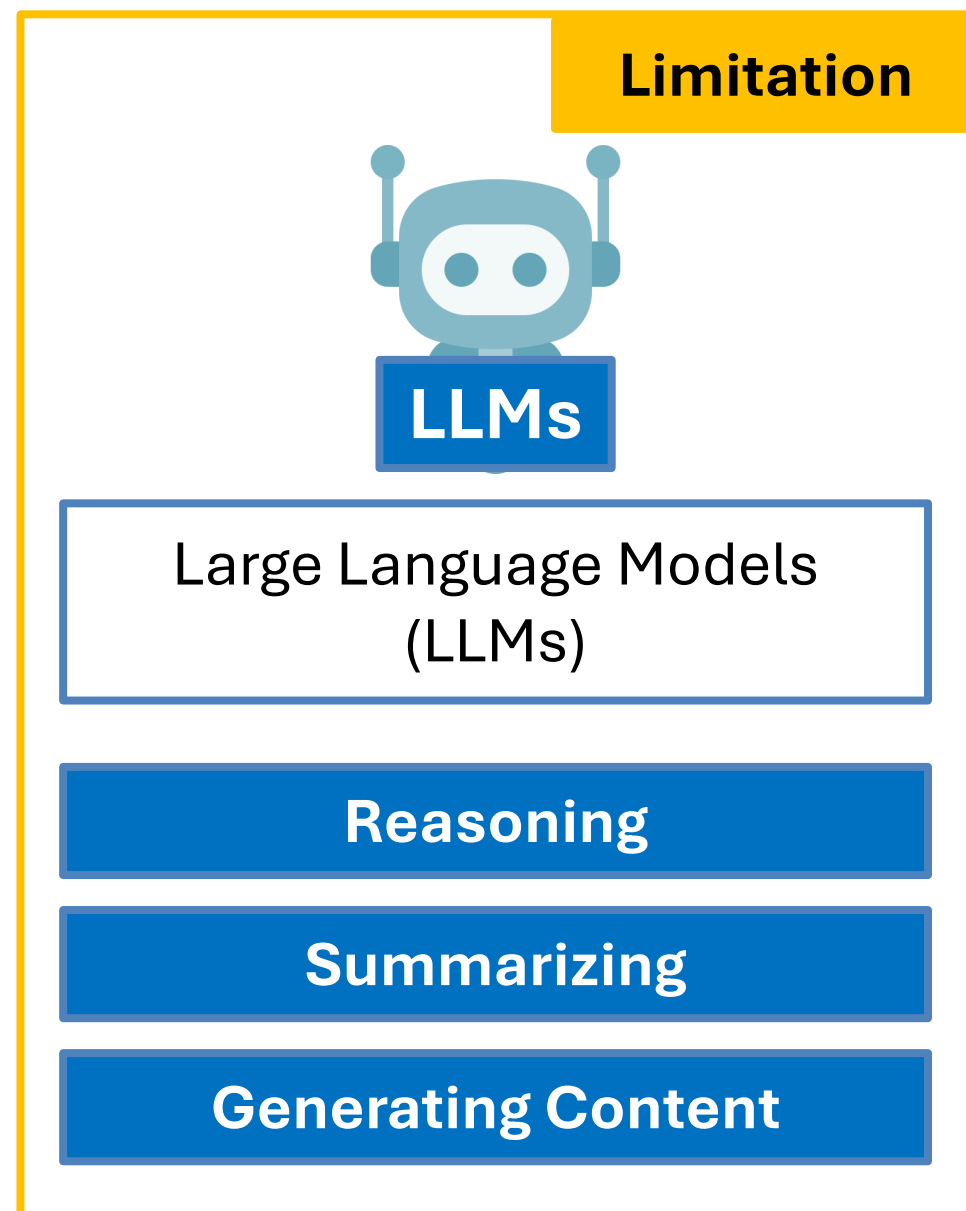
Understanding Model Context Protocol (MCP)



Understanding Model Context Protocol (MCP)

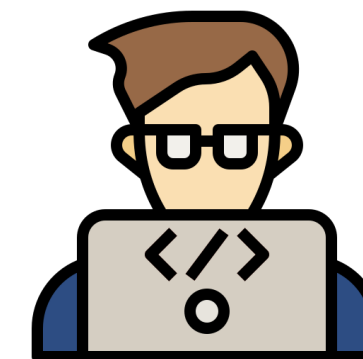


Why did we need MCP?



They only know what they were trained on

Current company data – Sales from last week



 Custom Connectors

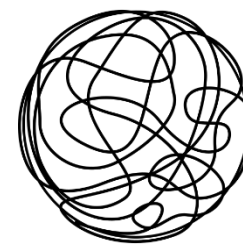
- Databases
- CRMs
- APIs

Understanding Model Context Protocol (MCP)



Problem

Each model and tool combination requires its own custom code

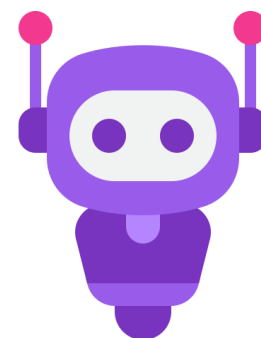


Messy

- ☐ Many integrations
- ☐ Inconsistent Standards
- ☐ Lot of Maintenance



- ☐ Common Standard
- ☐ Simple
- ☐ Secure way

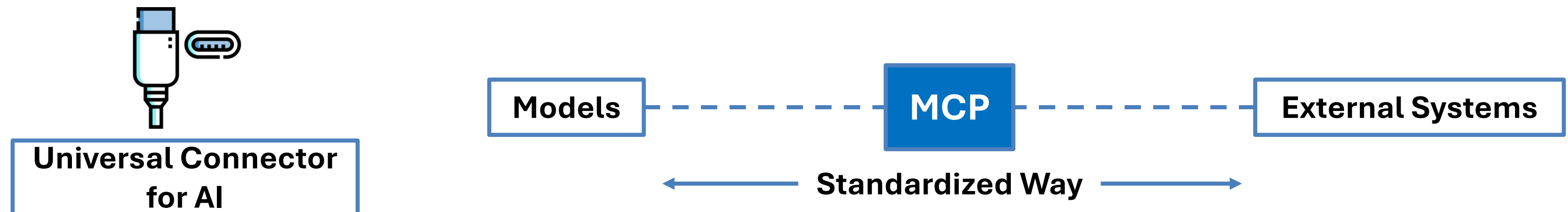


Any tool or data source without rebuilding everything from scratch

Model Context Protocol (MCP)

Understanding Model Context Protocol (MCP)

Model Context Protocol (MCP)



MCP

An open protocol that anyone can implement, defining how models communicate with tools, resources, and data sources safely and uniformly



Fetch Information



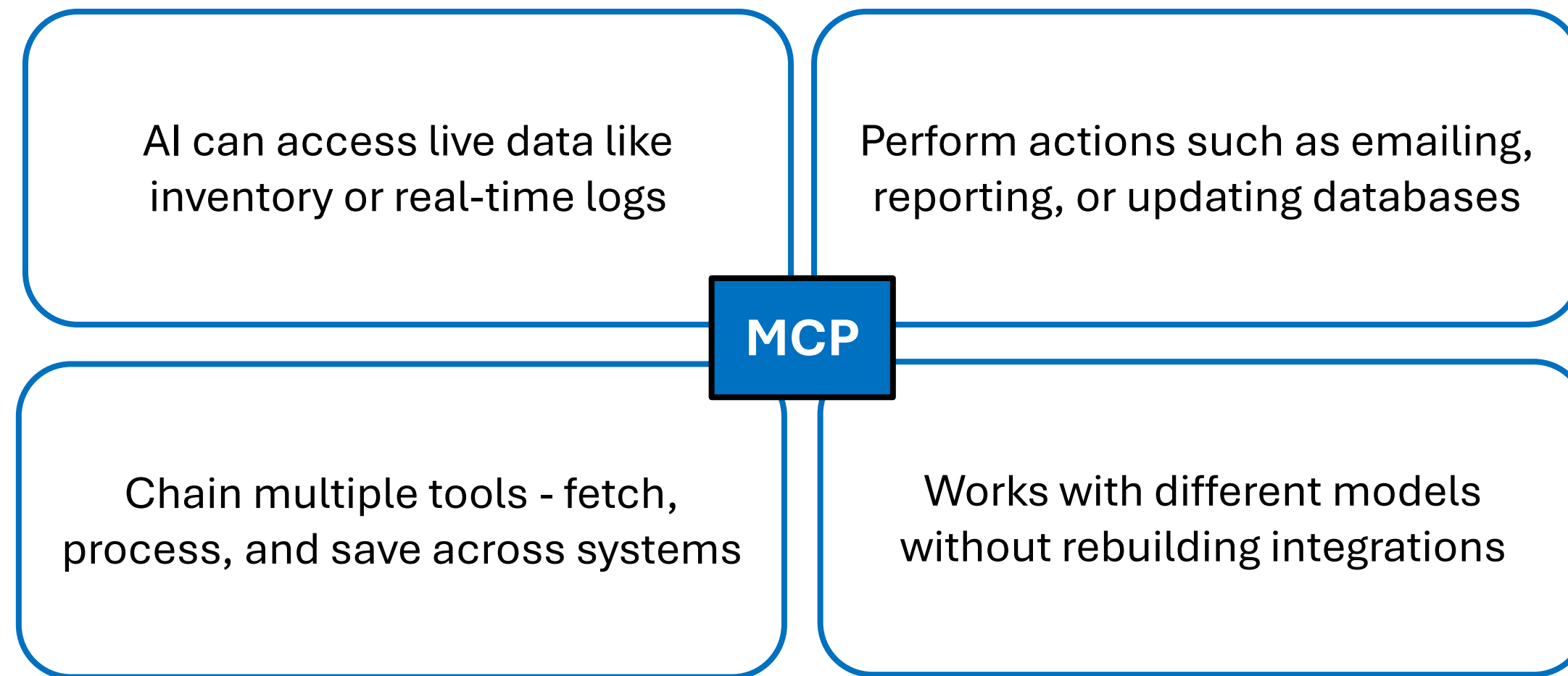
Perform Actions



Stay Up-to-date

Understanding Model Context Protocol (MCP)

What can MCP enable?



Understanding Model Context Protocol (MCP)

Benefits of MCP

Interoperability

One integration works with many models

Scalability

No redesign needed as tools and system grow

Consistency & Security

All interactions follow the same structured and safe format

Fresh & Reliable Data

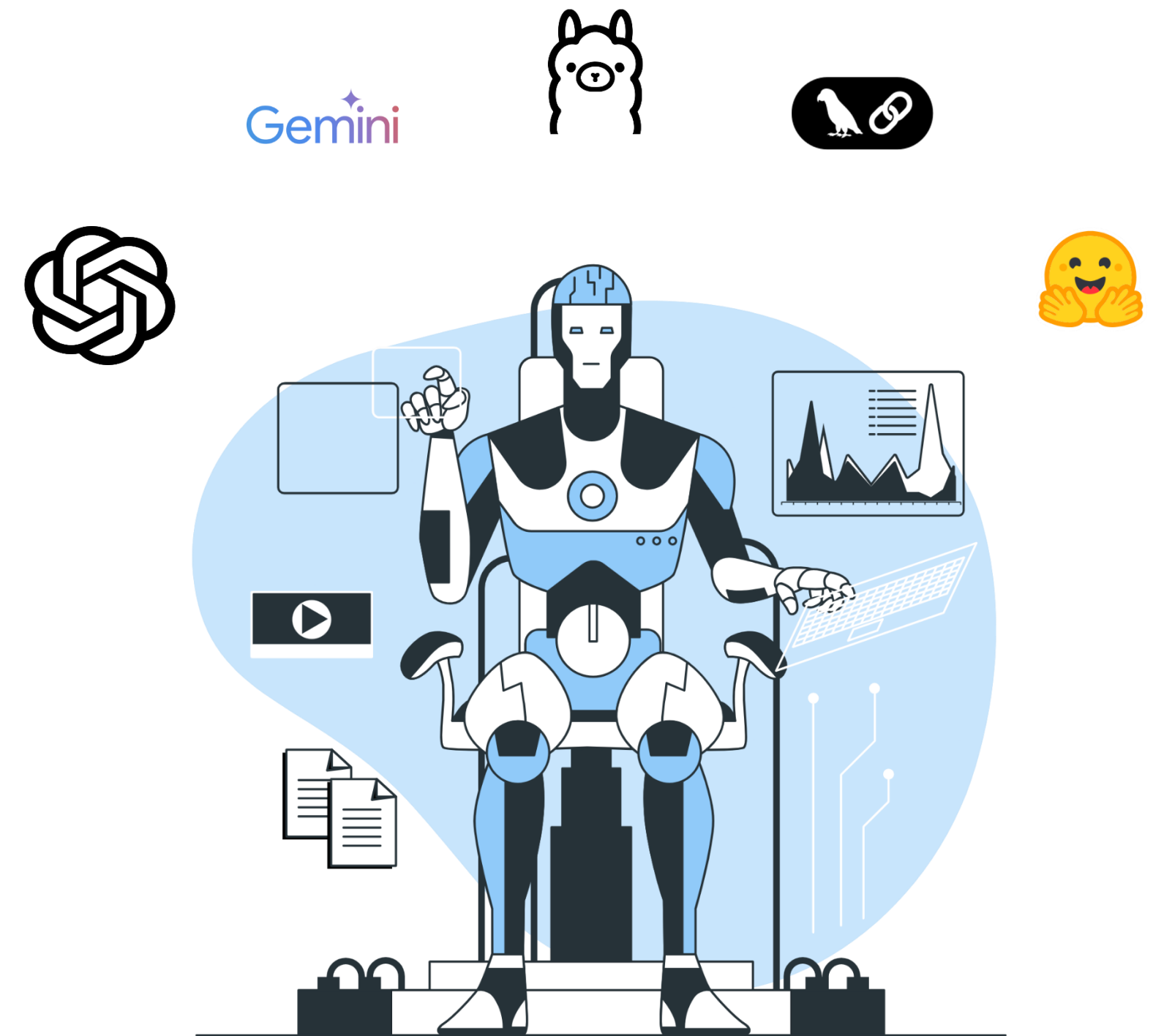
Model no longer relies on outdated training data

Ease of Development

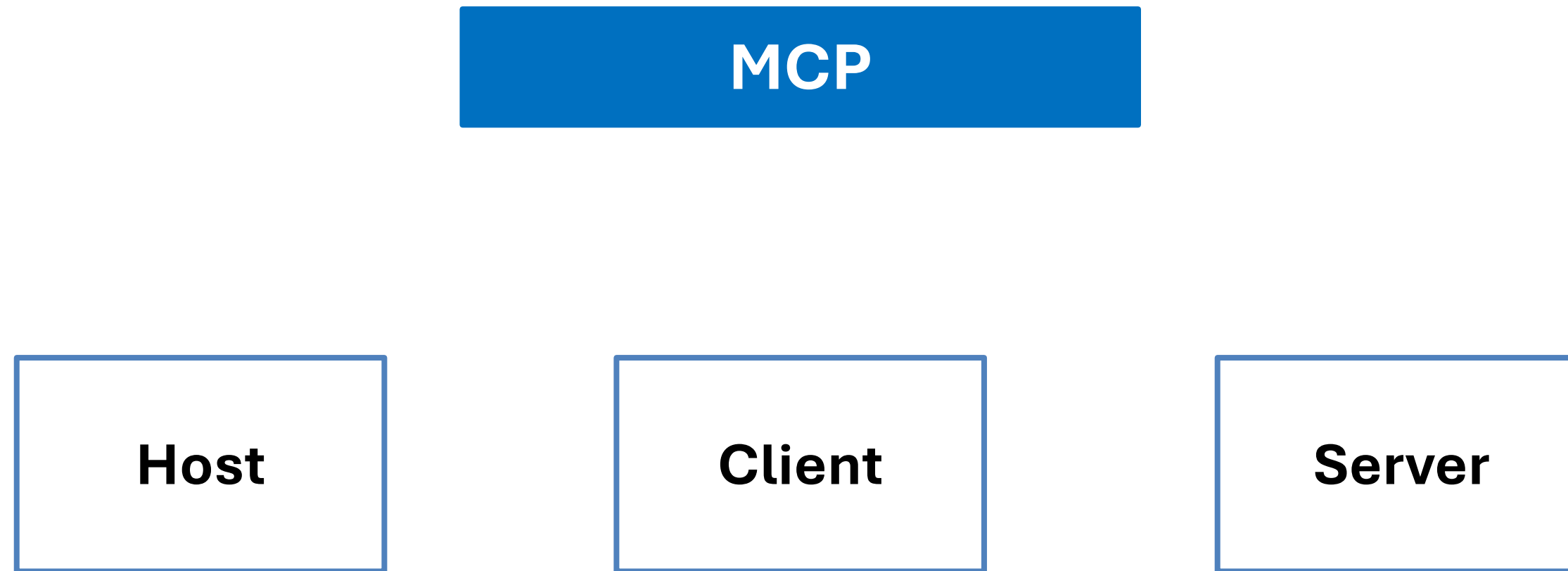
Less glue code, fewer bugs, faster time to build

Section 4.2

How does MCP work?



How does MCP work?



How does MCP work?



Host

AI Application or Agent

How does MCP work?



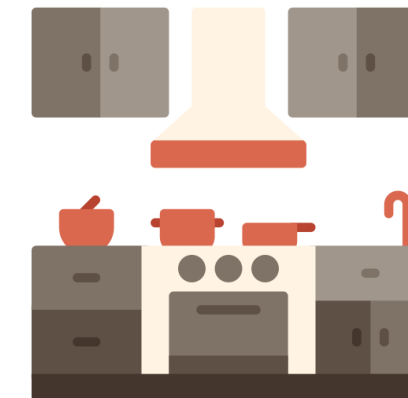
Host

AI Application or Agent



Client

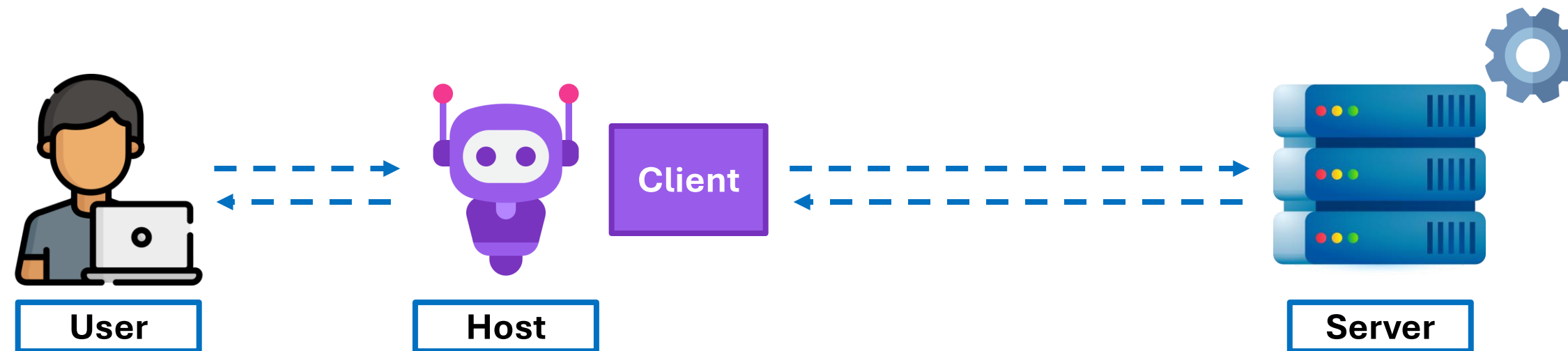
Standard Format



Server

Tools

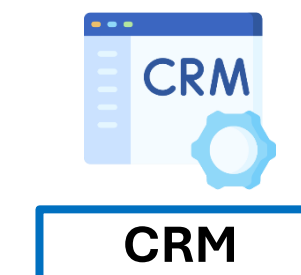
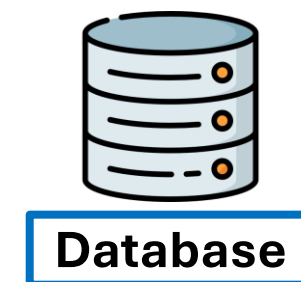
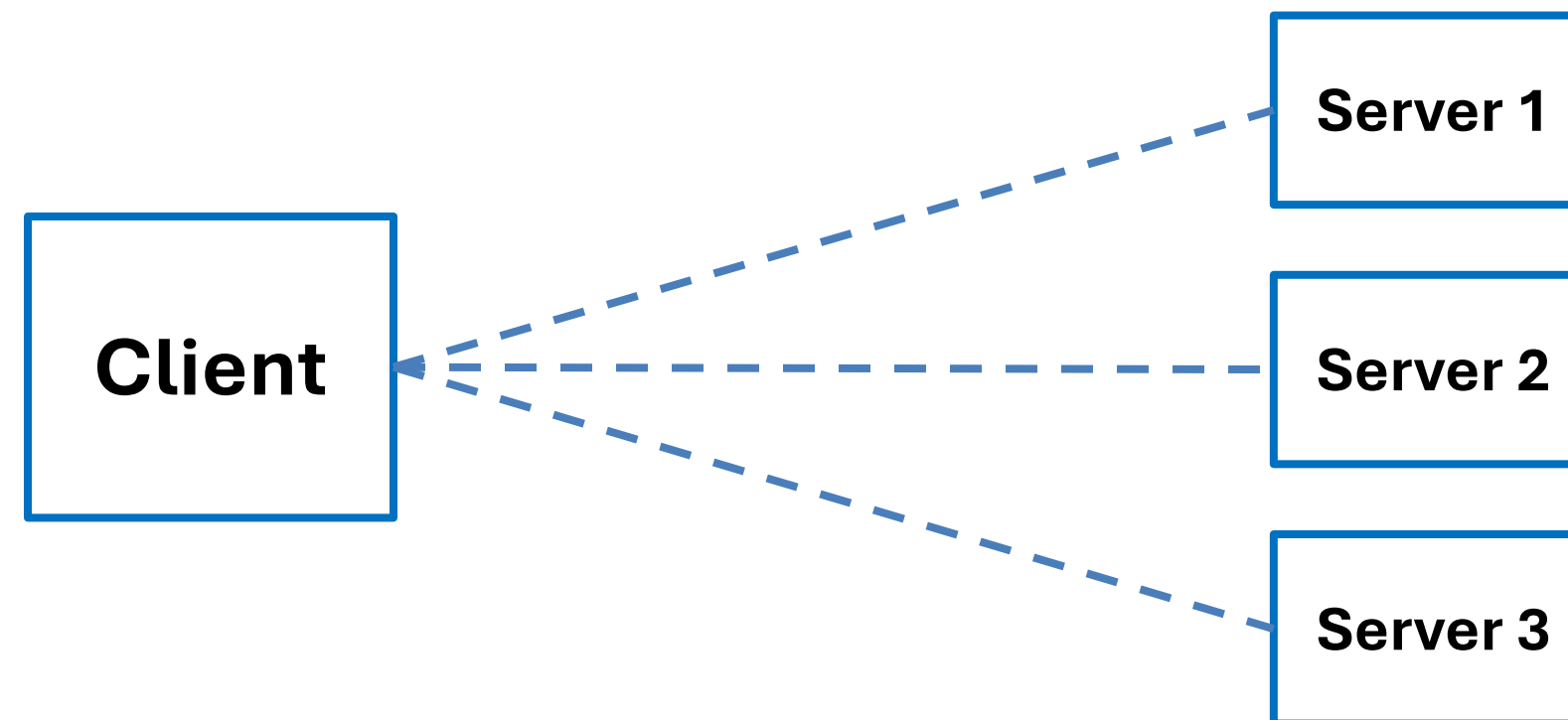
How does MCP work?



How does MCP work?

Working

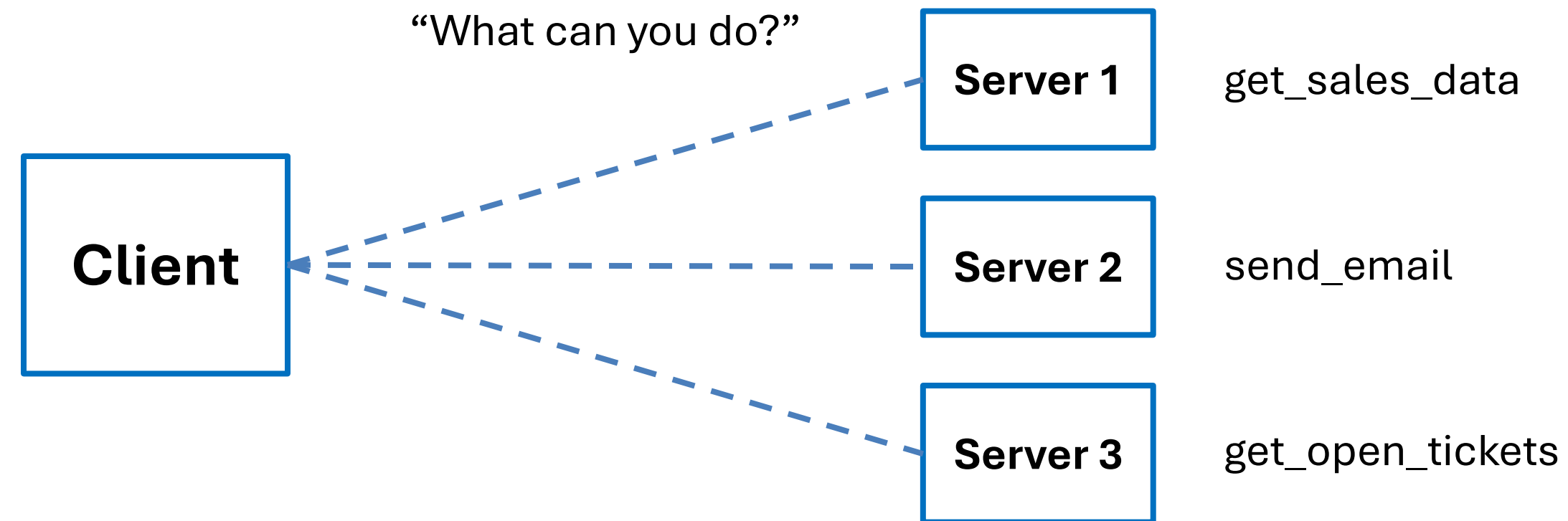
➤ Initialization



How does MCP work?

Working

- Initialization
- Discovery

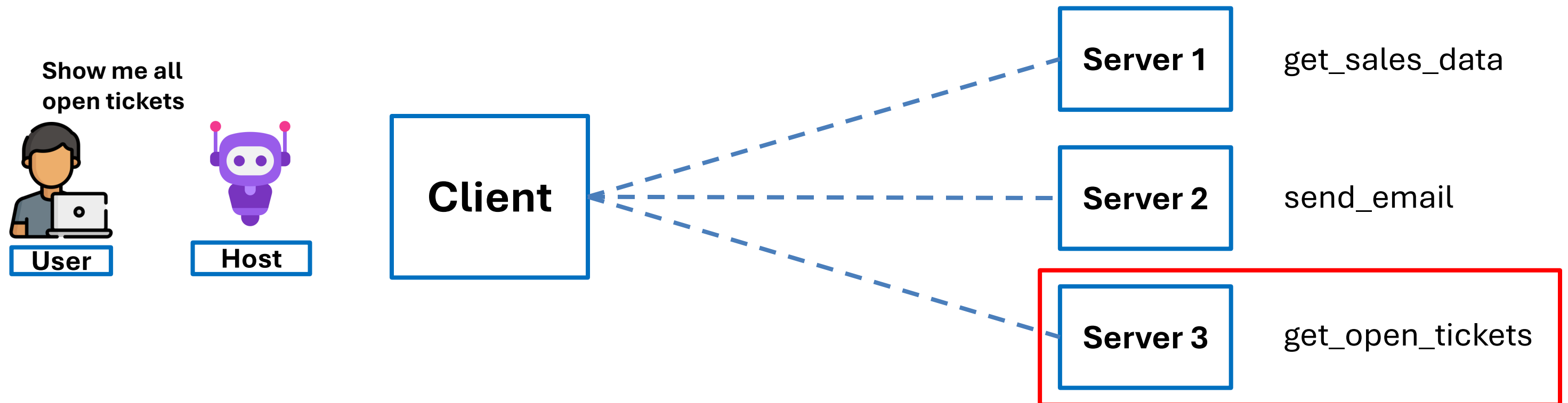


- ❑ Helps the model know what’s available before it starts thinking

How does MCP work?

Working

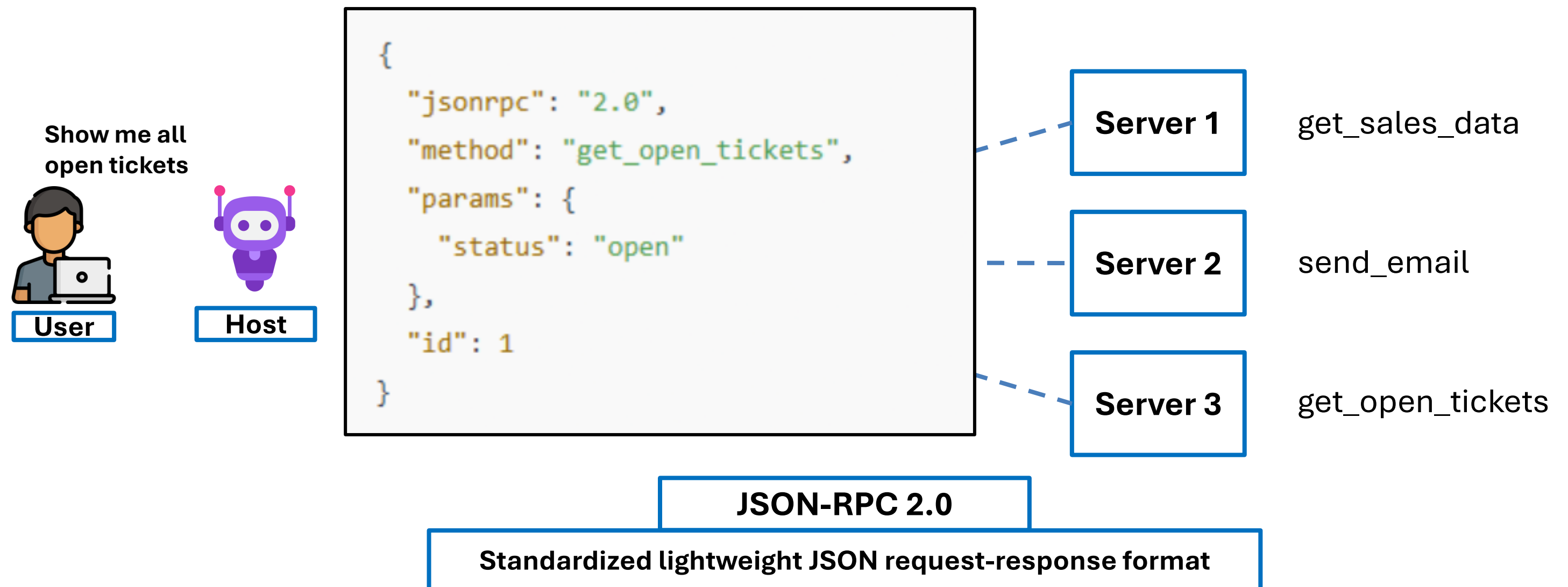
- Initialization
- Discovery
- Decision



How does MCP work?

Working

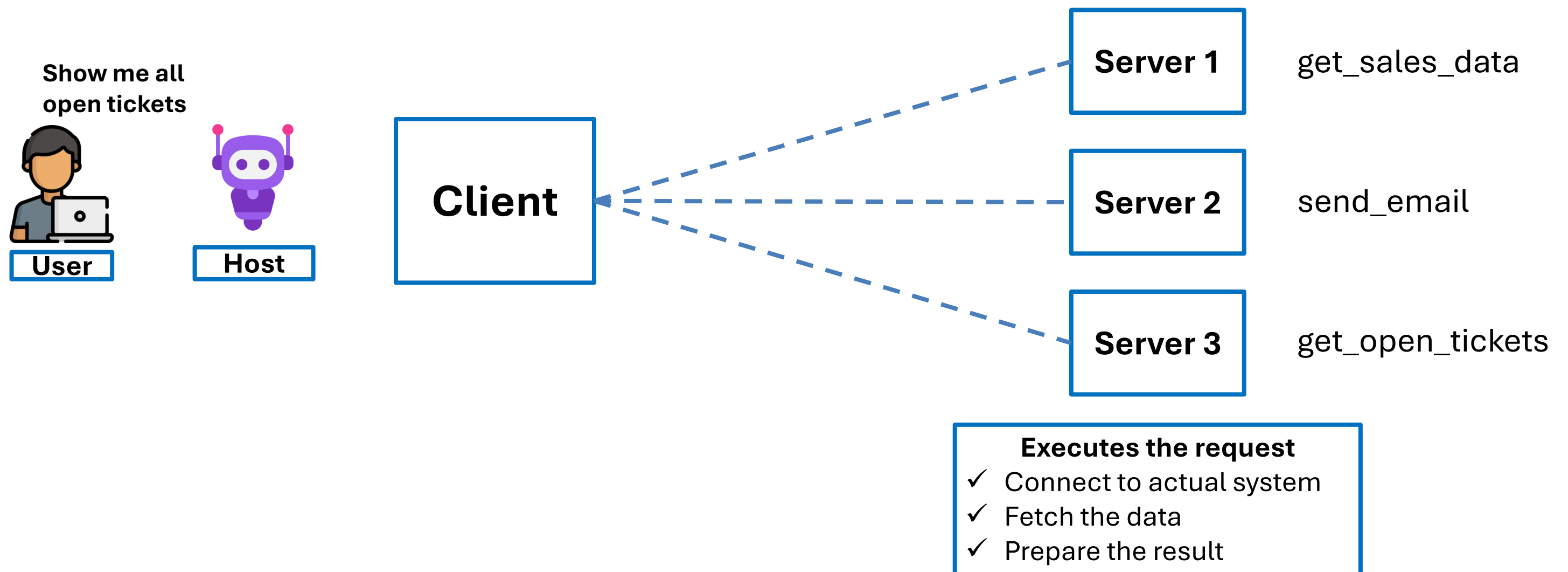
- Initialization
- Discovery
- Decision
- Invocation



How does MCP work?

Working

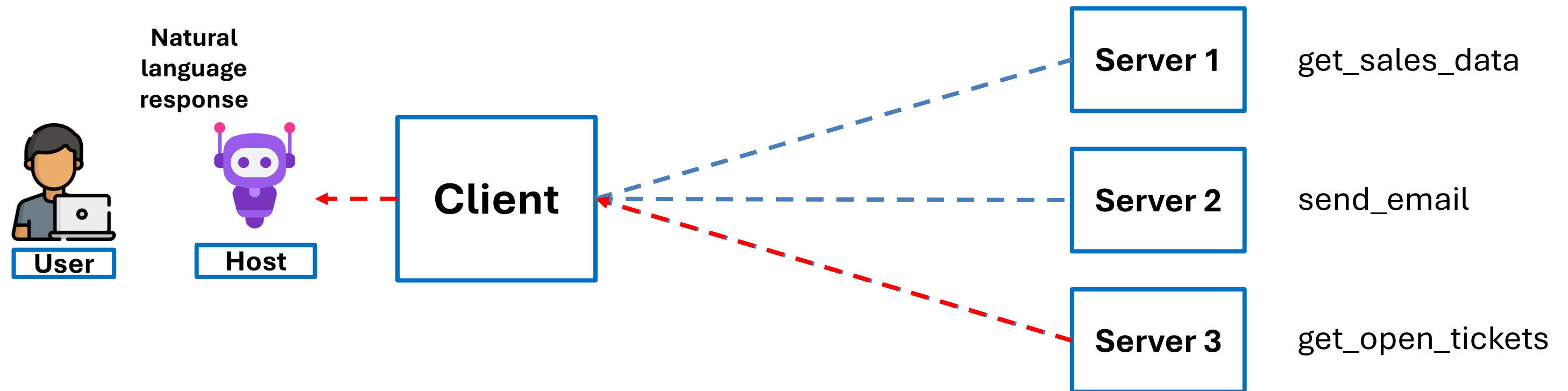
- Initialization
- Discovery
- Decision
- Invocation
- Execution



How does MCP work?

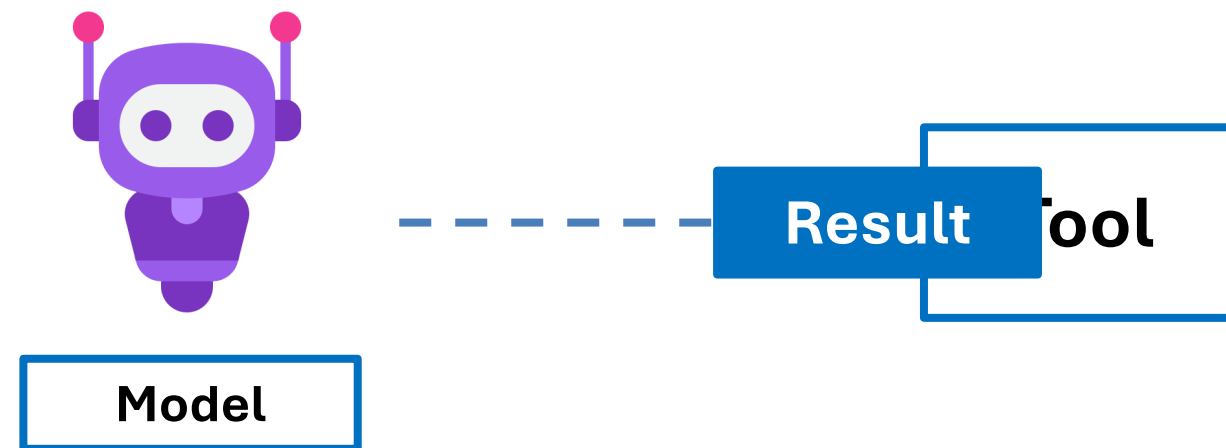
Working

- Initialization
- Discovery
- Decision
- Invocation
- Execution
- Response



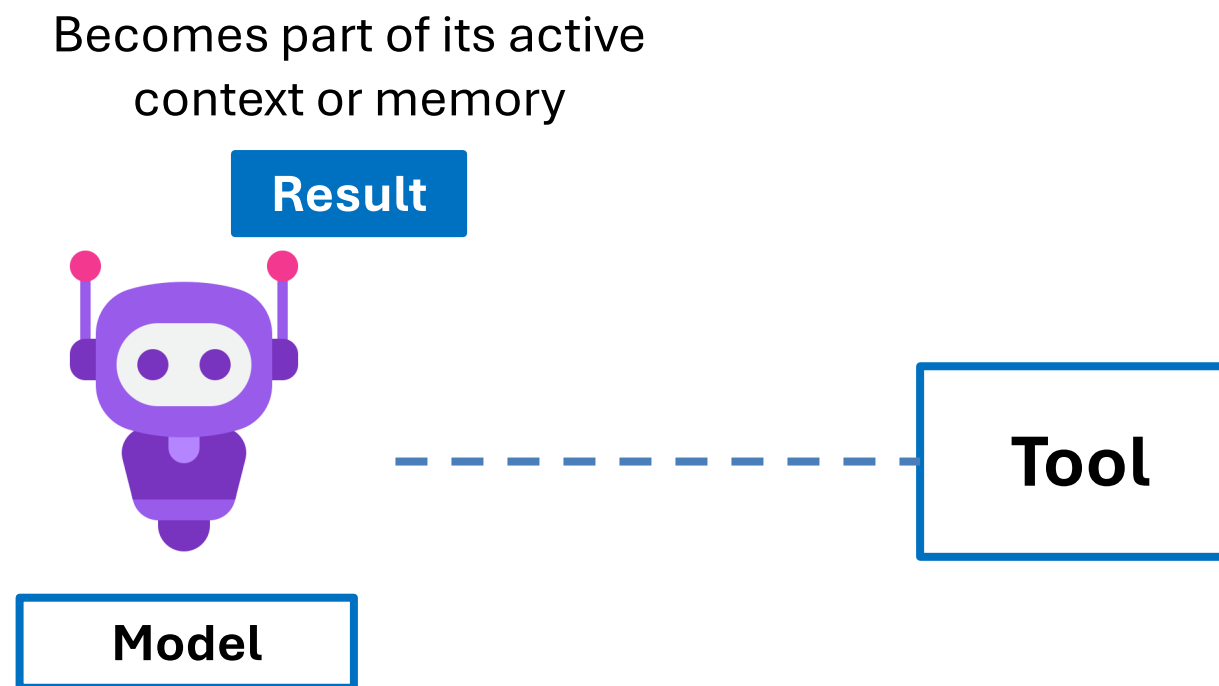
How does MCP work?

How Context flows



How does MCP work?

How Context flows

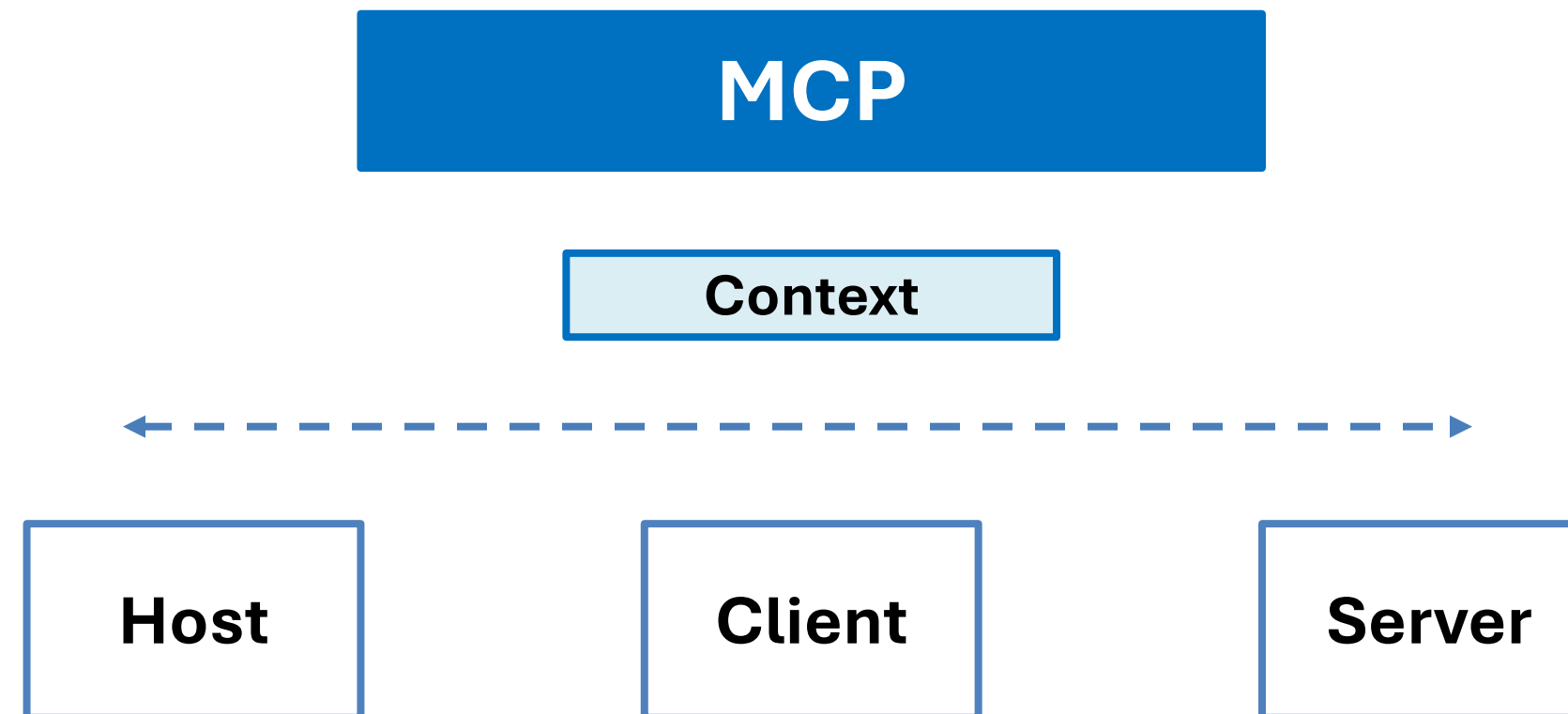


For example:

- ❑ First, the model might call a tool to fetch data
- ❑ Then, using that same data, it could call another tool to generate a chart
- ❑ Finally, it may send the chart by email using another tool

How does MCP work?

How Context flows



Maintains context across multiple tasks

How does MCP work?

Summary

Well-defined communication bridge between models and tools

- ☐ Client and server handle the talking
- ☐ Host manage the workflow
- ☐ Ensures everything stays consistent and secure

Section 4.3

Demonstration: Build an MCP server



Section 4.4

Demonstration: Building Agent on top of MCP server



Section 4.5

Demonstration: Docker MCP with Agentic AI



Section 4.6

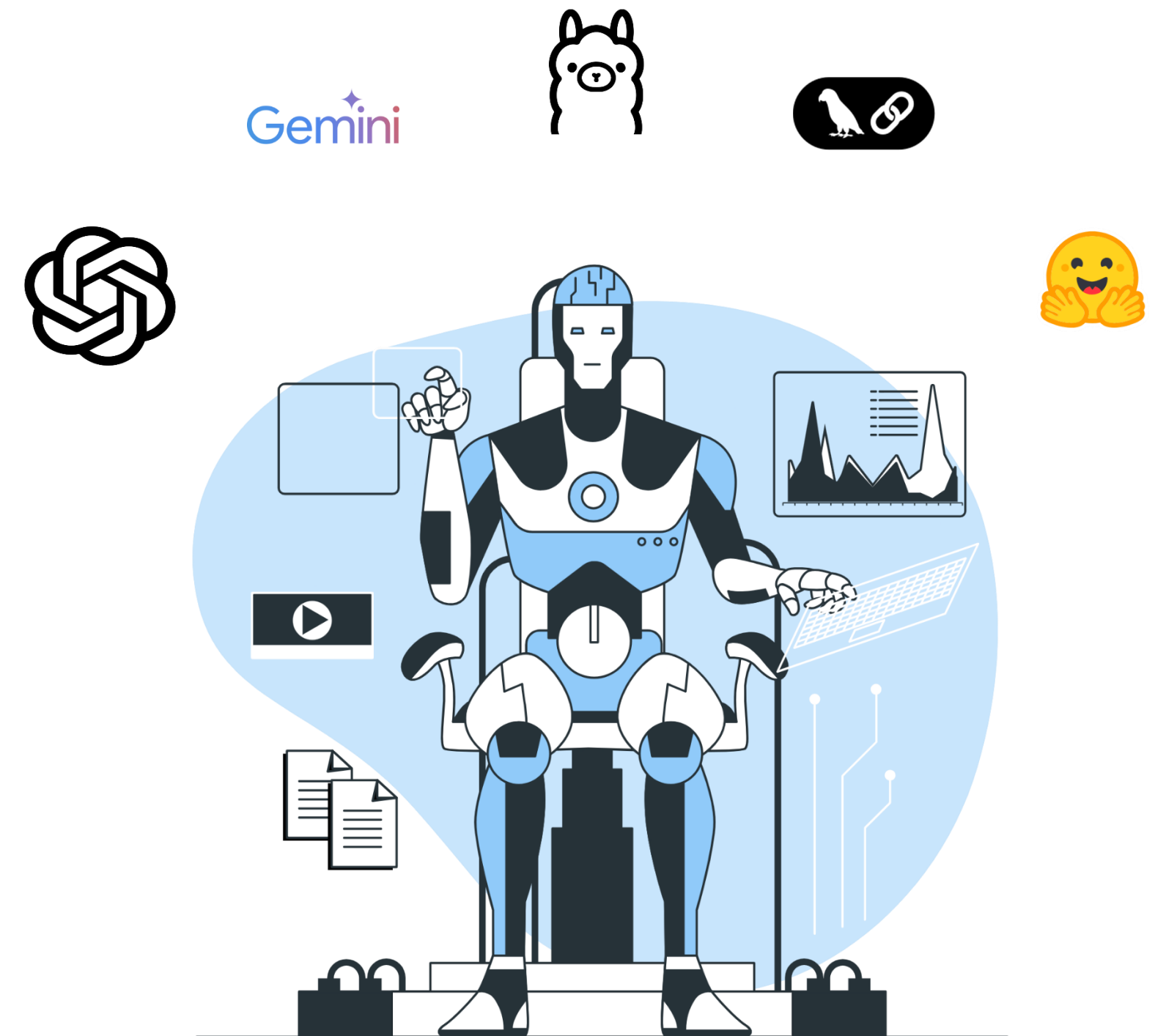
Summary

- ✓ Understood Model Context Protocol (MCP) and its working
- ✓ Hands-on demonstration:
 - ✓ Built a complete MCP server
 - ✓ Created agent over MCP server
 - ✓ Set up Docker MCP

Thank You

Section 5.0

Introduction to Multi-Agent Systems



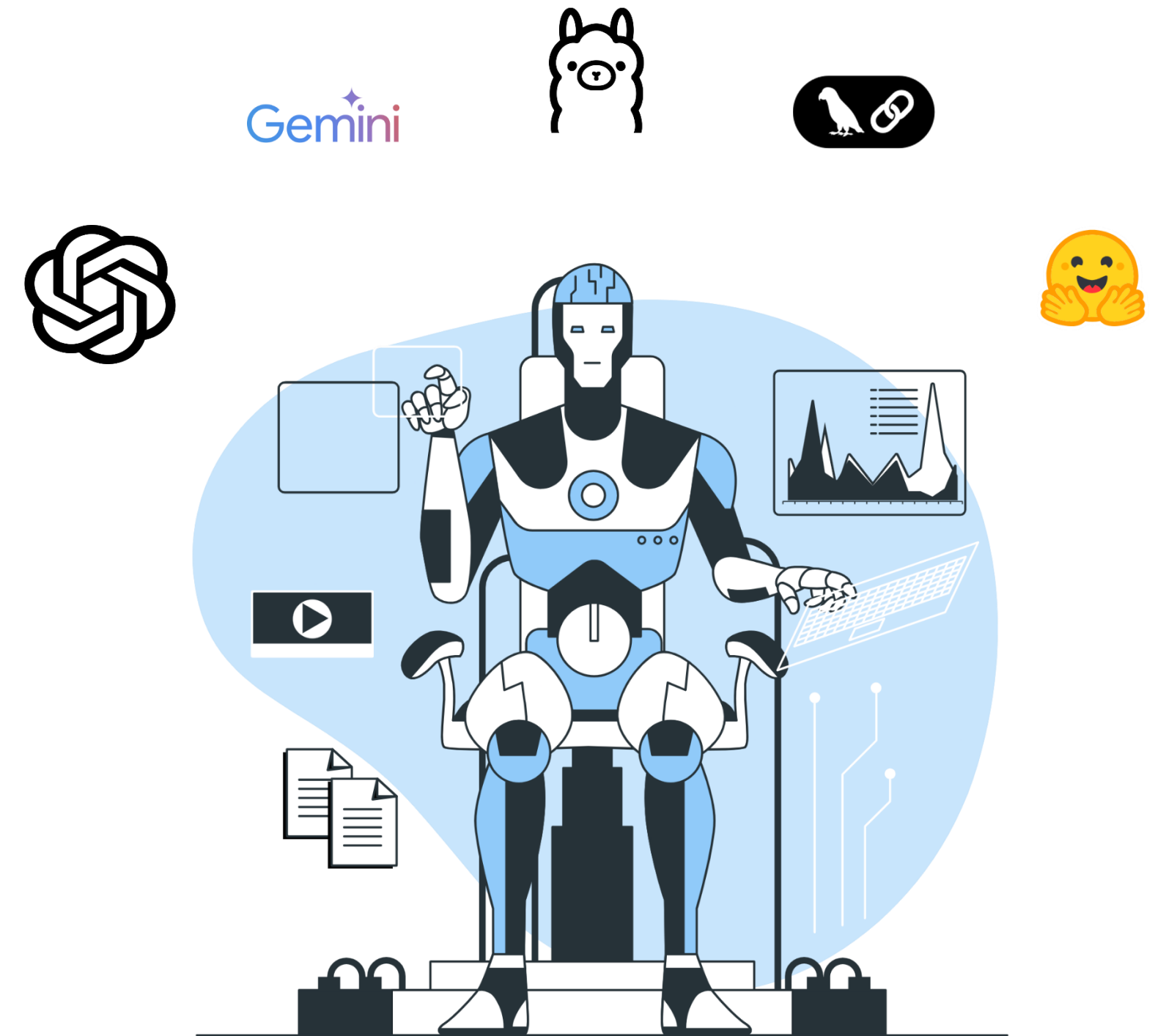
Introduction to Multi-Agent Systems

Section Overview

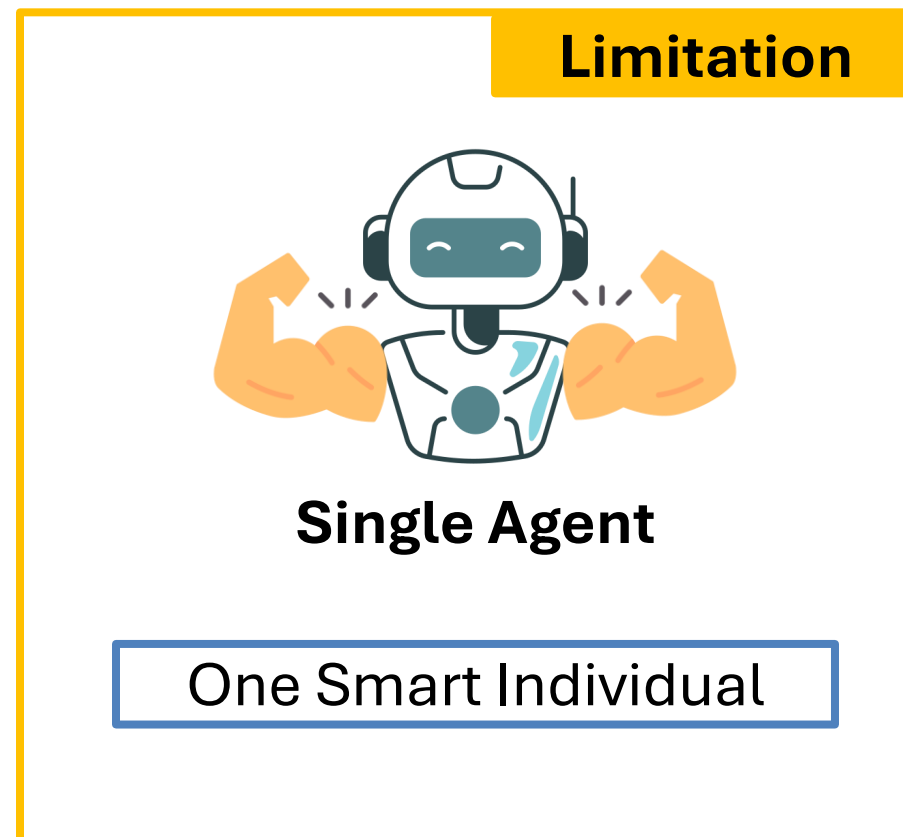
- ❑ Understand Multi-Agent System and its Architecture
- ❑ Popular Multi-Agent frameworks
- ❑ Hands-on demonstrations:
 - ❑ CrewAI installation
 - ❑ Basic & advanced workflows using CrewAI
 - ❑ No-Code Multi-Agent system using n8n
 - ❑ Intelligent AWS monitoring
 - ❑ Cross-cloud Agent-to-Agent coordination

Section 5.1

Multi-Agent System (MAS)



Multi-Agent System (MAS)



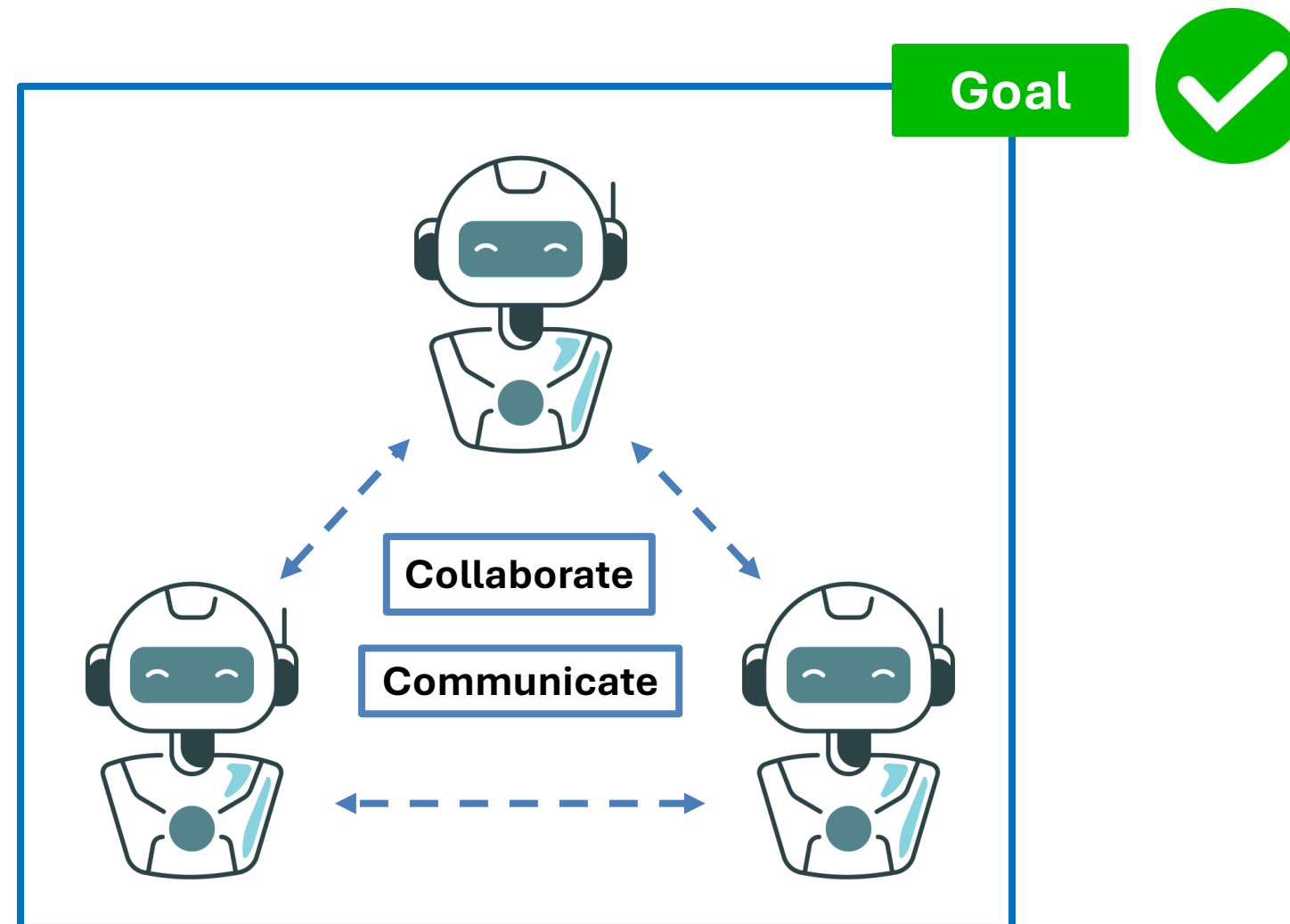
Bigger

Broader

Multiple Skills

Multi-Agent System (MAS)

Multi Agent Systems (MAS)



Multi-Agent System (MAS)

Multi Agent Systems (MAS)



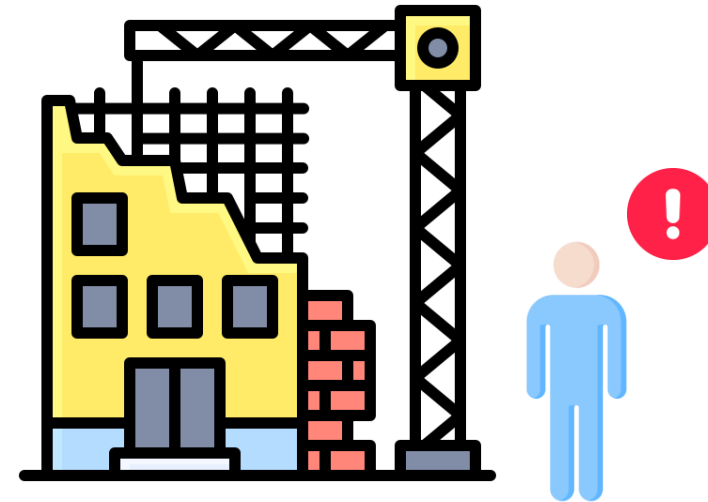
Specific Role

Specialized Capability

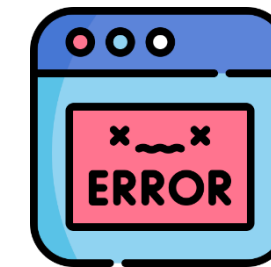
Clear Responsibility

Multi-Agent System (MAS)

- Design the house
- Get Approvals
- Manage Materials
- Wirings
- Handle Plumbing
- Paint the Interior
- Inspect everything



Very Slow

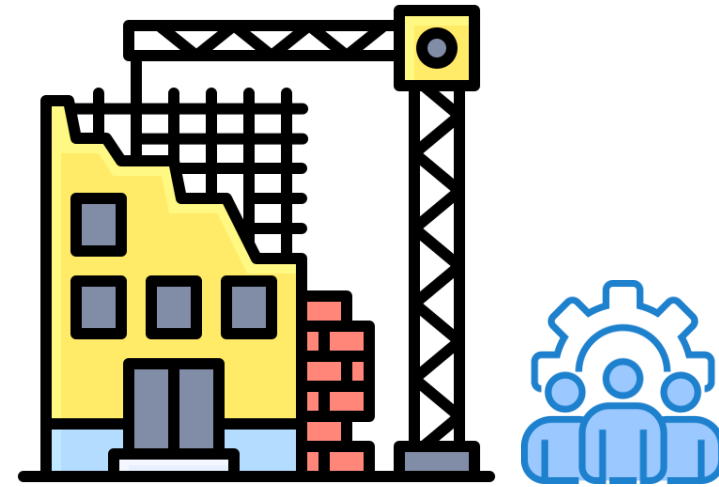


Error Prone



Exhausting

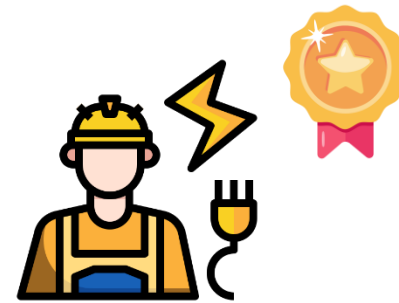
Multi-Agent System (MAS)



- ✓ Faster
- ✓ Better
- ✓ Highly reliable



Design the plan



Handle wirings



Fix the pipes



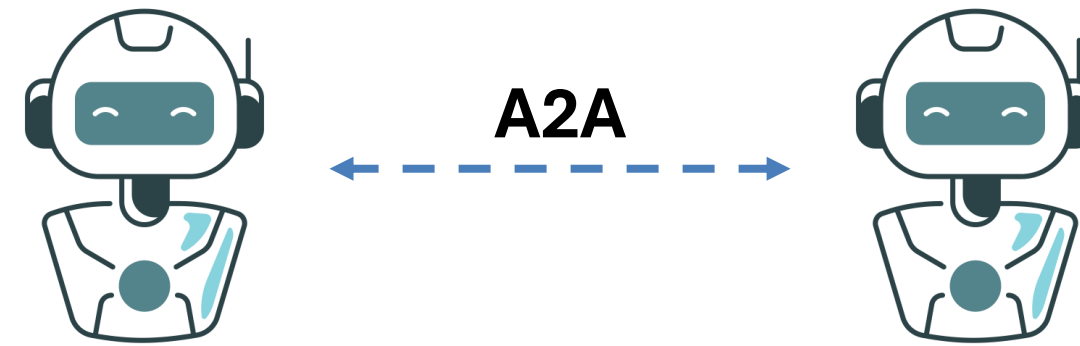
Handle finishing



Final checks

Communicate

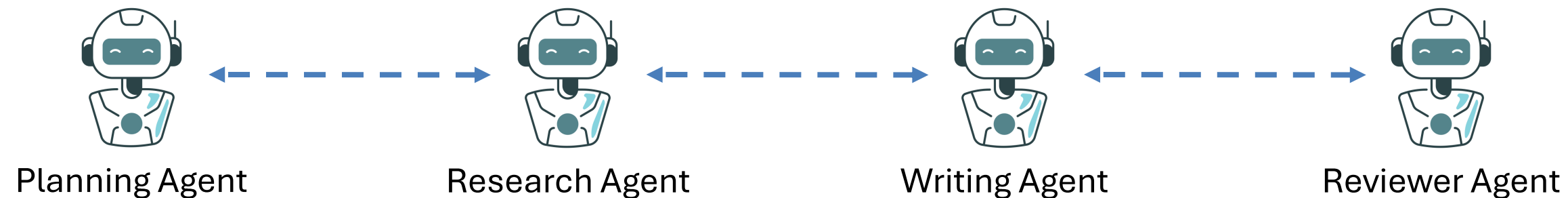
Multi-Agent System (MAS)



**Agent-to-Agent
Communication**

A2A

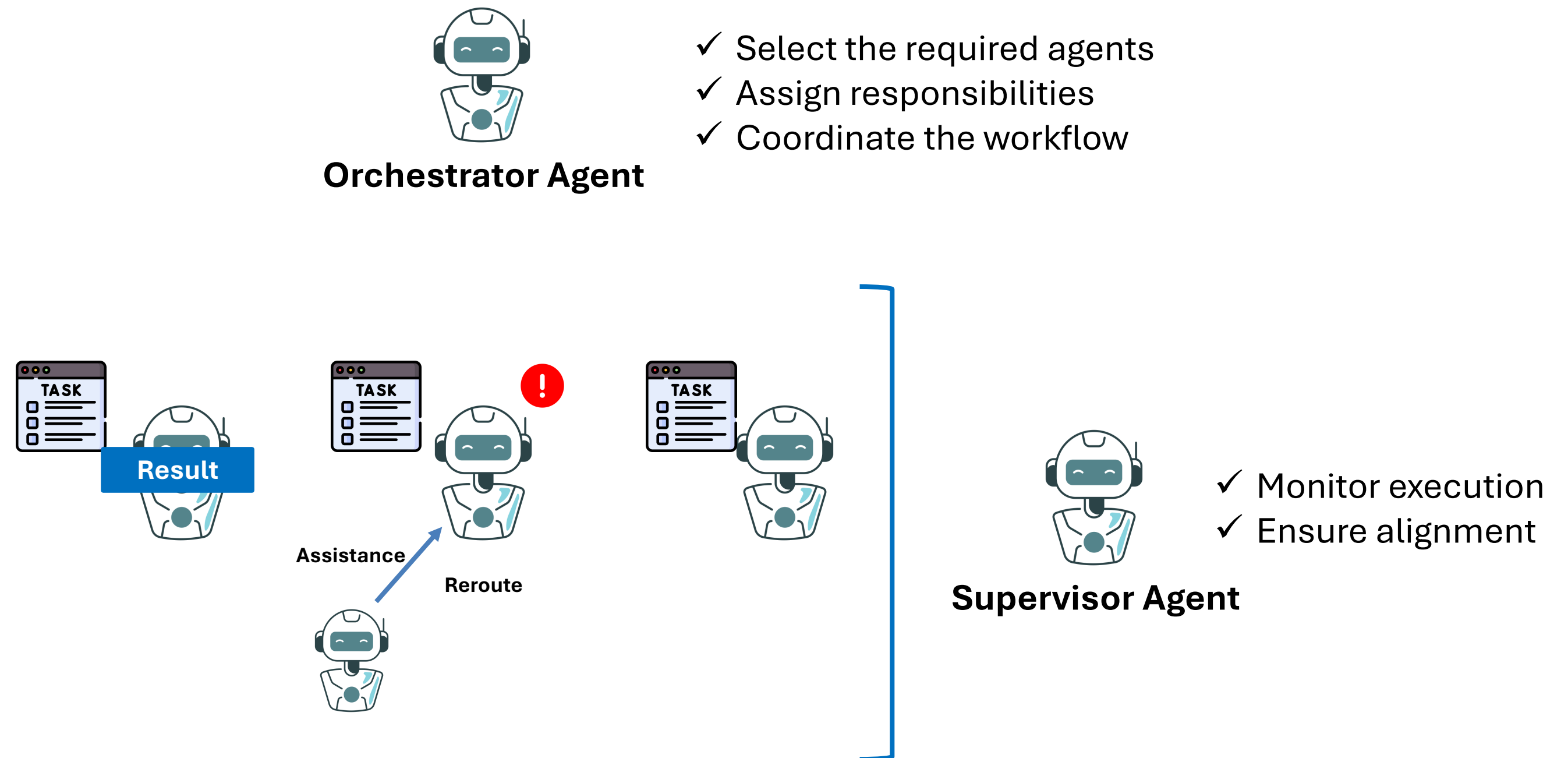
**Method by which agents talk to each other, share thoughts,
exchange intermediate results, or hand off subtasks**



Structured Format

Multi-Agent System (MAS)

Centralized Orchestration Approach



Multi-Agent System (MAS)

Benefits of MAS

Enhanced Efficiency & Productivity

- ❑ Each agent focus on its area of expertise
- ❑ Reduces errors & maximize productivity
- ❑ Fast and reliable complex operations completion

Superior Flexibility & Scalability

- ❑ Agents can be added, modified, or removed easily
- ❑ System capabilities improve without replacements or downtime

Deep Specialized Expertise

- ❑ Each agent has deep domain expertise
- ❑ Specialization handles multi-faceted problems better

Built-in Resilience & Fault Tolerance

- ❑ Other agents continue working seamlessly despite failure of one agent
- ❑ Ensures smooth & uninterrupted operations

Improved Interpretability & Management

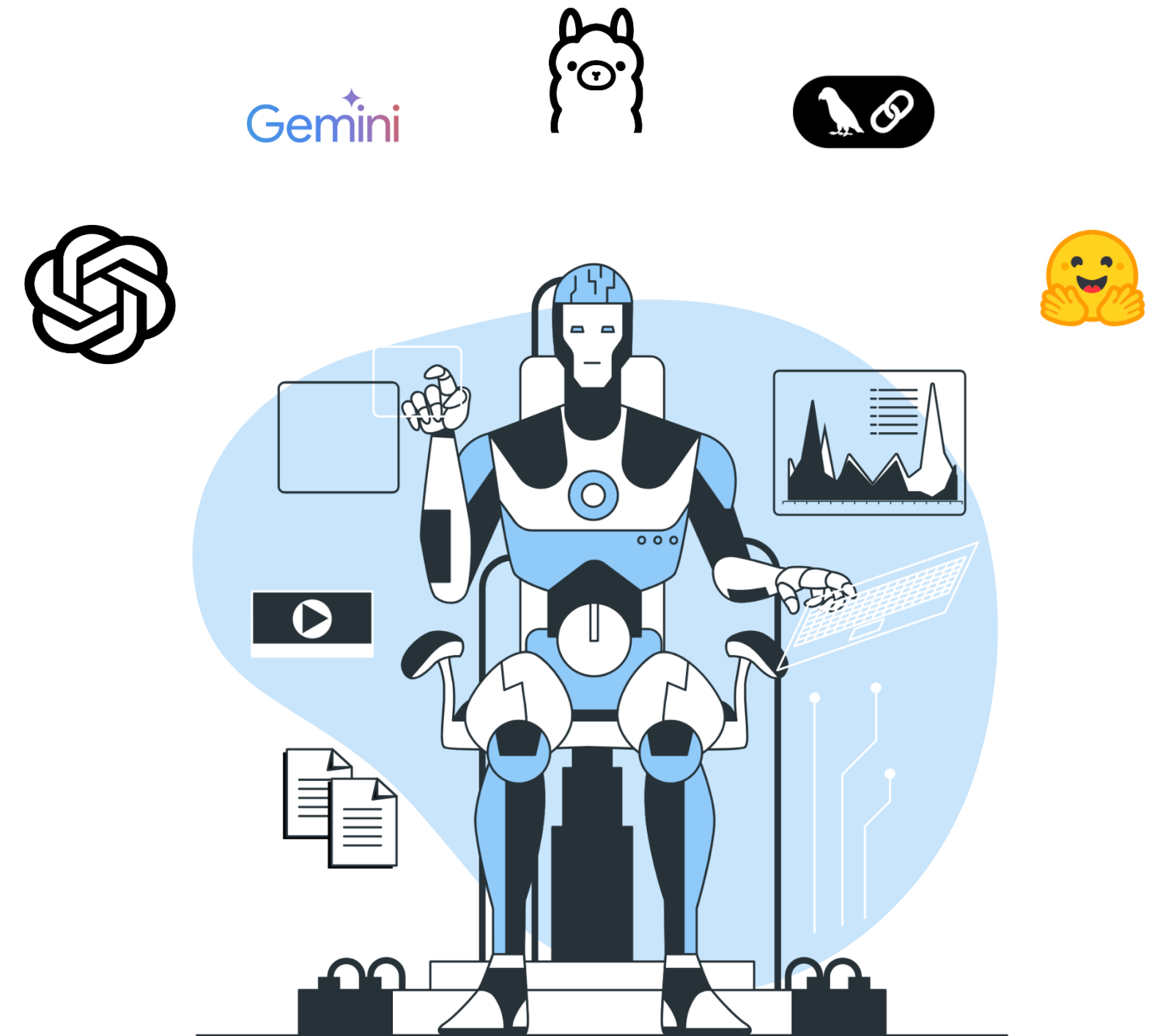
- ❑ Each agent has a clearly defined role
- ❑ Issues are easier to identify and fix without affection the whole system

Automated Workflows

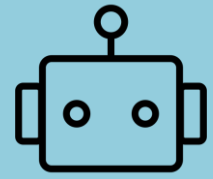
- ❑ Agents coordinate actions in a synchronized manner
- ❑ Consistent execution with fewer errors

Section 5.2

Multi-Agent System Architecture



Multi-Agent System Architecture



Agents

- ✓ Individual units each having their own abilities, knowledge, and goals



Environment

- ✓ Shared physical or virtual space where agents operate
- ✓ Define constraints and influences agent perception & actions



Interactions

- ✓ Define how agents interact with each other & the environment
- ✓ Determine task distribution, influence, and collective behavior



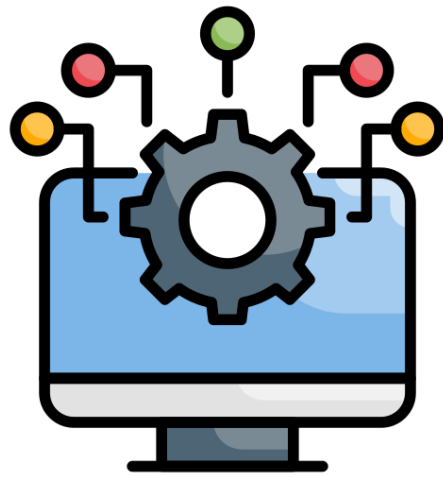
Communication

- ✓ Protocols or channels for agents to exchange information, negotiate, coordinate actions, share plans or resources

Multi-Agent System Architecture

Architecture

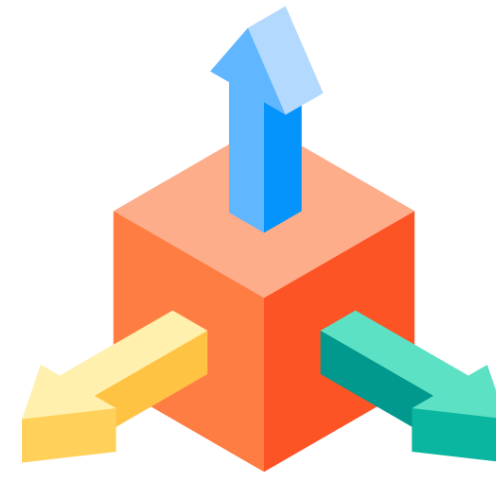
Define how agents are structured internally and how they make decisions



System Behaviour



Efficiency



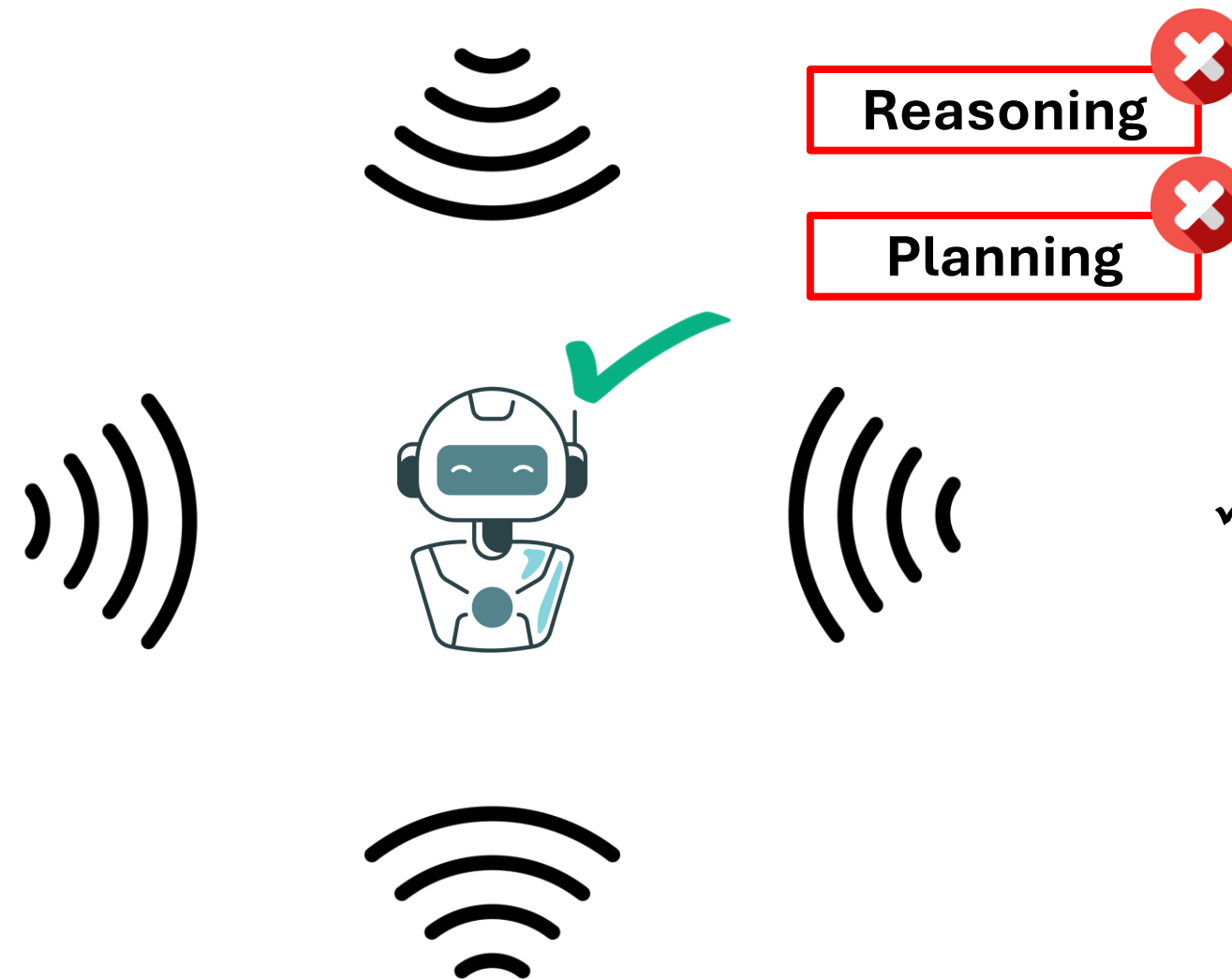
Scalability



Coordination

Multi-Agent System Architecture

Reactive Architecture



✓ Example - Obstacle-avoiding robot

Responses are instantaneous, based on simple perception-action rules

Multi-Agent System Architecture

Deliberative Architecture

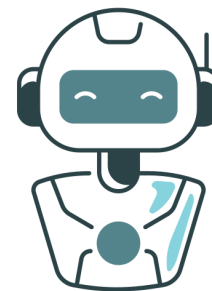
Cognitive

Maintain internal models

Plan ahead

Reason goal

Choose action

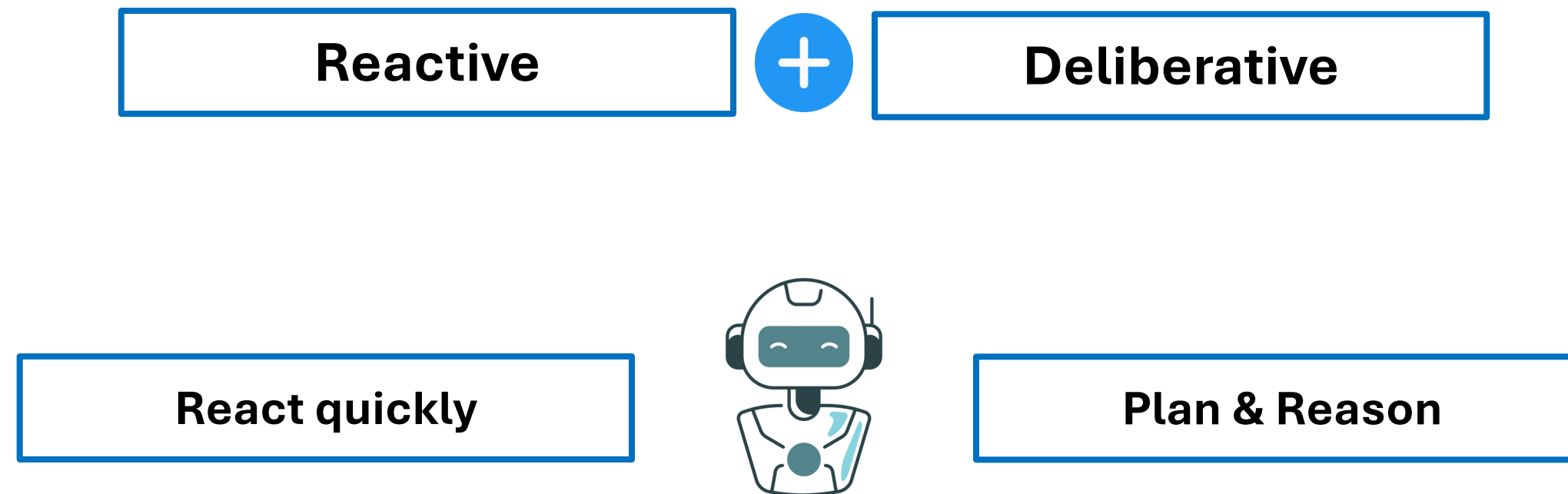


✓ Example - Intelligent personal assistants and planning agents

Allow to handle complex decision making, long-term goals, and dynamic environments

Multi-Agent System Architecture

Hybrid Architecture



✓ Example - Autonomous vehicles

Multi-Agent System Architecture

Selecting the Right Architecture

Depends on problem context:

- ☐ How dynamic the environment is?
- ☐ How much foresight is needed?
- ☐ How much autonomy and flexibility is required?
- ☐ How critical real-time responses are ?

Multi-Agent System Architecture

Types of MAS (based on Interaction Goals)

Cooperative MAS

- ☐ Agents work together toward a common goal
- ☐ Share information, resources, divide tasks, & work together
- ☐ Example: Coordinated search-and-rescue drones

Competitive MAS

- ☐ Agents have conflicting goals or limited resources
- ☐ Compete to achieve the best possible outcomes
- ☐ Common in gaming, auctions, and resource allocation scenarios

Hierarchical MAS

- ☐ Agents are organized into roles or levels
- ☐ Higher-level agents manage or coordinate others
- ☐ Useful for structured systems needing oversight and control

Heterogeneous MAS

- ☐ Agents have different capabilities or specializations
- ☐ Diverse agent roles handle data, analysis, and actions
- ☐ Improves flexibility and adaptability for complex tasks

Multi-Agent System Architecture

Structures

Refers to how agents are organized & how they communicate or delegate work

Multi-Agent System Architecture

Flat Structure

- ✓ All agents have equal status (no hierarchy)
- ✓ Agents interact directly with each other
- ✓ Simple, flexible, and decentralized

Hierarchical Structure

- ✓ Agents are organized into hierarchical layers
- ✓ Higher-level agents supervise and coordinate
- ✓ Lower-level agents handle execution or specialized tasks
- ✓ Useful when clear control is needed

Multi-Agent System Architecture

Holonic Structure

- ✓ Agents are grouped into autonomous holons
- ✓ Each holon acts as an agent at a higher level
- ✓ Supports modularity and scalability

Organizational / Network Structure

- ✓ Agents form networks or task-based groups
- ✓ Teams are dynamic with no fixed hierarchy
- ✓ Suitable for dynamic environments

Multi-Agent System Architecture

Use Cases

- ✓ Problems are too complex or dynamic for a single agent

Distributed Control

Swarm Robotics

**Distributed Resource
Management**

- ✓ Tasks need specialization, parallelism, or multiple perspectives

Teams of Agents - Each responsible for distinct subtasks

- ✓ Environments are dynamic, uncertain, or decentralized

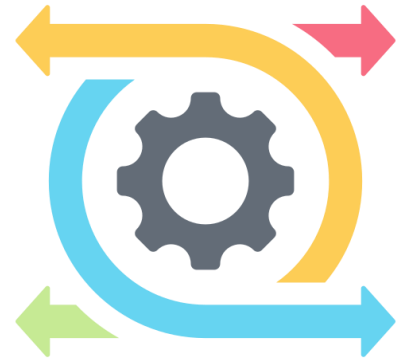
Traffic Management

Smart Grids

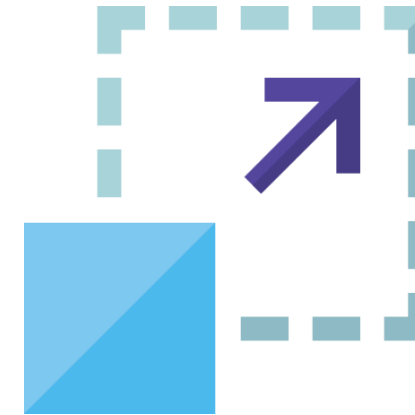
**Multi-robot
Coordination**

Distributed Sensors

Multi-Agent System Architecture



Flexibility

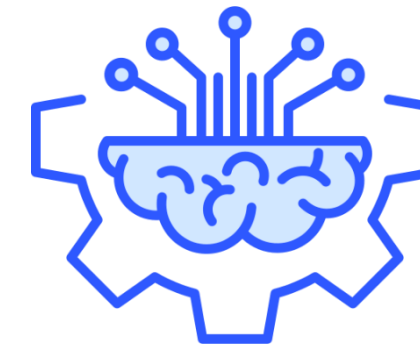


Scalability

Multi-Agent Systems



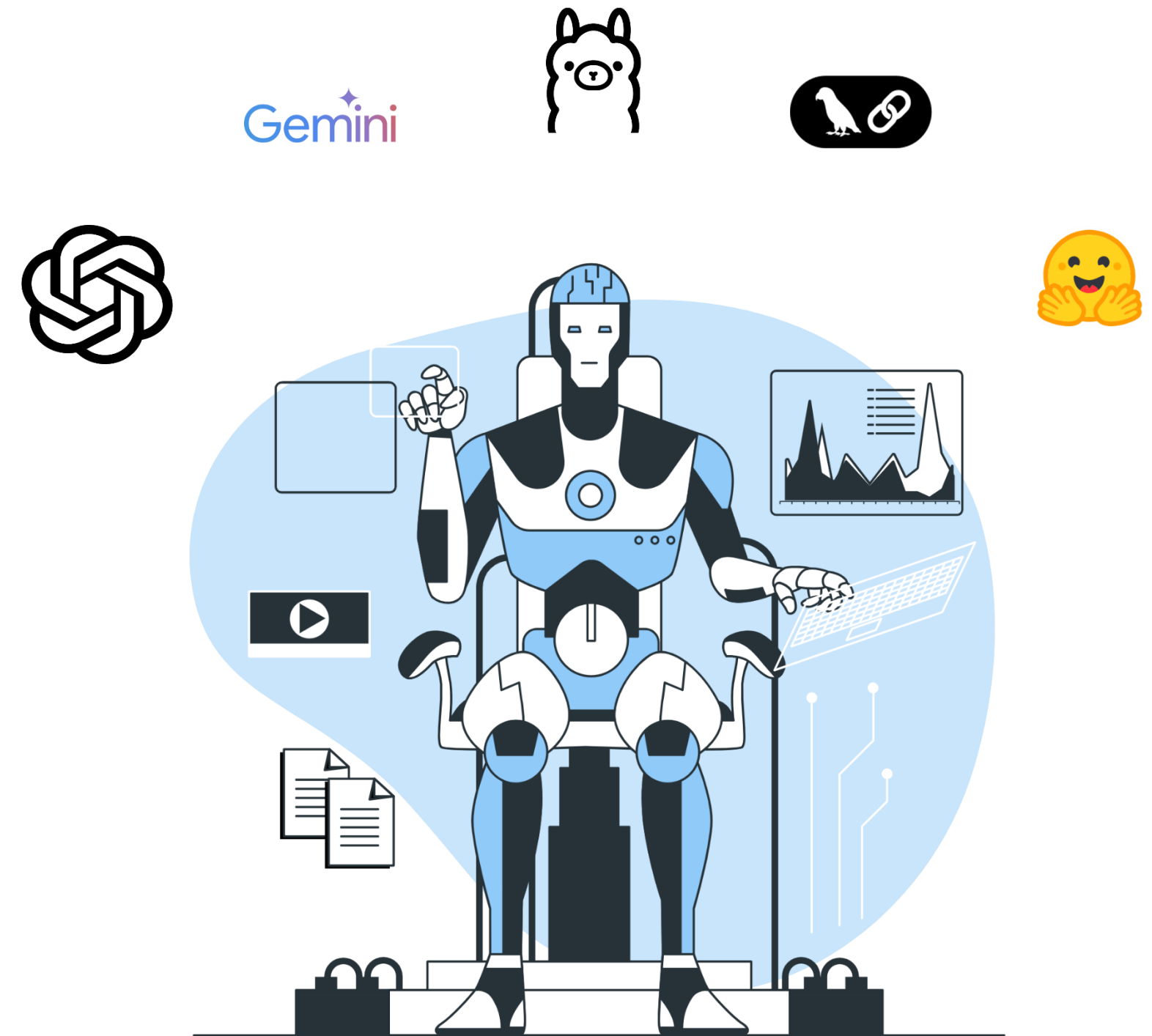
Resilience



Intelligence

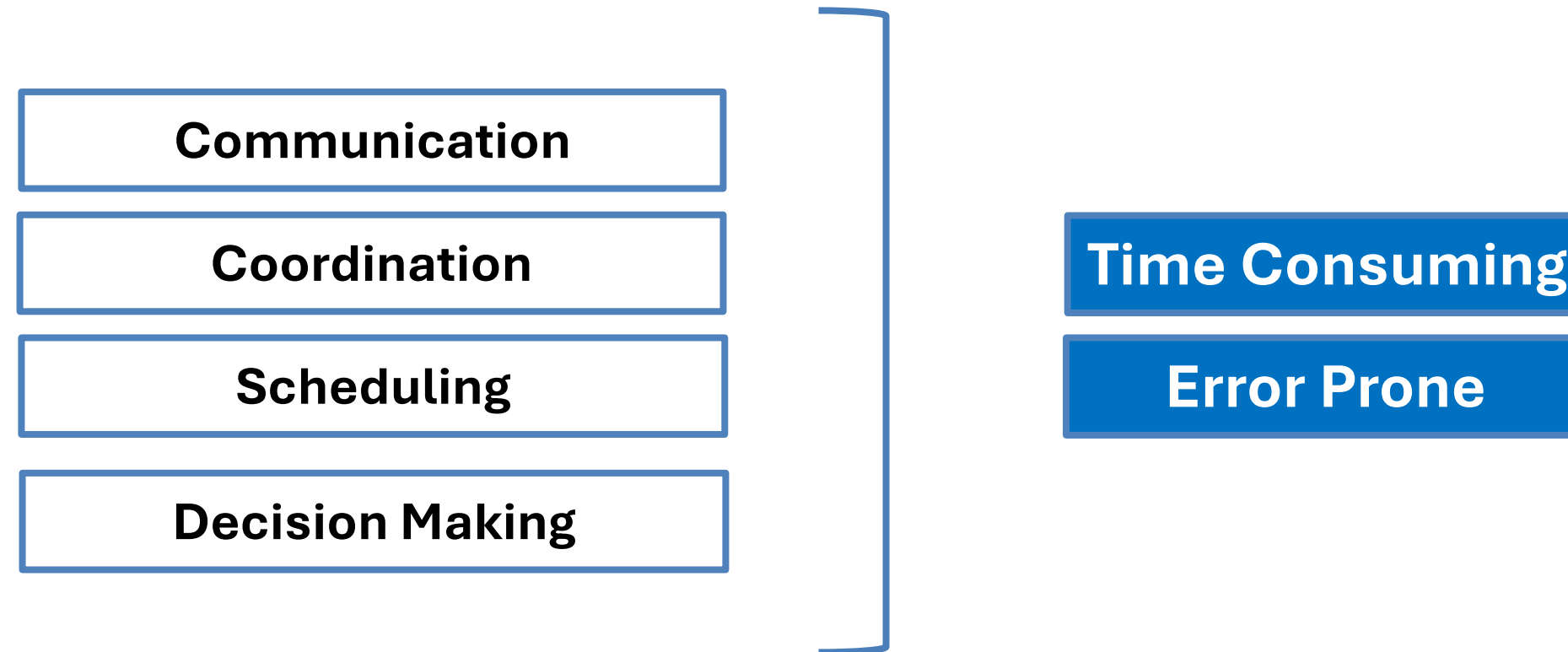
Section 5.3

Frameworks for Multi-Agent Systems



Frameworks for Multi-Agent Systems

Why Frameworks Matter in MAS



Framework provides:

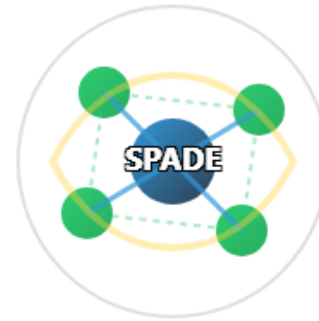
- ☐ Ready-made communication channels between agents
- ☐ Structures for defining roles, tasks, and behaviors
- ☐ Tools for orchestrating workflows
- ☐ Support for scalability and modularity

Frameworks for Multi-Agent Systems



**Oldest & most established
MAS frameworks**

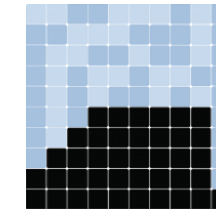
- ❑ Follows FIPA standards with built-in messaging and a robust agent lifecycle
- ❑ Widely used in research and academic simulations for standards-based systems



**Python-based framework that
uses XMPP for communication**

- ❑ Provides agent messaging, task execution, and monitoring for real-time distributed systems
- ❑ Simple, flexible, and well suited for Python ecosystems

Mesa



Python-based framework

- ❑ Commonly used for agent-based modeling and simulations in research domains
- ❑ Excellent for simulating thousands of interacting agents

Frameworks for Multi-Agent Systems



LangChain



LangGraph

Popular Frameworks

- ❑ LangChain provides building blocks for LLM apps
- ❑ LangGraph adds graph-structured workflows for multi-agent

Autogen



Developed by Microsoft for LLM-to-LLM collaboration

- ❑ Enables multiple agents to communicate and solve problems
- ❑ Well suited for experimentation and multi-agent research

CrewAI



Lean, lightning-fast Python framework

- ❑ Focuses on agentic AI with clear roles, goals, and collaboration
- ❑ Supports tool integration, structured workflows, and coordination
- ❑ Built for modern agents and practical automation use cases

Frameworks for Multi-Agent Systems

Choosing the Right Framework

Each framework has its strengths:

- ❑ JADE and SPADE excel in traditional agent communication
- ❑ Mesa is great for large simulations
- ❑ LangChain, LangGraph, and AutoGen shine in LLM-driven reasoning
- ❑ CrewAI stands out for building practical, production-ready agentic systems

Our goal is to create real-world AI agents that can collaborate, use tools, perform tasks, and automate workflows

CrewAI



Section 5.4

Demonstration: CrewAI Installation



Section 5.5

Demonstration: Building a Simple Multi-Agent System using CrewAI



Section 5.6

Demonstration: Building an Advanced Multi-Agent System using CrewAI



Section 5.7

Demonstration: Building Multi-Agent System using n8n



Section 5.8

Demonstration: Building AWS Monitoring Multi- Agent System



Section 5.9

Demonstration: Cross-cloud Multi- Agent System with Google Cloud and AWS



Section 5.10

Summary

- ✓ Understood Multi-Agent System and its architecture
- ✓ Explored frameworks for Multi-Agent workflow
- ✓ Hands-on demonstrations:
 - ✓ CrewAI installation
 - ✓ Simple & Advanced Multi-Agent Systems
 - ✓ No-Code Multi-Agent workflow using n8n
 - ✓ Intelligent AWS monitoring
 - ✓ Cross-cloud Agent-to-Agent coordination

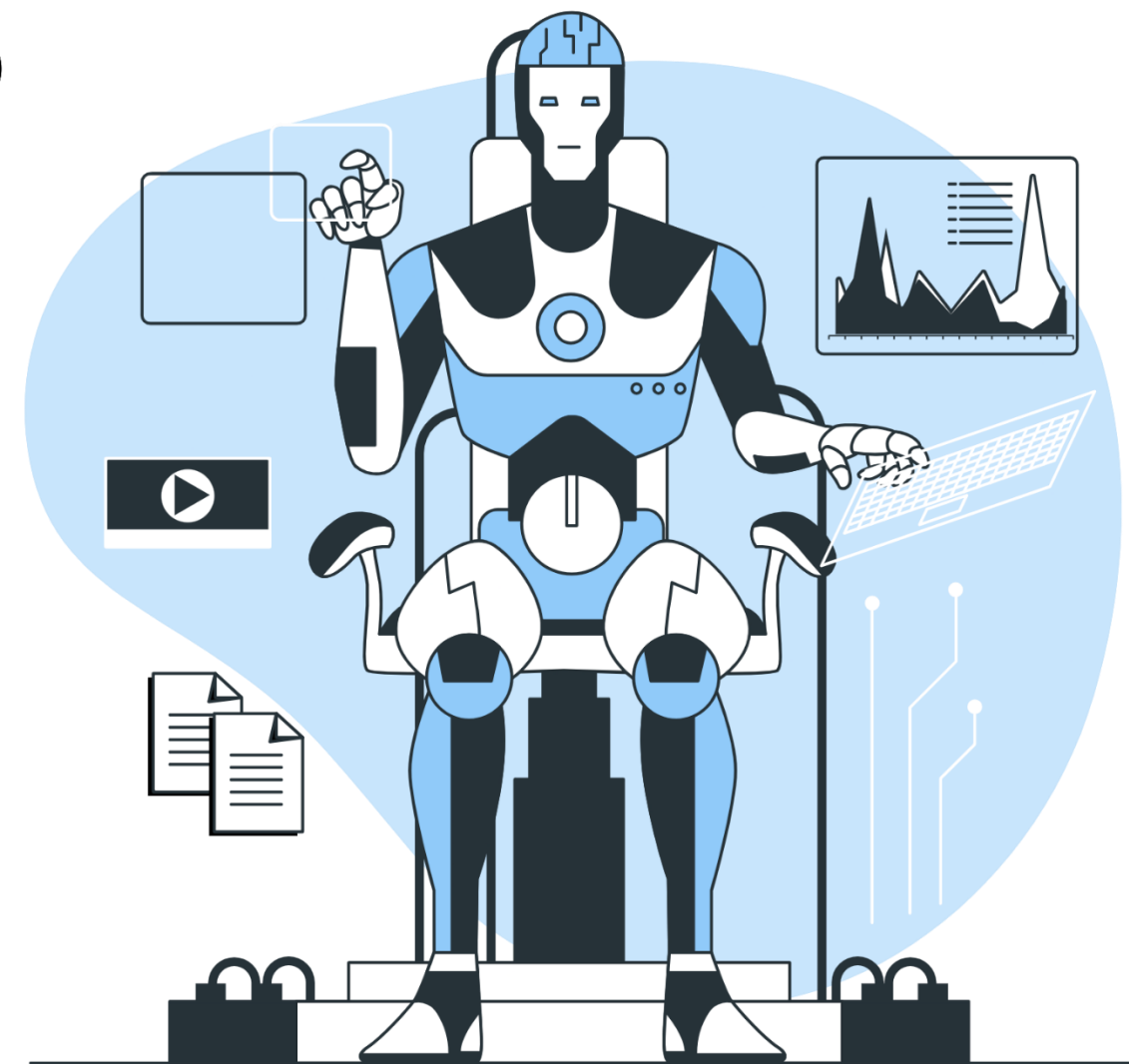
Thank You

Section 6.0

RAG-Enhanced Agents



Gemini



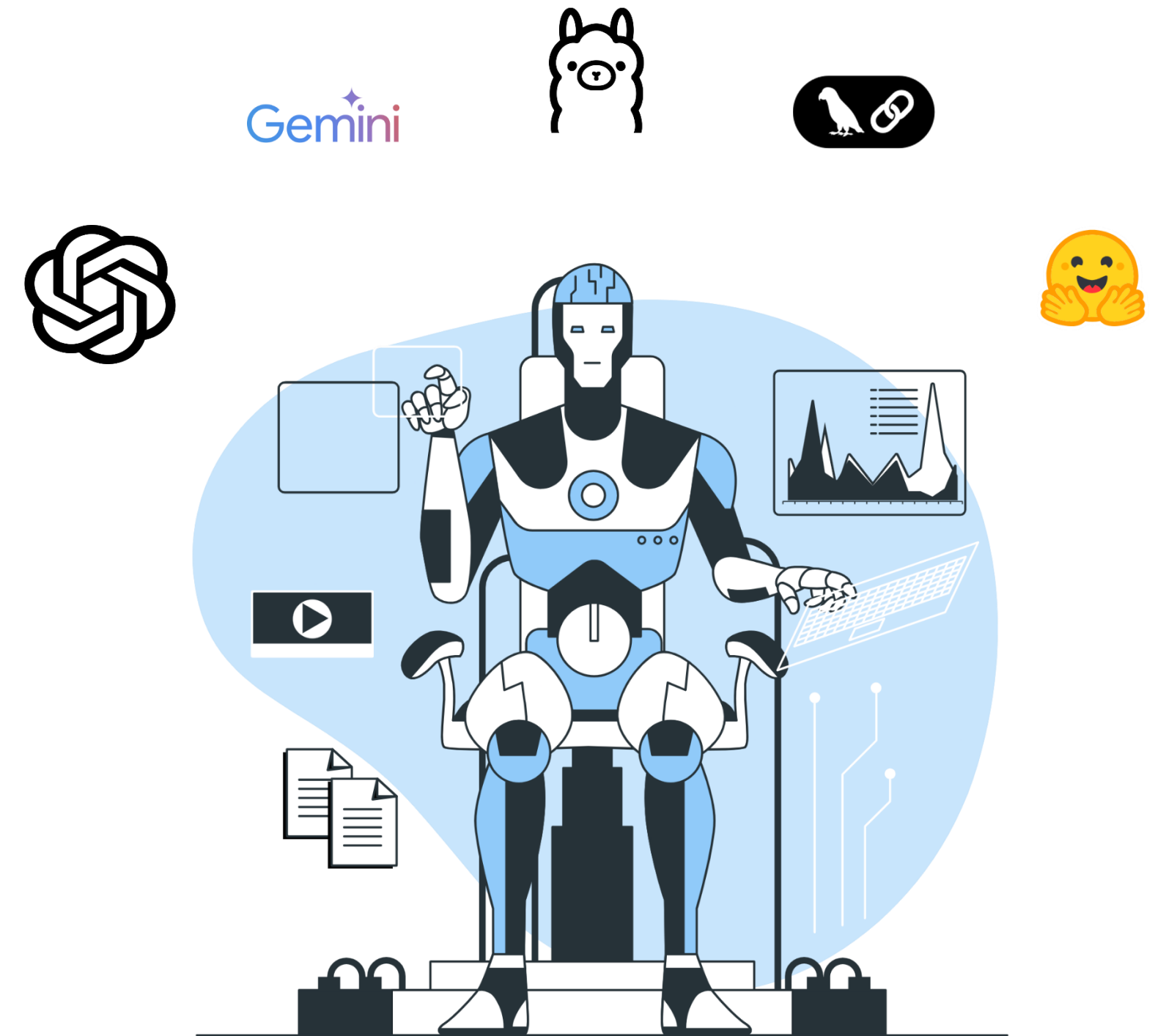
RAG-Enhanced Agents

Section Overview

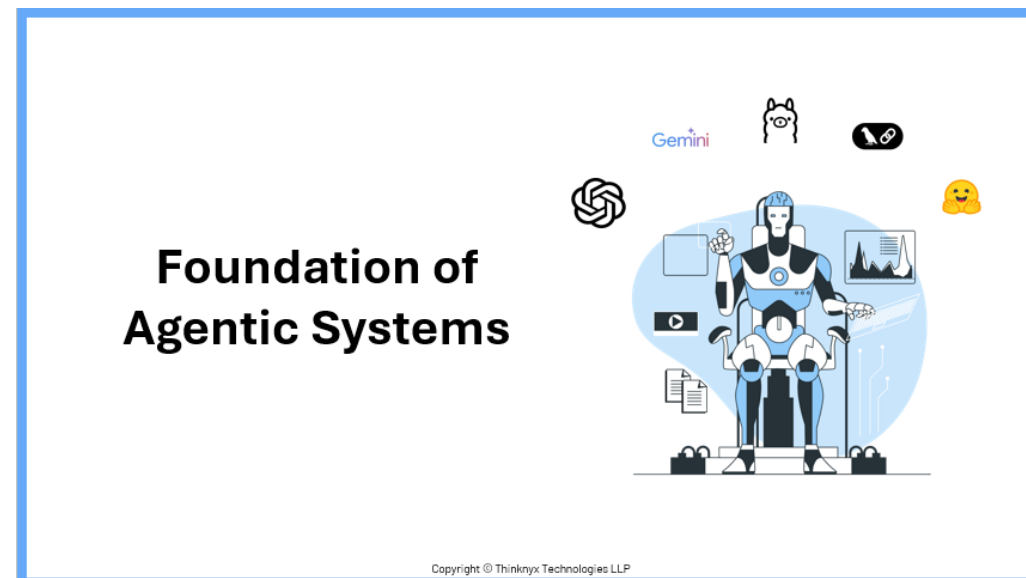
- ❑ Integration of RAG into Agents
- ❑ Hands-on demonstration:
 - ❑ RAG-based Q&A agent
 - ❑ Agentic RAG using CrewAI
 - ❑ Course intelligence
 - ❑ Project feasibility analysis

Section 6.1

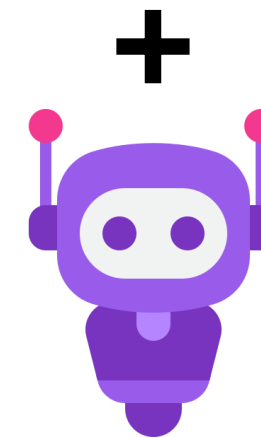
Integrating RAG into Agents



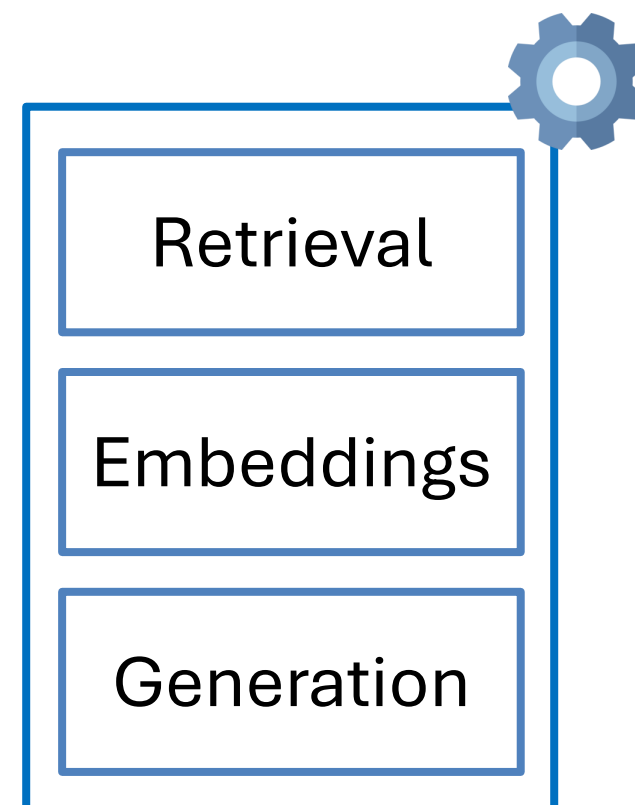
Integrating RAG into Agents



Retrieval Augmented Generation (RAG)



AI Agents



Hallucination



Accuracy



Domain Specific
Intelligence

Integrating RAG into Agents



System

- ✓ Reason
- ✓ Plan
- ✓ Act

Accurate, up-to-date information

Retrieval Augmented Generation (RAG)

Integrating RAG into Agents

How do we actually integrate RAG into an AI agent?

RAG

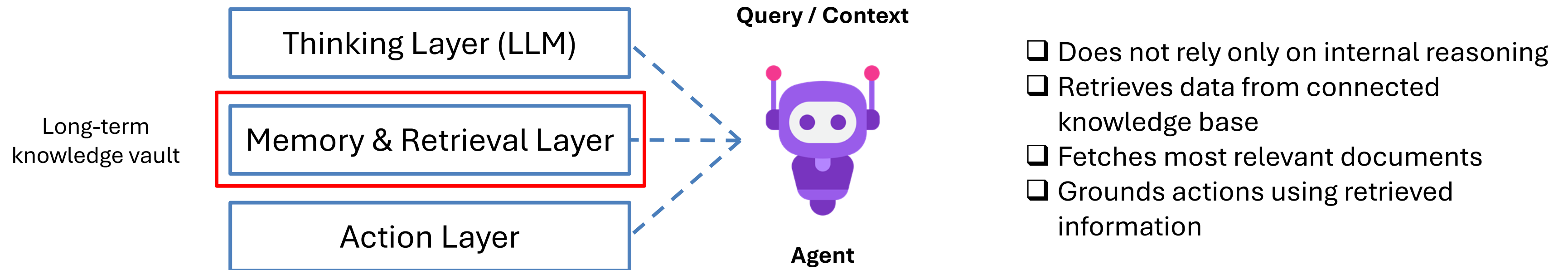


Agent

Response grounded in external knowledge before the agent takes action

Integrating RAG into Agents

Integration of RAG with AI agent



Integrating RAG into Agents

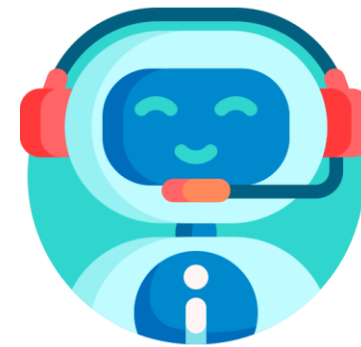
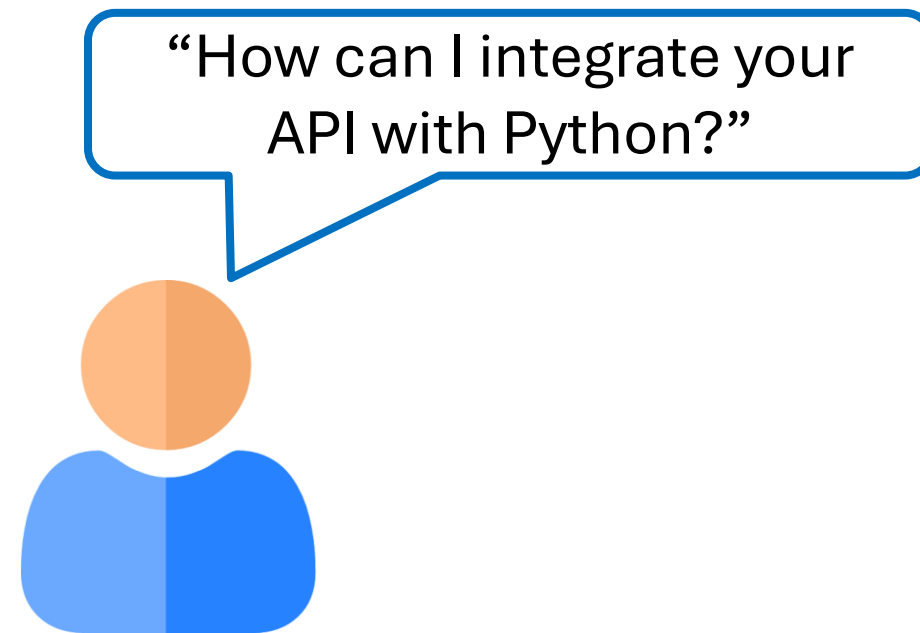
RAG

Enhances a model's answers by retrieving relevant data at query time

MCP

Standardizes how an AI agent connects to and uses external tools and data sources

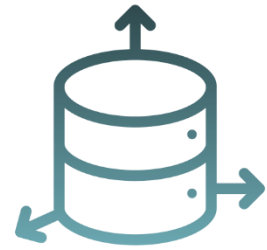
Integrating RAG into Agents



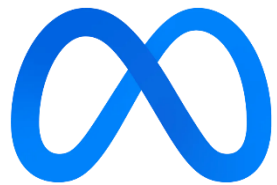
- ❑ Searches the company's documentation database
- ❑ Finds the API integration guide
- ❑ Retrieves the relevant paragraphs
- ❑ Uses that information to craft a step-by-step answer

Conversational responses grounded in real-time documentation

Integrating RAG into Agents



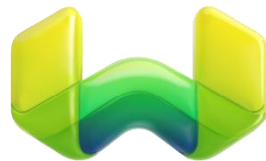
Vector Databases



FAISS



Pinecone



Weaviate

Store text embeddings

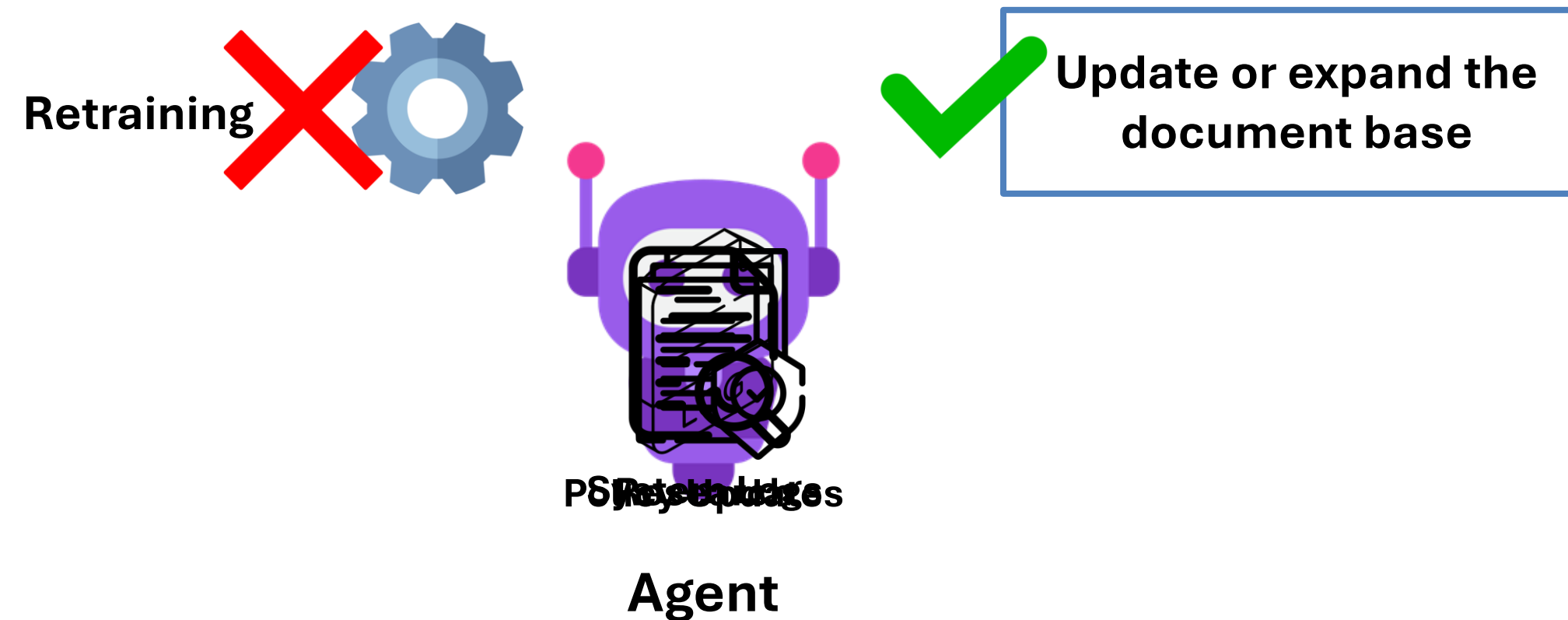


Agent

Generates reliable,
domain-aware
responses

- ☐ Takes user query as input
- ☐ Embeds query into vector space
- ☐ Performs similarity search
- ☐ Passes retrieved chunks as context

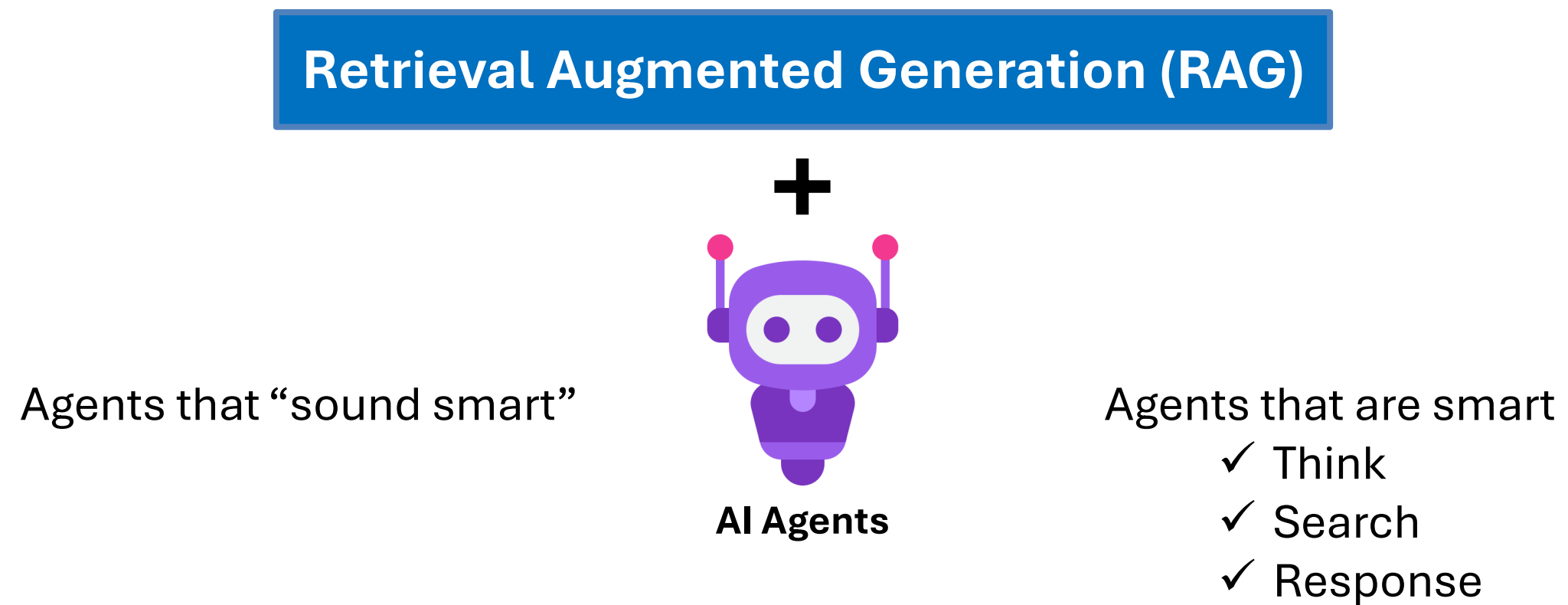
Integrating RAG into Agents



Keeps the agent's knowledge fresh and aligned with the latest information

Integrating RAG into Agents

Summary



Section 6.2

Demonstration: RAG-based Q&A Agent



Section 6.3

Demonstration: Agentic RAG using CrewAI



Section 6.4

Demonstration: Agentic RAG for Course Intelligence



Section 6.5

Demonstration: Project Feasibility Agent using Agentic RAG



Section 6.6

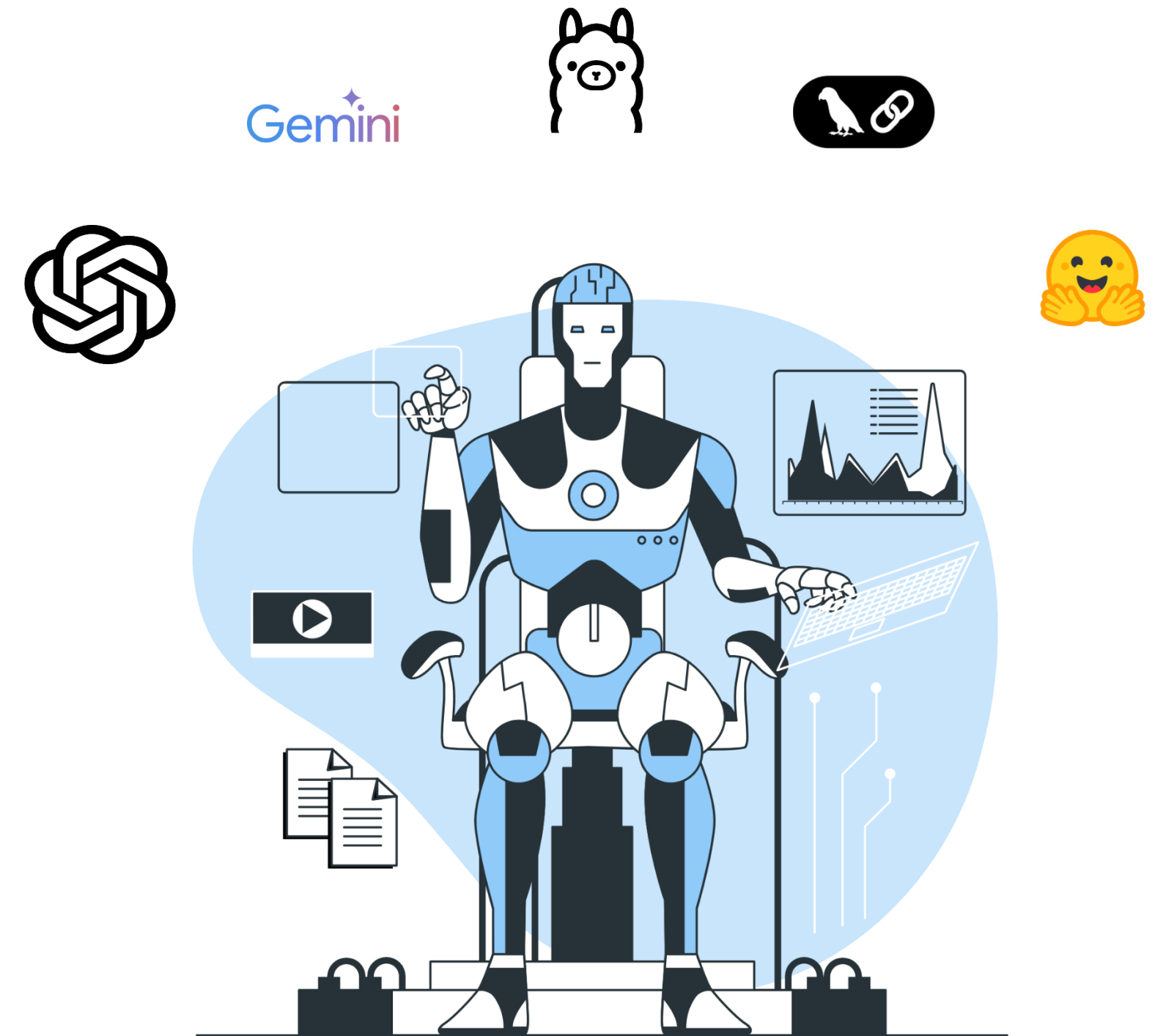
Summary

- ✓ Integrated RAG into Agents
- ✓ Hands-on demonstration:
 - ✓ Built RAG-based Q&A agent
 - ✓ Upgraded multi-agent RAG using CrewAI
 - ✓ Course intelligence
 - ✓ Project feasibility analysis

Thank You

Section 7.0

Reflection, Optimization & Deployment



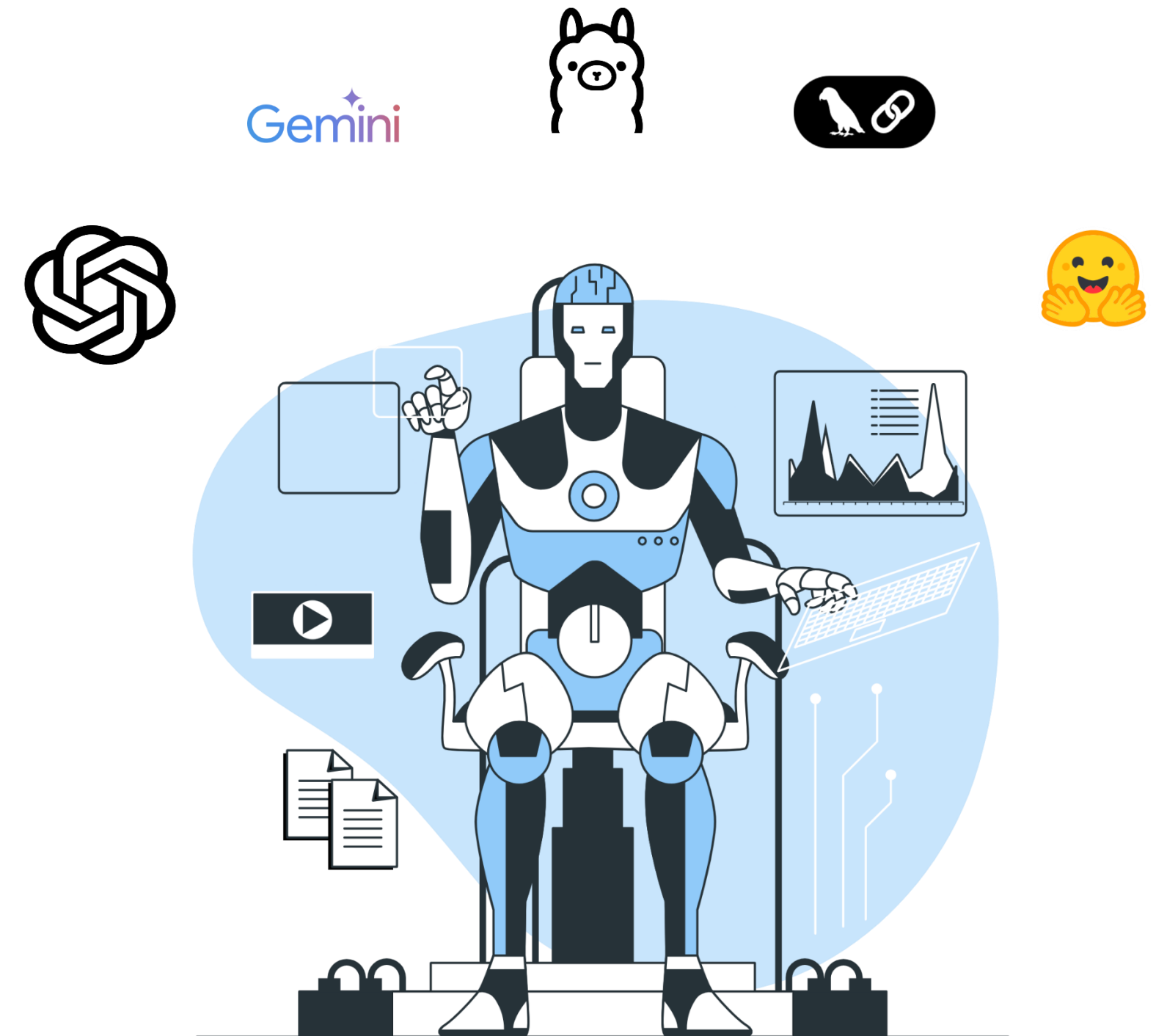
Reflection, Optimization & Deployment

Section Overview

- ❑ Explore Reflection and Self Improvement
- ❑ Understand Deploying Agents
- ❑ Learn about the Guardrails and Safety

Section 7.1

Reflection & Self-Improvement



Reflection & Self-Improvement

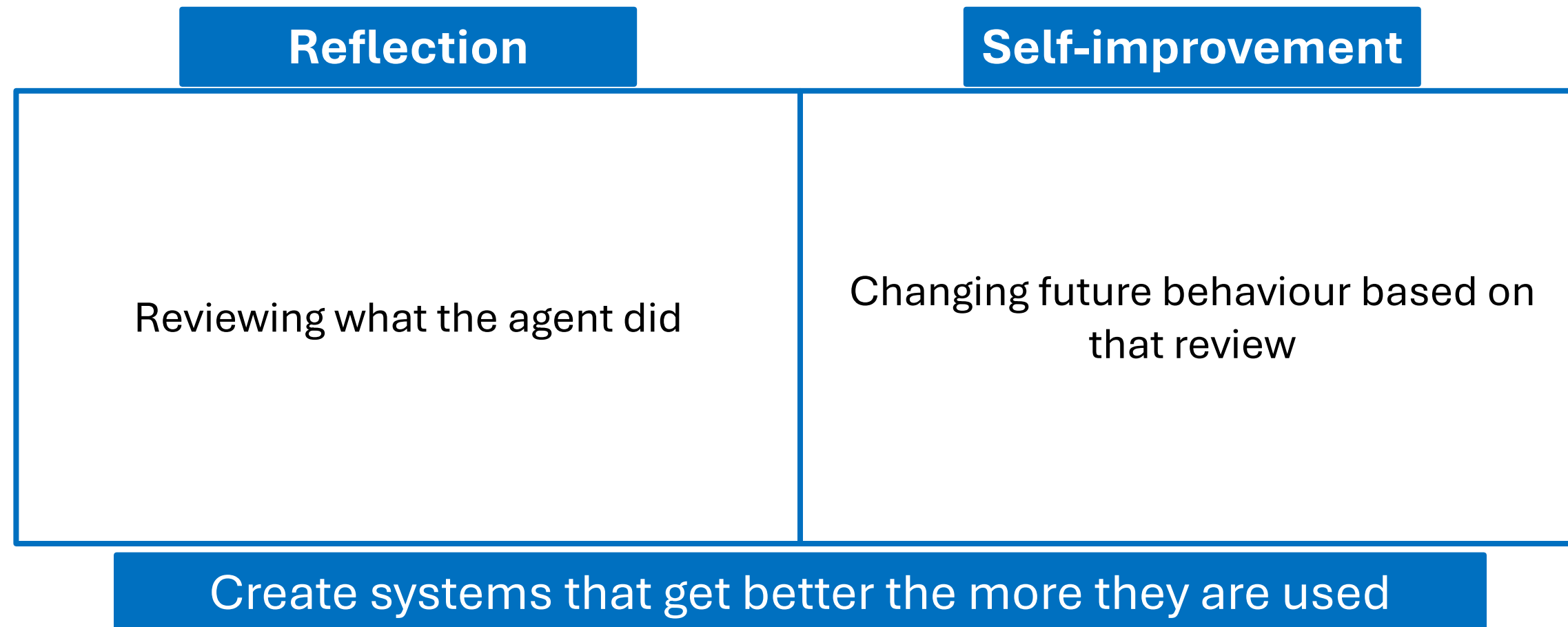
Reflection

Reviewing what the agent did

Self-improvement

Changing future behaviour based on
that review

Reflection & Self-Improvement



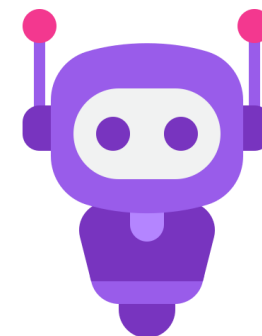
Reflection & Self-Improvement

Understanding Reflection in Agentic AI

Reflection

Process where an agent analyses its own reasoning and actions after completing a task

- ✓ Examine what happened
- ✓ Identify what worked



Agent

- ✓ Notice what failed
- ✓ Self-reflect to improve future performance

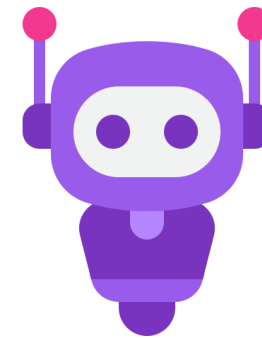
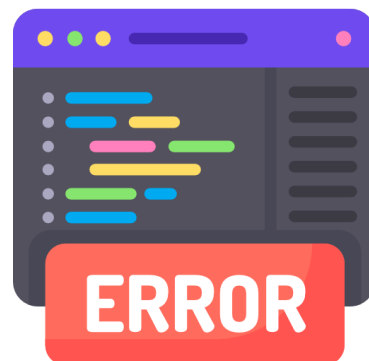
Adaptive

Intelligent

Reflection & Self-Improvement

Understanding Reflection in Agentic AI

Non-Agentic System



SQL Query
Agent

Reflective System

- ✓ Analyze error messages
- ✓ Review reasoning steps
- ✓ Refine future approach

Reflection & Self-Improvement

Levels of Reflection

Immediate

- ☐ Happens right after producing an output
- ☐ Reviews output for clarity, correctness & completeness

Episodic

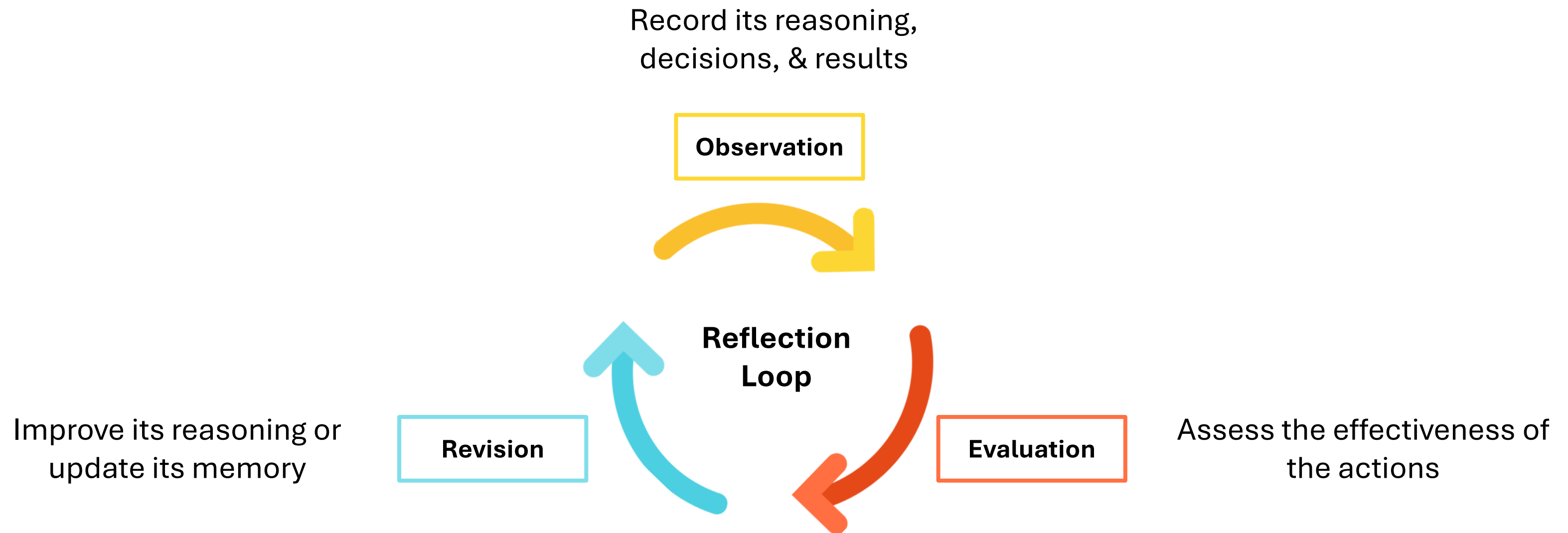
- ☐ Looks at the entire session once the task is complete
- ☐ Reviews both the user's intent and the steps it took along the way

Long-term

- ☐ Identifies patterns across many interactions
- ☐ Uses them to refine its strategies over days, weeks, or even months

Reflection & Self-Improvement

The Reflection Loop



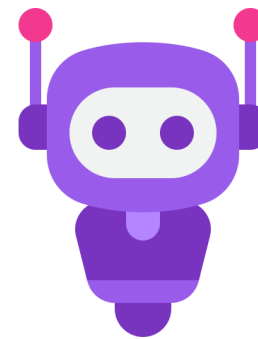
Reflection & Self-Improvement

Understanding Self-Improvement

Self-Improvement

Helps the agent change what will happen next

Reflection Insights



Agent

Modify:

- ✓ Strategies
- ✓ Prompts
- ✓ Memory
- ✓ Tool Selection

For Example:

- ☐ Notices repeated misunderstanding of date formats
- ☐ Stores this as a preference in long-term memory
- ☐ Automatically formats dates correctly in future conversations

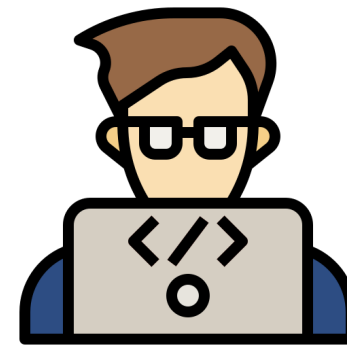
In advanced systems:

- ☐ Use feedback from previous failures
- ☐ Redesign task plans, reasoning chains, or tool choices
- ☐ Become more accurate, personalized, and efficient over time

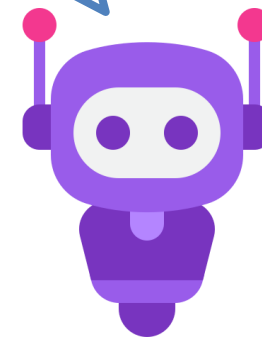
Reflection & Self-Improvement

Practical Implementation of Self-Improvement

Combine reflection prompts with persistent memory



Identify any steps that could be improved and store relevant learnings



Agent

- ✓ Save results in vector databases, memory graphs, or key-value stores
- ✓ Retrieve past learnings and applies them automatically

In Multi-agent setup:

- ✓ One agent perform the task
- ✓ Another agent critique the output
- ✓ Feedback is used for self-improvement

Reflection & Self-Improvement

Why Reflection & Self-Improvement matter

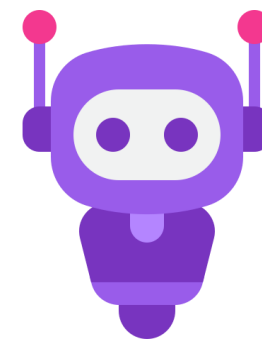
- ✓ Reflective agent catches incorrect assumptions and hallucinations
- ✓ Self-improving agent reduces repeated mistakes over time
- ✓ Enables continuous learning and consistent performance

Reflection & Self-Improvement

Summary

Reflection identifies what went wrong

Self-improvement drives better future outcomes

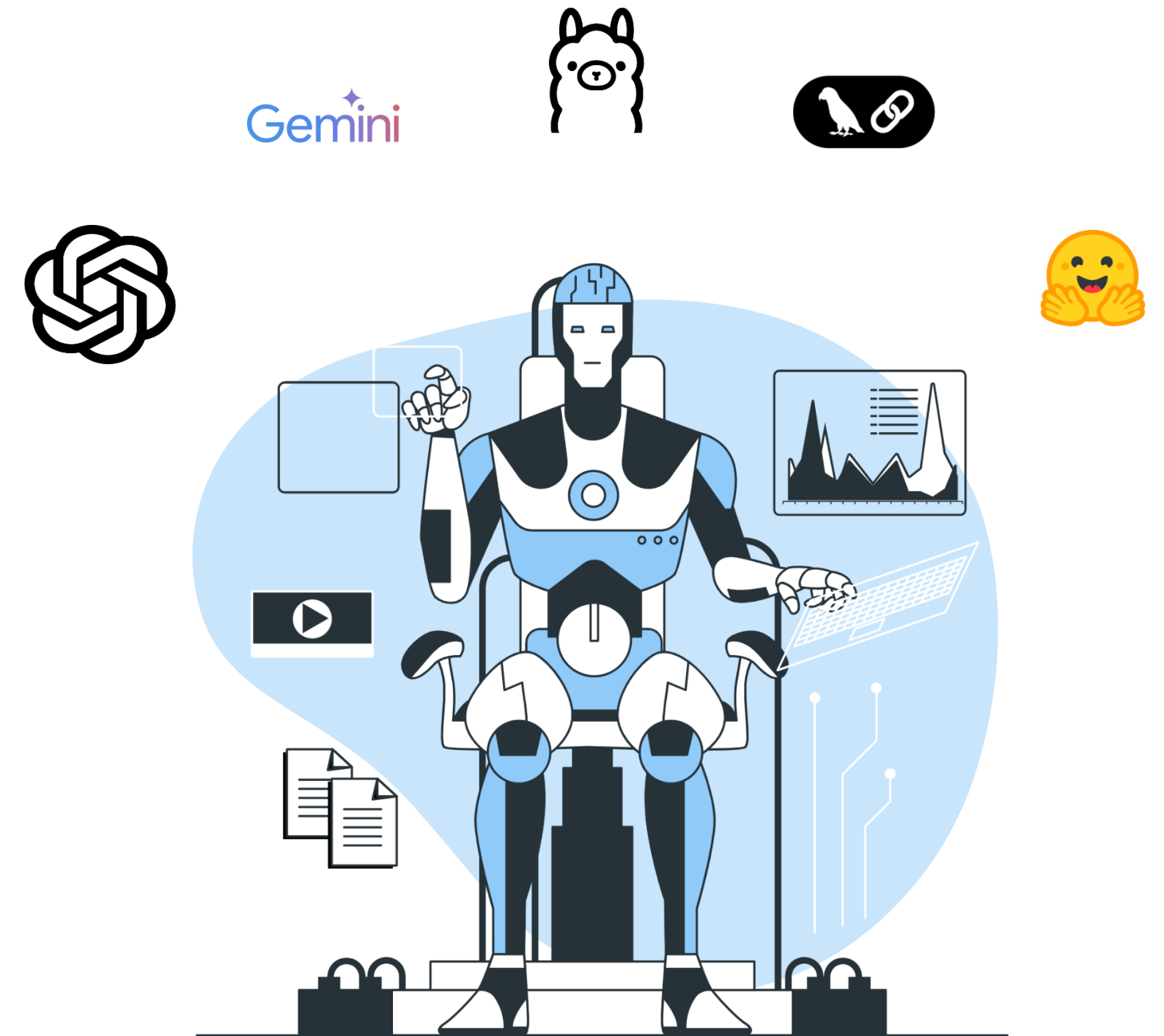


Agent

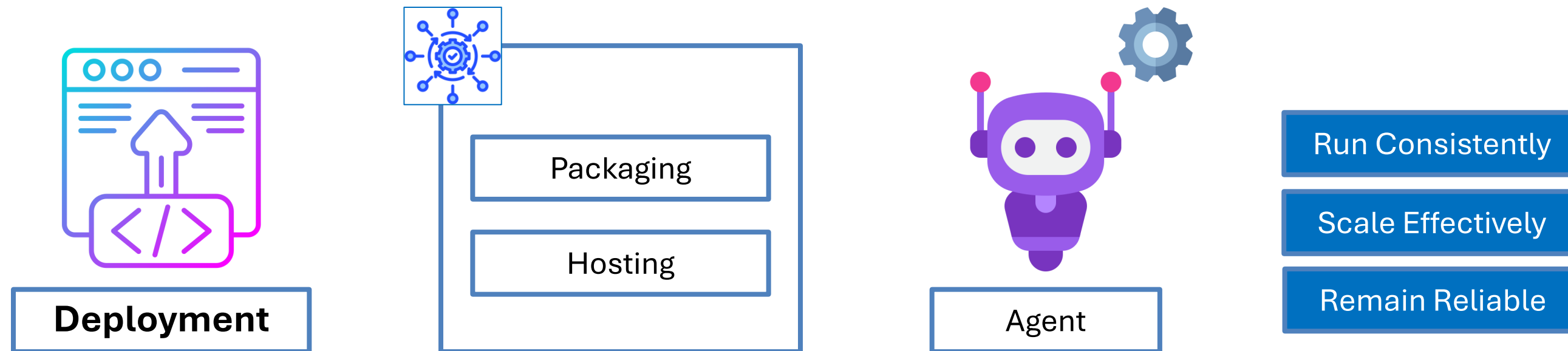
- ✓ Adapt to user needs
- ✓ Real-world environments
- ✓ Long-term workflows

Section 7.2

Deploying Agents

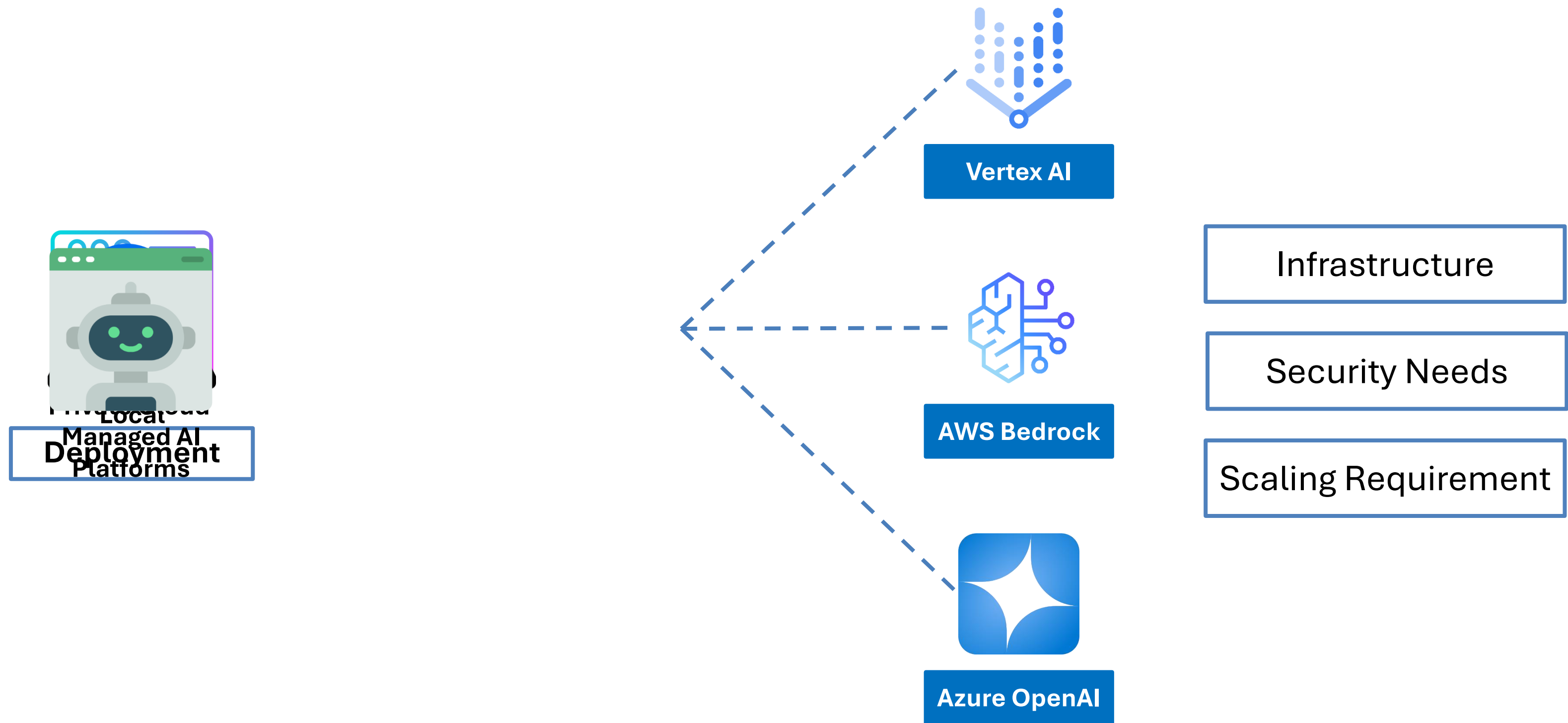


Deploying Agents



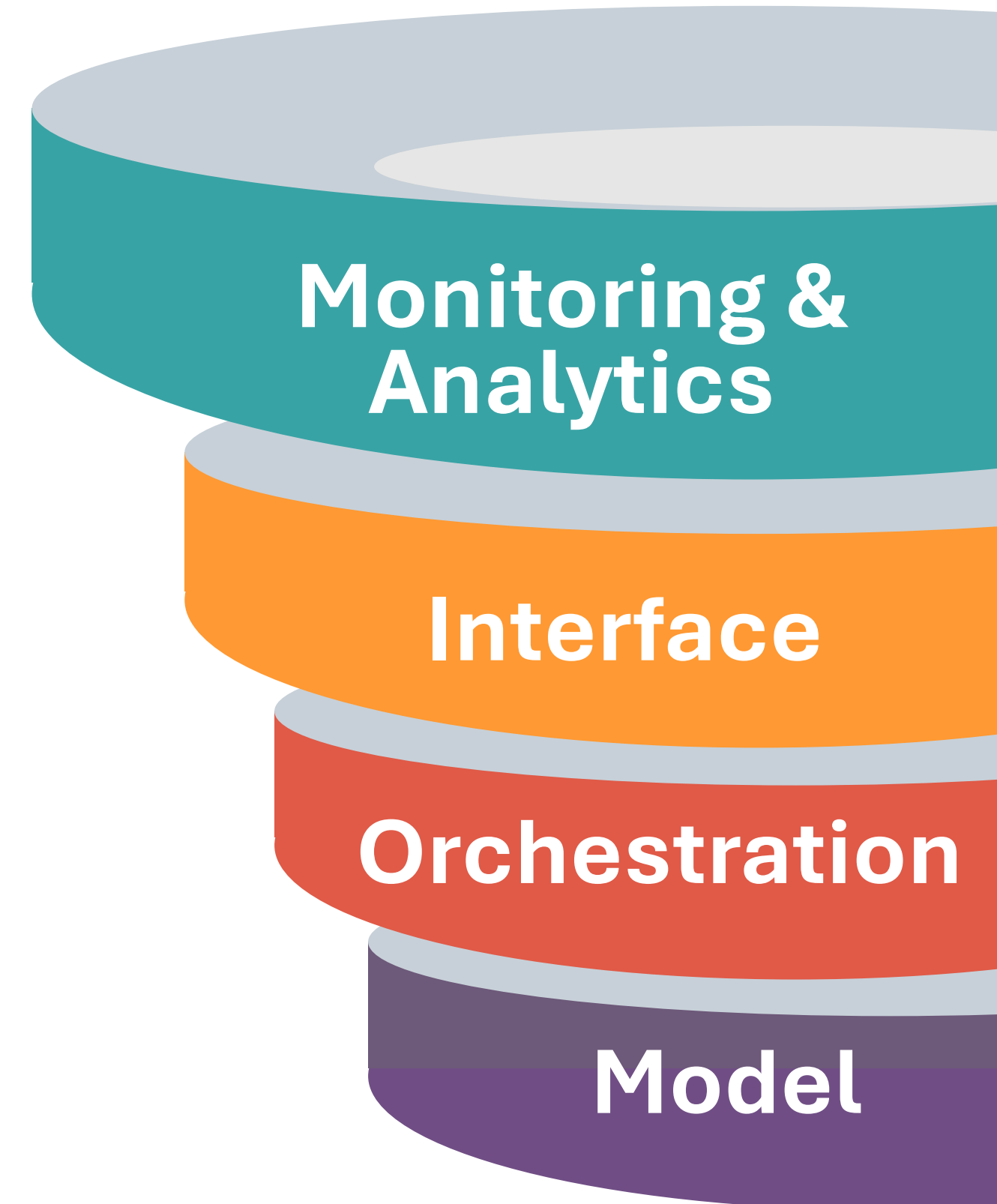
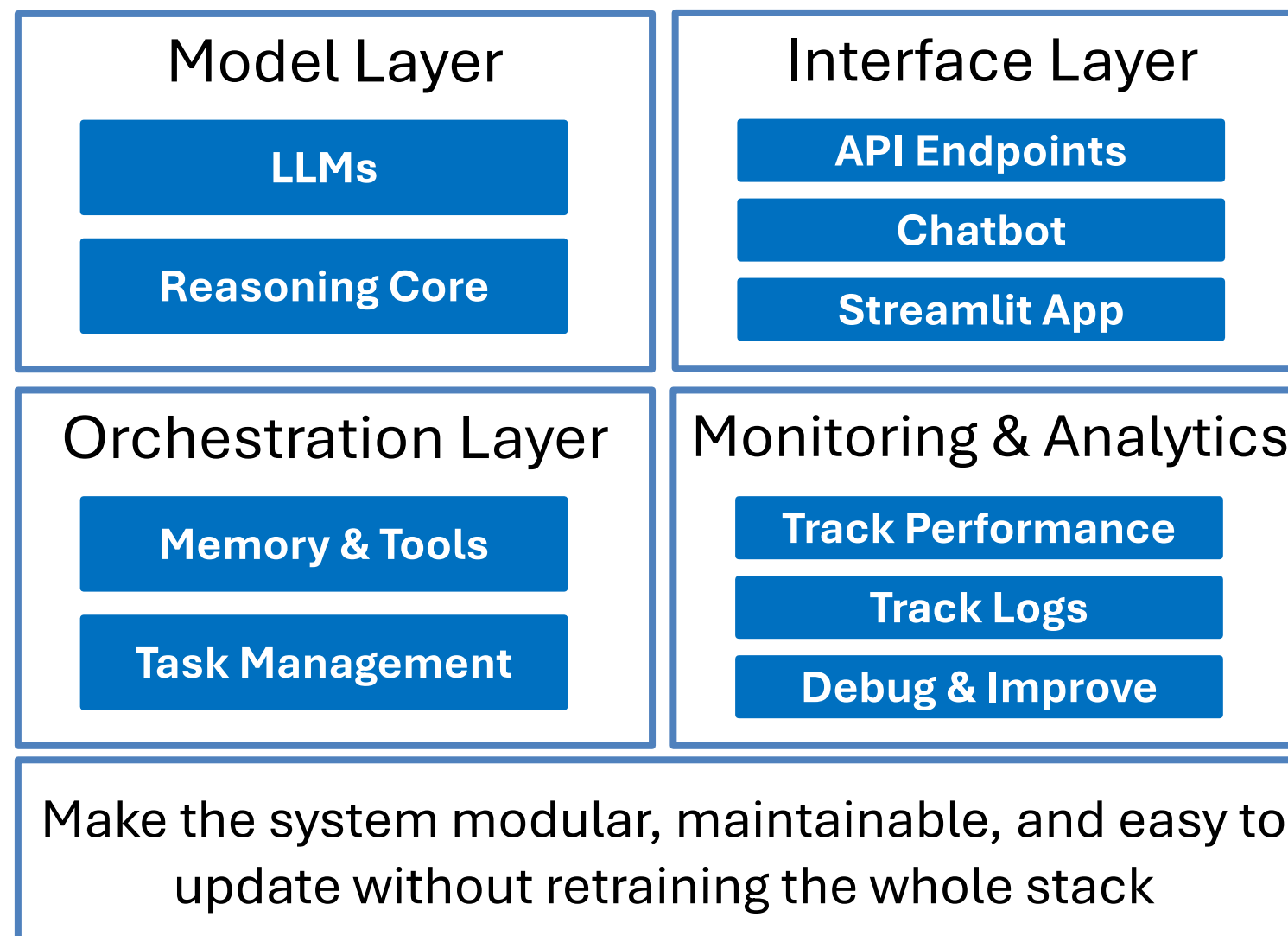
A well-deployed agent should be easy to update, monitor, and integrate with other applications

Deploying Agents



Deploying Agents

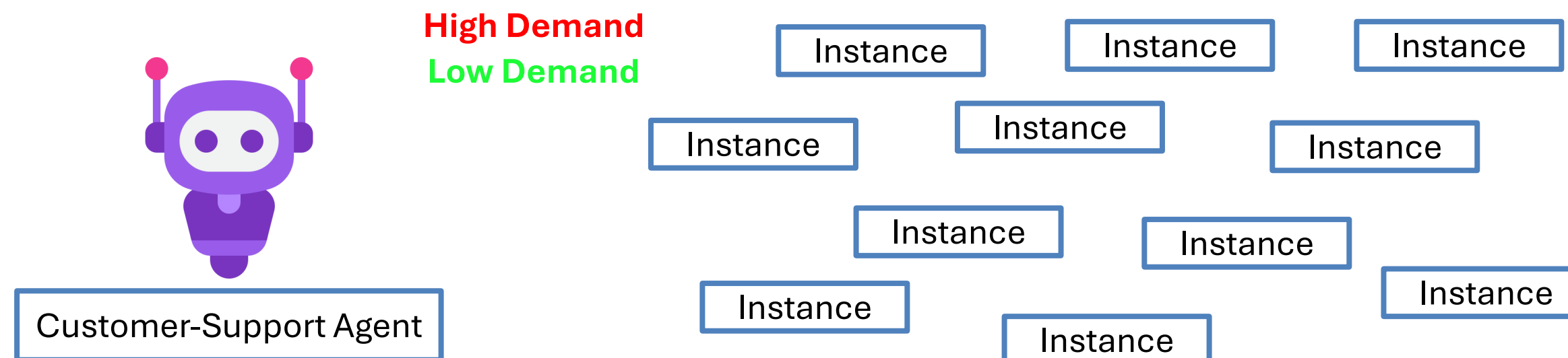
Deployment Architecture



Deploying Agents

Containerization and Scaling

- ❑ Agents in production are packaged in containers like Docker
- ❑ Keeps environments, dependencies, and configs consistent across servers
- ❑ With orchestration tools, agents can scale dynamically based on demand



Deploying Agents

Observability

- ☐ Agents shouldn't operate in isolation
- ☐ Every action, decision & interaction should be logged
- ☐ Monitoring tools track latency, uptime, error rates or API usage



Prometheus



Grafana



AWS
CloudWatch



Google Cloud
Monitoring

Local Log
Files

Deploying Agents

Security and Maintenance

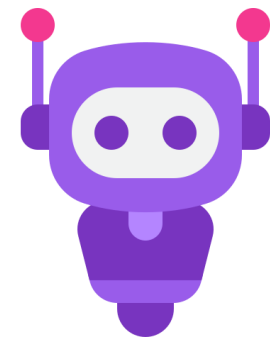
- ☐ Security is vital for agents accessing sensitive data or external systems
- ☐ Use auth tokens, access roles, and encrypted channels for safe operation
- ☐ Regularly update models, memory layers, and tools to align with new policies and improvements

Deploying Agents

Summary

Deployment

Where agent becomes truly operational

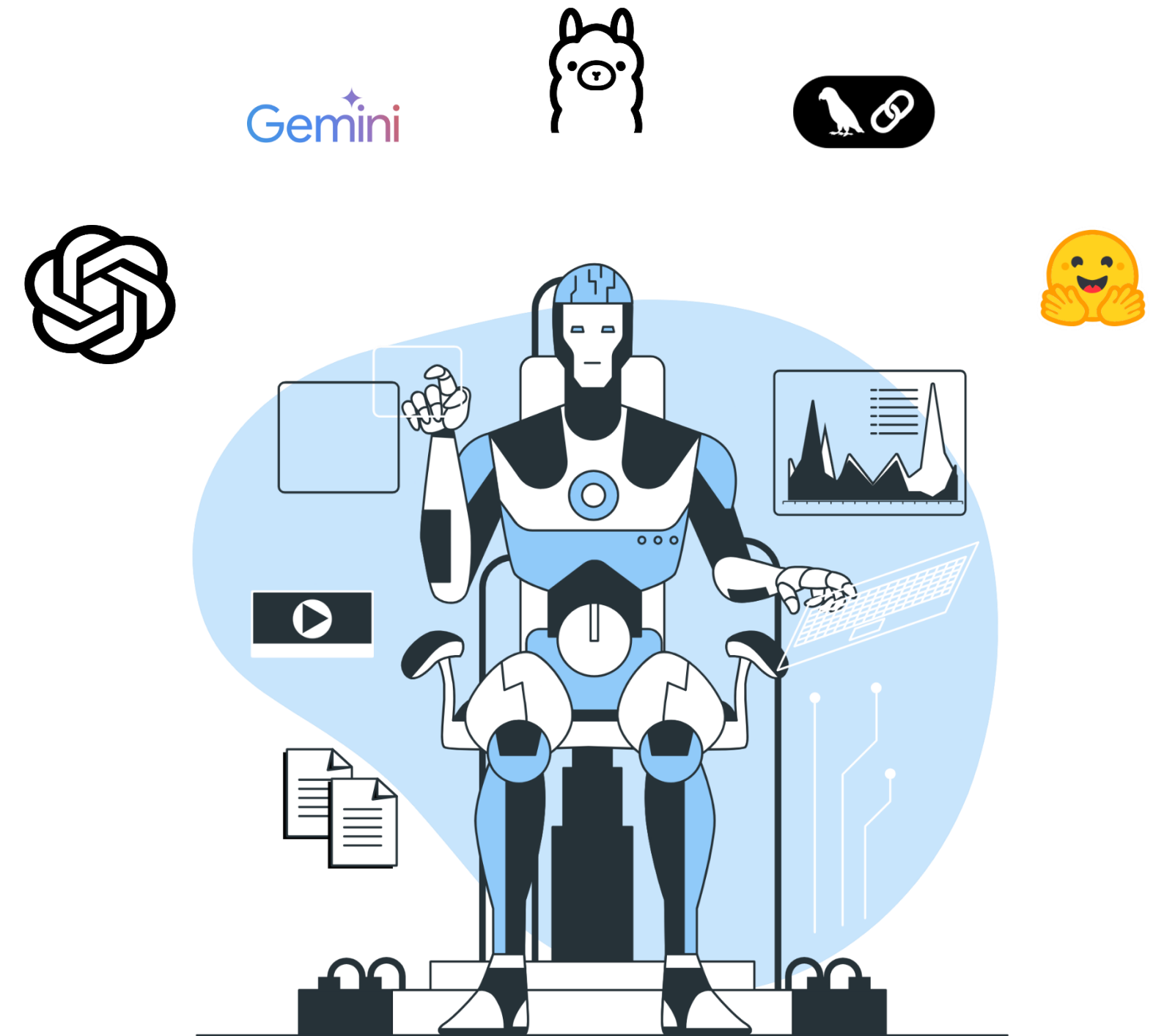


Agentic AI
System

- ✓ Adapt
- ✓ Scale
- ✓ Learn

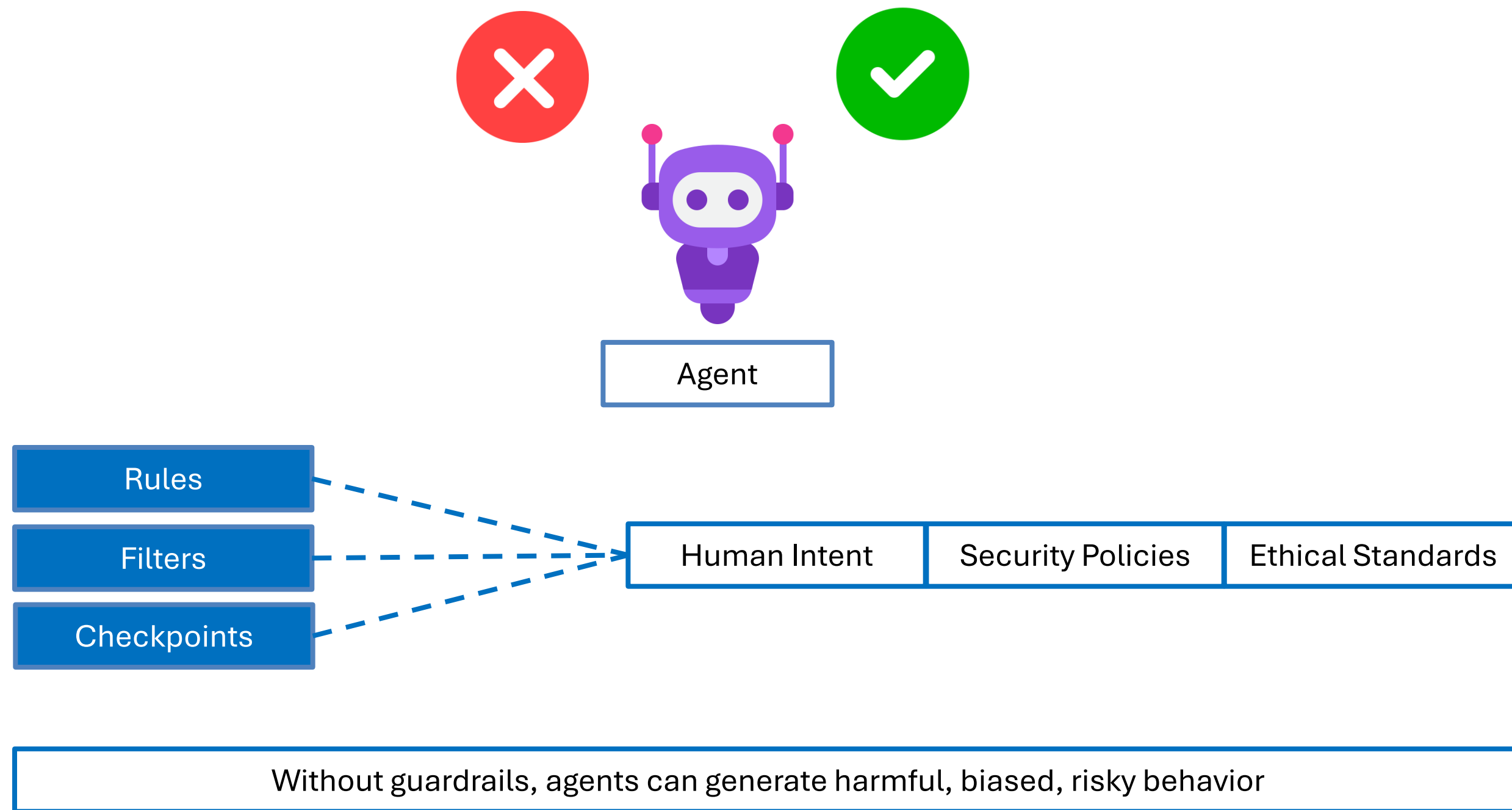
Section 7.3

Guardrails and Safety



Guardrails and Safety

The Need for Guardrails



Guardrails and Safety



Delete a resource



Modify sensitive data



Make financial decisions

Ensures the agent always operates within predefined limits

Guardrails and Safety

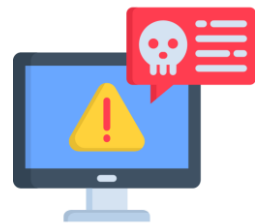
Types of Guardrails

Input-level Guardrails

- ❑ Filters or sanitizes user inputs before they reach the model
- ❑ Prevents:

PROMPT|

Prompt Injection



Malicious Commands



Data Leaks

Model-level Guardrails

- ❑ Agent checks its own reasoning before executing



Verify Facts



Check Biases



Confirm Compliances

Output-level Guardrails

- ❑ Review the final response before it reaches the user or system
- ❑ Ensure output is:



Safe



Accurate



Appropriate

Guardrails and Safety

Ethical and Policy Considerations

- ☐ Ethical alignment is as important as technical safety
- ☐ Agents must respect privacy, avoid misinformation, and stay transparent
- ☐ Organizations create ethical guidelines or compliance checklists
- ☐ Embedding these checks in the reasoning loop ensures accountability

Guardrails and Safety

Implementing Guardrails

- ❑ Developers can use safety libraries, moderation APIs, or policy frameworks
- ❑ Reflection loops can act as safety checks by evaluating reasoning
- ❑ Agents verify actions against ethical or operational constraints
- ❑ Mission-critical systems often use human-in-the-loop for final approval

Guardrails and Safety

Summary

- ❑ Guardrails don't limit intelligence; they guide it responsibly
- ❑ Ensure agents stay helpful, honest, and harmless as they grow
- ❑ The goal is to build trust in reliability, fairness, and human values

Section 7.4

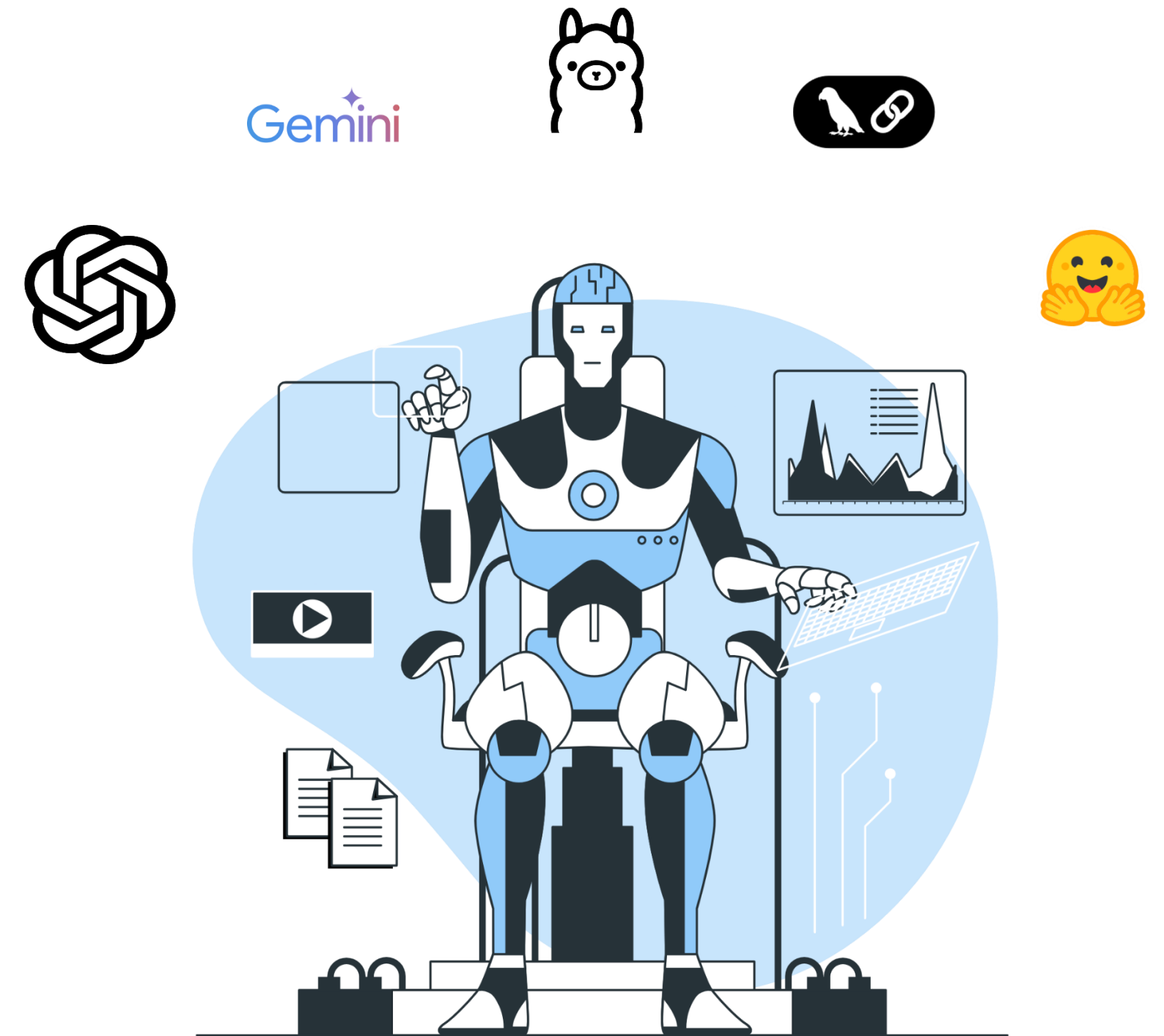
Summary

- ✓ Understood about Reflection and Self Improvement
- ✓ Learned the essentials of Deploying Agents
- ✓ Explored the Guardrails & Safety

Thank You

Section 8.0

Agentic Design Patterns



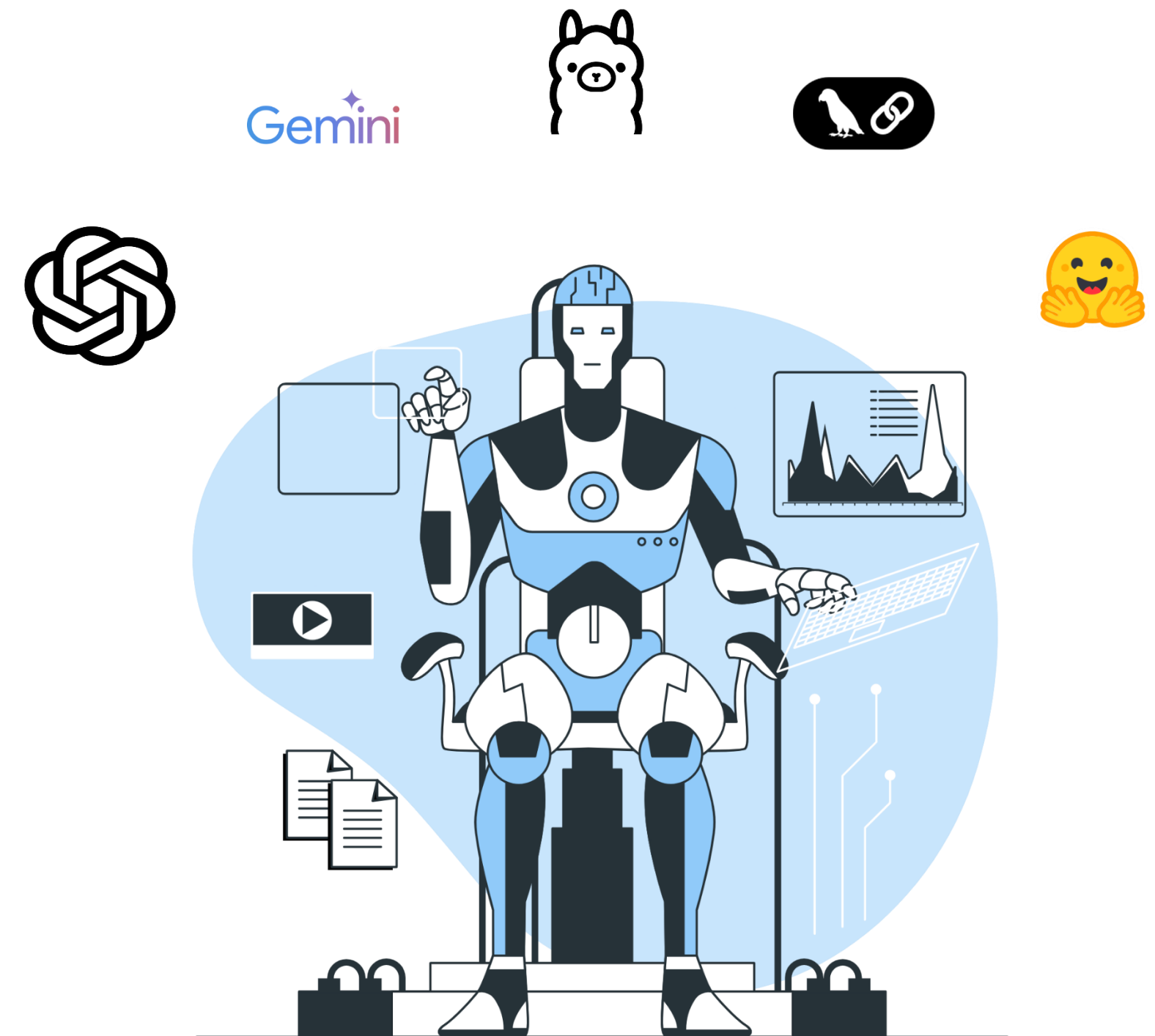
Agentic Design Patterns

Section Overview

- ☐ Understand Design Patterns
- ☐ Learn how to choose the right Agentic Design Pattern
- ☐ Explore Single & Multi-Agent Design Patterns

Section 8.1

Introduction to Agentic Design Patterns



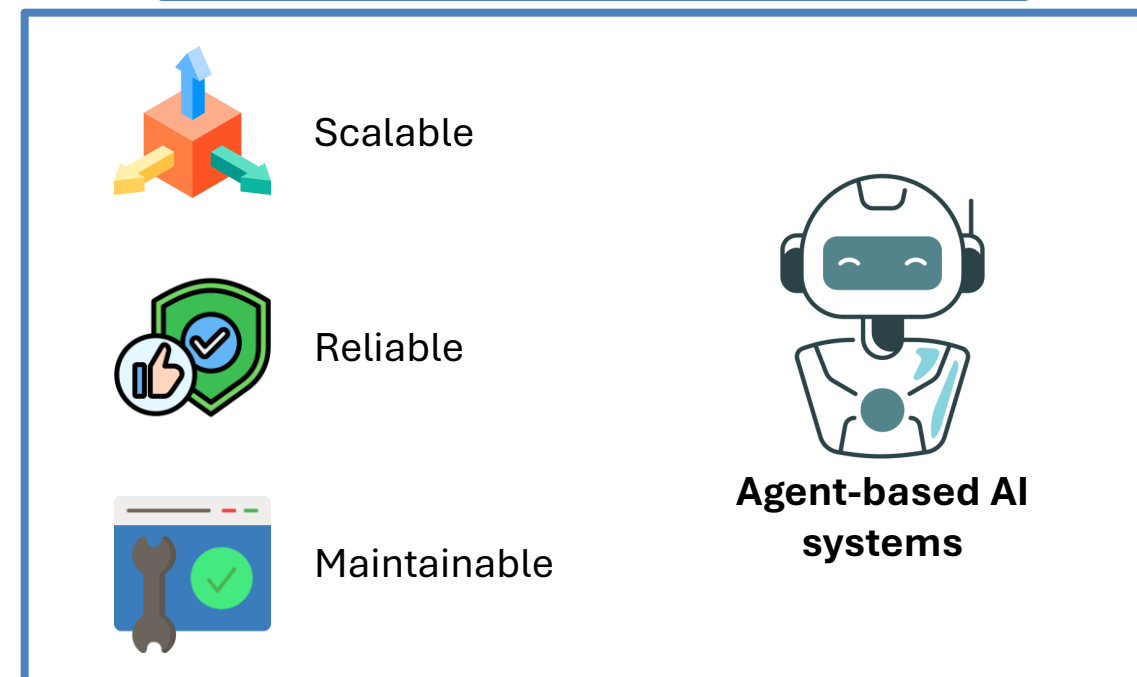
Introduction to Agentic Design Patterns

Agentic Design Patterns

Structured approaches to build scalable, reliable, and maintainable agent-based AI systems

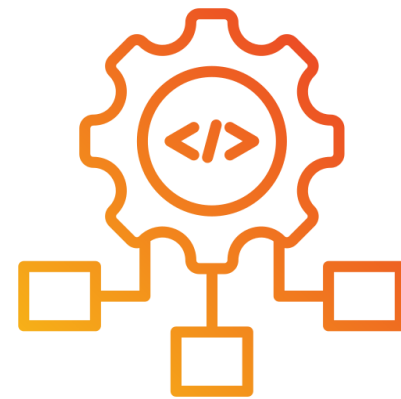
Introduction to Agentic Design Patterns

Agentic Design Patterns



Show how to move from simple demos to production-ready agentic systems

Introduction to Agentic Design Patterns



Architectural Templates

Design Patterns

Introduction to Agentic Design Patterns

Design Patterns

Reusable solution to a commonly occurring problem within a particular context



Copy-paste Code



Template or Blueprint

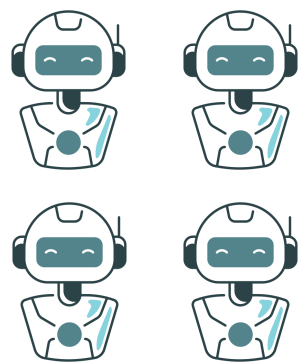
In software engineering

- ✓ Help organize code for better scalability, clarity, and flexibility
- ✓ For example - Separating object creation from usage, or organizing complex interactions between components

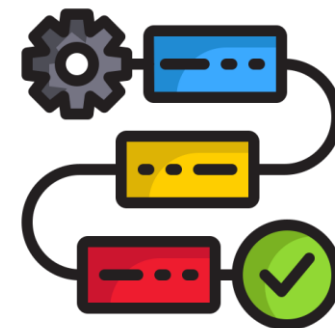
Introduction to Agentic Design Patterns

Why Patterns matter

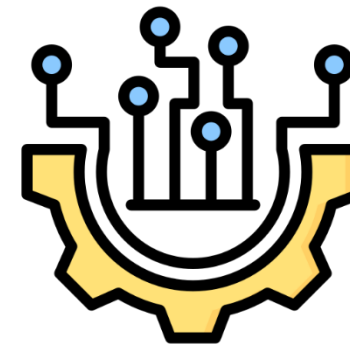
Agentic Systems



Agents



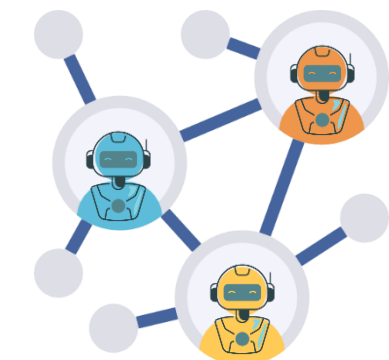
Workflows



Tool Integrations



Memory Services



Agent Coordination



Plan



Execute



Review

Patterns solve questions like:

- ☐ How should independent agents collaborate on a task?
- ☐ Should agents run simultaneously or one after another?
- ☐ How do we iteratively refine outputs when the first answer isn't good enough?
- ☐ How do we balance autonomy with safety or human oversight?

Introduction to Agentic Design Patterns

Importance of Agentic Design Patterns

**Capture best practices and
reduce reinventing the wheel**

Provide a common language

Manage complexity

**Improve reliability and
scalability**

Introduction to Agentic Design Patterns

How do design patterns simplify complex agent behaviour?

Task Decomposition

- ❑ Break a large problem into smaller tasks

Delegation

- ❑ Assign subtasks to specialized agents

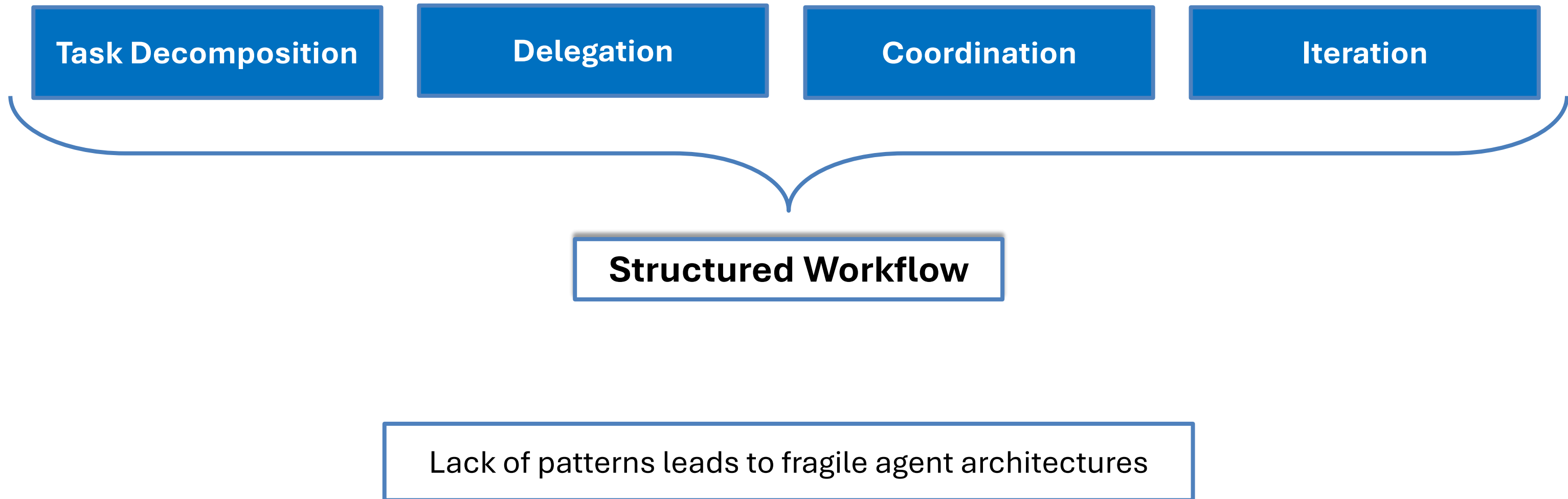
Coordination

- ❑ Ensure sub-agents don't work at cross-purposes

Iteration

- ❑ Refine outputs across multiple passes

Introduction to Agentic Design Patterns



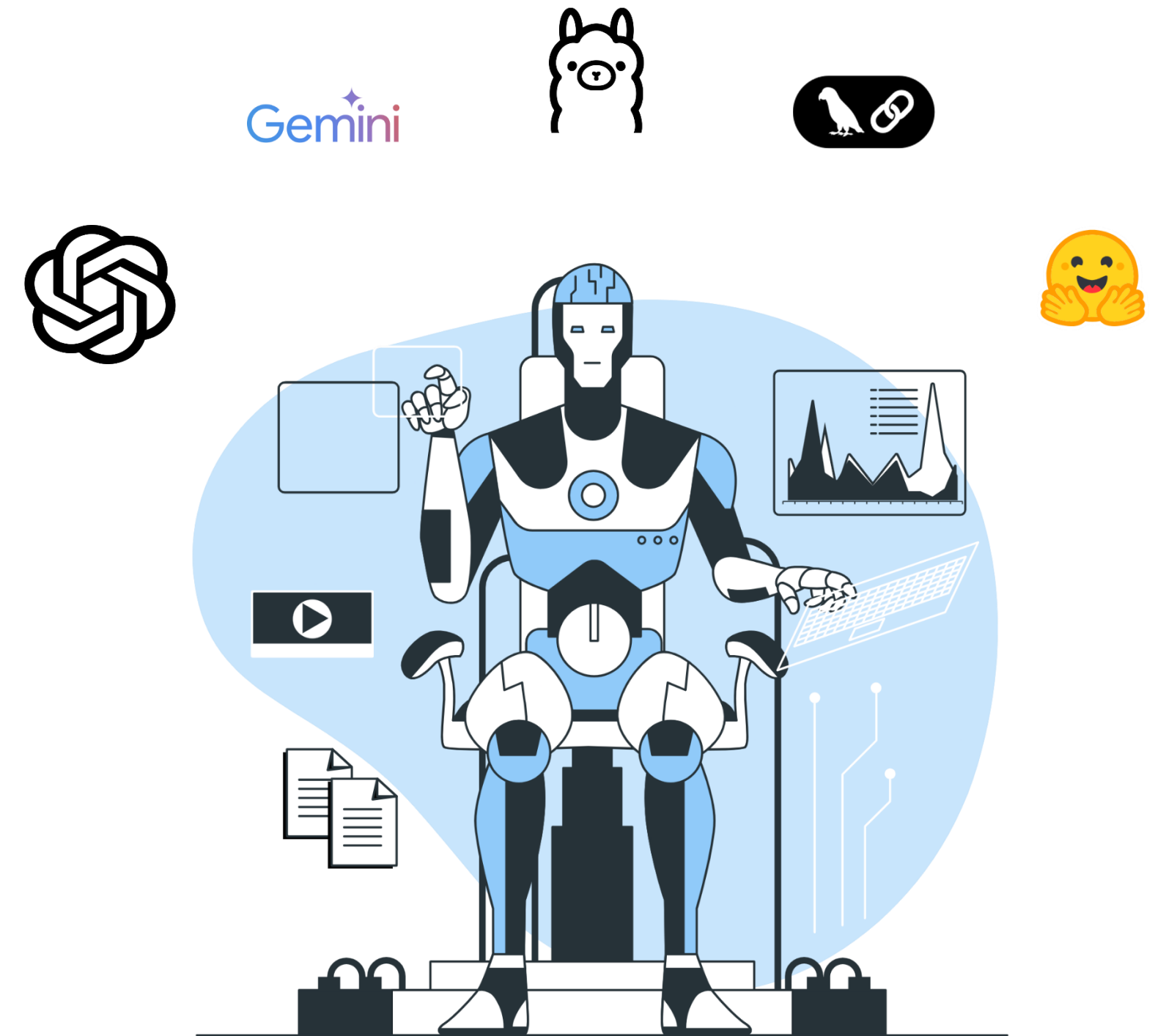
Introduction to Agentic Design Patterns

Summary

- ✓ Reusable, proven blueprints for structuring complex software systems
- ✓ Reliably organize agents, workflows, and complexity in agentic AI
- ✓ Simplify agent behavior; improve clarity, buildability, and maintainability

Section 8.2

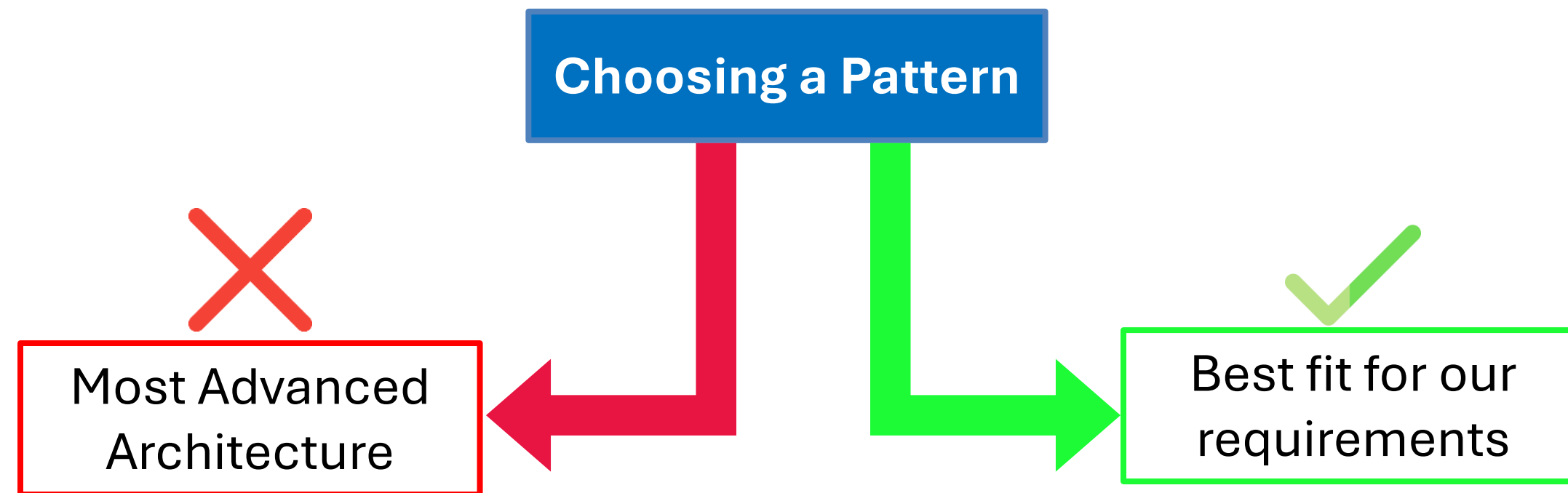
How to Choose the Right Agentic Design Pattern



How to Choose the Right Agentic Design Pattern



“How do we choose the right Agentic Design Pattern for our system?”

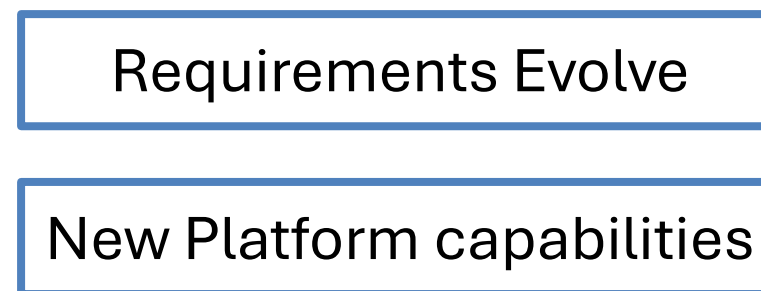


How to Choose the Right Agentic Design Pattern

Overview of Design process



The process is **Iterative**



Workload

Revisit & refine the architecture over time

How to Choose the Right Agentic Design Pattern

Define your Requirements

- ❑ Understanding **what kind of problem you're solving**, not about architecture yet

Nature of the Task

- Check whether the task is **well-structured with clear steps** or **open-ended and exploratory**
- Some problems need **planning, reasoning, and decision-making**
- Others can be solved in a **single, direct model call**

Latency & performance expectations

- Does the application need **fast, interactive responses**?
- Can it afford a **delay** in exchange for more **accurate or thorough results**?
- Different workloads have different **trade-offs** between **speed and quality**

Cost Constraints

- Involve multiple model calls for a **single request**
- Important to understand the **inference budget** and whether your use case can **support that overhead**

Human Involvement

- If the task involves:
 - High-stakes decisions
 - Safety concerns
 - Subjective judgment
- System may need human review or approval at certain points
- Important checkpoint is to ask: **Do you even need an agentic system for this problem?**

How to Choose the Right Agentic Design Pattern

Review the Available Design Patterns

Single-Agent systems

Multi-Agent systems

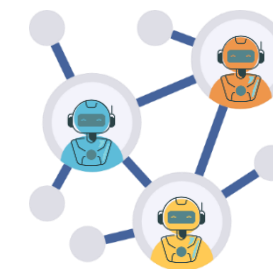
- ✓ Each pattern is designed to handle different workload characteristics



**Task
Complexity**



**Performance
needs**



**Coordination
Requirements**

How to Choose the Right Agentic Design Pattern

Select the Right Pattern

- Based on:
 - ☐ Task characteristics
 - ☐ Latency expectations
 - ☐ Cost constraints
 - ☐ Human involvement needs
- As your system evolves or new capabilities emerge, you can revisit and switch to a different pattern
- Good agentic architecture is designed to adapt over time

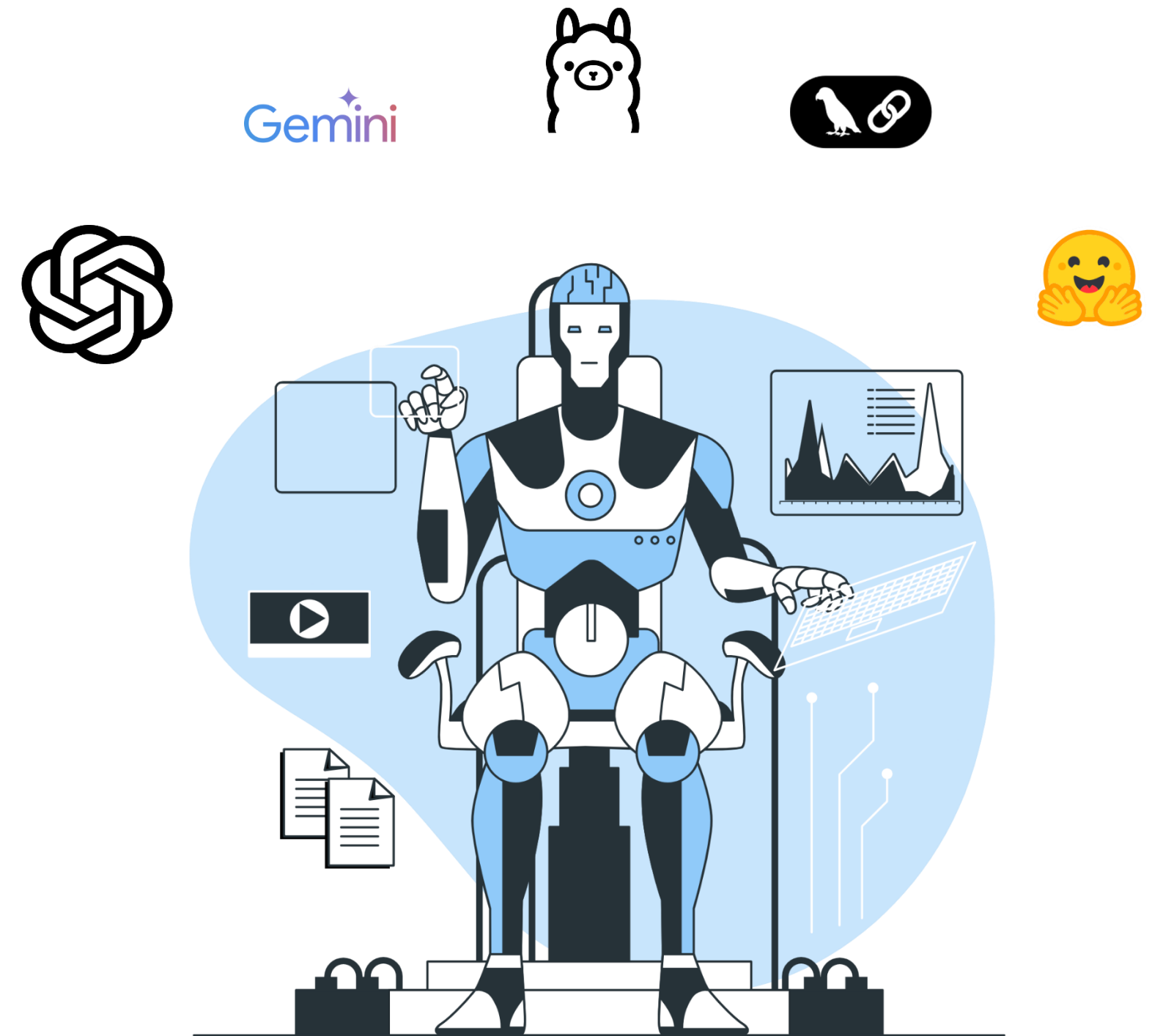
How to Choose the Right Agentic Design Pattern

Summary

- ✓ Defining your requirements
- ✓ Not every problem needs an agentic solution
- ✓ Design pattern selection is a structured, iterative process
- ✓ The best pattern is the one that fits your workload, not the most complex one

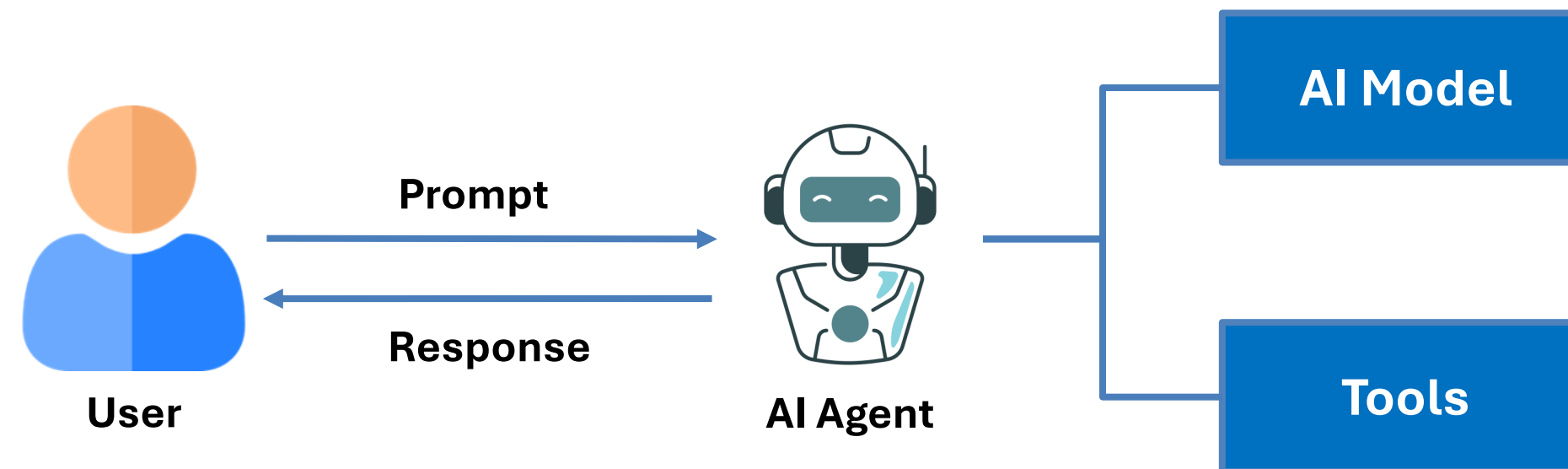
Section 8.3

Single-Agent Design Patterns



Single-Agent Design Patterns

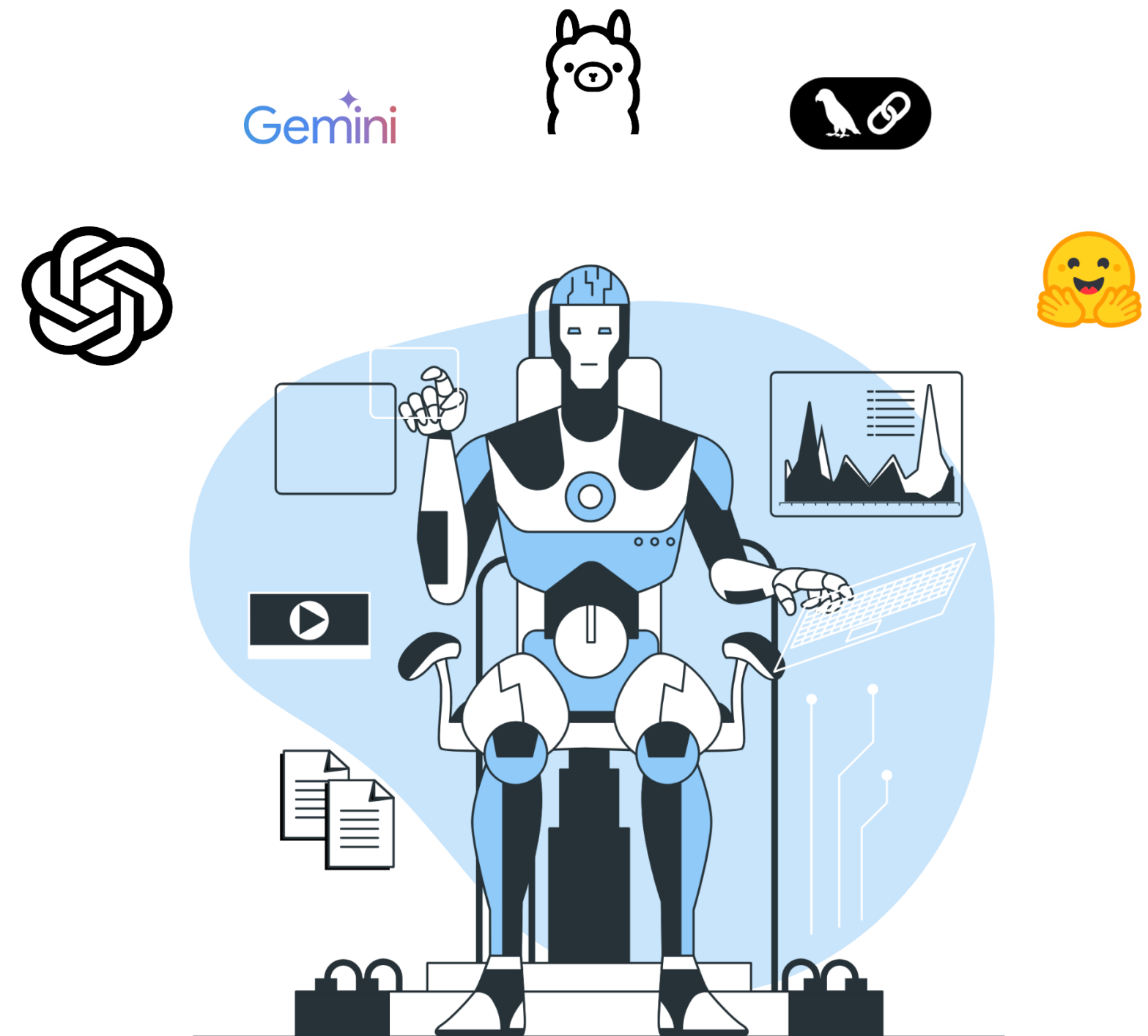
Single-Agent Design Patterns



System prompt defines the agent's role, behaviour, constraints, and tool-usage rules

Section 8.4

Multi-Agent Design Patterns Part 1



Multi-Agent Design Patterns

Multi-Agent Design Patterns

- ✓ A single agent is often insufficient for solving complex problems
- ✓ The overall goal is broken into smaller tasks, each handled by a specialized agent
- ✓ Agents collaborate, for example, collecting data, analyzing it, and validating results
- ✓ Structured collaboration makes systems scalable, reliable, and maintainable, guided by agentic design patterns

Multi-Agent Design Patterns

Multi-Agent Design Patterns

- ✓ Agents need the right information (context) to work correctly
- ✓ Context may include documents, rules, constraints, history, or other agents' outputs
- ✓ Context engineering decides what information to share and how to share it
- ✓ Good context engineering reduces confusion and cost, improves accuracy, and limits unnecessary data exposure

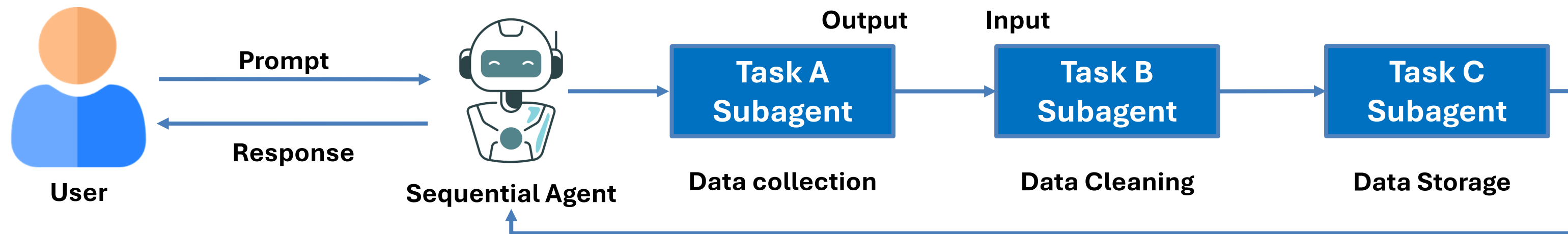
Multi-Agent Design Patterns

Multi-Agent Design Patterns

- ✓ Require more model calls, higher costs, stronger security, and tighter coordination
- ✓ Without structure, systems can become messy and hard to manage
- ✓ Provide proven ways to keep systems organized, predictable, and scalable

Multi-Agent Design Patterns

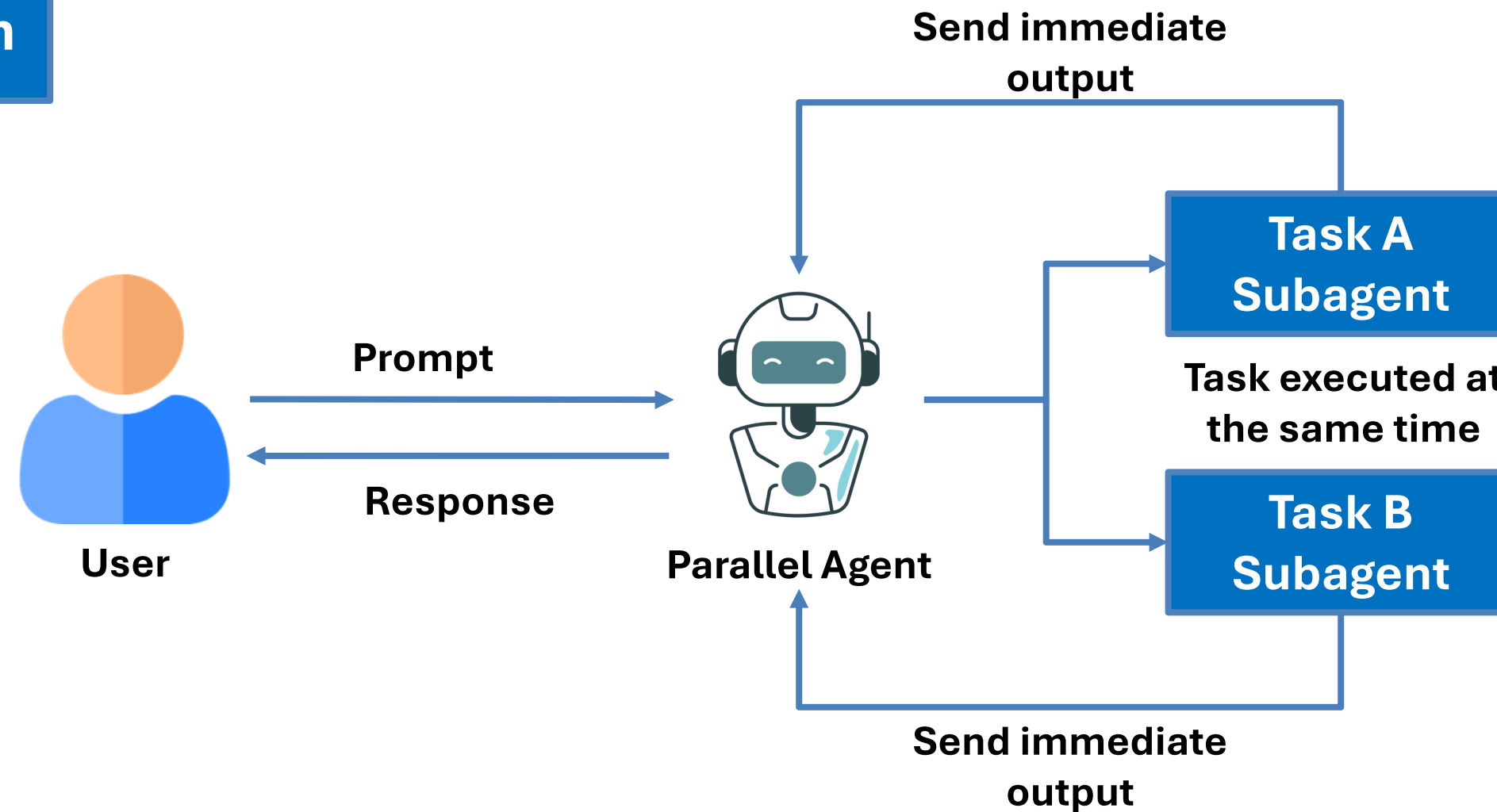
Sequential Pattern



- ✓ The linear flow makes the pattern simple, cost-effective, and easy to debug
- ✓ It is inflexible, all steps run regardless of need, and one slow agent delays the entire flow

Multi-Agent Design Patterns

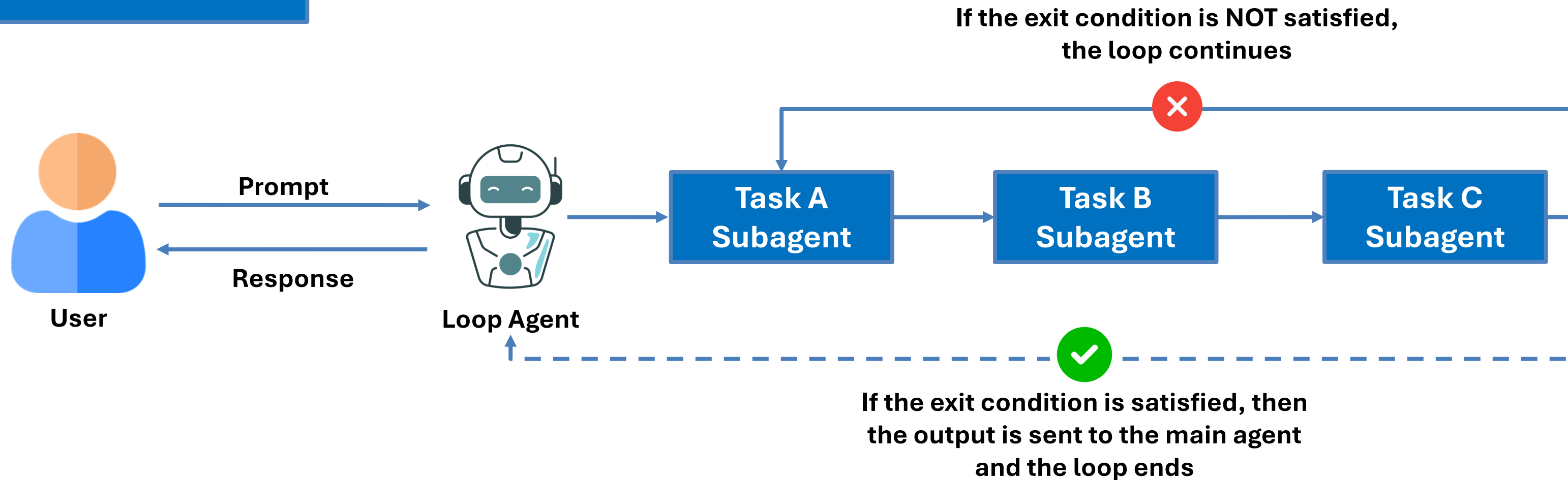
Parallel Pattern



- ✓ Works best when tasks are independent of each other
- ✓ Provides faster results and multiple viewpoints
- ✓ Comes with higher cost and complexity due to multiple agents and result coordination

Multi-Agent Design Patterns

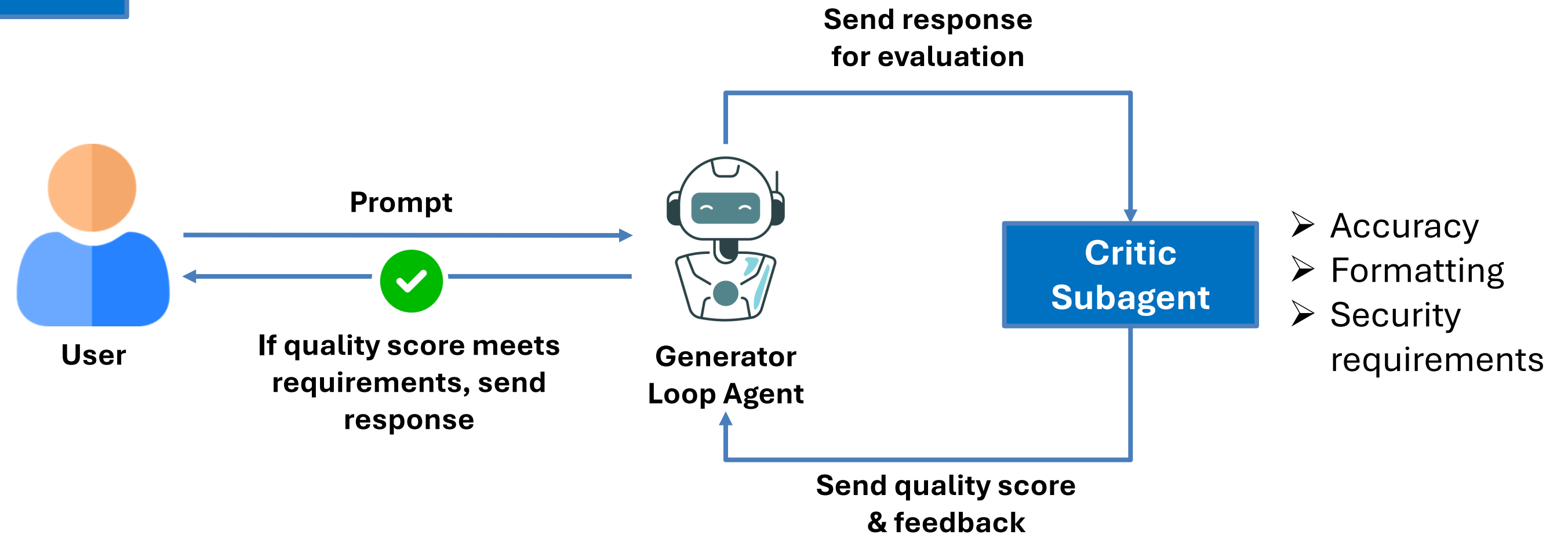
Loop Pattern



- ✓ The loop continues until a quality level is reached or a maximum number of attempts is exceeded
- ✓ Poor stopping rules can cause endless loops, high costs, and unstable behaviour

Multi-Agent Design Patterns

Review & Critique Pattern



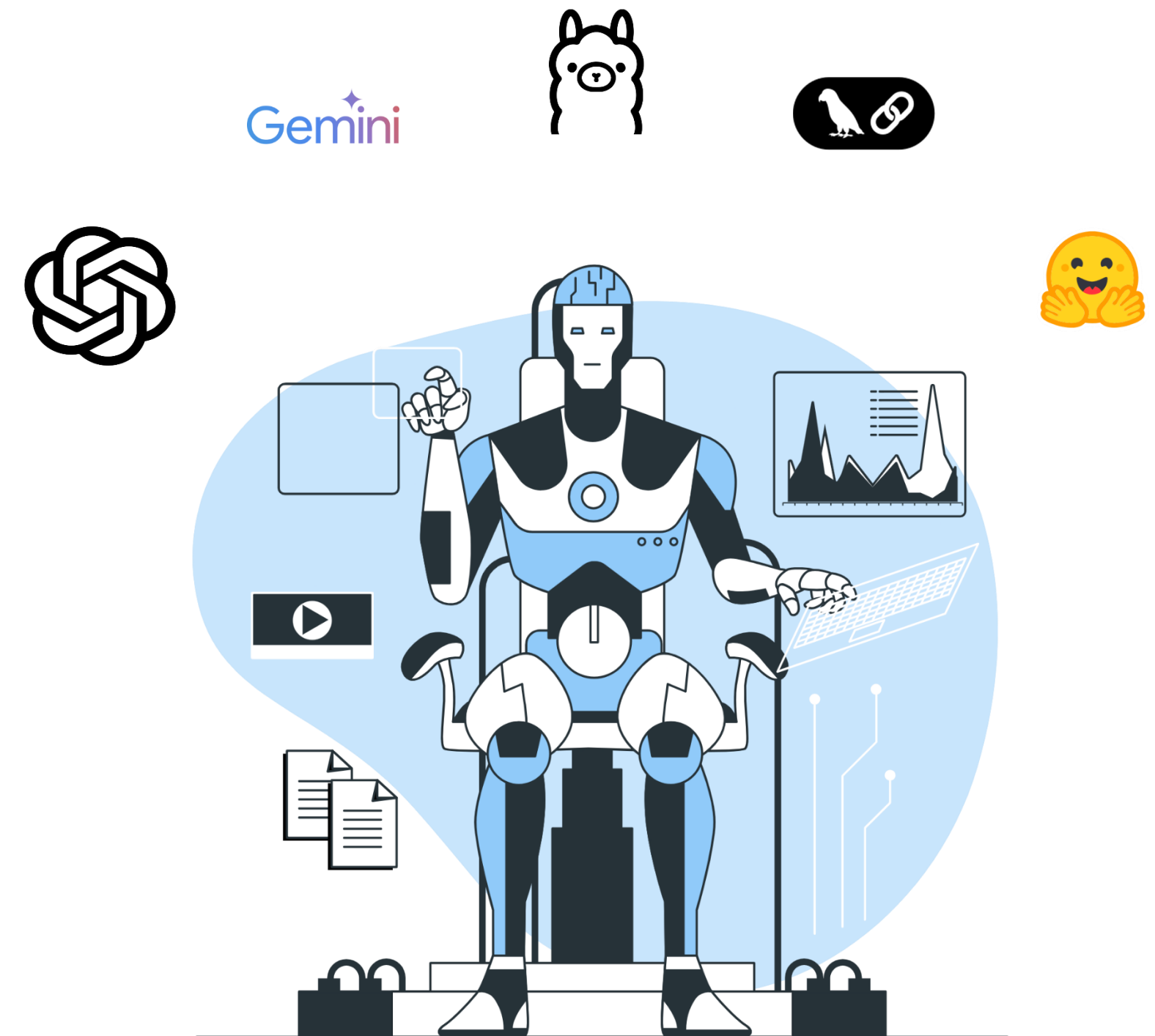
- ✓ Ideal for critical tasks like code generation or compliance-sensitive content
- ✓ Improves reliability but increases time and cost due to extra reviews and revisions

Multi-Agent Design Patterns

- ✓ The goal is not to memorize patterns, but to know when to use them
- ✓ Sequential patterns suit structured and predictable workflows
- ✓ Parallel patterns are best when speed and multiple perspectives are needed
- ✓ Loop patterns support refinement and continuous improvement
- ✓ Review and critique patterns help ensure quality and safety.

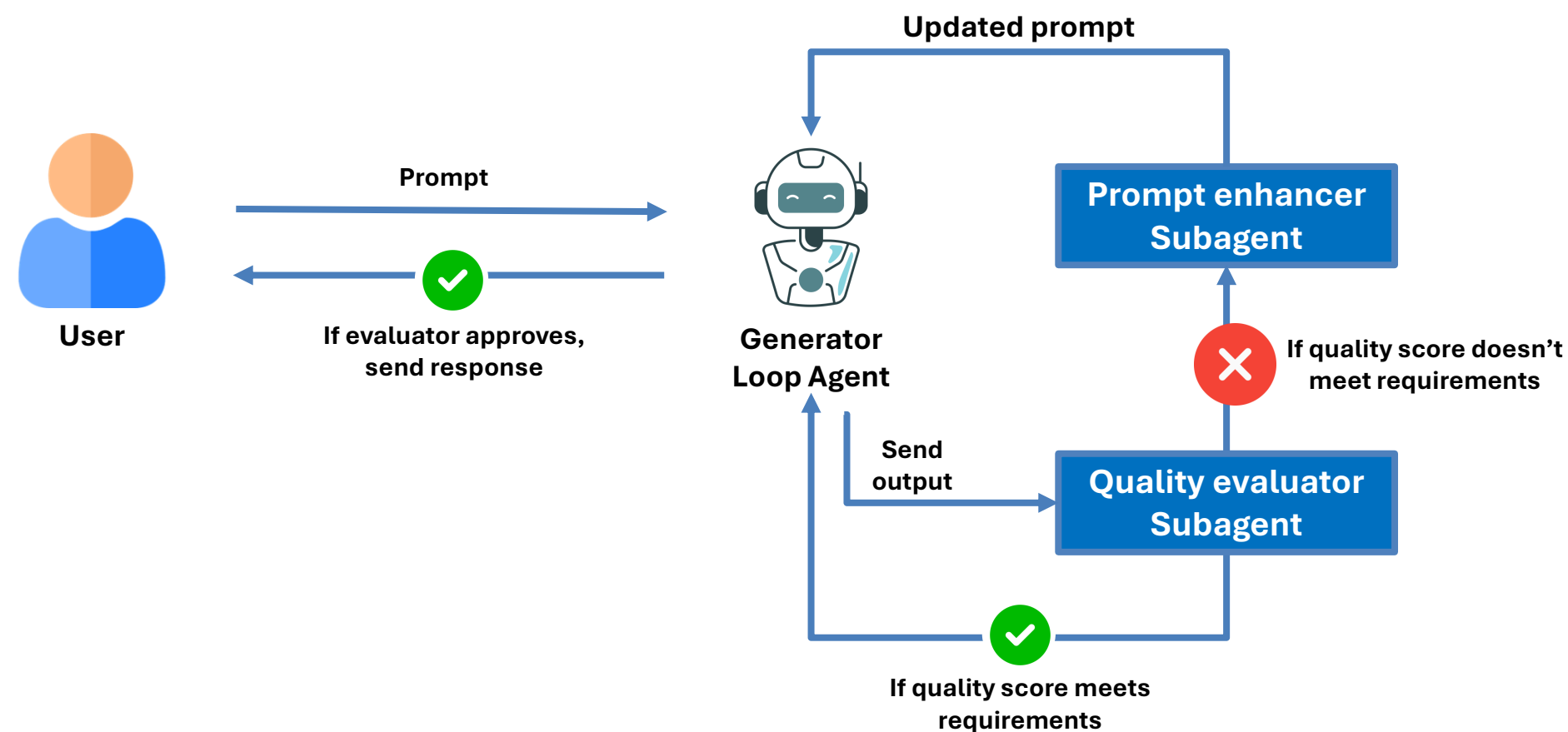
Section 8.5

Multi-Agent Design Patterns Part 2



Multi-Agent Design Patterns

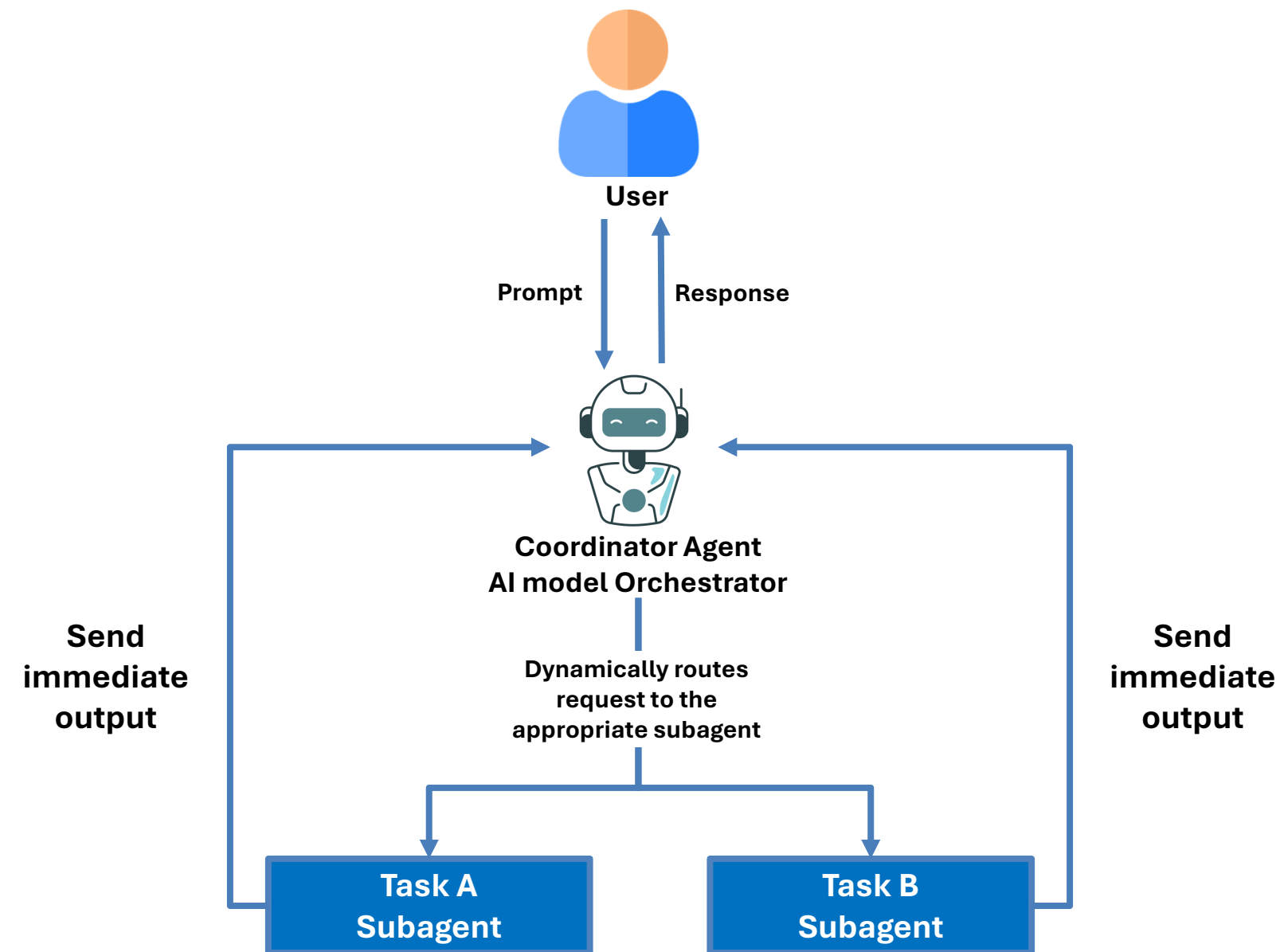
Iterative Refinement Pattern



- ✓ This pattern is used when high-quality output can't be achieved in a single step
- ✓ The system iteratively refines the output over multiple cycles
- ✓ Agents work in a loop, updating the output stored in system state
- ✓ After each cycle, a quality check decides whether to stop or continue
- ✓ Commonly used for code writing/debugging, long-form content, and detailed planning
- ✓ The key benefit is high-quality output through refinement
- ✓ Each iteration adds latency and cost
- ✓ Careful exit conditions are needed to prevent excessive execution

Multi-Agent Design Patterns

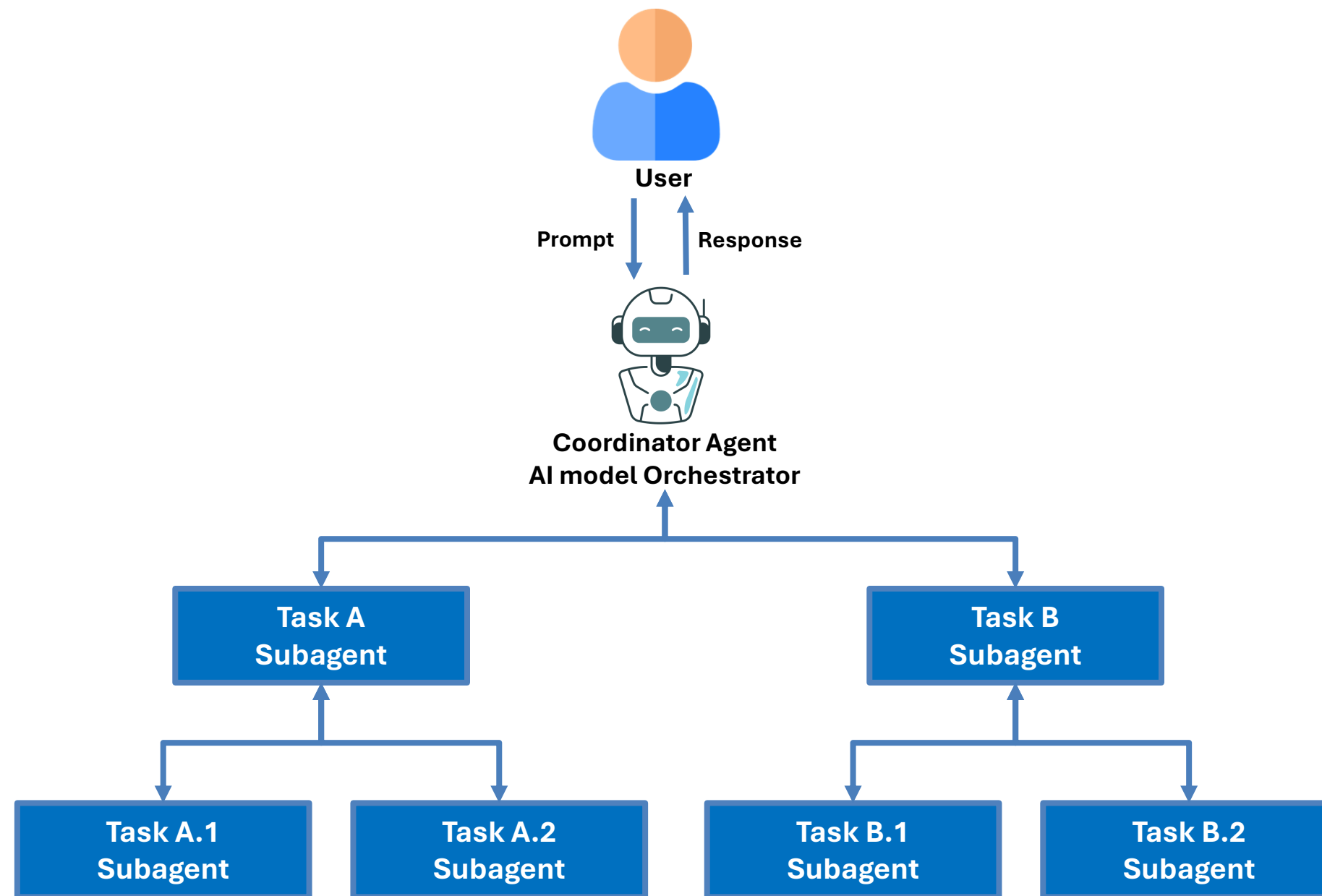
Coordinator Pattern



- ✓ A coordinator agent receives the request and breaks it into smaller sub-tasks
- ✓ Each sub-task is dynamically routed to the most suitable specialized agent
- ✓ The workflow is not hardcoded; routing decisions are made using an AI model
- ✓ Example: In customer support, the coordinator routes requests to order tracking, refund, or return agents
- ✓ This pattern is flexible and adaptive, but results in higher latency and cost due to extra model calls

Multi-Agent Design Patterns

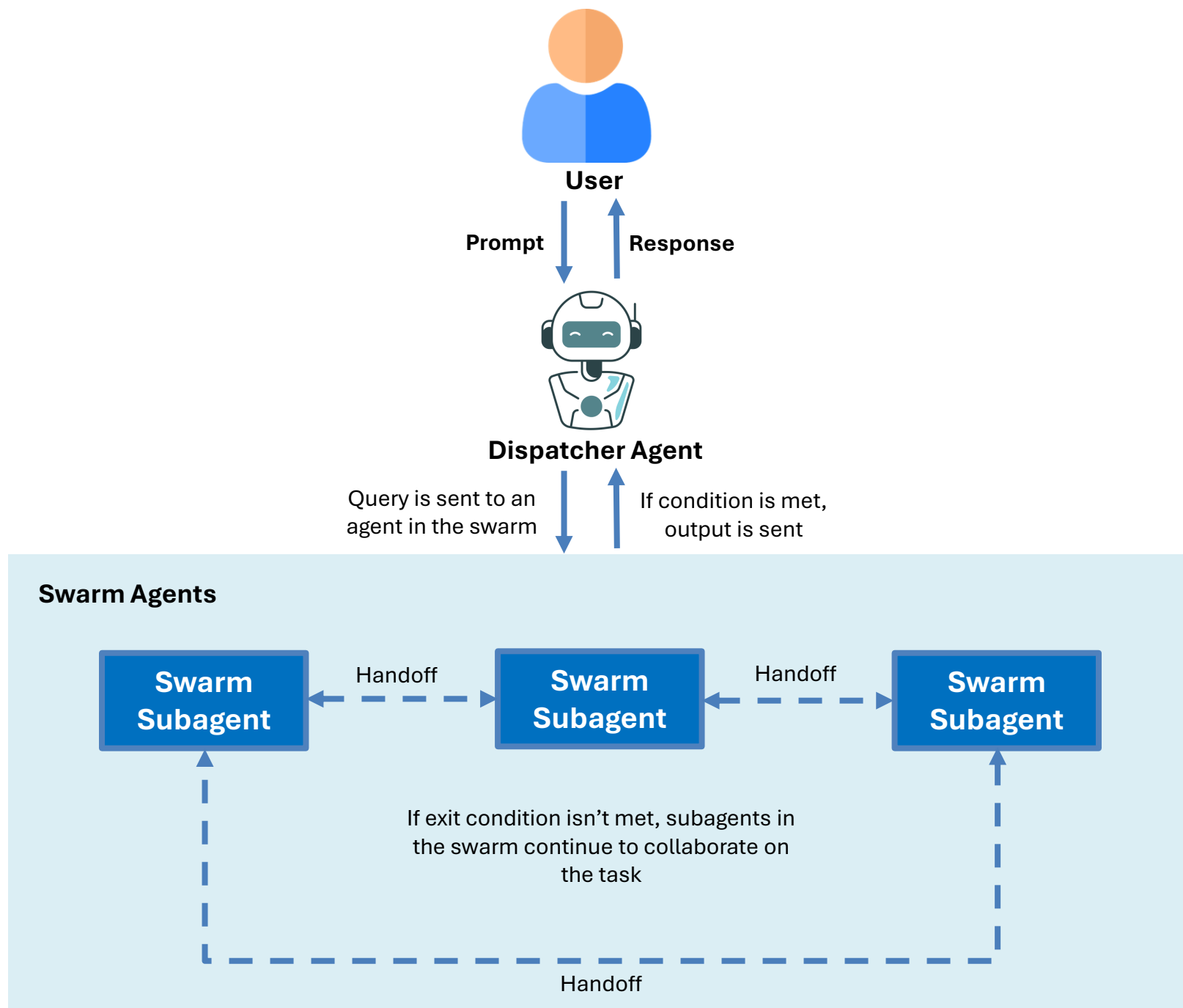
Hierarchical Task Decomposition Pattern



- ✓ Designed for very large and ambiguous problems
- ✓ A top-level agent decomposes tasks and delegates them to lower-level agents
- ✓ Well-suited for research, planning, and synthesis
- ✓ Delivers high-quality results but with high complexity, cost, and latency

Multi-Agent Design Patterns

Swarm Pattern



- ✓ The most collaborative and least structured agentic pattern
- ✓ A dispatcher sends the request, but agents communicate freely without strict control
- ✓ Best for creative and highly ambiguous problems like product design or strategy
- ✓ Produces innovative results, but is complex, costly, and hard to converge without clear exit rules

Multi-Agent Design Patterns

Summary

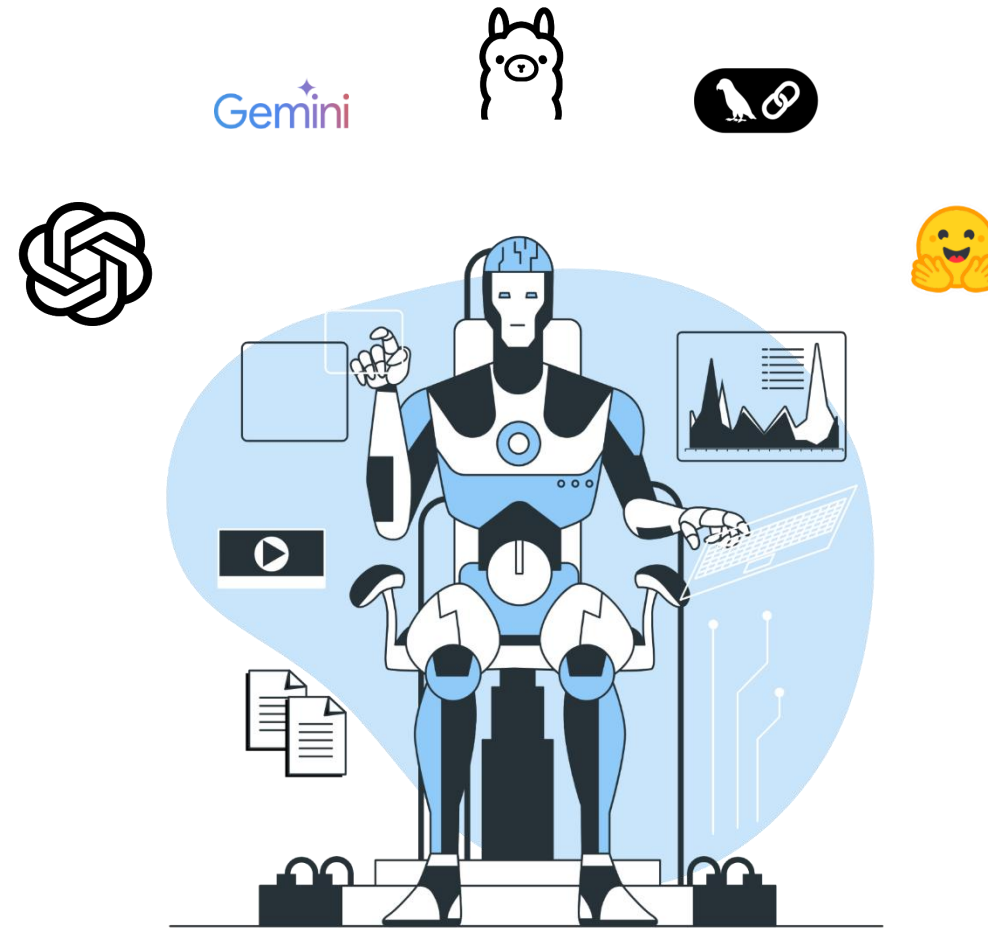
- ✓ Each pattern serves a distinct purpose in agentic systems
- ✓ Iterative refinement improves quality over time
- ✓ Coordinator pattern enables dynamic routing
- ✓ Hierarchical decomposition handles large and complex problems
- ✓ Swarm pattern supports deep collaboration and creativity

Summary

- ✓ Understood Agentic Design Patterns and its importance
- ✓ Learned how to choose the right Agentic Design Pattern
- ✓ Explored Single & Multi-Agent Design Patterns

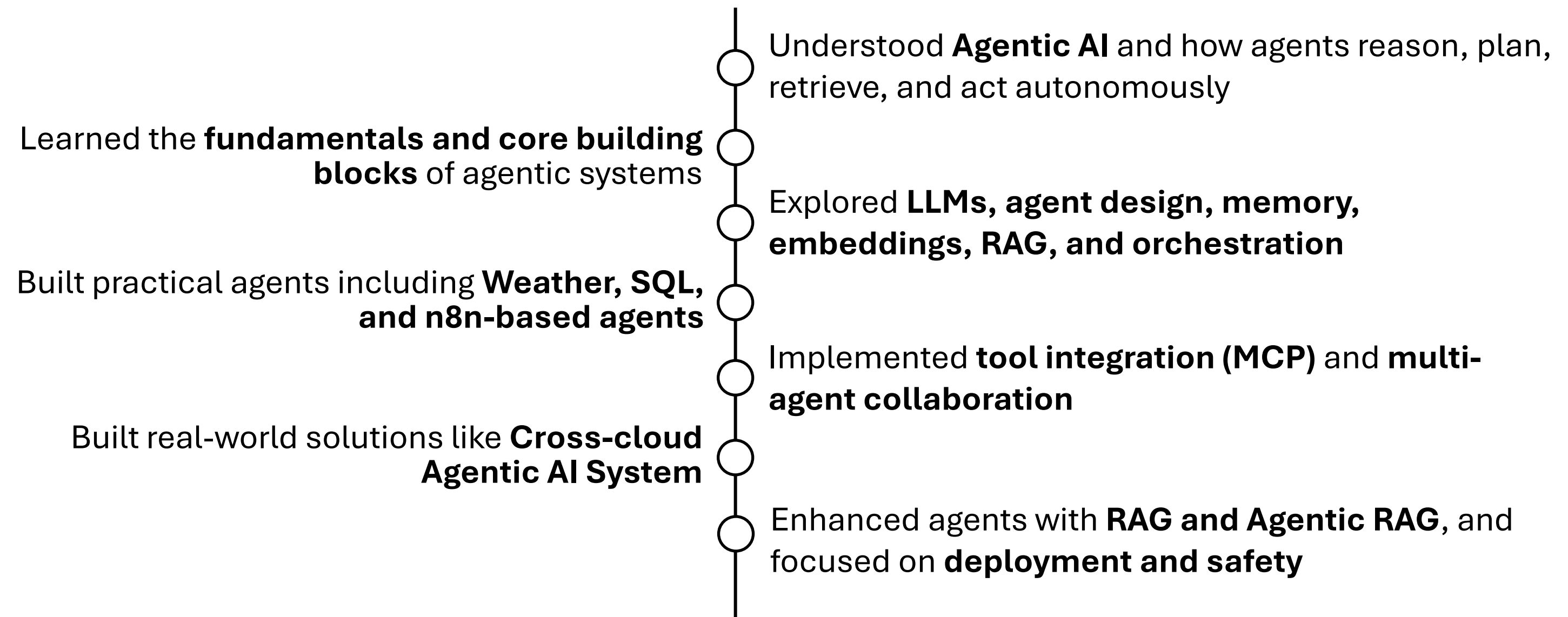
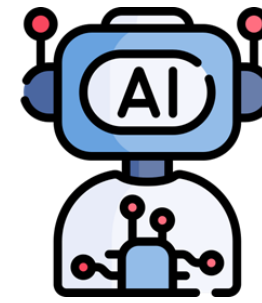
Thank You

Conclusion



Building Autonomous Agentic AI Systems for Beginners Hands-On

- By Thinknyx Technologies LLP





Microsoft Certified Azure AI Engineer Associate : AI-102

Learn Microsoft Azure AI-102 Skills to Build and Deploy AI Solutions on Azure

Yogesh Raheja, Thinknyx Technologies

Course ★ 4.6 265 ratings 3 total hours 46 lectures All Levels

₹449 ₹799



Introduction to Chatbots - Build Your First AI Assistant

Design, Develop, and Deploy Your First Intelligent Chatbot

Yogesh Raheja

Course ★ 4.8 98 ratings 2 total hours 20 lectures All Levels

₹449 ₹799



**THANK
YOU!**

Follow us on:



@thinknyx



@thinknyx



@thinknyx-technologies



@thinknyx



@thinknyx-technologies